Preface to the second edition

Welcome to the second edition of “R for Data Science”! This is a major reworking of the first edition, removing material we no longer think is useful, adding material we wish we included in the first edition, and generally updating the text and code to reflect changes in best practices. We’re also very excited to welcome a new co-author: Mine Çetinkaya-Rundel, a noted data science educator and one of our colleagues at Posit (the company formerly known as RStudio).

A brief summary of the biggest changes follows:

- The first part of the book has been renamed to “Whole game”. The goal of this section is to give you the rough details of the “whole game” of data science before we dive into the details.
- The second part of the book is “Visualize”. This part gives data visualization tools and best practices a more thorough coverage compared to the first edition. The best place to get all the details is still the ggplot2 book, but now R4DS covers more of the most important techniques.
- The third part of the book is now called “Transform” and gains new chapters on numbers, logical vectors, and missing values. These were previously parts of the data transformation chapter, but needed much more room to cover all the details.
- The fourth part of the book is called “Import”. It’s a new set of chapters that goes beyond reading flat text files to working with spreadsheets, getting data out of databases, working with big data, rectangling hierarchical data, and scraping data from web sites.
- The “Program” part remains, but has been rewritten from top-to-bottom to focus on the most important parts of function writing and iteration. Function writing now includes details on how to wrap tidyverse functions (dealing with the challenges of tidy evaluation), since this has become much easier and more important over the last few years. We’ve added a new chapter on important base R functions that you’re likely to see in wild-caught R code.
- The modeling part has been removed. We never had enough room to fully do modelling justice, and there are now much better resources available. We generally recommend using the tidymodels packages and reading Tidy Modeling with R by Max Kuhn and Julia Silge.
- The “Communicate” part remains, but has been thoroughly updated to feature Quarto instead of R Markdown. This edition of the book has been written in Quarto, and it’s clearly the tool of the future.

Welcome
Introduction

Introduction

Data science is an exciting discipline that allows you to transform raw data into understanding, insight, and knowledge. The goal of "R for Data Science" is to help you learn the most important tools in R that will allow you to do data science efficiently and reproducibly, and to have some fun along the way 😃. After reading this book, you'll have the tools to tackle a wide variety of data science challenges using the best parts of R.

What you will learn

Data science is a vast field, and there's no way you can master it all by reading a single book. This book aims to give you a solid foundation in the most important tools and enough knowledge to find the resources to learn more when necessary. Our model of the steps of a typical data science project looks something like Figure 1.

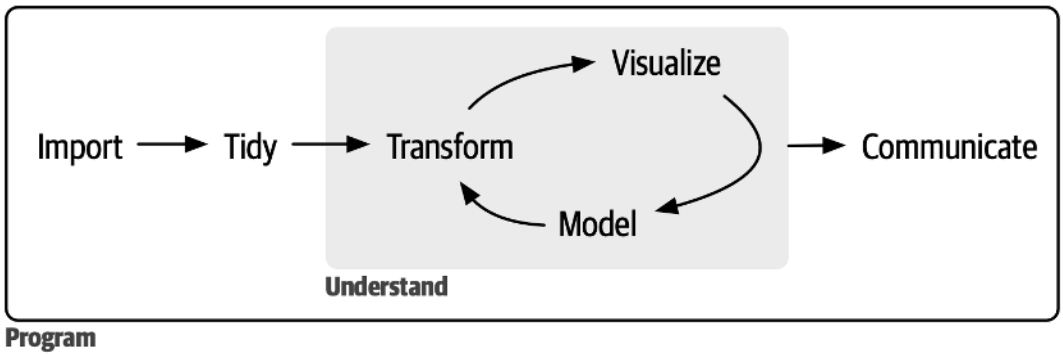


Figure 1: In our model of the data science process, you start with data import and tidying. Next, you understand your data with an iterative cycle of transforming, visualizing, and modeling. You finish the process by communicating your results to other humans.

First, you must **import** your data into R. This typically means that you take data stored in a file, database, or web application programming interface (API) and load it into a data frame in R. If you can't get your data into R, you can't do data science on it!

Once you've imported your data, it is a good idea to **tidy** it. Tidying your data means storing it in a consistent form that matches the semantics of the dataset with how it is stored. In brief, when your data is tidy, each column is a variable and each row is an observation. Tidy data is important because the consistent structure lets you focus your efforts on answering questions about the data, not fighting to get the data into the right form for different functions.

Once you have tidy data, a common next step is to **transform** it. Transformation includes narrowing in on observations of interest (like all people in one city or all data from the last year), creating new variables that

are functions of existing variables (like computing speed from distance and time), and calculating a set of summary statistics (like counts or means). Together, tidying and transforming are called **wrangling** because getting your data in a form that's natural to work with often feels like a fight!

Once you have tidy data with the variables you need, there are two main engines of knowledge generation: visualization and modeling. These have complementary strengths and weaknesses, so any real data analysis will iterate between them many times.

Visualization is a fundamentally human activity. A good visualization will show you things you did not expect or raise new questions about the data. A good visualization might also hint that you're asking the wrong question or that you need to collect different data. Visualizations can surprise you, but they don't scale particularly well because they require a human to interpret them.

Models are complementary tools to visualization. Once you have made your questions sufficiently precise, you can use a model to answer them. Models are fundamentally mathematical or computational tools, so they generally scale well. Even when they don't, it's usually cheaper to buy more computers than it is to buy more brains! But every model makes assumptions, and by its very nature, a model cannot question its own assumptions. That means a model cannot fundamentally surprise you.

The last step of data science is **communication**, an absolutely critical part of any data analysis project. It doesn't matter how well your models and visualization have led you to understand the data unless you can also communicate your results to others.

Surrounding all these tools is **programming**. Programming is a cross-cutting tool that you use in nearly every part of a data science project. You don't need to be an expert programmer to be a successful data scientist, but learning more about programming pays off because becoming a better programmer allows you to automate common tasks and solve new problems with greater ease.

You'll use these tools in every data science project, but they're not enough for most projects. There's a rough 80/20 rule at play: you can tackle about 80% of every project using the tools you'll learn in this book, but you'll need other tools to tackle the remaining 20%. Throughout this book, we'll point you to resources where you can learn more.

How this book is organized

The previous description of the tools of data science is organized roughly according to the order in which you use them in an analysis (although, of course, you'll iterate through them multiple times). In our experience, however, learning data importing and tidying first is suboptimal because, 80% of the time, it's routine and boring, and the other 20% of the time, it's weird and frustrating. That's a bad place to start learning a new subject! Instead, we'll start with visualization and transformation of data that's already been imported and tidied. That way, when you ingest and tidy your own data, your motivation will stay high because you know the pain is worth the effort.

Within each chapter, we try to adhere to a consistent pattern: start with some motivating examples so you can see the bigger picture, and then dive into the details. Each section of the book is paired with exercises to help you practice what you've learned. Although it can be tempting to skip the exercises, there's no better way to learn than by practicing on real problems.

What you won't learn

There are several important topics that this book doesn't cover. We believe it's important to stay ruthlessly focused on the essentials so you can get up and running as quickly as possible. That means this book can't cover every important topic.

Modeling

Modeling is super important for data science, but it's a big topic, and unfortunately, we just don't have the space to give it the coverage it deserves here. To learn more about modeling, we highly recommend Tidy Modeling with R by our colleagues Max Kuhn and Julia Silge. This book will teach you the tidymodels family of packages, which, as you might guess from the name, share many conventions with the tidyverse packages we use in this book.

Big data

This book proudly and primarily focuses on small, in-memory datasets. This is the right place to start because you can't tackle big data unless you have experience with small data. The tools you'll learn throughout the majority of this book will easily handle hundreds of megabytes of data, and with a bit of care, you can typically use them to work with a few gigabytes of data. We'll also show you how to get data out of databases and parquet files, both of which are often used to store big data. You won't necessarily be able to work with the entire dataset, but that's not a problem because you only need a subset or subsample to answer the question that you're interested in.

If you're routinely working with larger data (10–100 GB, say), we recommend learning more about data.table. We don't teach it here because it uses a different interface than the tidyverse and requires you to learn some different conventions. However, it is incredibly faster, and the performance payoff is worth investing some time in learning it if you're working with large data.

Python, Julia, and friends

In this book, you won't learn anything about Python, Julia, or any other programming language useful for data science. This isn't because we think these tools are bad. They're not! And in practice, most data science teams use a mix of languages, often at least R and Python. But we strongly believe that it's best to master one tool at a time, and R is a great place to start.

Prerequisites

We've made a few assumptions about what you already know to get the most out of this book. You should be generally numerically literate, and it's helpful if you have some basic programming experience already. If you've never programmed before, you might find Hands on Programming with R by Garrett to be a valuable adjunct to this book.

You need four things to run the code in this book: R, RStudio, a collection of R packages called the **tidyverse**, and a handful of other packages. Packages are the fundamental units of reproducible R code. They include reusable functions, documentation that describes how to use them, and sample data.

R

To download R, go to CRAN, the **c**omprehensive **R** **a**rchive **n**etwork, https://cloud.r-project.org. A new major version of R comes out once a year, and there are 2-3 minor releases each year. It's a good idea to update regularly. Upgrading can be a bit of a hassle, especially for major versions that require you to re-install all your packages, but putting it off only makes it worse. We recommend R 4.2.0 or later for this book.

RStudio

RStudio is an integrated development environment, or IDE, for R programming, which you can download from https://posit.co/download/rstudio-desktop/. RStudio is updated a couple of times a year, and it will automatically let you know when a new version is out, so there's no need to check back. It's a good idea to upgrade regularly to take advantage of the latest and greatest features. For this book, make sure you have at least RStudio 2022.02.0.

When you start RStudio, Figure 2, you'll see two key regions in the interface: the console pane and the output pane. For now, all you need to know is that you type the R code in the console pane and press enter to run it. You'll learn more as we go along![1]

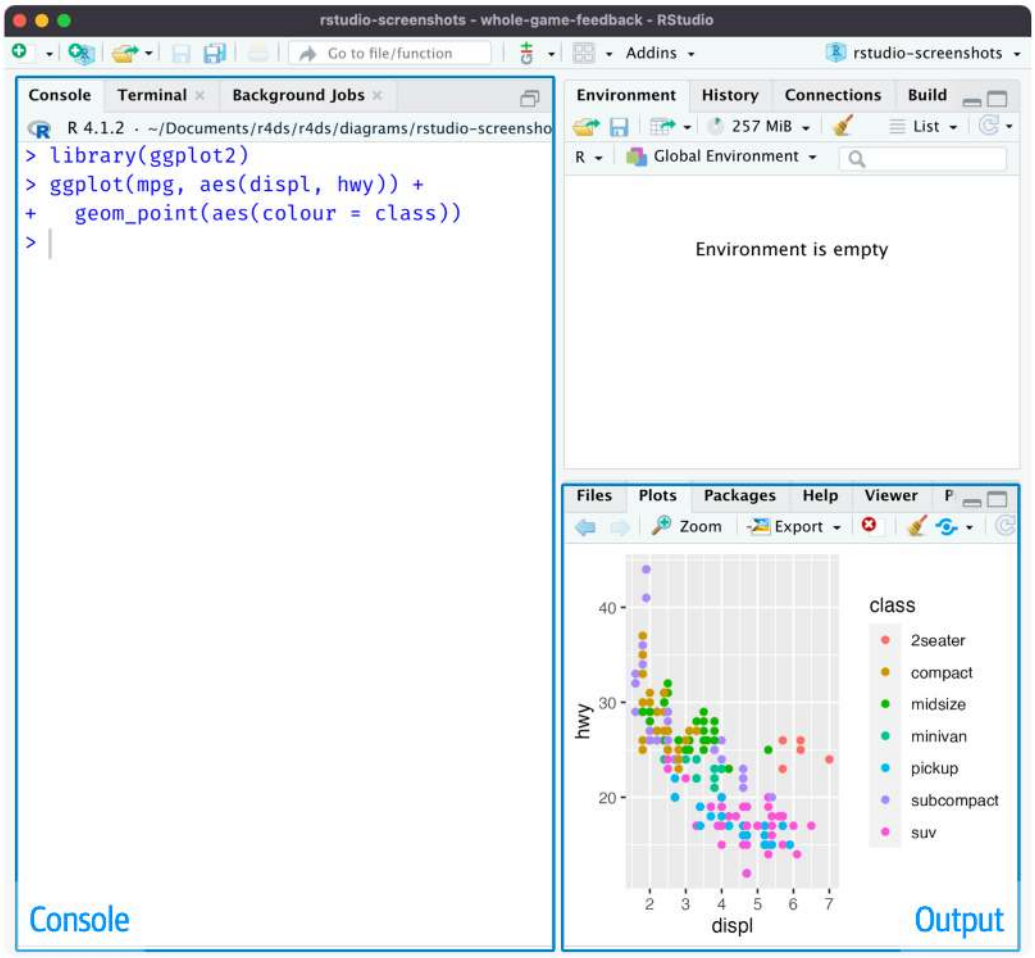


Figure 2: The RStudio IDE has two key regions: type R code in the console pane on the left, and look for plots in the output pane on the right.

The tidyverse

You'll also need to install some R packages. An R **package** is a collection of functions, data, and documentation that extends the capabilities of base R. Using packages is key to the successful use of R. The majority of the packages that you will learn in this book are part of the so-called tidyverse. All packages in the tidyverse share a common philosophy of data and R programming and are designed to work together.

You can install the complete tidyverse with a single line of code:

```
install.packages("tidyverse")
```

On your computer, type that line of code in the console, and then press enter to run it. R will download the packages from CRAN and install them on your computer.

You will not be able to use the functions, objects, or help files in a package until you load it with `library()`. Once you have installed a package, you can load it using the `library()` function:

```
library(tidyverse)
#> ── Attaching core tidyverse packages ──────────────────────── tidyverse 2.0.0
──
#> ✔ dplyr     1.1.4     ✔ readr     2.1.5
#> ✔ forcats   1.0.0     ✔ stringr   1.5.1
#> ✔ ggplot2   3.5.1     ✔ tibble    3.2.1
#> ✔ lubridate 1.9.3     ✔ tidyr     1.3.1
#> ✔ purrr     1.0.2
#> ── Conflicts ────────────────────────────────────────── tidyverse_conflicts()
──
#> ✖ dplyr::filter() masks stats::filter()
#> ✖ dplyr::lag()    masks stats::lag()
#> ℹ Use the conflicted package (<http://conflicted.r-lib.org/>) to force all
conflicts to become errors
```

This tells you that tidyverse loads nine packages: dplyr, forcats, ggplot2, lubridate, purrr, readr, stringr, tibble, tidyr. These are considered the **core** of the tidyverse because you'll use them in almost every analysis.

Packages in the tidyverse change fairly frequently. You can see if updates are available by running `tidyverse_update()`.

Other packages

There are many other excellent packages that are not part of the tidyverse because they solve problems in a different domain or are designed with a different set of underlying principles. This doesn't make them better or worse; it just makes them different. In other words, the complement to the tidyverse is not the messyverse but many other universes of interrelated packages. As you tackle more data science projects with R, you'll learn new packages and new ways of thinking about data.

We'll use many packages from outside the tidyverse in this book. For example, we'll use the following packages because they provide interesting datasets for us to work with in the process of learning R:

```
install.packages(
  c("arrow", "babynames", "curl", "duckdb", "gapminder",
    "ggrepel", "ggridges", "ggthemes", "hexbin", "janitor", "Lahman",
    "leaflet", "maps", "nycflights13", "openxlsx", "palmerpenguins",
    "repurrrsive", "tidymodels", "writexl")
  )
```

We'll also use a selection of other packages for one off examples. You don't need to install them now, just remember that whenever you see an error like this:

You need to run `install.packages("ggrepel")` to install the package.

Running R code

The previous section showed you several examples of running R code. The code in the book looks like this:

```
1 + 2
#> [1] 3
```

If you run the same code in your local console, it will look like this:

```
> 1 + 2
[1] 3
```

There are two main differences. In your console, you type after the `>`, called the **prompt**; we don't show the prompt in the book. In the book, the output is commented out with `#>`; in your console, it appears directly after your code. These two differences mean that if you're working with an electronic version of the book, you can easily copy code out of the book and paste it into the console.

Throughout the book, we use a consistent set of conventions to refer to code:

- Functions are displayed in a code font and followed by parentheses, like `sum()` or `mean()`.
- Other R objects (such as data or function arguments) are in a code font, without parentheses, like `flights` or `x`.
- Sometimes, to make it clear which package an object comes from, we'll use the package name followed by two colons, like `dplyr::mutate()` or `nycflights13::flights`. This is also valid R code.

Acknowledgments

This book isn't just the product of Hadley, Mine, and Garrett but is the result of many conversations (in person and online) that we've had with many people in the R community. We're incredibly grateful for all the conversations we've had with y'all; thank you so much!

This book was written in the open, and many people contributed via pull requests. A special thanks to all 259 of you who contributed improvements via GitHub pull requests (in alphabetical order by username): @a-rosenberg, Tim Becker (@a2800276), Abinash Satapathy (@Abinashbunty), Adam Gruer (@adam-gruer), adi pradhan (@adidoit), A. s. (@Adrianzo), Aep Hidyatuloh (@aephidayatuloh), Andrea Gilardi (@agila5), Ajay Deonarine (@ajay-d), @AlanFeder, Daihe Sui (@alansuidaihe), @alberto-agudo, @AlbertRapp, @aleloi, pete (@alonzi), Alex (@ALShum), Andrew M. (@amacfarland), Andrew Landgraf (@andland), @andyhuynh92, Angela Li (@angela-li), Antti Rask (@AnttiRask), LOU Xun (@aquarhead), @ariespirgel, @august-18, Michael Henry (@aviast), Azza Ahmed (@azzaea), Steven Moran (@bambooforest), Brian G. Barkley (@BarkleyBG), Mara Averick (@batpigandme), Oluwafemi OYEDELE (@BB1464), Brent

Brewington (@bbrewington), Bill Behrman (@behrman), Ben Herbertson (@benherbertson), Ben Marwick (@benmarwick), Ben Steinberg (@bensteinberg), Benjamin Yeh (@bentyeh), Betul Turkoglu (@betulturkoglu), Brandon Greenwell (@bgreenwell), Bianca Peterson (@BinxiePeterson), Birger Niklas (@BirgerNi), Brett Klamer (@bklamer), @boardtc, Christian (@c-hoh), Caddy (@caddycarine), Camille V Leonard (@camillevleonard), @canovasjm, Cedric Batailler (@cedricbatailler), Christina Wei (@christina-wei), Christian Mongeau (@chrMongeau), Cooper Morris (@coopermor), Colin Gillespie (@csgillespie), Rademeyer Vermaak (@csrvermaak), Chloe Thierstein (@cthierst), Chris Saunders (@ctsa), Abhinav Singh (@curious-abhinav), Curtis Alexander (@curtisalexander), Christian G. Warden (@cwarden), Charlotte Wickham (@cwickham), Kenny Darrell (@darrkj), David Kane (@davidkane9), David (@davidrsch), David Rubinger (@davidrubinger), David Clark (@DDClark), Derwin McGeary (@derwinmcgeary), Daniel Gromer (@dgromer), @Divider85, @djbirke, Danielle Navarro (@djnavarro), Russell Shean (@DOH-RPS1303), Zhuoer Dong (@dongzhuoer), Devin Pastoor (@dpastoor), @DSGeoff, Devarshi Thakkar (@dthakkar09), Julian During (@duju211), Dylan Cashman (@dylancashman), Dirk Eddelbuettel (@eddelbuettel), Edwin Thoen (@EdwinTh), Ahmed El-Gabbas (@elgabbas), Henry Webel (@enryH), Ercan Karadas (@ercan7), Eric Kitaif (@EricKit), Eric Watt (@ericwatt), Erik Erhardt (@erikerhardt), Etienne B. Racine (@etiennebr), Everett Robinson (@evjrob), @fellennert, Flemming Miguel (@flemmingmiguel), Floris Vanderhaeghe (@florisvdh), @funkybluehen, @gabrivera, Garrick Aden-Buie (@gadenbuie), Peter Ganong (@ganong123), Gerome Meyer (@GeroVanMi), Gleb Ebert (@gl-eb), Josh Goldberg (@GoldbergData), bahadir cankardes (@gridgrad), Gustav W Delius (@gustavdelius), Hao Chen (@hao-trivago), Harris McGehee (@harrismcgehee), @hendrikweisser, Hengni Cai (@hengnicai), Iain (@Iain-S), Ian Sealy (@iansealy), Ian Lyttle (@ijlyttle), Ivan Krukov (@ivan-krukov), Jacob Kaplan (@jacobkap), Jazz Weisman (@jazzlw), John Blischak (@jdblischak), John D. Storey (@jdstorey), Gregory Jefferis (@jefferis), Jeffrey Stevens (@JeffreyRStevens), 蒋雨蒙 (@JeldorPKU), Jennifer (Jenny) Bryan (@jennybc), Jen Ren (@jenren), Jeroen Janssens (@jeroenjanssens), @jeromecholewa, Janet Wesner (@jilmun), Jim Hester (@jimhester), JJ Chen (@jjchern), Jacek Kolacz (@jkolacz), Joanne Jang (@joannejang), @johannes4998, John Sears (@johnsears), @jonathanflint, Jon Calder (@jonmcalder), Jonathan Page (@jonpage), Jon Harmon (@jonthegeek), JooYoung Seo (@jooyoungseo), Justinas Petuchovas (@jpetuchovas), Jordan (@jrdnbradford), Jeffrey Arnold (@jrnold), Jose Roberto Ayala Solares (@jroberayalas), Joyce Robbins (@jtr13), @juandering, Julia Stewart Lowndes (@jules32), Sonja (@kaetschap), Kara Woo (@karawoo), Katrin Leinweber (@katrinleinweber), Karandeep Singh (@kdpsingh), Kevin Perese (@kevinxperese), Kevin Ferris (@kferris10), Kirill Sevastyanenko (@kirillseva), Jonathan Kitt (@KittJonathan), @koalabearski, Kirill Müller (@krlmlr), Rafał Kucharski (@kucharsky), Kevin Wright (@kwstat), Noah Landesberg (@landesbergn), Lawrence Wu (@lawwu), @lindbrook, Luke W Johnston (@lwjohnst86), Kara de la Marck (@MarckK), Kunal Marwaha (@marwahaha), Matan Hakim (@matanhakim), Matthias Liew (@MatthiasLiew), Matt Wittbrodt (@MattWittbrodt), Mauro Lepore (@maurolepore), Mark Beveridge (@mbeveridge), @mcewenkhundi, mcsnowface, PhD (@mcsnowface), Matt Herman (@mfherman), Michael Boerman (@michaelboerman), Mitsuo Shiota (@mitsuoxv), Matthew Hendrickson (@mjhendrickson), @MJMarshall, Misty Knight-Finley (@mkfin7), Mohammed Hamdy (@mmhamdy), Maxim Nazarov (@mnazarov), Maria Paula Caldas (@mpaulacaldas), Mustafa Ascha (@mustafaascha), Nelson Areal (@nareal), Nate Olson (@nate-d-olson), Nathanael (@nateaff), @nattalides, Ned Western (@NedJWestern), Nick Clark (@nickclark1000), @nickelas, Nirmal Patel (@nirmalpatel), Nischal Shrestha

(@nischalshrestha), Nicholas Tierney (@njtierney), Jakub Nowosad (@Nowosad), Nick Pullen (@nstjhp), @olivier6088, Olivier Cailloux (@oliviercailloux), Robin Penfold (@p0bs), Pablo E. Garcia (@pabloedug), Paul Adamson (@padamson), Penelope Y (@penelopeysm), Peter Hurford (@peterhurford), Peter Baumgartner (@petzi53), Patrick Kennedy (@pkq), Pooya Taherkhani (@pooyataher), Y. Yu (@PursuitOfDataScience), Radu Grosu (@radugrosu), Ranae Dietzel (@Ranae), Ralph Straumann (@rastrau), Rayna M Harris (@raynamharris), @ReeceGoding, Robin Gertenbach (@rgertenbach), Jajo (@RIngyao), Riva Quiroga (@rivaquiroga), Richard Knight (@RJHKnight), Richard Zijdeman (@rlzijdeman), @robertchu03, Robin Kohrs (@RobinKohrs), Robin (@Robinlovelace), Emily Robinson (@robinsones), Rob Tenorio (@robtenorio), Rod Mazloomi (@RodAli), Rohan Alexander (@RohanAlexander), Romero Morais (@RomeroBarata), Albert Y. Kim (@rudeboybert), Saghir (@saghirb), Hojjat Salmasian (@salmasian), Jonas (@sauercrowd), Vebash Naidoo (@sciencificity), Seamus McKinsey (@seamus-mckinsey), @seanpwilliams, Luke Smith (@seasmith), Matthew Sedaghatfar (@sedaghatfar), Sebastian Kraus (@sekR4), Sam Firke (@sfirke), Shannon Ellis (@ShanEllis), @shoili, Christian Heinrich (@Shurakai), S'busiso Mkhondwane (@sibusiso16), SM Raiyyan (@sm-raiyyan), Jakob Krigovsky (@sonicdoe), Stephan Koenig (@stephan-koenig), Stephen Balogun (@stephenbalogun), Steven M. Mortimer (@StevenMMortimer), Stéphane Guillou (@stragu), Sulgi Kim (@sulgik), Sergiusz Bleja (@svenski), Tal Galili (@talgalili), Alec Fisher (@Taurenamo), Todd Gerarden (@tgerarden), Tom Godfrey (@thomasggodfrey), Tim Broderick (@timbroderick), Tim Waterhouse (@timwaterhouse), TJ Mahr (@tjmahr), Thomas Klebel (@tklebel), Tom Prior (@tomjamesprior), Terence Teo (@tteo), @twgardner2, Ulrik Lyngs (@ulyngs), Shinya Uryu (@uribo), Martin Van der Linden (@vanderlindenma), Walter Somerville (@waltersom), @werkstattcodes, Will Beasley (@wibeasley), Yihui Xie (@yihui), Yiming (Paul) Li (@yimingli), @yingxingwu, Hiroaki Yutani (@yutannihilation), Yu Yu Aung (@yuyu-aung), Zach Bogart (@zachbogart), @zeal626, Zeki Akyol (@zekiakyol).

Colophon

An online version of this book is available at https://r4ds.hadley.nz. It will continue to evolve in between reprints of the physical book. The source of the book is available at https://github.com/hadley/r4ds. The book is powered by Quarto, which makes it easy to write books that combine text and executable code.

Whole game

Our goal in this part of the book is to give you a rapid overview of the main tools of data science: **importing**, **tidying**, **transforming**, and **visualizing data**, as shown in Figure 1. We want to show you the "whole game" of data science giving you just enough of all the major pieces so that you can tackle real, if simple, datasets. The later parts of the book will hit each of these topics in more depth, increasing the range of data science challenges that you can tackle.

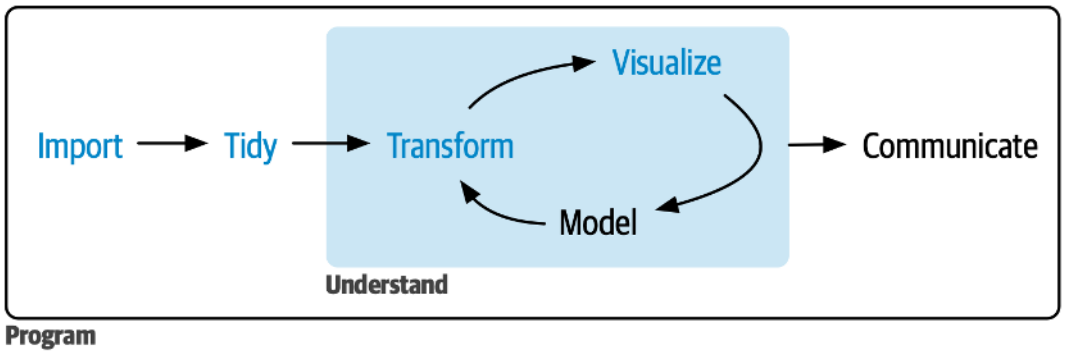


Figure 1: In this section of the book, you'll learn how to import, tidy, transform, and visualize data.

Four chapters focus on the tools of data science:

- Visualization is a great place to start with R programming, because the payoff is so clear: you get to make elegant and informative plots that help you understand data. In 1 Data visualization you'll dive into visualization, learning the basic structure of a ggplot2 plot, and powerful techniques for turning data into plots.

- Visualization alone is typically not enough, so in 3 Data transformation, you'll learn the key verbs that allow you to select important variables, filter out key observations, create new variables, and compute summaries.

- In 5 Data tidying, you'll learn about tidy data, a consistent way of storing your data that makes transformation, visualization, and modelling easier. You'll learn the underlying principles, and how to get your data into a tidy form.

- Before you can transform and visualize your data, you need to first get your data into R. In 7 Data import you'll learn the basics of getting `.csv` files into R.

Nestled among these chapters are four other chapters that focus on your R workflow. In 2 Workflow: basics, 4 Workflow: code style, and 6 Workflow: scripts and projects you'll learn good workflow practices for writing

and organizing your R code. These will set you up for success in the long run, as they'll give you the tools to stay organized when you tackle real projects. Finally, 8 Workflow: getting help will teach you how to get help and keep learning.

Introduction
1 Data visualization

1 Data visualization

1.1 Introduction

> "The simple graph has brought more information to the data analyst's mind than any other device." — John Tukey

R has several systems for making graphs, but ggplot2 is one of the most elegant and most versatile. ggplot2 implements the **grammar of graphics**, a coherent system for describing and building graphs. With ggplot2, you can do more and faster by learning one system and applying it in many places.

This chapter will teach you how to visualize your data using **ggplot2**. We will start by creating a simple scatterplot and use that to introduce aesthetic mappings and geometric objects – the fundamental building blocks of ggplot2. We will then walk you through visualizing distributions of single variables as well as visualizing relationships between two or more variables. We'll finish off with saving your plots and troubleshooting tips.

1.1.1 Prerequisites

This chapter focuses on ggplot2, one of the core packages in the tidyverse. To access the datasets, help pages, and functions used in this chapter, load the tidyverse by running:

```
library(tidyverse)
#> ── Attaching core tidyverse packages ──────────────────────── tidyverse 2.0.0 ──
#> ✔ dplyr     1.1.4     ✔ readr     2.1.5
#> ✔ forcats   1.0.0     ✔ stringr   1.5.1
#> ✔ ggplot2   3.5.1     ✔ tibble    3.2.1
#> ✔ lubridate 1.9.3     ✔ tidyr     1.3.1
#> ✔ purrr     1.0.2
#> ── Conflicts ────────────────────────────────────────── tidyverse_conflicts() ──
#> ✖ dplyr::filter() masks stats::filter()
#> ✖ dplyr::lag()    masks stats::lag()
#> ℹ Use the conflicted package (<http://conflicted.r-lib.org/>) to force all
conflicts to become errors
```

That one line of code loads the core tidyverse; the packages that you will use in almost every data analysis. It also tells you which functions from the tidyverse conflict with functions in base R (or from other packages you might have loaded)[1].

If you run this code and get the error message `there is no package called 'tidyverse'`, you'll need to first install it, then run `library()` once again.

You only need to install a package once, but you need to load it every time you start a new session.

In addition to tidyverse, we will also use the **palmerpenguins** package, which includes the `penguins` dataset containing body measurements for penguins on three islands in the Palmer Archipelago, and the ggthemes package, which offers a colorblind safe color palette.

```
library(palmerpenguins)
library(ggthemes)
```

1.2 First steps

Do penguins with longer flippers weigh more or less than penguins with shorter flippers? You probably already have an answer, but try to make your answer precise. What does the relationship between flipper length and body mass look like? Is it positive? Negative? Linear? Nonlinear? Does the relationship vary by the species of the penguin? How about by the island where the penguin lives? Let's create visualizations that we can use to answer these questions.

1.2.1 The `penguins` data frame

You can test your answers to those questions with the `penguins` **data frame** found in palmerpenguins (a.k.a. `palmerpenguins::penguins`). A data frame is a rectangular collection of variables (in the columns) and observations (in the rows). `penguins` contains 344 observations collected and made available by Dr. Kristen Gorman and the Palmer Station, Antarctica LTER[2].

To make the discussion easier, let's define some terms:

- A **variable** is a quantity, quality, or property that you can measure.
- A **value** is the state of a variable when you measure it. The value of a variable may change from measurement to measurement.
- An **observation** is a set of measurements made under similar conditions (you usually make all of the measurements in an observation at the same time and on the same object). An observation will contain several values, each associated with a different variable. We'll sometimes refer to an observation as a data point.
- **Tabular data** is a set of values, each associated with a variable and an observation. Tabular data is *tidy* if each value is placed in its own "cell", each variable in its own column, and each observation in its own row.

In this context, a variable refers to an attribute of all the penguins, and an observation refers to all the attributes of a single penguin.

Type the name of the data frame in the console and R will print a preview of its contents. Note that it says `tibble` on top of this preview. In the tidyverse, we use special data frames called **tibbles** that you will learn more about soon.

```
penguins
#> # A tibble: 344 × 8
#>   species island    bill_length_mm bill_depth_mm flipper_length_mm
#>   <fct>   <fct>              <dbl>         <dbl>             <int>
#> 1 Adelie  Torgersen           39.1          18.7               181
#> 2 Adelie  Torgersen           39.5          17.4               186
#> 3 Adelie  Torgersen           40.3          18                 195
#> 4 Adelie  Torgersen           NA            NA                  NA
#> 5 Adelie  Torgersen           36.7          19.3               193
#> 6 Adelie  Torgersen           39.3          20.6               190
#> # ℹ 338 more rows
#> # ℹ 3 more variables: body_mass_g <int>, sex <fct>, year <int>
```

This data frame contains 8 columns. For an alternative view, where you can see all variables and the first few observations of each variable, use `glimpse()`. Or, if you're in RStudio, run `View(penguins)` to open an interactive data viewer.

```
glimpse(penguins)
#> Rows: 344
#> Columns: 8
#> $ species           <fct> Adelie, Adelie, Adelie, Adelie, Adelie, Adelie, A…
#> $ island            <fct> Torgersen, Torgersen, Torgersen, Torgersen, Torge…
#> $ bill_length_mm    <dbl> 39.1, 39.5, 40.3, NA, 36.7, 39.3, 38.9, 39.2, 34.…
#> $ bill_depth_mm     <dbl> 18.7, 17.4, 18.0, NA, 19.3, 20.6, 17.8, 19.6, 18.…
#> $ flipper_length_mm <int> 181, 186, 195, NA, 193, 190, 181, 195, 193, 190, …
#> $ body_mass_g       <int> 3750, 3800, 3250, NA, 3450, 3650, 3625, 4675, 347…
#> $ sex               <fct> male, female, female, NA, female, male, female, m…
#> $ year              <int> 2007, 2007, 2007, 2007, 2007, 2007, 2007, 2007, 2…
```

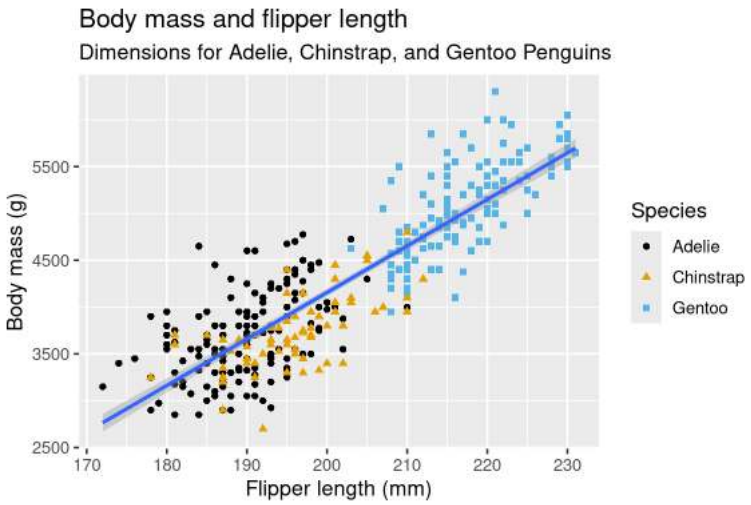
Among the variables in `penguins` are:

1. `species`: a penguin's species (Adelie, Chinstrap, or Gentoo).
2. `flipper_length_mm`: length of a penguin's flipper, in millimeters.
3. `body_mass_g`: body mass of a penguin, in grams.

To learn more about `penguins`, open its help page by running `?penguins`.

1.2.2 Ultimate goal

Our ultimate goal in this chapter is to recreate the following visualization displaying the relationship between flipper lengths and body masses of these penguins, taking into consideration the species of the penguin.



1.2.3 Creating a ggplot

Let's recreate this plot step-by-step.

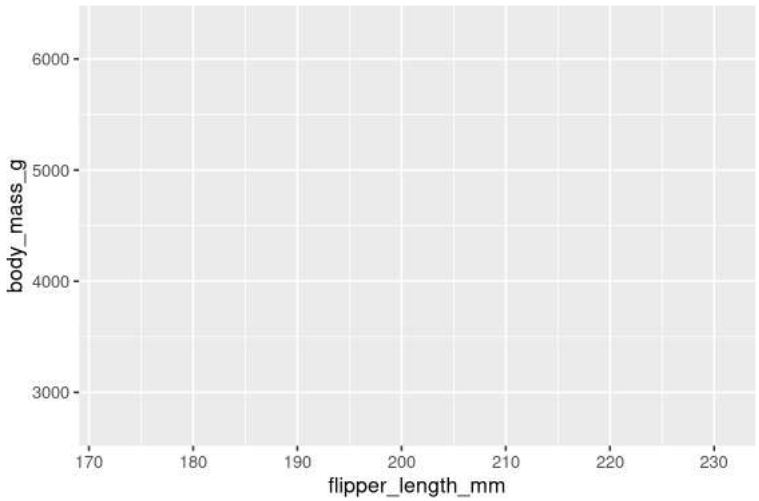
With ggplot2, you begin a plot with the function `ggplot()`, defining a plot object that you then add **layers** to. The first argument of `ggplot()` is the dataset to use in the graph and so `ggplot(data = penguins)` creates an empty graph that is primed to display the `penguins` data, but since we haven't told it how to visualize it yet, for now it's empty. This is not a very exciting plot, but you can think of it like an empty canvas you'll paint the remaining layers of your plot onto.

```
ggplot(data = penguins)
```


Next, we need to tell `ggplot()` how the information from our data will be visually represented. The `mapping` argument of the `ggplot()` function defines how variables in your dataset are mapped to visual properties (**aesthetics**) of your plot. The `mapping` argument is always defined in the `aes()` function, and the `x` and `y` arguments of `aes()` specify which variables to map to the x and y axes. For now, we will only map flipper length to the `x` aesthetic and body mass to the `y` aesthetic. ggplot2 looks for the mapped variables in the `data` argument, in this case, `penguins`.

The following plot shows the result of adding these mappings.

```
ggplot(
  data = penguins,
  mapping = aes(x = flipper_length_mm, y = body_mass_g)
)
```

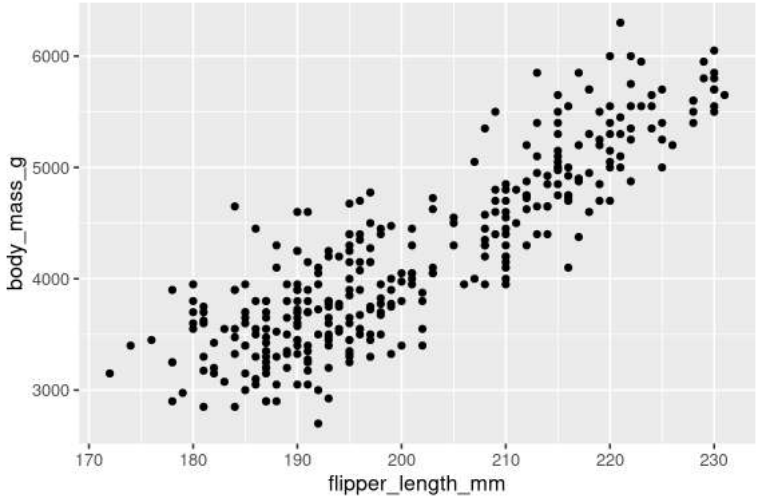


Our empty canvas now has more structure – it's clear where flipper lengths will be displayed (on the x-axis) and where body masses will be displayed (on the y-axis). But the penguins themselves are not yet on the plot. This is because we have not yet articulated, in our code, how to represent the observations from our data frame on our plot.

To do so, we need to define a **geom**: the geometrical object that a plot uses to represent data. These geometric objects are made available in ggplot2 with functions that start with `geom_`. People often describe plots by the type of geom that the plot uses. For example, bar charts use bar geoms (`geom_bar()`), line charts use line geoms (`geom_line()`), boxplots use boxplot geoms (`geom_boxplot()`), scatterplots use point geoms (`geom_point()`), and so on.

The function `geom_point()` adds a layer of points to your plot, which creates a scatterplot. ggplot2 comes with many geom functions that each adds a different type of layer to a plot. You'll learn a whole bunch of geoms throughout the book, particularly in Chapter 9.

```
ggplot(
  data = penguins,
  mapping = aes(x = flipper_length_mm, y = body_mass_g)
) +
  geom_point()
#> Warning: Removed 2 rows containing missing values or values outside the scale
range
#> (`geom_point()`).
```



Now we have something that looks like what we might think of as a "scatterplot". It doesn't yet match our "ultimate goal" plot, but using this plot we can start answering the question that motivated our exploration: "What does the relationship between flipper length and body mass look like?" The relationship appears to be positive (as flipper length increases, so does body mass), fairly linear (the points are clustered around a line instead of a curve), and

moderately strong (there isn't too much scatter around such a line). Penguins with longer flippers are generally larger in terms of their body mass.

Before we add more layers to this plot, let's pause for a moment and review the warning message we got:

> Removed 2 rows containing missing values (`geom_point()`).

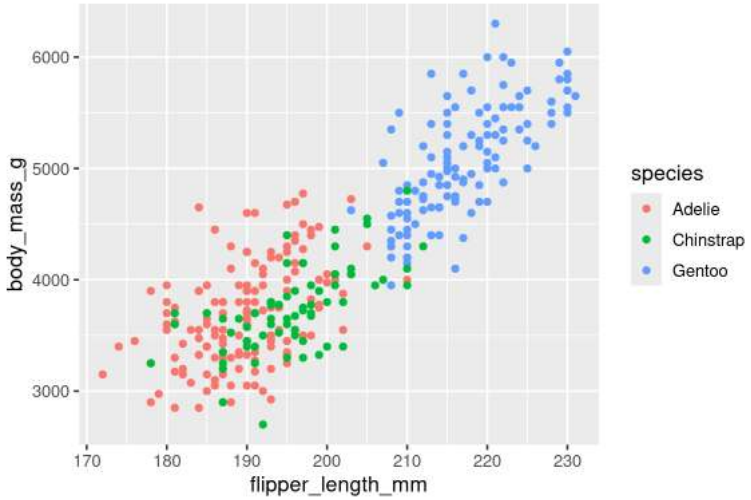
We're seeing this message because there are two penguins in our dataset with missing body mass and/or flipper length values and ggplot2 has no way of representing them on the plot without both of these values. Like R, ggplot2 subscribes to the philosophy that missing values should never silently go missing. This type of warning is probably one of the most common types of warnings you will see when working with real data – missing values are a very common issue and you'll learn more about them throughout the book, particularly in Chapter 18. For the remaining plots in this chapter we will suppress this warning so it's not printed alongside every single plot we make.

1.2.4 Adding aesthetics and layers

Scatterplots are useful for displaying the relationship between two numerical variables, but it's always a good idea to be skeptical of any apparent relationship between two variables and ask if there may be other variables that explain or change the nature of this apparent relationship. For example, does the relationship between flipper length and body mass differ by species? Let's incorporate species into our plot and see if this reveals any additional insights into the apparent relationship between these variables. We will do this by representing species with different colored points.

To achieve this, will we need to modify the aesthetic or the geom? If you guessed "in the aesthetic mapping, inside of `aes()`", you're already getting the hang of creating data visualizations with ggplot2! And if not, don't worry. Throughout the book you will make many more ggplots and have many more opportunities to check your intuition as you make them.

```
ggplot(
  data = penguins,
  mapping = aes(x = flipper_length_mm, y = body_mass_g, color = species)
) +
  geom_point()
```



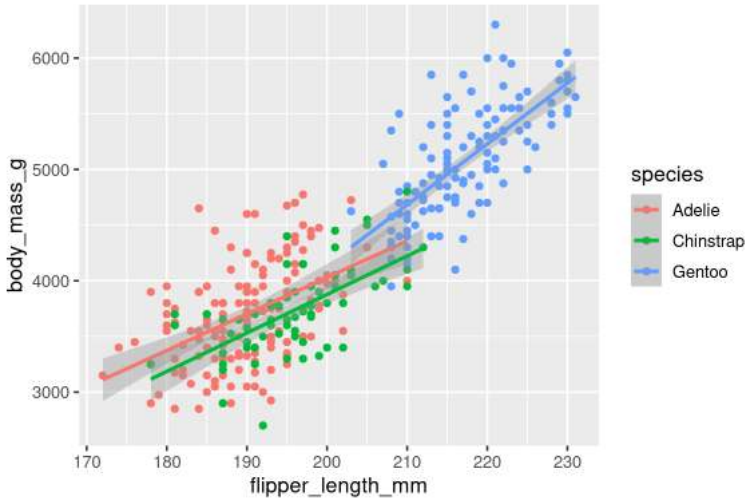
When a categorical variable is mapped to an aesthetic, ggplot2 will automatically assign a unique value of the aesthetic (here a unique color) to each unique level of the variable (each of the three species), a process known as **scaling**. ggplot2 will also add a legend that explains which values correspond to which levels.

Now let's add one more layer: a smooth curve displaying the relationship between body mass and flipper length. Before you proceed, refer back to the code above, and think about how we can add this to our existing plot.

Since this is a new geometric object representing our data, we will add a new geom as a layer on top of our point geom: `geom_smooth()`. And we will specify that we want to draw the line of best fit based on a `l`inear `m`odel with `method = "lm"`.

```
ggplot(
  data = penguins,
```

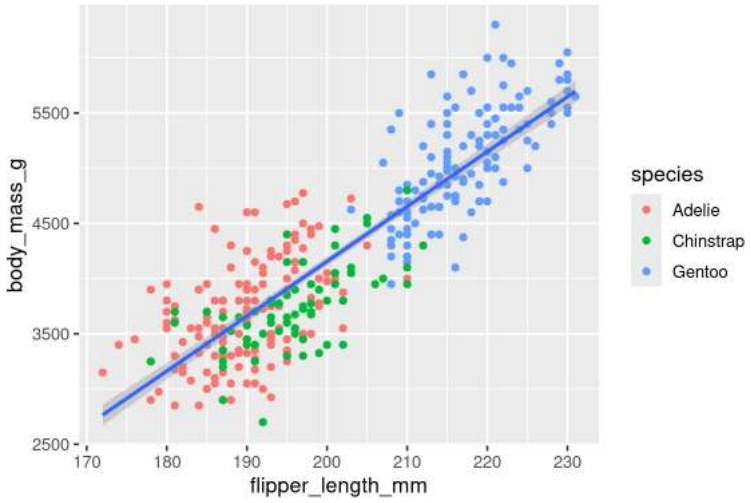
```
  mapping = aes(x = flipper_length_mm, y = body_mass_g, color = species)
) +
  geom_point() +
  geom_smooth(method = "lm")
```



We have successfully added lines, but this plot doesn't look like the plot from Section 1.2.2, which only has one line for the entire dataset as opposed to separate lines for each of the penguin species.

When aesthetic mappings are defined in `ggplot()`, at the *global* level, they're passed down to each of the subsequent geom layers of the plot. However, each geom function in ggplot2 can also take a `mapping` argument, which allows for aesthetic mappings at the *local* level that are added to those inherited from the global level. Since we want points to be colored based on species but don't want the lines to be separated out for them, we should specify `color = species` for `geom_point()` only.

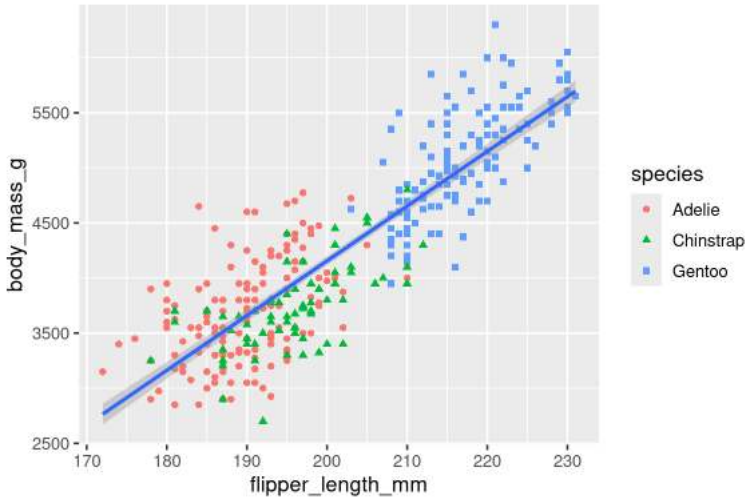
```
ggplot(
  data = penguins,
  mapping = aes(x = flipper_length_mm, y = body_mass_g)
) +
  geom_point(mapping = aes(color = species)) +
  geom_smooth(method = "lm")
```



Voila! We have something that looks very much like our ultimate goal, though it's not yet perfect. We still need to use different shapes for each species of penguins and improve labels.

It's generally not a good idea to represent information using only colors on a plot, as people perceive colors differently due to color blindness or other color vision differences. Therefore, in addition to color, we can also map `species` to the `shape` aesthetic.

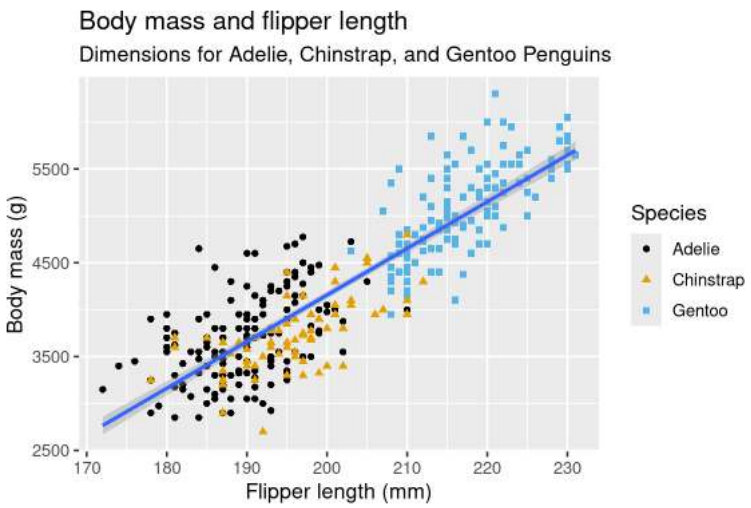
```
ggplot(
  data = penguins,
  mapping = aes(x = flipper_length_mm, y = body_mass_g)
) +
  geom_point(mapping = aes(color = species, shape = species)) +
  geom_smooth(method = "lm")
```



Note that the legend is automatically updated to reflect the different shapes of the points as well.

And finally, we can improve the labels of our plot using the `labs()` function in a new layer. Some of the arguments to `labs()` might be self explanatory: `title` adds a title and `subtitle` adds a subtitle to the plot. Other arguments match the aesthetic mappings, `x` is the x-axis label, `y` is the y-axis label, and `color` and `shape` define the label for the legend. In addition, we can improve the color palette to be colorblind safe with the `scale_color_colorblind()` function from the ggthemes package.

```
ggplot(
  data = penguins,
  mapping = aes(x = flipper_length_mm, y = body_mass_g)
) +
  geom_point(aes(color = species, shape = species)) +
  geom_smooth(method = "lm") +
  labs(
    title = "Body mass and flipper length",
    subtitle = "Dimensions for Adelie, Chinstrap, and Gentoo Penguins",
    x = "Flipper length (mm)", y = "Body mass (g)",
    color = "Species", shape = "Species"
  ) +
  scale_color_colorblind()
```



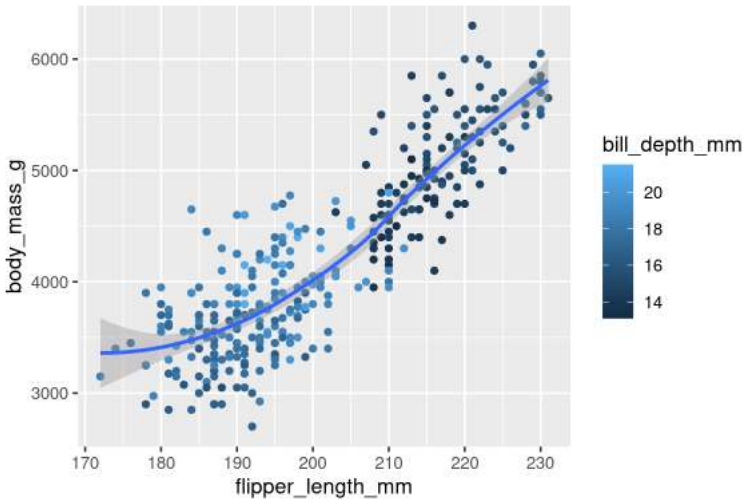
We finally have a plot that perfectly matches our "ultimate goal"!

1.2.5 Exercises

1. How many rows are in `penguins`? How many columns?
2. What does the `bill_depth_mm` variable in the `penguins` data frame describe? Read the help for `?penguins` to find out.
3. Make a scatterplot of `bill_depth_mm` vs. `bill_length_mm`. That is, make a scatterplot with `bill_depth_mm` on the y-axis and `bill_length_mm` on the x-axis. Describe the relationship between these two variables.
4. What happens if you make a scatterplot of `species` vs. `bill_depth_mm`? What might be a better choice of geom?
5. Why does the following give an error and how would you fix it?

```
ggplot(data = penguins) +
  geom_point()
```

6. What does the `na.rm` argument do in `geom_point()`? What is the default value of the argument? Create a scatterplot where you successfully use this argument set to `TRUE`.
7. Add the following caption to the plot you made in the previous exercise: "Data come from the palmerpenguins package." Hint: Take a look at the documentation for `labs()`.
8. Recreate the following visualization. What aesthetic should `bill_depth_mm` be mapped to? And should it be mapped at the global level or at the geom level?



9. Run this code in your head and predict what the output will look like. Then, run the code in R and check your predictions.

```
ggplot(
  data = penguins,
  mapping = aes(x = flipper_length_mm, y = body_mass_g, color = island)
) +
  geom_point() +
  geom_smooth(se = FALSE)
```

10. Will these two graphs look different? Why/why not?

```
ggplot(
  data = penguins,
  mapping = aes(x = flipper_length_mm, y = body_mass_g)
) +
  geom_point() +
  geom_smooth()

ggplot() +
  geom_point(
    data = penguins,
    mapping = aes(x = flipper_length_mm, y = body_mass_g)
  ) +
  geom_smooth(
    data = penguins,
    mapping = aes(x = flipper_length_mm, y = body_mass_g)
  )
```

1.3 ggplot2 calls

As we move on from these introductory sections, we'll transition to a more concise expression of ggplot2 code. So far we've been very explicit, which is helpful when you are learning:

```
ggplot(
  data = penguins,
  mapping = aes(x = flipper_length_mm, y = body_mass_g)
) +
  geom_point()
```

Typically, the first one or two arguments to a function are so important that you should know them by heart. The first two arguments to `ggplot()` are `data` and `mapping`, in the remainder of the book, we won't supply those names. That saves typing, and, by reducing the amount of extra text, makes it easier to see what's different between plots. That's a really important programming concern that we'll come back to in Chapter 25.

Rewriting the previous plot more concisely yields:

```
ggplot(penguins, aes(x = flipper_length_mm, y = body_mass_g)) +
  geom_point()
```

In the future, you'll also learn about the pipe, `|>`, which will allow you to create that plot with:

```
penguins |>
  ggplot(aes(x = flipper_length_mm, y = body_mass_g)) +
  geom_point()
```

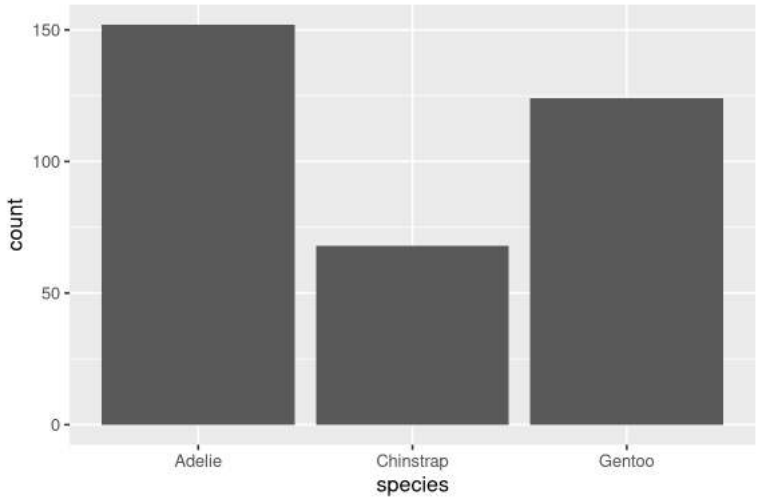
1.4 Visualizing distributions

How you visualize the distribution of a variable depends on the type of variable: categorical or numerical.

1.4.1 A categorical variable

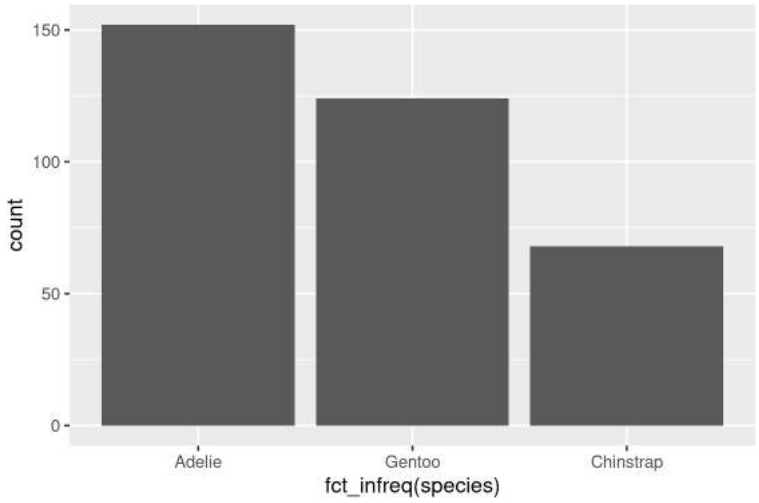
A variable is **categorical** if it can only take one of a small set of values. To examine the distribution of a categorical variable, you can use a bar chart. The height of the bars displays how many observations occurred with each `x` value.

```
ggplot(penguins, aes(x = species)) +
  geom_bar()
```



In bar plots of categorical variables with non-ordered levels, like the penguin `species` above, it's often preferable to reorder the bars based on their frequencies. Doing so requires transforming the variable to a factor (how R handles categorical data) and then reordering the levels of that factor.

```
ggplot(penguins, aes(x = fct_infreq(species))) +
  geom_bar()
```



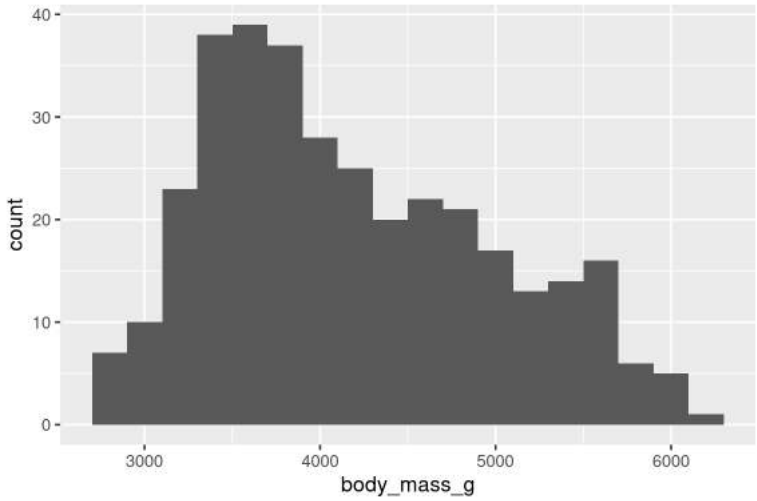
You will learn more about factors and functions for dealing with factors (like `fct_infreq()` shown above) in Chapter 16.

1.4.2 A numerical variable

A variable is **numerical** (or quantitative) if it can take on a wide range of numerical values, and it is sensible to add, subtract, or take averages with those values. Numerical variables can be continuous or discrete.

One commonly used visualization for distributions of continuous variables is a histogram.

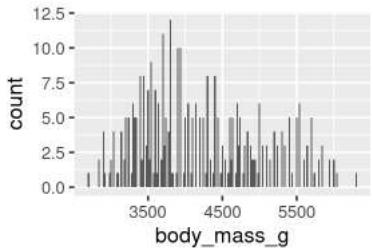
```
ggplot(penguins, aes(x = body_mass_g)) +
  geom_histogram(binwidth = 200)
```

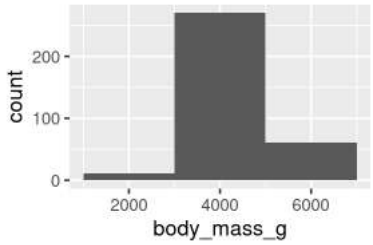


A histogram divides the x-axis into equally spaced bins and then uses the height of a bar to display the number of observations that fall in each bin. In the graph above, the tallest bar shows that 39 observations have a `body_mass_g` value between 3,500 and 3,700 grams, which are the left and right edges of the bar.

You can set the width of the intervals in a histogram with the binwidth argument, which is measured in the units of the `x` variable. You should always explore a variety of binwidths when working with histograms, as different binwidths can reveal different patterns. In the plots below a binwidth of 20 is too narrow, resulting in too many bars, making it difficult to determine the shape of the distribution. Similarly, a binwidth of 2,000 is too high, resulting in all data being binned into only three bars, and also making it difficult to determine the shape of the distribution. A binwidth of 200 provides a sensible balance.

```
ggplot(penguins, aes(x = body_mass_g)) +
  geom_histogram(binwidth = 20)
ggplot(penguins, aes(x = body_mass_g)) +
  geom_histogram(binwidth = 2000)
```

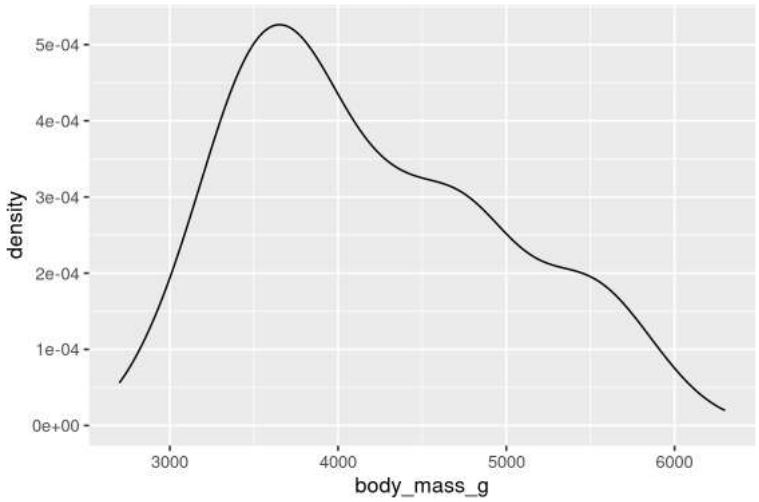




An alternative visualization for distributions of numerical variables is a density plot. A density plot is a smoothed-out version of a histogram and a practical alternative, particularly for continuous data that comes from an underlying smooth distribution. We won't go into how `geom_density()` estimates the density (you can read more about that in the function documentation), but let's explain how the density curve is drawn with an analogy. Imagine a histogram made out of wooden blocks. Then, imagine that you drop a cooked spaghetti string over it. The shape the spaghetti will take draped over blocks can be thought of as the shape of the density curve. It shows fewer details than a histogram but can make it easier to quickly glean the shape of the distribution, particularly with respect to modes and skewness.

```
ggplot(penguins, aes(x = body_mass_g)) +
  geom_density()
```

```
#> Warning: Removed 2 rows containing non-finite outside the scale range
#> (`stat_density()`).
```



1.4.3 Exercises

1. Make a bar plot of `species` of `penguins`, where you assign `species` to the `y` aesthetic. How is this plot different?
2. How are the following two plots different? Which aesthetic, `color` or `fill`, is more useful for changing the color of bars?
3. What does the `bins` argument in `geom_histogram()` do?
4. Make a histogram of the `carat` variable in the `diamonds` dataset that is available when you load the tidyverse package. Experiment with different binwidths. What binwidth reveals the most interesting patterns?

1.5 Visualizing relationships

To visualize a relationship we need to have at least two variables mapped to aesthetics of a plot. In the following sections you will learn about commonly used plots for visualizing relationships between two or more variables and the geoms used for creating them.

1.5.1 A numerical and a categorical variable

To visualize the relationship between a numerical and a categorical variable we can use side-by-side box plots. A **boxplot** is a type of visual shorthand for measures of position (percentiles) that describe a distribution. It is also useful for identifying potential outliers. As shown in Figure 1.1, each boxplot consists of:

- A box that indicates the range of the middle half of the data, a distance known as the interquartile range (IQR), stretching from the 25th percentile of the distribution to the 75th percentile. In the middle of the box is a line that displays the median, i.e. 50th percentile, of the distribution. These three lines give you a sense of the spread of the distribution and whether or not the distribution is symmetric about the median or skewed to one side.
- Visual points that display observations that fall more than 1.5 times the IQR from either edge of the box. These outlying points are unusual so are plotted individually.
- A line (or whisker) that extends from each end of the box and goes to the farthest non-outlier point in the distribution.

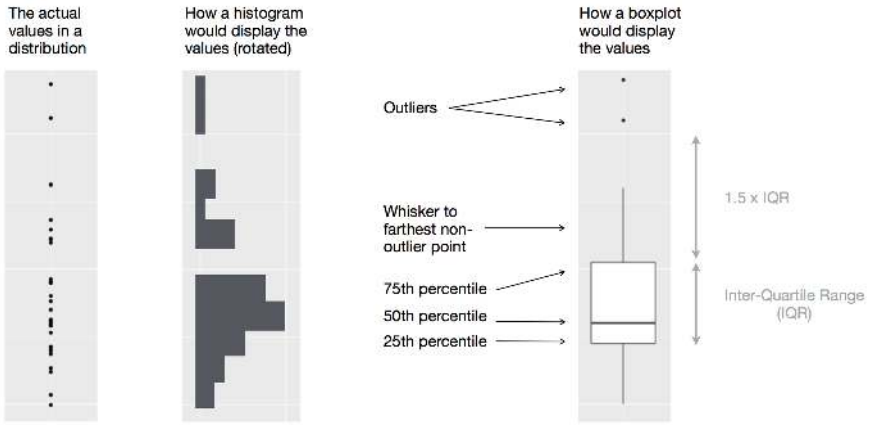
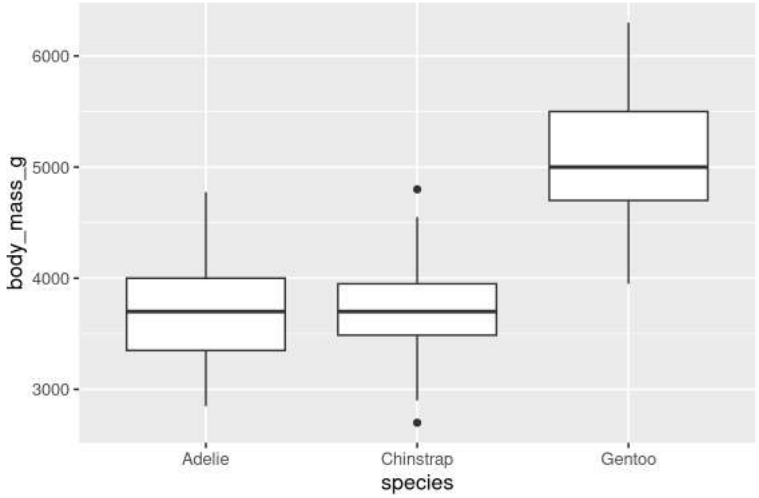


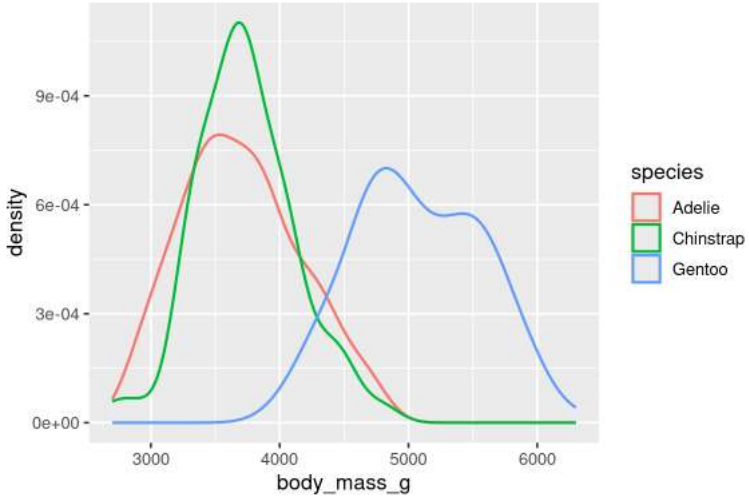
Figure 1.1: Diagram depicting how a boxplot is created.

Let's take a look at the distribution of body mass by species using `geom_boxplot()`:



Alternatively, we can make density plots with `geom_density()`.

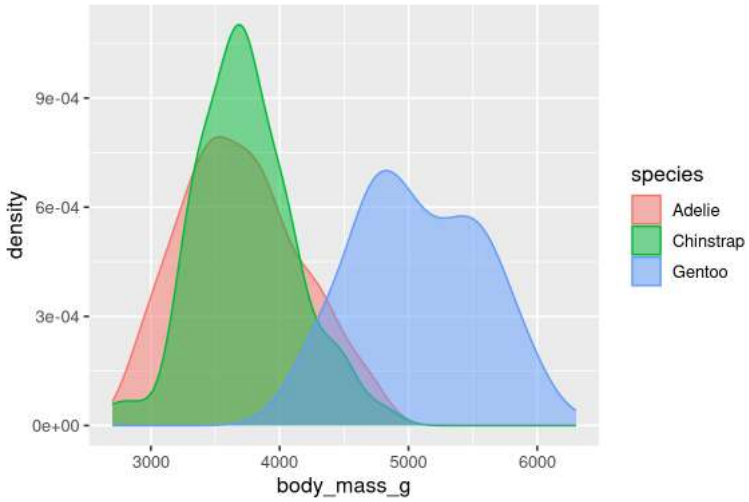
```
ggplot(penguins, aes(x = body_mass_g, color = species)) +
  geom_density(linewidth = 0.75)
```



We've also customized the thickness of the lines using the `linewidth` argument in order to make them stand out a bit more against the background.

Additionally, we can map `species` to both `color` and `fill` aesthetics and use the `alpha` aesthetic to add transparency to the filled density curves. This aesthetic takes values between 0 (completely transparent) and 1 (completely opaque). In the following plot it's *set* to 0.5.

```
ggplot(penguins, aes(x = body_mass_g, color = species, fill = species)) +
  geom_density(alpha = 0.5)
```



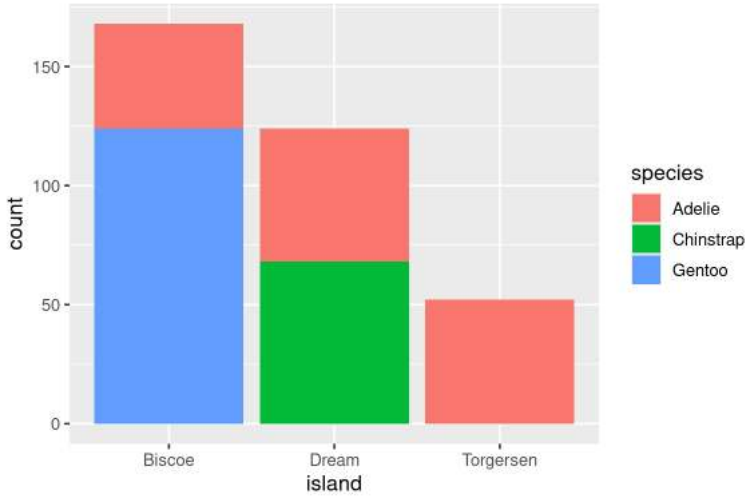
Note the terminology we have used here:

- We *map* variables to aesthetics if we want the visual attribute represented by that aesthetic to vary based on the values of that variable.
- Otherwise, we *set* the value of an aesthetic.

1.5.2 Two categorical variables

We can use stacked bar plots to visualize the relationship between two categorical variables. For example, the following two stacked bar plots both display the relationship between `island` and `species`, or specifically, visualizing the distribution of `species` within each island.

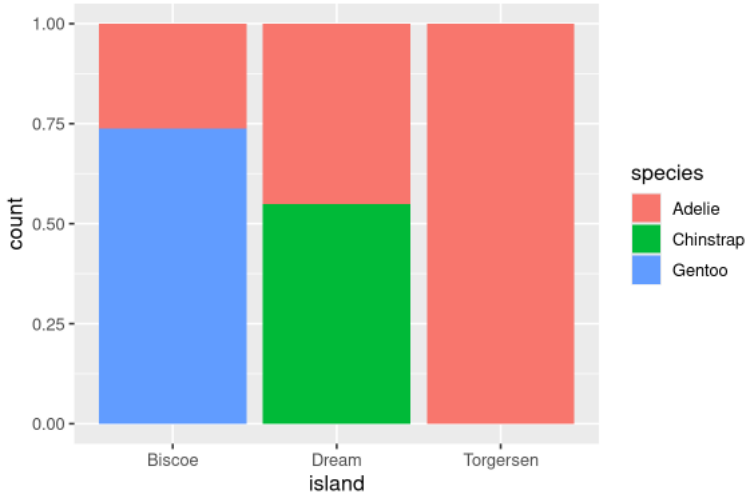
The first plot shows the frequencies of each species of penguins on each island. The plot of frequencies shows that there are equal numbers of Adelies on each island. But we don't have a good sense of the percentage balance within each island.



The second plot, a relative frequency plot created by setting `position = "fill"` in the geom, is more useful for comparing species distributions across islands since it's not affected by the unequal numbers of penguins across the

islands. Using this plot we can see that Gentoo penguins all live on Biscoe island and make up roughly 75% of the penguins on that island, Chinstrap all live on Dream island and make up roughly 50% of the penguins on that island, and Adelie live on all three islands and make up all of the penguins on Torgersen.

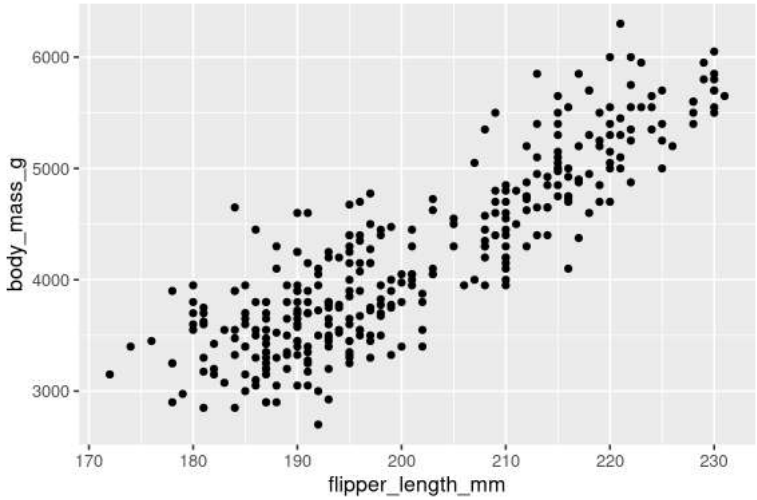
```
ggplot(penguins, aes(x = island, fill = species)) +
  geom_bar(position = "fill")
```



In creating these bar charts, we map the variable that will be separated into bars to the `x` aesthetic, and the variable that will change the colors inside the bars to the `fill` aesthetic.

1.5.3 Two numerical variables

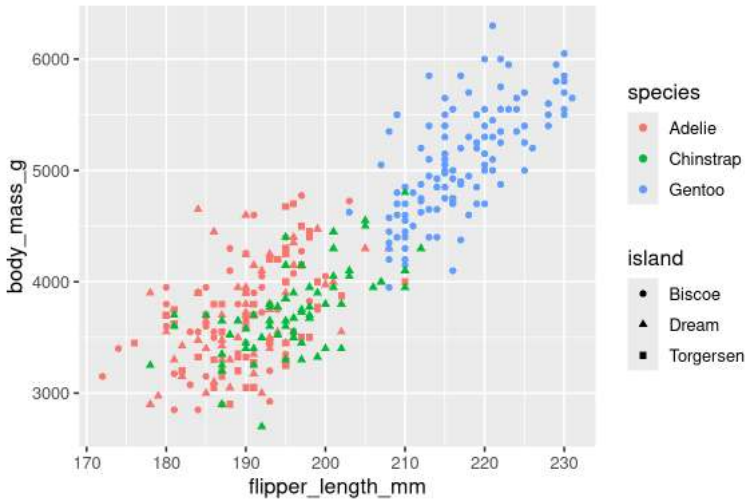
So far you've learned about scatterplots (created with `geom_point()`) and smooth curves (created with `geom_smooth()`) for visualizing the relationship between two numerical variables. A scatterplot is probably the most commonly used plot for visualizing the relationship between two numerical variables.



1.5.4 Three or more variables

As we saw in Section 1.2.4, we can incorporate more variables into a plot by mapping them to additional aesthetics. For example, in the following scatterplot the colors of points represent species and the shapes of points represent islands.

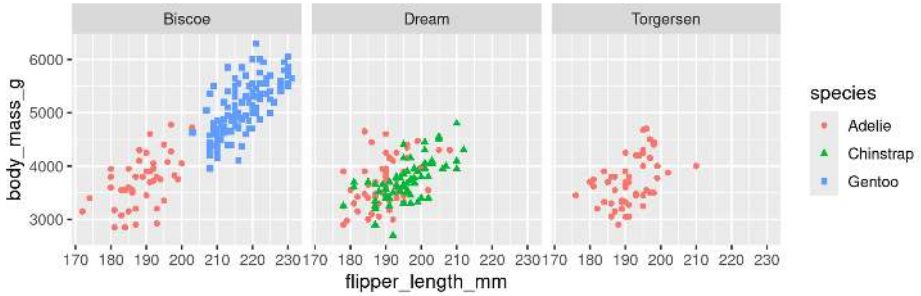
```
  geom_point(aes(color = species, shape = island))
```



However adding too many aesthetic mappings to a plot makes it cluttered and difficult to make sense of. Another way, which is particularly useful for categorical variables, is to split your plot into **facets**, subplots that each display one subset of the data.

To facet your plot by a single variable, use `facet_wrap()`. The first argument of `facet_wrap()` is a formula[3], which you create with ~ followed by a variable name. The variable that you pass to `facet_wrap()` should be categorical.

```
geom_point(aes(color = species, shape = species)) +
facet_wrap(~island)
```



You will learn about many other geoms for visualizing distributions of variables and relationships between them in Chapter 9.

1.5.5 Exercises

1. The `mpg` data frame that is bundled with the ggplot2 package contains 234 observations collected by the US Environmental Protection Agency on 38 car models. Which variables in `mpg` are categorical? Which variables are numerical? (Hint: Type `?mpg` to read the documentation for the dataset.) How can you see this information when you run `mpg`?

2. Make a scatterplot of `hwy` vs. `displ` using the `mpg` data frame. Next, map a third, numerical variable to `color`, then `size`, then both `color` and `size`, then `shape`. How do these aesthetics behave differently for categorical vs. numerical variables?

3. In the scatterplot of `hwy` vs. `displ`, what happens if you map a third variable to `linewidth`?

4. What happens if you map the same variable to multiple aesthetics?

5. Make a scatterplot of `bill_depth_mm` vs. `bill_length_mm` and color the points by `species`. What does adding coloring by species reveal about the relationship between these two variables? What about faceting by `species`?

6. Why does the following yield two separate legends? How would you fix it to combine the two legends?

```
ggplot(
  data = penguins,
  mapping = aes(
    x = bill_length_mm, y = bill_depth_mm,
    color = species, shape = species
  )
) +
  geom_point() +
  labs(color = "Species")
```

7. Create the two following stacked bar plots. Which question can you answer with the first one? Which question can you answer with the second one?

```
ggplot(penguins, aes(x = island, fill = species)) +
  geom_bar(position = "fill")
ggplot(penguins, aes(x = species, fill = island)) +
  geom_bar(position = "fill")
```

1.6 Saving your plots

Once you've made a plot, you might want to get it out of R by saving it as an image that you can use elsewhere. That's the job of `ggsave()`, which will save the plot most recently created to disk:

This will save your plot to your working directory, a concept you'll learn more about in Chapter 6.

If you don't specify the `width` and `height` they will be taken from the dimensions of the current plotting device. For reproducible code, you'll want to specify them. You can learn more about `ggsave()` in the documentation.

Generally, however, we recommend that you assemble your final reports using Quarto, a reproducible authoring system that allows you to interleave your code and your prose and automatically include your plots in your write-ups. You will learn more about Quarto in Chapter 28.

1.6.1 Exercises

1. Run the following lines of code. Which of the two plots is saved as `mpg-plot.png`? Why?

2. What do you need to change in the code above to save the plot as a PDF instead of a PNG? How could you find out what types of image files would work in `ggsave()`?

1.7 Common problems

As you start to run R code, you're likely to run into problems. Don't worry — it happens to everyone. We have all been writing R code for years, but every day we still write code that doesn't work on the first try!

Start by carefully comparing the code that you're running to the code in the book. R is extremely picky, and a misplaced character can make all the difference. Make sure that every `(` is matched with a `)` and every `"` is paired with another `"`. Sometimes you'll run the code and nothing happens. Check the left-hand of your console: if it's a `+`, it means that R doesn't think you've typed a complete expression and it's waiting for you to finish it. In this case, it's usually easy to start from scratch again by pressing ESCAPE to abort processing the current command.

One common problem when creating ggplot2 graphics is to put the `+` in the wrong place: it has to come at the end of the line, not the start. In other words, make sure you haven't accidentally written code like this:

```
ggplot(data = mpg)
```

If you're still stuck, try the help. You can get help about any R function by running `?function_name` in the console, or highlighting the function name and pressing F1 in RStudio. Don't worry if the help doesn't seem that helpful - instead skip down to the examples and look for code that matches what you're trying to do.

If that doesn't help, carefully read the error message. Sometimes the answer will be buried there! But when you're new to R, even if the answer is in the error message, you might not yet know how to understand it. Another great tool is Google: try googling the error message, as it's likely someone else has had the same problem, and has gotten help online.

1.8 Summary

In this chapter, you've learned the basics of data visualization with ggplot2. We started with the basic idea that underpins ggplot2: a visualization is a mapping from variables in your data to aesthetic properties like position, color, size and shape. You then learned about increasing the complexity and improving the presentation of your plots layer-by-layer. You also learned about commonly used plots for visualizing the distribution of a single variable as well as for visualizing relationships between two or more variables, by leveraging additional aesthetic mappings and/or splitting your plot into small multiples using faceting.

We'll use visualizations again and again throughout this book, introducing new techniques as we need them as well as do a deeper dive into creating visualizations with ggplot2 in Chapter 9 through Chapter 11.

With the basics of visualization under your belt, in the next chapter we're going to switch gears a little and give you some practical workflow advice. We intersperse workflow advice with data science tools throughout this part of the book because it'll help you stay organized as you write increasing amounts of R code.

Whole game
2 Workflow: basics

2 Workflow: basics

You now have some experience running R code. We didn't give you many details, but you've obviously figured out the basics, or you would've thrown this book away in frustration! Frustration is natural when you start programming in R because it is such a stickler for punctuation, and even one character out of place can cause it to complain. But while you should expect to be a little frustrated, take comfort in that this experience is typical and temporary: it happens to everyone, and the only way to get over it is to keep trying.

Before we go any further, let's ensure you've got a solid foundation in running R code and that you know some of the most helpful RStudio features.

2.1 Coding basics

Let's review some basics we've omitted so far in the interest of getting you plotting as quickly as possible. You can use R to do basic math calculations:

```
1 / 200 * 30
#> [1] 0.15
(59 + 73 + 2) / 3
#> [1] 44.66667
sin(pi / 2)
#> [1] 1
```

You can create new objects with the assignment operator `<-`:

```
x <- 3 * 4
```

Note that the value of `x` is not printed, it's just stored. If you want to view the value, type `x` in the console.

You can **c**ombine multiple elements into a vector with `c()`:

```
primes <- c(2, 3, 5, 7, 11, 13)
```

And basic arithmetic on vectors is applied to every element of of the vector:

```
primes * 2
#> [1]  4  6 10 14 22 26
primes - 1
#> [1]  1  2  4  6 10 12
```

All R statements where you create objects, **assignment** statements, have the same form:

```
object_name <- value
```

When reading that code, say “object name gets value” in your head.

You will make lots of assignments, and `<-` is a pain to type. You can save time with RStudio’s keyboard shortcut: Alt + - (the minus sign). Notice that RStudio automatically surrounds `<-` with spaces, which is a good code formatting practice. Code can be miserable to read on a good day, so giveyoureyesabreak and use spaces.

2.2 Comments

R will ignore any text after `#` for that line. This allows you to write **comments**, text that is ignored by R but read by other humans. We’ll sometimes include comments in examples explaining what’s happening with the code.

Comments can be helpful for briefly describing what the following code does.

```
# create vector of primes
primes <- c(2, 3, 5, 7, 11, 13)

# multiply primes by 2
primes * 2
#> [1]  4  6 10 14 22 26
```

With short pieces of code like this, leaving a comment for every single line of code might not be necessary. But as the code you’re writing gets more complex, comments can save you (and your collaborators) a lot of time figuring out what was done in the code.

Use comments to explain the *why* of your code, not the *how* or the *what*. The *what* and *how* of your code are always possible to figure out, even if it might be tedious, by carefully reading it. If you describe every step in the comments, and then change the code, you will have to remember to update the comments as well or it will be confusing when you return to your code in the future.

Figuring out *why* something was done is much more difficult, if not impossible. For example, `geom_smooth()` has an argument called `span`, which controls the smoothness of the curve, with larger values yielding a smoother curve. Suppose you decide to change the value of `span` from its default of 0.75 to 0.9: it’s easy for a future reader to understand *what* is happening, but unless you note your thinking in a comment, no one will understand *why* you changed the default.

For data analysis code, use comments to explain your overall plan of attack and record important insights as you encounter them. There's no way to re-capture this knowledge from the code itself.

2.3 What's in a name?

Object names must start with a letter and can only contain letters, numbers, _, and `.`. You want your object names to be descriptive, so you'll need to adopt a convention for multiple words. We recommend **snake_case**, where you separate lowercase words with _.

```
i_use_snake_case
otherPeopleUseCamelCase
some.people.use.periods
And_aFew.People_RENOUNCEconvention
```

We'll return to names again when we discuss code style in Chapter 4.

You can inspect an object by typing its name:

```
x
#> [1] 12
```

Make another assignment:

```
this_is_a_really_long_name <- 2.5
```

To inspect this object, try out RStudio's completion facility: type "this", press TAB, add characters until you have a unique prefix, then press return.

Let's assume you made a mistake, and that the value of `this_is_a_really_long_name` should be 3.5, not 2.5. You can use another keyboard shortcut to help you fix it. For example, you can press ↑ to bring the last command you typed and edit it. Or, type "this" then press Cmd/Ctrl + ↑ to list all the commands you've typed that start with those letters. Use the arrow keys to navigate, then press enter to retype the command. Change 2.5 to 3.5 and rerun.

Make yet another assignment:

```
r_rocks <- 2^3
```

Let's try to inspect it:

```
r_rock
#> Error: object 'r_rock' not found
```

```
R_rocks
#> Error: object 'R_rocks' not found
```

This illustrates the implied contract between you and R: R will do the tedious computations for you, but in exchange, you must be completely precise in your instructions. If not, you're likely to get an error that says the object you're looking for was not found. Typos matter; R can't read your mind and say, "oh, they probably meant `r_rocks` when they typed `r_rock`". Case matters; similarly, R can't read your mind and say, "oh, they probably meant `r_rocks` when they typed `R_rocks`".

2.4 Calling functions

R has a large collection of built-in functions that are called like this:

```
function_name(argument1 = value1, argument2 = value2, ...)
```

Let's try using `seq()`, which makes regular **seq**uences of numbers, and while we're at it, learn more helpful features of RStudio. Type `se` and hit TAB. A popup shows you possible completions. Specify `seq()` by typing more (a `q`) to disambiguate or by using ↑/↓ arrows to select. Notice the floating tooltip that pops up, reminding you of the function's arguments and purpose. If you want more help, press F1 to get all the details in the help tab in the lower right pane.

When you've selected the function you want, press TAB again. RStudio will add matching opening (`(`) and closing (`)`) parentheses for you. Type the name of the first argument, `from`, and set it equal to `1`. Then, type the name of the second argument, `to`, and set it equal to `10`. Finally, hit return.

```
seq(from = 1, to = 10)
#>  [1]  1  2  3  4  5  6  7  8  9 10
```

We often omit the names of the first several arguments in function calls, so we can rewrite this as follows:

```
seq(1, 10)
#>  [1]  1  2  3  4  5  6  7  8  9 10
```

Type the following code and notice that RStudio provides similar assistance with the paired quotation marks:

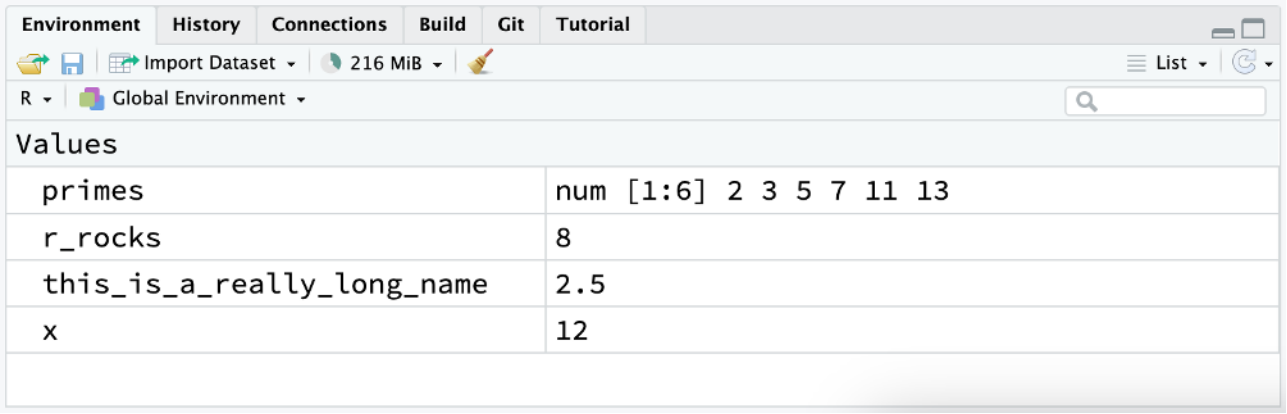
```
x <- "hello world"
```

Quotation marks and parentheses must always come in a pair. RStudio does its best to help you, but it's still possible to mess up and end up with a mismatch. If this happens, R will show you the continuation character "**+**":

```
> x <- "hello
+
```

The + tells you that R is waiting for more input; it doesn't think you're done yet. Usually, this means you've forgotten either a " or a). Either add the missing pair, or press ESCAPE to abort the expression and try again.

Note that the environment tab in the upper right pane displays all of the objects that you've created:



2.5 Exercises

1. Why does this code not work?

   ```
   my_variable <- 10
   my_varıable
   #> Error in eval(expr, envir, enclos): object 'my_varıable' not found
   ```

   Look carefully! (This may seem like an exercise in pointlessness, but training your brain to notice even the tiniest difference will pay off when programming.)

2. Tweak each of the following R commands so that they run correctly:

   ```
   libary(todyverse)

   ggplot(dTA = mpg) +
     geom_point(maping = aes(x = displ y = hwy)) +
     geom_smooth(method = "lm)
   ```

3. Press Option + Shift + K / Alt + Shift + K. What happens? How can you get to the same place using the menus?

4. Let's revisit an exercise from the Section 1.6. Run the following lines of code. Which of the two plots is saved as `mpg-plot.png`? Why?

```
my_bar_plot <- ggplot(mpg, aes(x = class)) +
  geom_bar()
my_scatter_plot <- ggplot(mpg, aes(x = cty, y = hwy)) +
  geom_point()
ggsave(filename = "mpg-plot.png", plot = my_bar_plot)
```

2.6 Summary

Now that you've learned a little more about how R code works, and some tips to help you understand your code when you come back to it in the future. In the next chapter, we'll continue your data science journey by teaching you about dplyr, the tidyverse package that helps you transform data, whether it's selecting important variables, filtering down to rows of interest, or computing summary statistics.

3 Data transformation

3.1 Introduction

Visualization is an important tool for generating insight, but it's rare that you get the data in exactly the right form you need to make the graph you want. Often you'll need to create some new variables or summaries to answer your questions with your data, or maybe you just want to rename the variables or reorder the observations to make the data a little easier to work with. You'll learn how to do all that (and more!) in this chapter, which will introduce you to data transformation using the **dplyr** package and a new dataset on flights that departed from New York City in 2013.

The goal of this chapter is to give you an overview of all the key tools for transforming a data frame. We'll start with functions that operate on rows and then columns of a data frame, then circle back to talk more about the pipe, an important tool that you use to combine verbs. We will then introduce the ability to work with groups. We will end the chapter with a case study that showcases these functions in action and we'll come back to the functions in more detail in later chapters, as we start to dig into specific types of data (e.g., numbers, strings, dates).

3.1.1 Prerequisites

In this chapter we'll focus on the dplyr package, another core member of the tidyverse. We'll illustrate the key ideas using data from the nycflights13 package, and use ggplot2 to help us understand the data.

```
library(nycflights13)
library(tidyverse)
#> ── Attaching core tidyverse packages ──────────────────────── tidyverse 2.0.0
──
#> ✔ dplyr     1.1.4     ✔ readr     2.1.5
#> ✔ forcats   1.0.0     ✔ stringr   1.5.1
#> ✔ ggplot2   3.5.1     ✔ tibble    3.2.1
#> ✔ lubridate 1.9.3     ✔ tidyr     1.3.1
#> ✔ purrr     1.0.2
#> ── Conflicts ────────────────────────────────────────── tidyverse_conflicts()
──
#> ✖ dplyr::filter() masks stats::filter()
#> ✖ dplyr::lag()    masks stats::lag()
#> ℹ Use the conflicted package (<http://conflicted.r-lib.org/>) to force all
conflicts to become errors
```

Take careful note of the conflicts message that's printed when you load the tidyverse. It tells you that dplyr overwrites some functions in base R. If you want to use the base version of these functions after loading dplyr, you'll need to use their full names: `stats::filter()` and `stats::lag()`. So far we've mostly ignored which package a function comes from because most of the time it doesn't matter. However, knowing the package can help you find help and find related functions, so when we need to be precise about which package a function comes from, we'll use the same syntax as R: `packagename::functionname()`.

3.1.2 nycflights13

To explore the basic dplyr verbs, we're going to use `nycflights13::flights`. This dataset contains all 336,776 flights that departed from New York City in 2013. The data comes from the US Bureau of Transportation Statistics, and is documented in `?flights`.

```
flights
#> # A tibble: 336,776 × 19
#>    year month   day dep_time sched_dep_time dep_delay arr_time
sched_arr_time
#>   <int> <int> <int>    <int>          <int>     <dbl>    <int>
<int>
#> 1  2013     1     1      517            515         2      830
819
#> 2  2013     1     1      533            529         4      850
830
#> 3  2013     1     1      542            540         2      923
850
#> 4  2013     1     1      544            545        -1     1004
1022
#> 5  2013     1     1      554            600        -6      812
837
#> 6  2013     1     1      554            558        -4      740
728
#> # ℹ 336,770 more rows
#> # ℹ 11 more variables: arr_delay <dbl>, carrier <chr>, flight <int>, …
```

`flights` is a tibble, a special type of data frame used by the tidyverse to avoid some common gotchas. The most important difference between tibbles and data frames is the way tibbles print; they are designed for large datasets, so they only show the first few rows and only the columns that fit on one screen. There are a few options to see everything. If you're using RStudio, the most convenient is probably `View(flights)`, which will open an interactive scrollable and filterable view. Otherwise you can use `print(flights, width = Inf)` to show all columns, or use `glimpse()`:

```
glimpse(flights)
#> Rows: 336,776
#> Columns: 19
#> $ year           <int> 2013, 2013, 2013, 2013, 2013, 2013, 2013, 2013,
2013…
#> $ month          <int> 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
1…
#> $ day            <int> 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
1…
#> $ dep_time       <int> 517, 533, 542, 544, 554, 554, 555, 557, 557, 558,
55…
#> $ sched_dep_time <int> 515, 529, 540, 545, 600, 558, 600, 600, 600, 600,
60…
#> $ dep_delay      <dbl> 2, 4, 2, -1, -6, -4, -5, -3, -3, -2, -2, -2, -2,
-2,…
#> $ arr_time       <int> 830, 850, 923, 1004, 812, 740, 913, 709, 838, 753,
8…
#> $ sched_arr_time <int> 819, 830, 850, 1022, 837, 728, 854, 723, 846, 745,
8…
#> $ arr_delay      <dbl> 11, 20, 33, -18, -25, 12, 19, -14, -8, 8, -2, -3,
7,…
#> $ carrier        <chr> "UA", "UA", "AA", "B6", "DL", "UA", "B6", "EV",
"B6"…
#> $ flight         <int> 1545, 1714, 1141, 725, 461, 1696, 507, 5708, 79,
301…
#> $ tailnum        <chr> "N14228", "N24211", "N619AA", "N804JB", "N668DN",
"N…
#> $ origin         <chr> "EWR", "LGA", "JFK", "JFK", "LGA", "EWR", "EWR",
"LG…
#> $ dest           <chr> "IAH", "IAH", "MIA", "BQN", "ATL", "ORD", "FLL",
"IA…
#> $ air_time       <dbl> 227, 227, 160, 183, 116, 150, 158, 53, 140, 138,
149…
#> $ distance       <dbl> 1400, 1416, 1089, 1576, 762, 719, 1065, 229, 944,
73…
#> $ hour           <dbl> 5, 5, 5, 5, 6, 5, 6, 6, 6, 6, 6, 6, 6, 6, 6, 5, 6,
6…
#> $ minute         <dbl> 15, 29, 40, 45, 0, 58, 0, 0, 0, 0, 0, 0, 0, 0, 0,
59…
#> $ time_hour      <dttm> 2013-01-01 05:00:00, 2013-01-01 05:00:00, 2013-01-
0…
```

In both views, the variables names are followed by abbreviations that tell you the type of each variable: `<int>` is short for integer, `<dbl>` is short for double (aka real numbers), `<chr>` for character (aka strings), and `<dttm>` for date-time. These are important because the operations you can perform on a column depend so much on its "type".

3.1.3 dplyr basics

You're about to learn the primary dplyr verbs (functions) which will allow you to solve the vast majority of your data manipulation challenges. But before we discuss their individual differences, it's worth stating what they have in common:

1. The first argument is always a data frame.
2. The subsequent arguments typically describe which columns to operate on, using the variable names (without quotes).
3. The output is always a new data frame.

Because each verb does one thing well, solving complex problems will usually require combining multiple verbs, and we'll do so with the pipe, `|>`. We'll discuss the pipe more in Section 3.4, but in brief, the pipe takes the thing on its left and passes it along to the function on its right so that `x |> f(y)` is equivalent to `f(x, y)`, and `x |> f(y) |> g(z)` is equivalent to `g(f(x, y), z)`. The easiest way to pronounce the pipe is "then". That makes it possible to get a sense of the following code even though you haven't yet learned the details:

```
flights |>
  filter(dest == "IAH") |>
  group_by(year, month, day) |>
  summarize(
    arr_delay = mean(arr_delay, na.rm = TRUE)
  )
```

dplyr's verbs are organized into four groups based on what they operate on: **rows**, **columns**, **groups**, or **tables**. In the following sections you'll learn the most important verbs for rows, columns, and groups, then we'll come back to the join verbs that work on tables in Chapter 19. Let's dive in!

3.2 Rows

The most important verbs that operate on rows of a dataset are `filter()`, which changes which rows are present without changing their order, and `arrange()`, which changes the order of the rows without changing which are present. Both functions only affect the rows, and the columns are left unchanged. We'll also discuss `distinct()` which finds rows with unique values but unlike `arrange()` and `filter()` it can also optionally modify the columns.

3.2.1 `filter()`

`filter()` allows you to keep rows based on the values of the columns[1]. The first argument is the data frame. The second and subsequent arguments are the conditions that must be true to keep the row. For example, we could find all flights that departed more than 120 minutes (two hours) late:

```
flights |>
  filter(dep_delay > 120)
#> # A tibble: 9,723 × 19
#>    year month   day dep_time sched_dep_time dep_delay arr_time
sched_arr_time
#>   <int> <int> <int>    <int>          <int>     <dbl>    <int>
<int>
#> 1  2013     1     1      848           1835       853     1001
1950
#> 2  2013     1     1      957            733       144     1056
853
#> 3  2013     1     1     1114            900       134     1447
1222
#> 4  2013     1     1     1540           1338       122     2020
1825
#> 5  2013     1     1     1815           1325       290     2120
1542
#> 6  2013     1     1     1842           1422       260     1958
1535
#> # ℹ 9,717 more rows
#> # ℹ 11 more variables: arr_delay <dbl>, carrier <chr>, flight <int>, …
```

As well as > (greater than), you can use >= (greater than or equal to), < (less than), <= (less than or equal to), == (equal to), and != (not equal to). You can also combine conditions with `&` or `,` to indicate "and" (check for both conditions) or with `|` to indicate "or" (check for either condition):

```
# Flights that departed on January 1
flights |>
  filter(month == 1 & day == 1)
#> # A tibble: 842 × 19
#>    year month   day dep_time sched_dep_time dep_delay arr_time
sched_arr_time
#>   <int> <int> <int>    <int>          <int>     <dbl>    <int>
<int>
#> 1  2013     1     1      517            515         2      830
```

```
819
#> 2  2013     1     1      533            529         4      850
830
#> 3  2013     1     1      542            540         2      923
850
#> 4  2013     1     1      544            545        -1     1004
1022
#> 5  2013     1     1      554            600        -6      812
837
#> 6  2013     1     1      554            558        -4      740
728
#> # i 836 more rows
#> # i 11 more variables: arr_delay <dbl>, carrier <chr>, flight <int>, …

# Flights that departed in January or February
flights |>
  filter(month == 1 | month == 2)
#> # A tibble: 51,955 × 19
#>    year month   day dep_time sched_dep_time dep_delay arr_time
sched_arr_time
#>   <int> <int> <int>    <int>          <int>     <dbl>    <int>
<int>
#> 1  2013     1     1      517            515         2      830
819
#> 2  2013     1     1      533            529         4      850
830
#> 3  2013     1     1      542            540         2      923
850
#> 4  2013     1     1      544            545        -1     1004
1022
#> 5  2013     1     1      554            600        -6      812
837
#> 6  2013     1     1      554            558        -4      740
728
#> # i 51,949 more rows
#> # i 11 more variables: arr_delay <dbl>, carrier <chr>, flight <int>, …
```

There's a useful shortcut when you're combining | and ==: `%in%`. It keeps rows where the variable equals one of the values on the right:

```
# A shorter way to select flights that departed in January or February
flights |>
```

```
  filter(month %in% c(1, 2))
#> # A tibble: 51,955 × 19
#>    year month   day dep_time sched_dep_time dep_delay arr_time
sched_arr_time
#>   <int> <int> <int>    <int>          <int>     <dbl>    <int>
<int>
#> 1  2013     1     1      517            515         2      830
819
#> 2  2013     1     1      533            529         4      850
830
#> 3  2013     1     1      542            540         2      923
850
#> 4  2013     1     1      544            545        -1     1004
1022
#> 5  2013     1     1      554            600        -6      812
837
#> 6  2013     1     1      554            558        -4      740
728
#> # ℹ 51,949 more rows
#> # ℹ 11 more variables: arr_delay <dbl>, carrier <chr>, flight <int>, …
```

We'll come back to these comparisons and logical operators in more detail in Chapter 12.

When you run `filter()` dplyr executes the filtering operation, creating a new data frame, and then prints it. It doesn't modify the existing `flights` dataset because dplyr functions never modify their inputs. To save the result, you need to use the assignment operator, `<-`:

```
jan1 <- flights |>
  filter(month == 1 & day == 1)
```

3.2.2 Common mistakes

When you're starting out with R, the easiest mistake to make is to use = instead of == when testing for equality. `filter()` will let you know when this happens:

```
flights |>
  filter(month = 1)
#> Error in `filter()`:
#> ! We detected a named input.
#> ℹ This usually means that you've used `=` instead of `==`.
#> ℹ Did you mean `month == 1`?
```

Another mistakes is you write “or” statements like you would in English:

```
flights |>
  filter(month == 1 | 2)
```

This “works”, in the sense that it doesn’t throw an error, but it doesn’t do what you want because `|` first checks the condition `month == 1` and then checks the condition `2`, which is not a sensible condition to check. We’ll learn more about what’s happening here and why in Section 15.6.2.

3.2.3 `arrange()`

`arrange()` changes the order of the rows based on the value of the columns. It takes a data frame and a set of column names (or more complicated expressions) to order by. If you provide more than one column name, each additional column will be used to break ties in the values of preceding columns. For example, the following code sorts by the departure time, which is spread over four columns. We get the earliest years first, then within a year the earliest months, etc.

```
flights |>
  arrange(year, month, day, dep_time)
#> # A tibble: 336,776 × 19
#>    year month   day dep_time sched_dep_time dep_delay arr_time
sched_arr_time
#>   <int> <int> <int>    <int>          <int>     <dbl>    <int>
<int>
#> 1  2013     1     1      517            515         2      830
819
#> 2  2013     1     1      533            529         4      850
830
#> 3  2013     1     1      542            540         2      923
850
#> 4  2013     1     1      544            545        -1     1004
1022
#> 5  2013     1     1      554            600        -6      812
837
#> 6  2013     1     1      554            558        -4      740
728
#> # ℹ 336,770 more rows
#> # ℹ 11 more variables: arr_delay <dbl>, carrier <chr>, flight <int>, …
```

You can use `desc()` on a column inside of `arrange()` to re-order the data frame based on that column in descending (big-to-small) order. For example, this code orders flights from most to least delayed:

```
flights |>
  arrange(desc(dep_delay))
#> # A tibble: 336,776 × 19
#>    year month   day dep_time sched_dep_time dep_delay arr_time
sched_arr_time
#>   <int> <int> <int>    <int>          <int>     <dbl>    <int>
<int>
#> 1  2013     1     9      641            900      1301     1242
1530
#> 2  2013     6    15     1432           1935      1137     1607
2120
#> 3  2013     1    10     1121           1635      1126     1239
1810
#> 4  2013     9    20     1139           1845      1014     1457
2210
#> 5  2013     7    22      845           1600      1005     1044
1815
#> 6  2013     4    10     1100           1900       960     1342
2211
#> # ℹ 336,770 more rows
#> # ℹ 11 more variables: arr_delay <dbl>, carrier <chr>, flight <int>, …
```

Note that the number of rows has not changed – we're only arranging the data, we're not filtering it.

3.2.4 `distinct()`

`distinct()` finds all the unique rows in a dataset, so in a technical sense, it primarily operates on the rows. Most of the time, however, you'll want the distinct combination of some variables, so you can also optionally supply column names:

```
# Remove duplicate rows, if any
flights |>
  distinct()
#> # A tibble: 336,776 × 19
#>    year month   day dep_time sched_dep_time dep_delay arr_time
sched_arr_time
#>   <int> <int> <int>    <int>          <int>     <dbl>    <int>
<int>
#> 1  2013     1     1      517            515         2      830
819
#> 2  2013     1     1      533            529         4      850
830
```

```
#> 3  2013     1     1      542            540         2      923
850
#> 4  2013     1     1      544            545        -1     1004
1022
#> 5  2013     1     1      554            600        -6      812
837
#> 6  2013     1     1      554            558        -4      740
728
#> # i 336,770 more rows
#> # i 11 more variables: arr_delay <dbl>, carrier <chr>, flight <int>, …

# Find all unique origin and destination pairs
flights |>
  distinct(origin, dest)
#> # A tibble: 224 × 2
#>   origin dest
#>   <chr>  <chr>
#> 1 EWR    IAH
#> 2 LGA    IAH
#> 3 JFK    MIA
#> 4 JFK    BQN
#> 5 LGA    ATL
#> 6 EWR    ORD
#> # i 218 more rows
```

Alternatively, if you want to the keep other columns when filtering for unique rows, you can use the `.keep_all = TRUE` option.

```
flights |>
  distinct(origin, dest, .keep_all = TRUE)
#> # A tibble: 224 × 19
#>    year month   day dep_time sched_dep_time dep_delay arr_time
sched_arr_time
#>   <int> <int> <int>    <int>          <int>     <dbl>    <int>
<int>
#> 1  2013     1     1      517            515         2      830
819
#> 2  2013     1     1      533            529         4      850
830
#> 3  2013     1     1      542            540         2      923
850
#> 4  2013     1     1      544            545        -1     1004
```

```
1022
#> 5  2013     1     1      554            600        -6      812
837
#> 6  2013     1     1      554            558        -4      740
728
#> # ℹ 218 more rows
#> # ℹ 11 more variables: arr_delay <dbl>, carrier <chr>, flight <int>, …
```

It's not a coincidence that all of these distinct flights are on January 1: `distinct()` will find the first occurrence of a unique row in the dataset and discard the rest.

If you want to find the number of occurrences instead, you're better off swapping `distinct()` for `count()`, and with the `sort = TRUE` argument you can arrange them in descending order of number of occurrences. You'll learn more about count in Section 13.3.

```
flights |>
  count(origin, dest, sort = TRUE)
#> # A tibble: 224 × 3
#>   origin dest      n
#>   <chr>  <chr> <int>
#> 1 JFK    LAX   11262
#> 2 LGA    ATL   10263
#> 3 LGA    ORD    8857
#> 4 JFK    SFO    8204
#> 5 LGA    CLT    6168
#> 6 EWR    ORD    6100
#> # ℹ 218 more rows
```

3.2.5 Exercises

1. In a single pipeline for each condition, find all flights that meet the condition:
    - Had an arrival delay of two or more hours
    - Flew to Houston (`IAH` or `HOU`)
    - Were operated by United, American, or Delta
    - Departed in summer (July, August, and September)
    - Arrived more than two hours late, but didn't leave late
    - Were delayed by at least an hour, but made up over 30 minutes in flight
2. Sort `flights` to find the flights with longest departure delays. Find the flights that left earliest in the morning.
3. Sort `flights` to find the fastest flights. (Hint: Try including a math calculation inside of your function.)

4. Was there a flight on every day of 2013?

5. Which flights traveled the farthest distance? Which traveled the least distance?

6. Does it matter what order you used `filter()` and `arrange()` if you're using both? Why/why not? Think about the results and how much work the functions would have to do.

3.3 Columns

There are four important verbs that affect the columns without changing the rows: `mutate()` creates new columns that are derived from the existing columns, `select()` changes which columns are present, `rename()` changes the names of the columns, and `relocate()` changes the positions of the columns.

3.3.1 `mutate()`

The job of `mutate()` is to add new columns that are calculated from the existing columns. In the transform chapters, you'll learn a large set of functions that you can use to manipulate different types of variables. For now, we'll stick with basic algebra, which allows us to compute the `gain`, how much time a delayed flight made up in the air, and the `speed` in miles per hour:

```
flights |>
  mutate(
    gain = dep_delay - arr_delay,
    speed = distance / air_time * 60
  )
#> # A tibble: 336,776 × 21
#>    year month   day dep_time sched_dep_time dep_delay arr_time
sched_arr_time
#>   <int> <int> <int>    <int>          <int>     <dbl>    <int>
<int>
#> 1  2013     1     1      517            515         2      830
819
#> 2  2013     1     1      533            529         4      850
830
#> 3  2013     1     1      542            540         2      923
850
#> 4  2013     1     1      544            545        -1     1004
1022
#> 5  2013     1     1      554            600        -6      812
837
#> 6  2013     1     1      554            558        -4      740
728
```

```
#> # ℹ 336,770 more rows
#> # ℹ 13 more variables: arr_delay <dbl>, carrier <chr>, flight <int>, …
```

By default, `mutate()` adds new columns on the right hand side of your dataset, which makes it difficult to see what's happening here. We can use the `.before` argument to instead add the variables to the left hand side[2]:

```
flights |>
  mutate(
    gain = dep_delay - arr_delay,
    speed = distance / air_time * 60,
    .before = 1
  )
#> # A tibble: 336,776 × 21
#>    gain speed  year month   day dep_time sched_dep_time dep_delay arr_time
#>   <dbl> <dbl> <int> <int> <int>    <int>          <int>     <dbl>    <int>
#> 1    -9  370.  2013     1     1      517            515         2      830
#> 2   -16  374.  2013     1     1      533            529         4      850
#> 3   -31  408.  2013     1     1      542            540         2      923
#> 4    17  517.  2013     1     1      544            545        -1     1004
#> 5    19  394.  2013     1     1      554            600        -6      812
#> 6   -16  288.  2013     1     1      554            558        -4      740
#> # ℹ 336,770 more rows
#> # ℹ 12 more variables: sched_arr_time <int>, arr_delay <dbl>, …
```

The `.` is a sign that `.before` is an argument to the function, not the name of a third new variable we are creating. You can also use `.after` to add after a variable, and in both `.before` and `.after` you can use the variable name instead of a position. For example, we could add the new variables after `day`:

```
flights |>
  mutate(
    gain = dep_delay - arr_delay,
    speed = distance / air_time * 60,
    .after = day
  )
```

Alternatively, you can control which variables are kept with the `.keep` argument. A particularly useful argument is `"used"` which specifies that we only keep the columns that were involved or created in the `mutate()` step. For example, the following output will contain only the variables `dep_delay`, `arr_delay`, `air_time`, `gain`, `hours`, and `gain_per_hour`.

```
flights |>
  mutate(
    gain = dep_delay - arr_delay,
    hours = air_time / 60,
    gain_per_hour = gain / hours,
    .keep = "used"
  )
```

Note that since we haven't assigned the result of the above computation back to `flights`, the new variables `gain`, `hours`, and `gain_per_hour` will only be printed but will not be stored in a data frame. And if we want them to be available in a data frame for future use, we should think carefully about whether we want the result to be assigned back to `flights`, overwriting the original data frame with many more variables, or to a new object. Often, the right answer is a new object that is named informatively to indicate its contents, e.g., `delay_gain`, but you might also have good reasons for overwriting `flights`.

3.3.2 `select()`

It's not uncommon to get datasets with hundreds or even thousands of variables. In this situation, the first challenge is often just focusing on the variables you're interested in. `select()` allows you to rapidly zoom in on a useful subset using operations based on the names of the variables:

- Select columns by name:

  ```
  flights |>
    select(year, month, day)
  ```

- Select all columns between year and day (inclusive):

  ```
  flights |>
    select(year:day)
  ```

- Select all columns except those from year to day (inclusive):

  ```
  flights |>
    select(!year:day)
  ```

  Historically this operation was done with `-` instead of `!`, so you're likely to see that in the wild. These two operators serve the same purpose but with subtle differences in behavior. We recommend using `!` because it reads as "not" and combines well with `&` and `|`.

- Select all columns that are characters:

```
flights |>
  select(where(is.character))
```

There are a number of helper functions you can use within `select()`:

- `starts_with("abc")`: matches names that begin with “abc”.
- `ends_with("xyz")`: matches names that end with “xyz”.
- `contains("ijk")`: matches names that contain “ijk”.
- `num_range("x", 1:3)`: matches `x1`, `x2` and `x3`.

See `?select` for more details. Once you know regular expressions (the topic of Chapter 15) you’ll also be able to use `matches()` to select variables that match a pattern.

You can rename variables as you `select()` them by using =. The new name appears on the left hand side of the =, and the old variable appears on the right hand side:

```
flights |>
  select(tail_num = tailnum)
#> # A tibble: 336,776 × 1
#>   tail_num
#>   <chr>
#> 1 N14228
#> 2 N24211
#> 3 N619AA
#> 4 N804JB
#> 5 N668DN
#> 6 N39463
#> # ℹ 336,770 more rows
```

3.3.3 `rename()`

If you want to keep all the existing variables and just want to rename a few, you can use `rename()` instead of `select()`:

```
flights |>
  rename(tail_num = tailnum)
#> # A tibble: 336,776 × 19
#>    year month   day dep_time sched_dep_time dep_delay arr_time
sched_arr_time
#>   <int> <int> <int>    <int>          <int>     <dbl>    <int>
<int>
#> 1  2013     1     1      517            515         2      830
```

```
819
#> 2  2013     1     1      533            529         4      850
830
#> 3  2013     1     1      542            540         2      923
850
#> 4  2013     1     1      544            545        -1     1004
1022
#> 5  2013     1     1      554            600        -6      812
837
#> 6  2013     1     1      554            558        -4      740
728
#> # i 336,770 more rows
#> # i 11 more variables: arr_delay <dbl>, carrier <chr>, flight <int>, …
```

If you have a bunch of inconsistently named columns and it would be painful to fix them all by hand, check out `janitor::clean_names()` which provides some useful automated cleaning.

3.3.4 `relocate()`

Use `relocate()` to move variables around. You might want to collect related variables together or move important variables to the front. By default `relocate()` moves variables to the front:

```
flights |>
  relocate(time_hour, air_time)
#> # A tibble: 336,776 × 19
#>   time_hour           air_time  year month   day dep_time sched_dep_time
#>   <dttm>                 <dbl> <int> <int> <int>    <int>          <int>
#> 1 2013-01-01 05:00:00      227  2013     1     1      517            515
#> 2 2013-01-01 05:00:00      227  2013     1     1      533            529
#> 3 2013-01-01 05:00:00      160  2013     1     1      542            540
#> 4 2013-01-01 05:00:00      183  2013     1     1      544            545
#> 5 2013-01-01 06:00:00      116  2013     1     1      554            600
#> 6 2013-01-01 05:00:00      150  2013     1     1      554            558
#> # i 336,770 more rows
#> # i 12 more variables: dep_delay <dbl>, arr_time <int>, …
```

You can also specify where to put them using the `.before` and `.after` arguments, just like in `mutate()`:

```
flights |>
  relocate(year:dep_time, .after = time_hour)
flights |>
  relocate(starts_with("arr"), .before = dep_time)
```

3.3.5 Exercises

1. Compare `dep_time`, `sched_dep_time`, and `dep_delay`. How would you expect those three numbers to be related?

2. Brainstorm as many ways as possible to select `dep_time`, `dep_delay`, `arr_time`, and `arr_delay` from `flights`.

3. What happens if you specify the name of the same variable multiple times in a `select()` call?

4. What does the `any_of()` function do? Why might it be helpful in conjunction with this vector?

```
variables <- c("year", "month", "day", "dep_delay", "arr_delay")
```

5. Does the result of running the following code surprise you? How do the select helpers deal with upper and lower case by default? How can you change that default?

```
flights |> select(contains("TIME"))
```

6. Rename `air_time` to `air_time_min` to indicate units of measurement and move it to the beginning of the data frame.

7. Why doesn't the following work, and what does the error mean?

```
flights |>
  select(tailnum) |>
  arrange(arr_delay)
#> Error in `arrange()`:
#> ℹ In argument: `..1 = arr_delay`.
#> Caused by error:
#> ! object 'arr_delay' not found
```

3.4 The pipe

We've shown you simple examples of the pipe above, but its real power arises when you start to combine multiple verbs. For example, imagine that you wanted to find the fastest flights to Houston's IAH airport: you need to combine `filter()`, `mutate()`, `select()`, and `arrange()`:

```
flights |>
  filter(dest == "IAH") |>
  mutate(speed = distance / air_time * 60) |>
  select(year:day, dep_time, carrier, flight, speed) |>
  arrange(desc(speed))
```

```
#> # A tibble: 7,198 × 7
#>    year month   day dep_time carrier flight speed
#>   <int> <int> <int>    <int> <chr>    <int> <dbl>
#> 1  2013     7     9      707 UA         226  522.
#> 2  2013     8    27     1850 UA        1128  521.
#> 3  2013     8    28      902 UA        1711  519.
#> 4  2013     8    28     2122 UA        1022  519.
#> 5  2013     6    11     1628 UA        1178  515.
#> 6  2013     8    27     1017 UA         333  515.
#> # ℹ 7,192 more rows
```

Even though this pipeline has four steps, it's easy to skim because the verbs come at the start of each line: start with the `flights` data, then filter, then mutate, then select, then arrange.

What would happen if we didn't have the pipe? We could nest each function call inside the previous call:

```
arrange(
  select(
    mutate(
      filter(
        flights,
        dest == "IAH"
      ),
      speed = distance / air_time * 60
    ),
    year:day, dep_time, carrier, flight, speed
  ),
  desc(speed)
)
```

Or we could use a bunch of intermediate objects:

```
flights1 <- filter(flights, dest == "IAH")
flights2 <- mutate(flights1, speed = distance / air_time * 60)
flights3 <- select(flights2, year:day, dep_time, carrier, flight, speed)
arrange(flights3, desc(speed))
```

While both forms have their time and place, the pipe generally produces data analysis code that is easier to write and read.

To add the pipe to your code, we recommend using the built-in keyboard shortcut Ctrl/Cmd + Shift + M. You'll need to make one change to your RStudio options to use `|>` instead of `%>%` as shown in Figure 3.1; more on

`%>%` shortly.

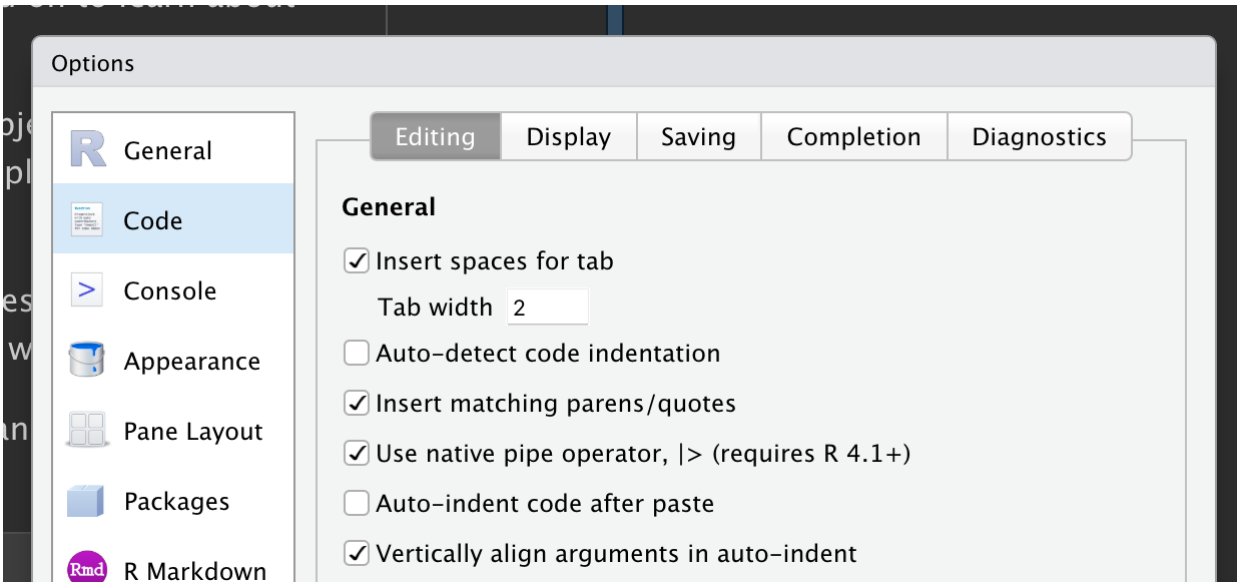


Figure 3.1: To insert `|>`, make sure the "Use native pipe operator" option is checked.

If you've been using the tidyverse for a while, you might be familiar with the `%>%` pipe provided by the **magrittr** package. The magrittr package is included in the core tidyverse, so you can use `%>%` whenever you load the tidyverse:

```
library(tidyverse)

mtcars %>%
  group_by(cyl) %>%
  summarize(n = n())
```

For simple cases, `|>` and `%>%` behave identically. So why do we recommend the base pipe? Firstly, because it's part of base R, it's always available for you to use, even when you're not using the tidyverse. Secondly, `|>` is quite a bit simpler than `%>%`: in the time between the invention of `%>%` in 2014 and the inclusion of `|>` in R 4.1.0 in 2021, we gained a better understanding of the pipe. This allowed the base implementation to jettison infrequently used and less important features.

3.5 Groups

So far you've learned about functions that work with rows and columns. dplyr gets even more powerful when you add in the ability to work with groups. In this section, we'll focus on the most important functions: `group_by()`, `summarize()`, and the slice family of functions.

3.5.1 `group_by()`

Use `group_by()` to divide your dataset into groups meaningful for your analysis:

```
flights |>
  group_by(month)
#> # A tibble: 336,776 × 19
#> # Groups:   month [12]
#>    year month   day dep_time sched_dep_time dep_delay arr_time
sched_arr_time
#>   <int> <int> <int>    <int>          <int>     <dbl>    <int>
<int>
#> 1  2013     1     1      517            515         2      830
819
#> 2  2013     1     1      533            529         4      850
830
#> 3  2013     1     1      542            540         2      923
850
#> 4  2013     1     1      544            545        -1     1004
1022
#> 5  2013     1     1      554            600        -6      812
837
#> 6  2013     1     1      554            558        -4      740
728
#> # ℹ 336,770 more rows
#> # ℹ 11 more variables: arr_delay <dbl>, carrier <chr>, flight <int>, …
```

`group_by()` doesn't change the data but, if you look closely at the output, you'll notice that the output indicates that it is "grouped by" month (`Groups: month [12]`). This means subsequent operations will now work "by month". `group_by()` adds this grouped feature (referred to as class) to the data frame, which changes the behavior of the subsequent verbs applied to the data.

3.5.2 `summarize()`

The most important grouped operation is a summary, which, if being used to calculate a single summary statistic, reduces the data frame to have a single row for each group. In dplyr, this operation is performed by `summarize()`[3], as shown by the following example, which computes the average departure delay by month:

```
flights |>
  group_by(month) |>
  summarize(
    avg_delay = mean(dep_delay)
  )
#> # A tibble: 12 × 2
```

```
#>   month avg_delay
#>   <int>     <dbl>
#> 1     1        NA
#> 2     2        NA
#> 3     3        NA
#> 4     4        NA
#> 5     5        NA
#> 6     6        NA
#> # i 6 more rows
```

Uhoh! Something has gone wrong and all of our results are `NA`s (pronounced "N-A"), R's symbol for missing value. This happened because some of the observed flights had missing data in the delay column, and so when we calculated the mean including those values, we got an `NA` result. We'll come back to discuss missing values in detail in Chapter 18, but for now we'll tell the `mean()` function to ignore all missing values by setting the argument `na.rm` to `TRUE`:

```
flights |>
  group_by(month) |>
  summarize(
    avg_delay = mean(dep_delay, na.rm = TRUE)
  )
#> # A tibble: 12 × 2
#>   month avg_delay
#>   <int>     <dbl>
#> 1     1      10.0
#> 2     2      10.8
#> 3     3      13.2
#> 4     4      13.9
#> 5     5      13.0
#> 6     6      20.8
#> # i 6 more rows
```

You can create any number of summaries in a single call to `summarize()`. You'll learn various useful summaries in the upcoming chapters, but one very useful summary is `n()`, which returns the number of rows in each group:

```
flights |>
  group_by(month) |>
  summarize(
    avg_delay = mean(dep_delay, na.rm = TRUE),
    n = n()
```

```
  )
#> # A tibble: 12 × 3
#>   month avg_delay     n
#>   <int>     <dbl> <int>
#> 1     1      10.0 27004
#> 2     2      10.8 24951
#> 3     3      13.2 28834
#> 4     4      13.9 28330
#> 5     5      13.0 28796
#> 6     6      20.8 28243
#> # ℹ 6 more rows
```

Means and counts can get you a surprisingly long way in data science!

3.5.3 The `slice_` functions

There are five handy functions that allow you extract specific rows within each group:

- `df |> slice_head(n = 1)` takes the first row from each group.
- `df |> slice_tail(n = 1)` takes the last row in each group.
- `df |> slice_min(x, n = 1)` takes the row with the smallest value of column `x`.
- `df |> slice_max(x, n = 1)` takes the row with the largest value of column `x`.
- `df |> slice_sample(n = 1)` takes one random row.

You can vary `n` to select more than one row, or instead of `n =`, you can use `prop = 0.1` to select (e.g.) 10% of the rows in each group. For example, the following code finds the flights that are most delayed upon arrival at each destination:

```
flights |>
  group_by(dest) |>
  slice_max(arr_delay, n = 1) |>
  relocate(dest)
#> # A tibble: 108 × 19
#> # Groups:   dest [105]
#>   dest   year month   day dep_time sched_dep_time dep_delay arr_time
#>   <chr> <int> <int> <int>    <int>          <int>     <dbl>    <int>
#> 1 ABQ    2013     7    22     2145           2007        98      132
#> 2 ACK    2013     7    23     1139            800       219     1250
#> 3 ALB    2013     1    25      123           2000       323      229
#> 4 ANC    2013     8    17     1740           1625        75     2042
#> 5 ATL    2013     7    22     2257            759       898      121
#> 6 AUS    2013     7    10     2056           1505       351     2347
```

```
#> # ℹ 102 more rows
#> # ℹ 11 more variables: sched_arr_time <int>, arr_delay <dbl>, …
```

Note that there are 105 destinations but we get 108 rows here. What's up? `slice_min()` and `slice_max()` keep tied values so `n = 1` means give us all rows with the highest value. If you want exactly one row per group you can set `with_ties = FALSE`.

This is similar to computing the max delay with `summarize()`, but you get the whole corresponding row (or rows if there's a tie) instead of the single summary statistic.

3.5.4 Grouping by multiple variables

You can create groups using more than one variable. For example, we could make a group for each date.

```
daily <- flights |>
  group_by(year, month, day)
daily
#> # A tibble: 336,776 × 19
#> # Groups:   year, month, day [365]
#>    year month   day dep_time sched_dep_time dep_delay arr_time
sched_arr_time
#>   <int> <int> <int>    <int>          <int>     <dbl>    <int>
<int>
#> 1  2013     1     1      517            515         2      830
819
#> 2  2013     1     1      533            529         4      850
830
#> 3  2013     1     1      542            540         2      923
850
#> 4  2013     1     1      544            545        -1     1004
1022
#> 5  2013     1     1      554            600        -6      812
837
#> 6  2013     1     1      554            558        -4      740
728
#> # ℹ 336,770 more rows
#> # ℹ 11 more variables: arr_delay <dbl>, carrier <chr>, flight <int>, …
```

When you summarize a tibble grouped by more than one variable, each summary peels off the last group. In hindsight, this wasn't a great way to make this function work, but it's difficult to change without breaking existing code. To make it obvious what's happening, dplyr displays a message that tells you how you can change this behavior:

```
daily_flights <- daily |>
  summarize(n = n())
#> `summarise()` has grouped output by 'year', 'month'. You can override
using
#> the `.groups` argument.
```

If you're happy with this behavior, you can explicitly request it in order to suppress the message:

```
daily_flights <- daily |>
  summarize(
    n = n(),
    .groups = "drop_last"
  )
```

Alternatively, change the default behavior by setting a different value, e.g., `"drop"` to drop all grouping or `"keep"` to preserve the same groups.

3.5.5 Ungrouping

You might also want to remove grouping from a data frame without using `summarize()`. You can do this with `ungroup()`.

```
daily |>
  ungroup()
#> # A tibble: 336,776 × 19
#>    year month   day dep_time sched_dep_time dep_delay arr_time
sched_arr_time
#>   <int> <int> <int>    <int>          <int>     <dbl>    <int>
<int>
#> 1  2013     1     1      517            515         2      830
819
#> 2  2013     1     1      533            529         4      850
830
#> 3  2013     1     1      542            540         2      923
850
#> 4  2013     1     1      544            545        -1     1004
1022
#> 5  2013     1     1      554            600        -6      812
837
#> 6  2013     1     1      554            558        -4      740
728
```

```
#> # ℹ 336,770 more rows
#> # ℹ 11 more variables: arr_delay <dbl>, carrier <chr>, flight <int>, …
```

Now let's see what happens when you summarize an ungrouped data frame.

```
daily |>
  ungroup() |>
  summarize(
    avg_delay = mean(dep_delay, na.rm = TRUE),
    flights = n()
  )
#> # A tibble: 1 × 2
#>   avg_delay flights
#>       <dbl>   <int>
#> 1      12.6  336776
```

You get a single row back because dplyr treats all the rows in an ungrouped data frame as belonging to one group.

3.5.6 `.by`

dplyr 1.1.0 includes a new, experimental, syntax for per-operation grouping, the `.by` argument. `group_by()` and `ungroup()` aren't going away, but you can now also use the `.by` argument to group within a single operation:

```
flights |>
  summarize(
    delay = mean(dep_delay, na.rm = TRUE),
    n = n(),
    .by = month
  )
```

Or if you want to group by multiple variables:

```
flights |>
  summarize(
    delay = mean(dep_delay, na.rm = TRUE),
    n = n(),
    .by = c(origin, dest)
  )
```

`.by` works with all verbs and has the advantage that you don't need to use the `.groups` argument to suppress the grouping message or `ungroup()` when you're done.

We didn't focus on this syntax in this chapter because it was very new when we wrote the book. We did want to mention it because we think it has a lot of promise and it's likely to be quite popular. You can learn more about it in the dplyr 1.1.0 blog post.

3.5.7 Exercises

1. Which carrier has the worst average delays? Challenge: can you disentangle the effects of bad airports vs. bad carriers? Why/why not? (Hint: think about `flights |> group_by(carrier, dest) |> summarize(n())`)

2. Find the flights that are most delayed upon departure from each destination.

3. How do delays vary over the course of the day. Illustrate your answer with a plot.

4. What happens if you supply a negative `n` to `slice_min()` and friends?

5. Explain what `count()` does in terms of the dplyr verbs you just learned. What does the `sort` argument to `count()` do?

6. Suppose we have the following tiny data frame:

```
df <- tibble(
  x = 1:5,
  y = c("a", "b", "a", "a", "b"),
  z = c("K", "K", "L", "L", "K")
)
```

   a. Write down what you think the output will look like, then check if you were correct, and describe what `group_by()` does.

```
df |>
  group_by(y)
```

   b. Write down what you think the output will look like, then check if you were correct, and describe what `arrange()` does. Also comment on how it's different from the `group_by()` in part (a).

```
df |>
  arrange(y)
```

   c. Write down what you think the output will look like, then check if you were correct, and describe what the pipeline does.

d. Write down what you think the output will look like, then check if you were correct, and describe what the pipeline does. Then, comment on what the message says.

e. Write down what you think the output will look like, then check if you were correct, and describe what the pipeline does. How is the output different from the one in part (d)?

f. Write down what you think the outputs will look like, then check if you were correct, and describe what each pipeline does. How are the outputs of the two pipelines different?

3.6 Case study: aggregates and sample size

Whenever you do any aggregation, it's always a good idea to include a count (`n()`). That way, you can ensure that you're not drawing conclusions based on very small amounts of data. We'll demonstrate this with some baseball data from the **Lahman** package. Specifically, we will compare what proportion of times a player gets a hit (`H`) vs. the number of times they try to put the ball in play (`AB`):

```
batters <- Lahman::Batting |>
  group_by(playerID) |>
  summarize(
    performance = sum(H, na.rm = TRUE) / sum(AB, na.rm = TRUE),
    n = sum(AB, na.rm = TRUE)
  )
batters
#> # A tibble: 20,469 × 3
#>   playerID  performance     n
#>   <chr>           <dbl> <int>
#> 1 aardsda01       0         4
#> 2 aaronha01       0.305 12364
#> 3 aaronto01       0.229   944
#> 4 aasedo01        0         5
#> 5 abadan01        0.0952   21
#> 6 abadfe01        0.111     9
#> # ℹ 20,463 more rows
```

When we plot the skill of the batter (measured by the batting average, `performance`) against the number of opportunities to hit the ball (measured by times at bat, `n`), you see two patterns:

1. The variation in `performance` is larger among players with fewer at-bats. The shape of this plot is very characteristic: whenever you plot a mean (or other summary statistics) vs. group size, you'll see that the variation decreases as the sample size increases[4].

2. There's a positive correlation between skill (`performance`) and opportunities to hit the ball (`n`) because teams want to give their best batters the most opportunities to hit the ball.

Note the handy pattern for combining ggplot2 and dplyr. You just have to remember to switch from `|>`, for dataset processing, to `+` for adding layers to your plot.

This also has important implications for ranking. If you naively sort on `desc(performance)`, the people with the best batting averages are clearly the ones who tried to put the ball in play very few times and happened to get a hit, they're not necessarily the most skilled players:

```
batters |>
  arrange(desc(performance))
#> # A tibble: 20,469 × 3
#>   playerID  performance     n
#>   <chr>           <dbl> <int>
#> 1 abramge01           1     1
#> 2 alberan01           1     1
#> 3 banisje01           1     1
#> 4 bartocl01           1     1
#> 5 bassdo01            1     1
#> 6 birasst01           1     2
#> # ℹ 20,463 more rows
```

You can find a good explanation of this problem and how to overcome it at http://varianceexplained.org/r/empirical_bayes_baseball/ and https://www.evanmiller.org/how-not-to-sort-by-average-rating.html.

3.7 Summary

In this chapter, you've learned the tools that dplyr provides for working with data frames. The tools are roughly grouped into three categories: those that manipulate the rows (like `filter()` and `arrange()`), those that manipulate the columns (like `select()` and `mutate()`), and those that manipulate groups (like `group_by()` and `summarize()`). In this chapter, we've focused on these "whole data frame" tools, but you haven't yet learned much about what you can do with the individual variable. We'll come back to that in the Transform part of the book, where each chapter will give you tools for a specific type of variable.

In the next chapter, we'll pivot back to workflow to discuss the importance of code style, keeping your code well organized in order to make it easy for you and others to read and understand your code.

4 Workflow: code style

Good coding style is like correct punctuation: you can manage without it, butitsuremakesthingseasiertoread. Even as a very new programmer, it's a good idea to work on your code style. Using a consistent style makes it easier for others (including future-you!) to read your work and is particularly important if you need to get help from someone else. This chapter will introduce the most important points of the tidyverse style guide, which is used throughout this book.

Styling your code will feel a bit tedious to start with, but if you practice it, it will soon become second nature. Additionally, there are some great tools to quickly restyle existing code, like the **styler** package by Lorenz Walthert. Once you've installed it with `install.packages("styler")`, an easy way to use it is via RStudio's **command palette**. The command palette lets you use any built-in RStudio command and many addins provided by packages. Open the palette by pressing Cmd/Ctrl + Shift + P, then type "styler" to see all the shortcuts offered by styler. Figure 4.1 shows the results.



Figure 4.1: RStudio's command palette makes it easy to access every RStudio command using only the keyboard.

We'll use the tidyverse and nycflights13 packages for code examples in this chapter.

```
library(tidyverse)
library(nycflights13)
```

4.1 Names

We talked briefly about names in Section 2.3. Remember that variable names (those created by <- and those created by `mutate()`) should use only lowercase letters, numbers, and _. Use _ to separate words within a name.

```
# Strive for:
short_flights <- flights |> filter(air_time < 60)

# Avoid:
SHORTFLIGHTS <- flights |> filter(air_time < 60)
```

As a general rule of thumb, it's better to prefer long, descriptive names that are easy to understand rather than concise names that are fast to type. Short names save relatively little time when writing code (especially since autocomplete will help you finish typing them), but it can be time-consuming when you come back to old code and are forced to puzzle out a cryptic abbreviation.

If you have a bunch of names for related things, do your best to be consistent. It's easy for inconsistencies to arise when you forget a previous convention, so don't feel bad if you have to go back and rename things. In general, if you have a bunch of variables that are a variation on a theme, you're better off giving them a common prefix rather than a common suffix because autocomplete works best on the start of a variable.

4.2 Spaces

Put spaces on either side of mathematical operators apart from `^` (i.e. `+`, `-`, `==`, `<`, …), and around the assignment operator (`<-`).

```
# Strive for
z <- (a + b)^2 / d

# Avoid
z<-( a + b ) ^ 2/d
```

Don't put spaces inside or outside parentheses for regular function calls. Always put a space after a comma, just like in standard English.

```
# Strive for
mean(x, na.rm = TRUE)

# Avoid
mean (x ,na.rm=TRUE)
```

It's OK to add extra spaces if it improves alignment. For example, if you're creating multiple variables in `mutate()`, you might want to add spaces so that all the `=` line up.[1] This makes it easier to skim the code.

```
flights |>
  mutate(
    speed      = distance / air_time,
```

```
    dep_hour   = dep_time %/% 100,
    dep_minute = dep_time %%  100
  )
```

4.3 Pipes

`|>` should always have a space before it and should typically be the last thing on a line. This makes it easier to add new steps, rearrange existing steps, modify elements within a step, and get a 10,000 ft view by skimming the verbs on the left-hand side.

```
# Strive for
flights |>
  filter(!is.na(arr_delay), !is.na(tailnum)) |>
  count(dest)

# Avoid
flights|>filter(!is.na(arr_delay), !is.na(tailnum))|>count(dest)
```

If the function you're piping into has named arguments (like `mutate()` or `summarize()`), put each argument on a new line. If the function doesn't have named arguments (like `select()` or `filter()`), keep everything on one line unless it doesn't fit, in which case you should put each argument on its own line.

```
# Strive for
flights |>
  group_by(tailnum) |>
  summarize(
    delay = mean(arr_delay, na.rm = TRUE),
    n = n()
  )

# Avoid
flights |>
  group_by(
    tailnum
  ) |>
  summarize(delay = mean(arr_delay, na.rm = TRUE), n = n())
```

After the first step of the pipeline, indent each line by two spaces. RStudio will automatically put the spaces in for you after a line break following a `|>`. If you're putting each argument on its own line, indent by an extra two spaces. Make sure `)` is on its own line, and un-indented to match the horizontal position of the function name.

```
# Strive for
flights |>
  group_by(tailnum) |>
  summarize(
    delay = mean(arr_delay, na.rm = TRUE),
    n = n()
  )

# Avoid
flights|>
  group_by(tailnum) |>
  summarize(
              delay = mean(arr_delay, na.rm = TRUE),
              n = n()
            )

# Avoid
flights|>
  group_by(tailnum) |>
  summarize(
  delay = mean(arr_delay, na.rm = TRUE),
  n = n()
  )
```

It's OK to shirk some of these rules if your pipeline fits easily on one line. But in our collective experience, it's common for short snippets to grow longer, so you'll usually save time in the long run by starting with all the vertical space you need.

```
# This fits compactly on one line
df |> mutate(y = x + 1)

# While this takes up 4x as many lines, it's easily extended to
# more variables and more steps in the future
df |>
  mutate(
    y = x + 1
  )
```

Finally, be wary of writing very long pipes, say longer than 10-15 lines. Try to break them up into smaller sub-tasks, giving each task an informative name. The names will help cue the reader into what's happening and makes it easier to check that intermediate results are as expected. Whenever you can give something an informative name, you should give it an informative name, for example when you fundamentally change the

structure of the data, e.g., after pivoting or summarizing. Don't expect to get it right the first time! This means breaking up long pipelines if there are intermediate states that can get good names.

4.4 ggplot2

The same basic rules that apply to the pipe also apply to ggplot2; just treat `+` the same way as `|>`.

Again, if you can't fit all of the arguments to a function on to a single line, put each argument on its own line:

```
flights |>
  group_by(dest) |>
  summarize(
    distance = mean(distance),
    speed = mean(distance / air_time, na.rm = TRUE)
  ) |>
  ggplot(aes(x = distance, y = speed)) +
  geom_smooth(
    method = "loess",
    span = 0.5,
    se = FALSE,
    color = "white",
    linewidth = 4
  ) +
  geom_point()
```

Watch for the transition from `|>` to `+`. We wish this transition wasn't necessary, but unfortunately, ggplot2 was written before the pipe was discovered.

4.5 Sectioning comments

As your scripts get longer, you can use **sectioning** comments to break up your file into manageable pieces:

```
# Load data ------------------------------------

# Plot data ------------------------------------
```

RStudio provides a keyboard shortcut to create these headers (Cmd/Ctrl + Shift + R), and will display them in the code navigation drop-down at the bottom-left of the editor, as shown in Figure 4.2.

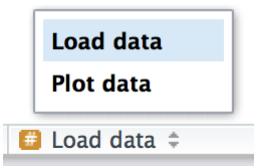


Figure 4.2: After adding sectioning comments to your script, you can easily navigate to them using the code navigation tool in the bottom-left of the script editor.

4.6 Exercises

1. Restyle the following pipelines following the guidelines above.

```
flights|>filter(dest=="IAH")|>group_by(year,month,day)|>summarize(n=n(),
delay=mean(arr_delay,na.rm=TRUE))|>filter(n>10)

flights|>filter(carrier=="UA",dest%in%c("IAH","HOU"),sched_dep_time>
0900,sched_arr_time<2000)|>group_by(flight)|>summarize(delay=mean(
arr_delay,na.rm=TRUE),cancelled=sum(is.na(arr_delay)),n=n())|>filter(n>10)
```

4.7 Summary

In this chapter, you've learned the most important principles of code style. These may feel like a set of arbitrary rules to start with (because they are!) but over time, as you write more code, and share code with more people, you'll see how important a consistent style is. And don't forget about the styler package: it's a great way to quickly improve the quality of poorly styled code.

In the next chapter, we switch back to data science tools, learning about tidy data. Tidy data is a consistent way of organizing your data frames that is used throughout the tidyverse. This consistency makes your life easier because once you have tidy data, it just works with the vast majority of tidyverse functions. Of course, life is never easy, and most datasets you encounter in the wild will not already be tidy. So we'll also teach you how to use the tidyr package to tidy your untidy data.

5 Data tidying

5.1 Introduction

> "Happy families are all alike; every unhappy family is unhappy in its own way."
> — Leo Tolstoy

> "Tidy datasets are all alike, but every messy dataset is messy in its own way."
> — Hadley Wickham

In this chapter, you will learn a consistent way to organize your data in R using a system called **tidy data**. Getting your data into this format requires some work up front, but that work pays off in the long term. Once you have tidy data and the tidy tools provided by packages in the tidyverse, you will spend much less time munging data from one representation to another, allowing you to spend more time on the data questions you care about.

In this chapter, you'll first learn the definition of tidy data and see it applied to a simple toy dataset. Then we'll dive into the primary tool you'll use for tidying data: pivoting. Pivoting allows you to change the form of your data without changing any of the values.

5.1.1 Prerequisites

In this chapter, we'll focus on tidyr, a package that provides a bunch of tools to help tidy up your messy datasets. tidyr is a member of the core tidyverse.

```
library(tidyverse)
```

From this chapter on, we'll suppress the loading message from `library(tidyverse)`.

5.2 Tidy data

You can represent the same underlying data in multiple ways. The example below shows the same data organized in three different ways. Each dataset shows the same values of four variables: *country*, *year*, *population*, and number of documented *cases* of TB (tuberculosis), but each dataset organizes the values in a different way.

```
table1
#> # A tibble: 6 × 4
#>   country      year  cases population
```

```
#>   <chr>       <dbl>  <dbl>      <dbl>
#> 1 Afghanistan  1999    745   19987071
#> 2 Afghanistan  2000   2666   20595360
#> 3 Brazil       1999  37737  172006362
#> 4 Brazil       2000  80488  174504898
#> 5 China        1999 212258 1272915272
#> 6 China        2000 213766 1280428583

table2
#> # A tibble: 12 × 4
#>   country      year type           count
#>   <chr>       <dbl> <chr>          <dbl>
#> 1 Afghanistan  1999 cases            745
#> 2 Afghanistan  1999 population  19987071
#> 3 Afghanistan  2000 cases           2666
#> 4 Afghanistan  2000 population  20595360
#> 5 Brazil       1999 cases          37737
#> 6 Brazil       1999 population 172006362
#> # i 6 more rows

table3
#> # A tibble: 6 × 3
#>   country      year rate
#>   <chr>       <dbl> <chr>
#> 1 Afghanistan  1999 745/19987071
#> 2 Afghanistan  2000 2666/20595360
#> 3 Brazil       1999 37737/172006362
#> 4 Brazil       2000 80488/174504898
#> 5 China        1999 212258/1272915272
#> 6 China        2000 213766/1280428583
```

These are all representations of the same underlying data, but they are not equally easy to use. One of them, `table1`, will be much easier to work with inside the tidyverse because it's **tidy**.

There are three interrelated rules that make a dataset tidy:

1. Each variable is a column; each column is a variable.
2. Each observation is a row; each row is an observation.
3. Each value is a cell; each cell is a single value.

Figure 5.1 shows the rules visually.

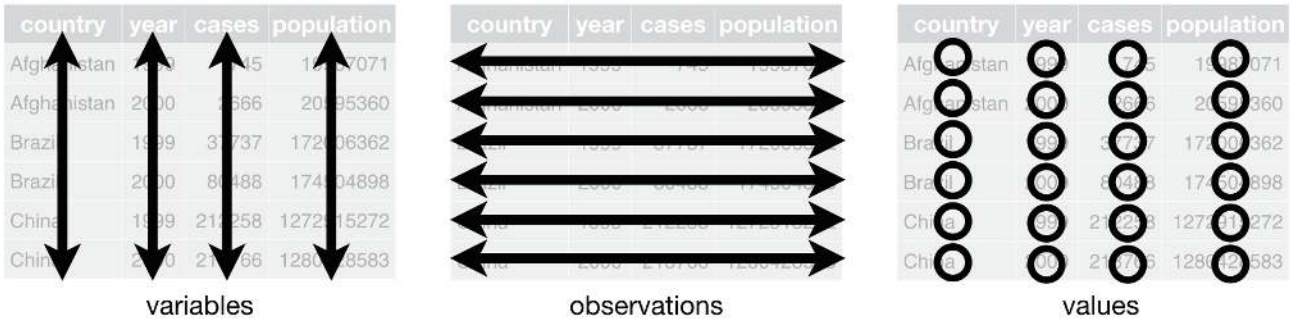


Figure 5.1: The following three rules make a dataset tidy: variables are columns, observations are rows, and values are cells.

Why ensure that your data is tidy? There are two main advantages:

1. There's a general advantage to picking one consistent way of storing data. If you have a consistent data structure, it's easier to learn the tools that work with it because they have an underlying uniformity.

2. There's a specific advantage to placing variables in columns because it allows R's vectorized nature to shine. As you learned in Section 3.3.1 and Section 3.5.2, most built-in R functions work with vectors of values. That makes transforming tidy data feel particularly natural.

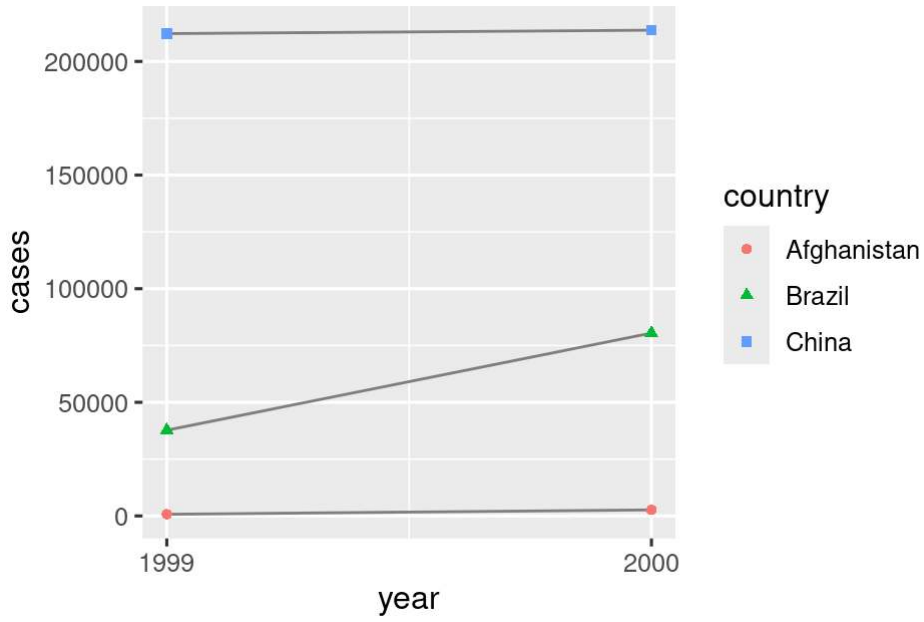
dplyr, ggplot2, and all the other packages in the tidyverse are designed to work with tidy data. Here are a few small examples showing how you might work with `table1`.

```
# Compute rate per 10,000
table1 |>
  mutate(rate = cases / population * 10000)
#> # A tibble: 6 × 5
#>   country      year  cases population  rate
#>   <chr>       <dbl>  <dbl>      <dbl> <dbl>
#> 1 Afghanistan  1999    745   19987071 0.373
#> 2 Afghanistan  2000   2666   20595360 1.29
#> 3 Brazil       1999  37737  172006362 2.19
#> 4 Brazil       2000  80488  174504898 4.61
#> 5 China        1999 212258 1272915272 1.67
#> 6 China        2000 213766 1280428583 1.67

# Compute total cases per year
table1 |>
  group_by(year) |>
  summarize(total_cases = sum(cases))
```

```
#> # A tibble: 2 × 2
#>    year total_cases
#>   <dbl>       <dbl>
#> 1  1999      250740
#> 2  2000      296920

# Visualize changes over time
ggplot(table1, aes(x = year, y = cases)) +
  geom_line(aes(group = country), color = "grey50") +
  geom_point(aes(color = country, shape = country)) +
  scale_x_continuous(breaks = c(1999, 2000)) # x-axis breaks at 1999 and 2000
```



5.2.1 Exercises

1. For each of the sample tables, describe what each observation and each column represents.

2. Sketch out the process you'd use to calculate the `rate` for `table2` and `table3`. You will need to perform four operations:

    a. Extract the number of TB cases per country per year.
    b. Extract the matching population per country per year.
    c. Divide cases by population, and multiply by 10000.
    d. Store back in the appropriate place.

    You haven't yet learned all the functions you'd need to actually perform these operations, but you should still be able to think through the transformations you'd need.

5.3 Lengthening data

The principles of tidy data might seem so obvious that you wonder if you'll ever encounter a dataset that isn't tidy. Unfortunately, however, most real data is untidy. There are two main reasons:

1. Data is often organized to facilitate some goal other than analysis. For example, it's common for data to be structured to make data entry, not analysis, easy.

2. Most people aren't familiar with the principles of tidy data, and it's hard to derive them yourself unless you spend a lot of time working with data.

This means that most real analyses will require at least a little tidying. You'll begin by figuring out what the underlying variables and observations are. Sometimes this is easy; other times you'll need to consult with the people who originally generated the data. Next, you'll **pivot** your data into a tidy form, with variables in the columns and observations in the rows.

tidyr provides two functions for pivoting data: `pivot_longer()` and `pivot_wider()`. We'll first start with `pivot_longer()` because it's the most common case. Let's dive into some examples.

5.3.1 Data in column names

The `billboard` dataset records the billboard rank of songs in the year 2000:

```
billboard
#> # A tibble: 317 × 79
#>   artist       track                date.entered   wk1   wk2   wk3   wk4
wk5
#>   <chr>        <chr>                <date>       <dbl> <dbl> <dbl> <dbl>
<dbl>
#> 1 2 Pac        Baby Don't Cry (Ke… 2000-02-26      87    82    72    77
87
#> 2 2Ge+her      The Hardest Part O… 2000-09-02      91    87    92    NA
NA
#> 3 3 Doors Down Kryptonite          2000-04-08      81    70    68    67
66
#> 4 3 Doors Down Loser               2000-10-21      76    76    72    69
67
#> 5 504 Boyz     Wobble Wobble       2000-04-15      57    34    25    17
17
#> 6 98^0         Give Me Just One N… 2000-08-19      51    39    34    26
26
#> # ℹ 311 more rows
#> # ℹ 71 more variables: wk6 <dbl>, wk7 <dbl>, wk8 <dbl>, wk9 <dbl>, …
```

In this dataset, each observation is a song. The first three columns (`artist`, `track` and `date.entered`) are variables that describe the song. Then we have 76 columns (`wk1`-`wk76`) that describe the rank of the song in each week[1]. Here, the column names are one variable (the `week`) and the cell values are another (the `rank`).

To tidy this data, we'll use `pivot_longer()`:

```
billboard |>
  pivot_longer(
    cols = starts_with("wk"),
    names_to = "week",
    values_to = "rank"
  )
#> # A tibble: 24,092 × 5
#>    artist track                   date.entered week   rank
#>    <chr>  <chr>                   <date>       <chr> <dbl>
#>  1 2 Pac  Baby Don't Cry (Keep... 2000-02-26   wk1      87
#>  2 2 Pac  Baby Don't Cry (Keep... 2000-02-26   wk2      82
#>  3 2 Pac  Baby Don't Cry (Keep... 2000-02-26   wk3      72
#>  4 2 Pac  Baby Don't Cry (Keep... 2000-02-26   wk4      77
#>  5 2 Pac  Baby Don't Cry (Keep... 2000-02-26   wk5      87
#>  6 2 Pac  Baby Don't Cry (Keep... 2000-02-26   wk6      94
#>  7 2 Pac  Baby Don't Cry (Keep... 2000-02-26   wk7      99
#>  8 2 Pac  Baby Don't Cry (Keep... 2000-02-26   wk8      NA
#>  9 2 Pac  Baby Don't Cry (Keep... 2000-02-26   wk9      NA
#> 10 2 Pac  Baby Don't Cry (Keep... 2000-02-26   wk10     NA
#> # ℹ 24,082 more rows
```

After the data, there are three key arguments:

- `cols` specifies which columns need to be pivoted, i.e. which columns aren't variables. This argument uses the same syntax as `select()` so here we could use `!c(artist, track, date.entered)` or `starts_with("wk")`.
- `names_to` names the variable stored in the column names, we named that variable `week`.
- `values_to` names the variable stored in the cell values, we named that variable `rank`.

Note that in the code `"week"` and `"rank"` are quoted because those are new variables we're creating, they don't yet exist in the data when we run the `pivot_longer()` call.

Now let's turn our attention to the resulting, longer data frame. What happens if a song is in the top 100 for less than 76 weeks? Take 2 Pac's "Baby Don't Cry", for example. The above output suggests that it was only in the top 100 for 7 weeks, and all the remaining weeks are filled in with missing values. These `NA`s don't

really represent unknown observations; they were forced to exist by the structure of the dataset[2], so we can ask `pivot_longer()` to get rid of them by setting `values_drop_na = TRUE`:

```
billboard |>
  pivot_longer(
    cols = starts_with("wk"),
    names_to = "week",
    values_to = "rank",
    values_drop_na = TRUE
  )
#> # A tibble: 5,307 × 5
#>   artist track                   date.entered week   rank
#>   <chr>  <chr>                   <date>       <chr> <dbl>
#> 1 2 Pac  Baby Don't Cry (Keep... 2000-02-26   wk1      87
#> 2 2 Pac  Baby Don't Cry (Keep... 2000-02-26   wk2      82
#> 3 2 Pac  Baby Don't Cry (Keep... 2000-02-26   wk3      72
#> 4 2 Pac  Baby Don't Cry (Keep... 2000-02-26   wk4      77
#> 5 2 Pac  Baby Don't Cry (Keep... 2000-02-26   wk5      87
#> 6 2 Pac  Baby Don't Cry (Keep... 2000-02-26   wk6      94
#> # ℹ 5,301 more rows
```

The number of rows is now much lower, indicating that many rows with `NA`s were dropped.

You might also wonder what happens if a song is in the top 100 for more than 76 weeks? We can't tell from this data, but you might guess that additional columns `wk77`, `wk78`, … would be added to the dataset.

This data is now tidy, but we could make future computation a bit easier by converting values of `week` from character strings to numbers using `mutate()` and `readr::parse_number()`. `parse_number()` is a handy function that will extract the first number from a string, ignoring all other text.

```
billboard_longer <- billboard |>
  pivot_longer(
    cols = starts_with("wk"),
    names_to = "week",
    values_to = "rank",
    values_drop_na = TRUE
  ) |>
  mutate(
    week = parse_number(week)
  )
billboard_longer
#> # A tibble: 5,307 × 5
```

```
#>   artist track                   date.entered  week  rank
#>   <chr>  <chr>                   <date>       <dbl> <dbl>
#> 1 2 Pac  Baby Don't Cry (Keep... 2000-02-26       1    87
#> 2 2 Pac  Baby Don't Cry (Keep... 2000-02-26       2    82
#> 3 2 Pac  Baby Don't Cry (Keep... 2000-02-26       3    72
#> 4 2 Pac  Baby Don't Cry (Keep... 2000-02-26       4    77
#> 5 2 Pac  Baby Don't Cry (Keep... 2000-02-26       5    87
#> 6 2 Pac  Baby Don't Cry (Keep... 2000-02-26       6    94
#> # i 5,301 more rows
```

Now that we have all the week numbers in one variable and all the rank values in another, we're in a good position to visualize how song ranks vary over time. The code is shown below and the result is in Figure 5.2. We can see that very few songs stay in the top 100 for more than 20 weeks.

```
billboard_longer |>
  ggplot(aes(x = week, y = rank, group = track)) +
  geom_line(alpha = 0.25) +
  scale_y_reverse()
```

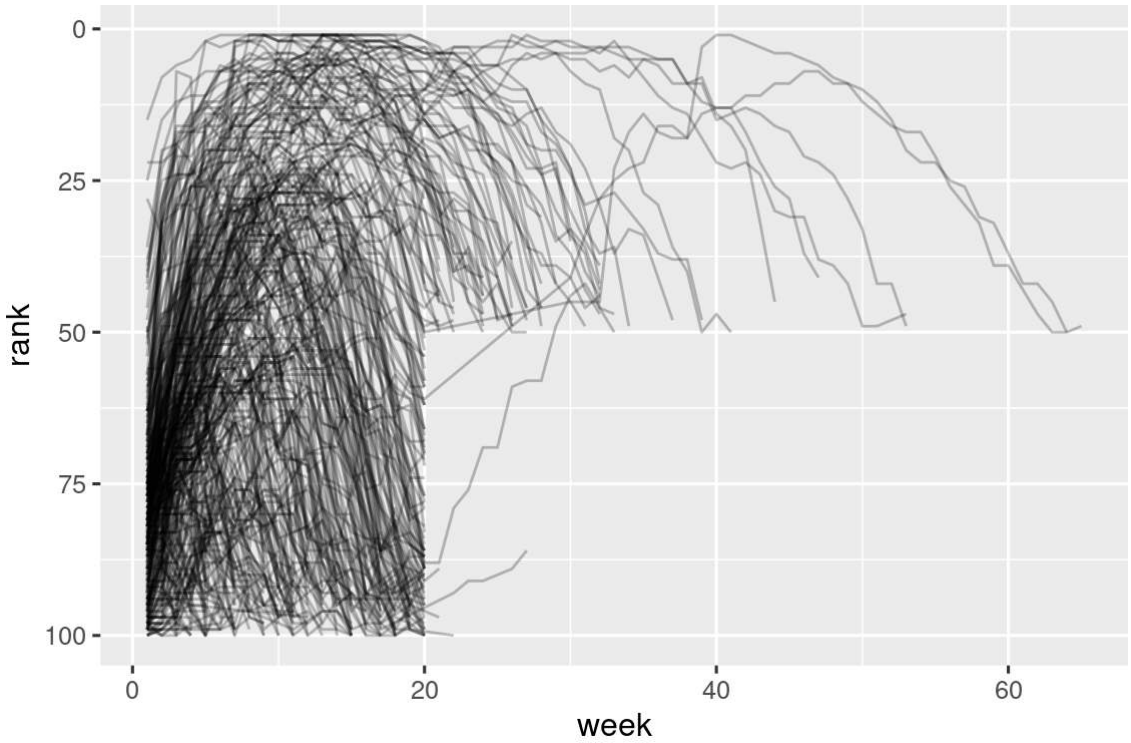


Figure 5.2: A line plot showing how the rank of a song changes over time.

5.3.2 How does pivoting work?

Now that you've seen how we can use pivoting to reshape our data, let's take a little time to gain some intuition about what pivoting does to the data. Let's start with a very simple dataset to make it easier to see

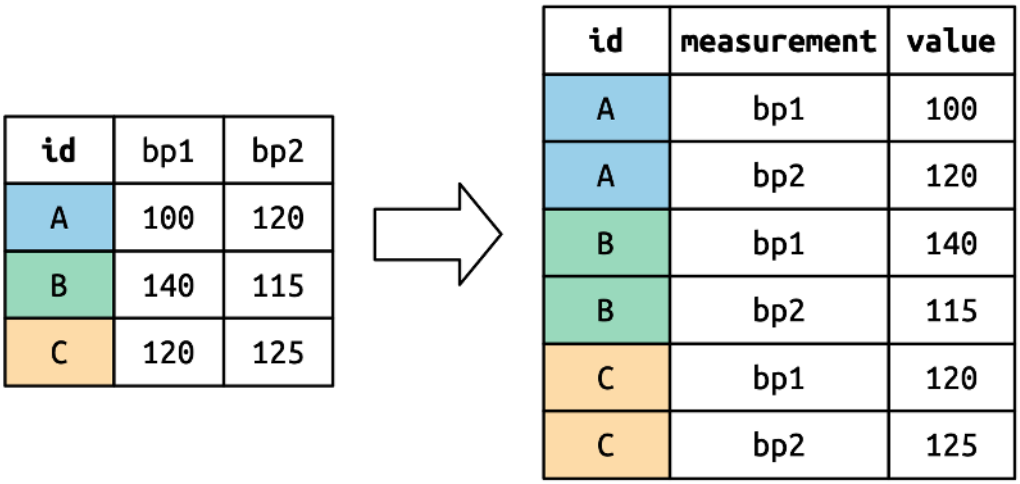
what's happening. Suppose we have three patients with `id`s A, B, and C, and we take two blood pressure measurements on each patient. We'll create the data with `tribble()`, a handy function for constructing small tibbles by hand:

```
df <- tribble(
  ~id,  ~bp1, ~bp2,
   "A",  100,  120,
   "B",  140,  115,
   "C",  120,  125
)
```

We want our new dataset to have three variables: `id` (already exists), `measurement` (the column names), and `value` (the cell values). To achieve this, we need to pivot `df` longer:

```
df |>
  pivot_longer(
    cols = bp1:bp2,
    names_to = "measurement",
    values_to = "value"
  )
#> # A tibble: 6 × 3
#>   id    measurement value
#>   <chr> <chr>       <dbl>
#> 1 A     bp1           100
#> 2 A     bp2           120
#> 3 B     bp1           140
#> 4 B     bp2           115
#> 5 C     bp1           120
#> 6 C     bp2           125
```

How does the reshaping work? It's easier to see if we think about it column by column. As shown in Figure 5.3, the values in a column that was already a variable in the original dataset (`id`) need to be repeated, once for each column that is pivoted.

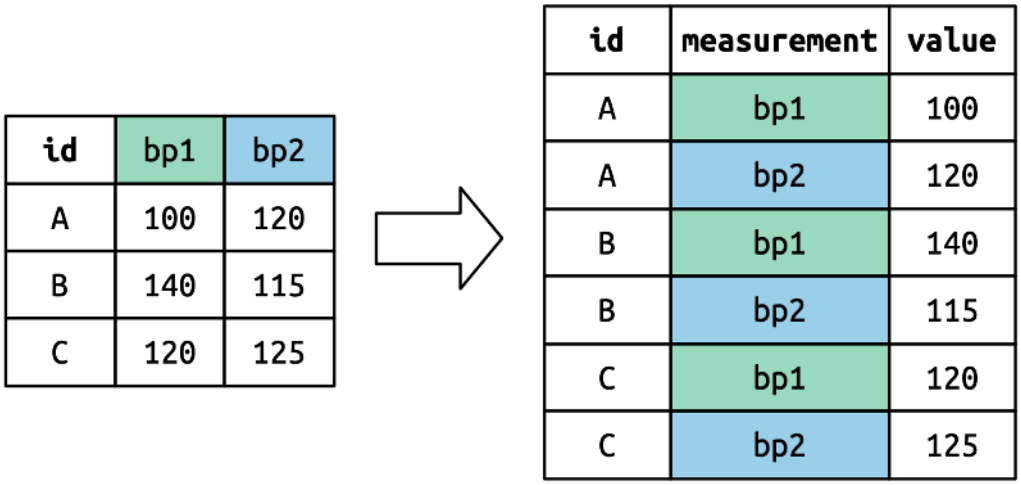


id	bp1	bp2
A	100	120
B	140	115
C	120	125

id	measurement	value
A	bp1	100
A	bp2	120
B	bp1	140
B	bp2	115
C	bp1	120
C	bp2	125

Figure 5.3: Columns that are already variables need to be repeated, once for each column that is pivoted.

The column names become values in a new variable, whose name is defined by `names_to`, as shown in Figure 5.4. They need to be repeated once for each row in the original dataset.

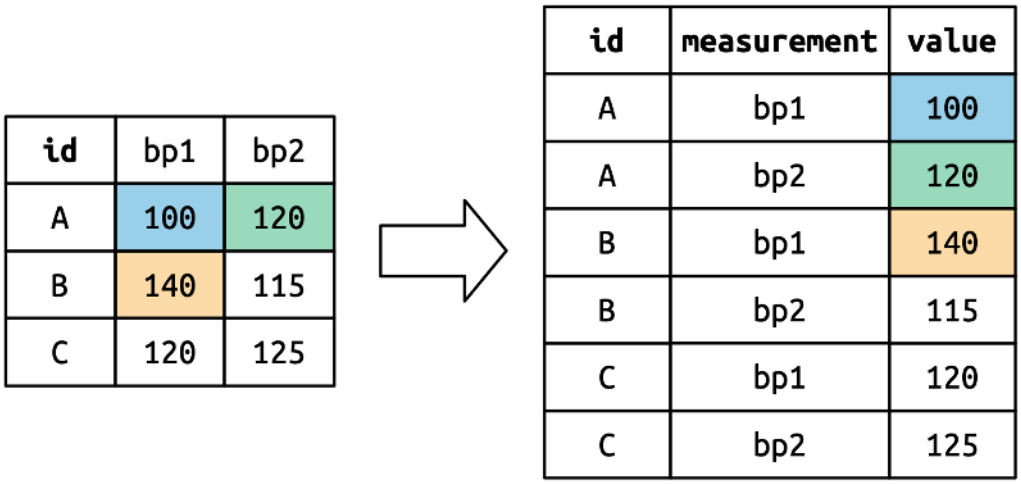


id	bp1	bp2
A	100	120
B	140	115
C	120	125

id	measurement	value
A	bp1	100
A	bp2	120
B	bp1	140
B	bp2	115
C	bp1	120
C	bp2	125

Figure 5.4: The column names of pivoted columns become values in a new column. The values need to be repeated once for each row of the original dataset.

The cell values also become values in a new variable, with a name defined by `values_to`. They are unwound row by row. Figure 5.5 illustrates the process.



id	bp1	bp2
A	100	120
B	140	115
C	120	125

id	measurement	value
A	bp1	100
A	bp2	120
B	bp1	140
B	bp2	115
C	bp1	120
C	bp2	125

Figure 5.5: The number of values is preserved (not repeated), but unwound row-by-row.

5.3.3 Many variables in column names

A more challenging situation occurs when you have multiple pieces of information crammed into the column names, and you would like to store these in separate new variables. For example, take the `who2` dataset, the source of `table1` and friends that you saw above:

```
who2
#> # A tibble: 7,240 × 58
#>   country      year sp_m_014 sp_m_1524 sp_m_2534 sp_m_3544 sp_m_4554
#>   <chr>       <dbl>    <dbl>     <dbl>     <dbl>     <dbl>     <dbl>
#> 1 Afghanistan  1980       NA        NA        NA        NA        NA
#> 2 Afghanistan  1981       NA        NA        NA        NA        NA
#> 3 Afghanistan  1982       NA        NA        NA        NA        NA
#> 4 Afghanistan  1983       NA        NA        NA        NA        NA
#> 5 Afghanistan  1984       NA        NA        NA        NA        NA
#> 6 Afghanistan  1985       NA        NA        NA        NA        NA
#> # ℹ 7,234 more rows
#> # ℹ 51 more variables: sp_m_5564 <dbl>, sp_m_65 <dbl>, sp_f_014 <dbl>, …
```

This dataset, collected by the World Health Organisation, records information about tuberculosis diagnoses. There are two columns that are already variables and are easy to interpret: `country` and `year`. They are followed by 56 columns like `sp_m_014`, `ep_m_4554`, and `rel_m_3544`. If you stare at these columns for long enough, you'll notice there's a pattern. Each column name is made up of three pieces separated by `_`. The first piece, `sp`/`rel`/`ep`, describes the method used for the diagnosis, the second piece, `m`/`f` is the `gender` (coded as a binary variable in this dataset), and the third piece, `014`/`1524`/`2534`/`3544`/`4554`/`5564`/`65` is the `age` range (`014` represents 0-14, for example).

So in this case we have six pieces of information recorded in `who2`: the country and the year (already columns); the method of diagnosis, the gender category, and the age range category (contained in the other column names); and the count of patients in that category (cell values). To organize these six pieces of information in six separate columns, we use `pivot_longer()` with a vector of column names for `names_to` and instructors for splitting the original variable names into pieces for `names_sep` as well as a column name for `values_to`:

```
who2 |>
  pivot_longer(
    cols = !(country:year),
    names_to = c("diagnosis", "gender", "age"),
    names_sep = "_",
    values_to = "count"
  )
#> # A tibble: 405,440 × 6
#>   country      year diagnosis gender age   count
#>   <chr>       <dbl> <chr>     <chr>  <chr> <dbl>
#> 1 Afghanistan  1980 sp        m      014      NA
#> 2 Afghanistan  1980 sp        m      1524     NA
#> 3 Afghanistan  1980 sp        m      2534     NA
#> 4 Afghanistan  1980 sp        m      3544     NA
#> 5 Afghanistan  1980 sp        m      4554     NA
#> 6 Afghanistan  1980 sp        m      5564     NA
#> # ℹ 405,434 more rows
```

An alternative to `names_sep` is `names_pattern`, which you can use to extract variables from more complicated naming scenarios, once you've learned about regular expressions in Chapter 15.

Conceptually, this is only a minor variation on the simpler case you've already seen. Figure 5.6 shows the basic idea: now, instead of the column names pivoting into a single column, they pivot into multiple columns. You can imagine this happening in two steps (first pivoting and then separating) but under the hood it happens in a single step because that's faster.

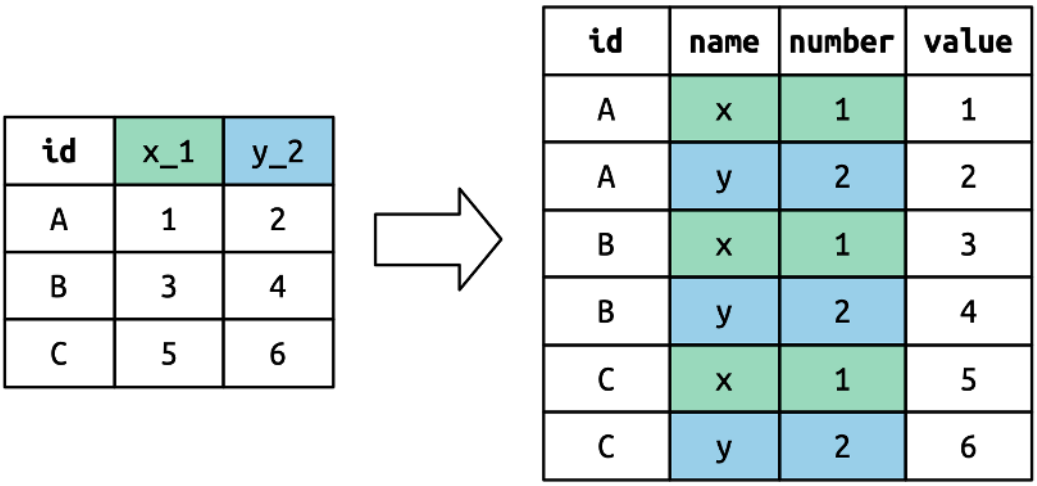


Figure 5.6: Pivoting columns with multiple pieces of information in the names means that each column name now fills in values in multiple output columns.

5.4 Widening data

So far we've used `pivot_longer()` to solve the common class of problems where values have ended up in column names. Next we'll pivot (HA HA) to `pivot_wider()`, which makes datasets **wider** by increasing columns and reducing rows and helps when one observation is spread across multiple rows. This seems to arise less commonly in the wild, but it does seem to crop up a lot when dealing with governmental data.

We'll start by looking at `cms_patient_experience`, a dataset from the Centers of Medicare and Medicaid services that collects data about patient experiences:

```
cms_patient_experience
#> # A tibble: 500 × 5
#>   org_pac_id org_nm                      measure_cd   measure_title
prf_rate
#>   <chr>      <chr>                       <chr>        <chr>
<dbl>
#> 1 0446157747 USC CARE MEDICAL GROUP INC CAHPS_GRP_1  CAHPS for MIPS…
63
#> 2 0446157747 USC CARE MEDICAL GROUP INC CAHPS_GRP_2  CAHPS for MIPS…
87
#> 3 0446157747 USC CARE MEDICAL GROUP INC CAHPS_GRP_3  CAHPS for MIPS…
86
#> 4 0446157747 USC CARE MEDICAL GROUP INC CAHPS_GRP_5  CAHPS for MIPS…
57
#> 5 0446157747 USC CARE MEDICAL GROUP INC CAHPS_GRP_8  CAHPS for MIPS…
85
#> 6 0446157747 USC CARE MEDICAL GROUP INC CAHPS_GRP_12 CAHPS for MIPS…
```

```
24
#> # ℹ 494 more rows
```

The core unit being studied is an organization, but each organization is spread across six rows, with one row for each measurement taken in the survey organization. We can see the complete set of values for `measure_cd` and `measure_title` by using `distinct()`:

```
cms_patient_experience |>
  distinct(measure_cd, measure_title)
#> # A tibble: 6 × 2
#>   measure_cd   measure_title
#>   <chr>        <chr>
#> 1 CAHPS_GRP_1  CAHPS for MIPS SSM: Getting Timely Care, Appointments, and
In…
#> 2 CAHPS_GRP_2  CAHPS for MIPS SSM: How Well Providers Communicate
#> 3 CAHPS_GRP_3  CAHPS for MIPS SSM: Patient's Rating of Provider
#> 4 CAHPS_GRP_5  CAHPS for MIPS SSM: Health Promotion and Education
#> 5 CAHPS_GRP_8  CAHPS for MIPS SSM: Courteous and Helpful Office Staff
#> 6 CAHPS_GRP_12 CAHPS for MIPS SSM: Stewardship of Patient Resources
```

Neither of these columns will make particularly great variable names: `measure_cd` doesn't hint at the meaning of the variable and `measure_title` is a long sentence containing spaces. We'll use `measure_cd` as the source for our new column names for now, but in a real analysis you might want to create your own variable names that are both short and meaningful.

`pivot_wider()` has the opposite interface to `pivot_longer()`: instead of choosing new column names, we need to provide the existing columns that define the values (`values_from`) and the column name (`names_from`):

```
cms_patient_experience |>
  pivot_wider(
    names_from = measure_cd,
    values_from = prf_rate
  )
#> # A tibble: 500 × 9
#>   org_pac_id org_nm                   measure_title   CAHPS_GRP_1
CAHPS_GRP_2
#>   <chr>      <chr>                    <chr>                 <dbl>
<dbl>
#> 1 0446157747 USC CARE MEDICAL GROUP … CAHPS for MIPS…          63
NA
#> 2 0446157747 USC CARE MEDICAL GROUP … CAHPS for MIPS…          NA
```

```
87
#> 3 0446157747 USC CARE MEDICAL GROUP … CAHPS for MIPS…          NA
NA
#> 4 0446157747 USC CARE MEDICAL GROUP … CAHPS for MIPS…          NA
NA
#> 5 0446157747 USC CARE MEDICAL GROUP … CAHPS for MIPS…          NA
NA
#> 6 0446157747 USC CARE MEDICAL GROUP … CAHPS for MIPS…          NA
NA
#> # ℹ 494 more rows
#> # ℹ 4 more variables: CAHPS_GRP_3 <dbl>, CAHPS_GRP_5 <dbl>, …
```

The output doesn't look quite right; we still seem to have multiple rows for each organization. That's because, we also need to tell `pivot_wider()` which column or columns have values that uniquely identify each row; in this case those are the variables starting with `"org"`:

```
cms_patient_experience |>
  pivot_wider(
    id_cols = starts_with("org"),
    names_from = measure_cd,
    values_from = prf_rate
  )
#> # A tibble: 95 × 8
#>   org_pac_id org_nm           CAHPS_GRP_1 CAHPS_GRP_2 CAHPS_GRP_3
CAHPS_GRP_5
#>   <chr>      <chr>                  <dbl>       <dbl>       <dbl>
<dbl>
#> 1 0446157747 USC CARE MEDICA…          63          87          86
57
#> 2 0446162697 ASSOCIATION OF …          59          85          83
63
#> 3 0547164295 BEAVER MEDICAL …          49          NA          75
44
#> 4 0749333730 CAPE PHYSICIANS…          67          84          85
65
#> 5 0840104360 ALLIANCE PHYSIC…          66          87          87
64
#> 6 0840109864 REX HOSPITAL INC          73          87          84
67
#> # ℹ 89 more rows
#> # ℹ 2 more variables: CAHPS_GRP_8 <dbl>, CAHPS_GRP_12 <dbl>
```

This gives us the output that we're looking for.

5.4.1 How does `pivot_wider()` work?

To understand how `pivot_wider()` works, let's again start with a very simple dataset. This time we have two patients with `id`s A and B, we have three blood pressure measurements on patient A and two on patient B:

```
df <- tribble(
  ~id, ~measurement, ~value,
  "A",        "bp1",    100,
  "B",        "bp1",    140,
  "B",        "bp2",    115,
  "A",        "bp2",    120,
  "A",        "bp3",    105
)
```

We'll take the values from the `value` column and the names from the `measurement` column:

```
df |>
  pivot_wider(
    names_from = measurement,
    values_from = value
  )
#> # A tibble: 2 × 4
#>   id      bp1   bp2   bp3
#>   <chr> <dbl> <dbl> <dbl>
#> 1 A       100   120   105
#> 2 B       140   115    NA
```

To begin the process `pivot_wider()` needs to first figure out what will go in the rows and columns. The new column names will be the unique values of `measurement`.

```
df |>
  distinct(measurement) |>
  pull()
#> [1] "bp1" "bp2" "bp3"
```

By default, the rows in the output are determined by all the variables that aren't going into the new names or values. These are called the `id_cols`. Here there is only one column, but in general there can be any number.

```
df |>
  select(-measurement, -value) |>
  distinct()
#> # A tibble: 2 × 1
#>   id
#>   <chr>
#> 1 A
#> 2 B
```

`pivot_wider()` then combines these results to generate an empty data frame:

```
df |>
  select(-measurement, -value) |>
  distinct() |>
  mutate(x = NA, y = NA, z = NA)
#> # A tibble: 2 × 4
#>   id    x     y     z
#>   <chr> <lgl> <lgl> <lgl>
#> 1 A     NA    NA    NA
#> 2 B     NA    NA    NA
```

It then fills in all the missing values using the data in the input. In this case, not every cell in the output has a corresponding value in the input as there's no third blood pressure measurement for patient B, so that cell remains missing. We'll come back to this idea that `pivot_wider()` can "make" missing values in Chapter 18.

You might also wonder what happens if there are multiple rows in the input that correspond to one cell in the output. The example below has two rows that correspond to `id` "A" and `measurement` "bp1":

```
df <- tribble(
  ~id, ~measurement, ~value,
  "A",        "bp1",    100,
  "A",        "bp1",    102,
  "A",        "bp2",    120,
  "B",        "bp1",    140,
  "B",        "bp2",    115
)
```

If we attempt to pivot this we get an output that contains list-columns, which you'll learn more about in Chapter 23:

```
df |>
  pivot_wider(
    names_from = measurement,
    values_from = value
  )
#> Warning: Values from `value` are not uniquely identified; output will
contain
#> list-cols.
#> • Use `values_fn = list` to suppress this warning.
#> • Use `values_fn = {summary_fun}` to summarise duplicates.
#> • Use the following dplyr code to identify duplicates.
#>   {data} |>
#>   dplyr::summarise(n = dplyr::n(), .by = c(id, measurement)) |>
#>   dplyr::filter(n > 1L)
#> # A tibble: 2 × 3
#>   id    bp1       bp2
#>   <chr> <list>    <list>
#> 1 A     <dbl [2]> <dbl [1]>
#> 2 B     <dbl [1]> <dbl [1]>
```

Since you don't know how to work with this sort of data yet, you'll want to follow the hint in the warning to figure out where the problem is:

```
df |>
  group_by(id, measurement) |>
  summarize(n = n(), .groups = "drop") |>
  filter(n > 1)
#> # A tibble: 1 × 3
#>   id    measurement     n
#>   <chr> <chr>       <int>
#> 1 A     bp1             2
```

It's then up to you to figure out what's gone wrong with your data and either repair the underlying damage or use your grouping and summarizing skills to ensure that each combination of row and column values only has a single row.

5.5 Summary

In this chapter you learned about tidy data: data that has variables in columns and observations in rows. Tidy data makes working in the tidyverse easier, because it's a consistent structure understood by most functions, the main challenge is transforming the data from whatever structure you receive it in to a tidy format. To that end, you learned about `pivot_longer()` and `pivot_wider()` which allow you to tidy up many untidy

datasets. The examples we presented here are a selection of those from `vignette("pivot", package = "tidyr")`, so if you encounter a problem that this chapter doesn't help you with, that vignette is a good place to try next.

Another challenge is that, for a given dataset, it can be impossible to label the longer or the wider version as the "tidy" one. This is partly a reflection of our definition of tidy data, where we said tidy data has one variable in each column, but we didn't actually define what a variable is (and it's surprisingly hard to do so). It's totally fine to be pragmatic and to say a variable is whatever makes your analysis easiest. So if you're stuck figuring out how to do some computation, consider switching up the organisation of your data; don't be afraid to untidy, transform, and re-tidy as needed!

If you enjoyed this chapter and want to learn more about the underlying theory, you can learn more about the history and theoretical underpinnings in the Tidy Data paper published in the Journal of Statistical Software.

Now that you're writing a substantial amount of R code, it's time to learn more about organizing your code into files and directories. In the next chapter, you'll learn all about the advantages of scripts and projects, and some of the many tools that they provide to make your life easier.

6 Workflow: scripts and projects

This chapter will introduce you to two essential tools for organizing your code: scripts and projects.

6.1 Scripts

So far, you have used the console to run code. That's a great place to start, but you'll find it gets cramped pretty quickly as you create more complex ggplot2 graphics and longer dplyr pipelines. To give yourself more room to work, use the script editor. Open it up by clicking the File menu, selecting New File, then R script, or using the keyboard shortcut Cmd/Ctrl + Shift + N. Now you'll see four panes, as in Figure 6.1. The script editor is a great place to experiment with your code. When you want to change something, you don't have to re-type the whole thing, you can just edit the script and re-run it. And once you have written code that works and does what you want, you can save it as a script file to easily return to later.

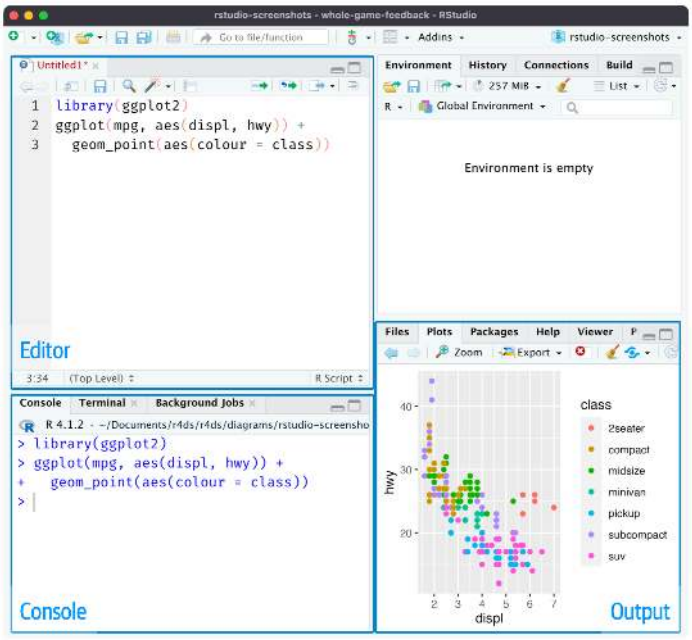


Figure 6.1: Opening the script editor adds a new pane at the top-left of the IDE.

6.1.1 Running code

The script editor is an excellent place for building complex ggplot2 plots or long sequences of dplyr manipulations. The key to using the script editor effectively is to memorize one of the most important keyboard shortcuts: Cmd/Ctrl + Enter. This executes the current R expression in the console. For example, take the code below.

```
library(dplyr)
library(nycflights13)

not_cancelled <- flights |>
  filter(!is.na(dep_delay)█, !is.na(arr_delay))

not_cancelled |>
  group_by(year, month, day) |>
  summarize(mean = mean(dep_delay))
```

If your cursor is at █, pressing Cmd/Ctrl + Enter will run the complete command that generates `not_cancelled`. It will also move the cursor to the following statement (beginning with `not_cancelled |>`). That makes it easy to step through your complete script by repeatedly pressing Cmd/Ctrl + Enter.

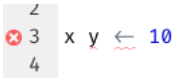
Instead of running your code expression-by-expression, you can also execute the complete script in one step with Cmd/Ctrl + Shift + S. Doing this regularly is a great way to ensure that you've captured all the important parts of your code in the script.

We recommend you always start your script with the packages you need. That way, if you share your code with others, they can easily see which packages they need to install. Note, however, that you should never include `install.packages()` in a script you share. It's inconsiderate to hand off a script that will change something on their computer if they're not being careful!

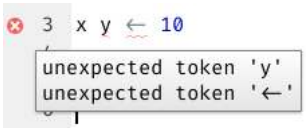
When working through future chapters, we highly recommend starting in the script editor and practicing your keyboard shortcuts. Over time, sending code to the console in this way will become so natural that you won't even think about it.

6.1.2 RStudio diagnostics

In the script editor, RStudio will highlight syntax errors with a red squiggly line and a cross in the sidebar:



Hover over the cross to see what the problem is:



RStudio will also let you know about potential problems:



6.1.3 Saving and naming

RStudio automatically saves the contents of the script editor when you quit, and automatically reloads it when you re-open. Nevertheless, it's a good idea to avoid Untitled1, Untitled2, Untitled3, and so on and instead save your scripts and to give them informative names.

It might be tempting to name your files `code.R` or `myscript.R`, but you should think a bit harder before choosing a name for your file. Three important principles for file naming are as follows:

1. File names should be **machine** readable: avoid spaces, symbols, and special characters. Don't rely on case sensitivity to distinguish files.
2. File names should be **human** readable: use file names to describe what's in the file.
3. File names should play well with default ordering: start file names with numbers so that alphabetical sorting puts them in the order they get used.

For example, suppose you have the following files in a project folder.

```
alternative model.R
code for exploratory analysis.r
finalreport.qmd
FinalReport.qmd
fig 1.png
Figure_02.png
model_first_try.R
run-first.r
temp.txt
```

There are a variety of problems here: it's hard to find which file to run first, file names contain spaces, there are two files with the same name but different capitalization (`finalreport` vs. `FinalReport`[1]), and some names don't describe their contents (`run-first` and `temp`).

Here's a better way of naming and organizing the same set of files:

```
01-load-data.R
02-exploratory-analysis.R
03-model-approach-1.R
04-model-approach-2.R
fig-01.png
fig-02.png
report-2022-03-20.qmd
report-2022-04-02.qmd
report-draft-notes.txt
```

Numbering the key scripts make it obvious in which order to run them and a consistent naming scheme makes it easier to see what varies. Additionally, the figures are labelled similarly, the reports are distinguished by dates included in the file names, and `temp` is renamed to `report-draft-notes` to better describe its contents. If you have a lot of files in a directory, taking organization one step further and placing different types of files (scripts, figures, etc.) in different directories is recommended.

6.2 Projects

One day, you will need to quit R, go do something else, and return to your analysis later. One day, you will be working on multiple analyses simultaneously and you want to keep them separate. One day, you will need to bring data from the outside world into R and send numerical results and figures from R back out into the world.

To handle these real life situations, you need to make two decisions:

1. What is the source of truth? What will you save as your lasting record of what happened?

2. Where does your analysis live?

6.2.1 What is the source of truth?

As a beginner, it's okay to rely on your current Environment to contain all the objects you have created throughout your analysis. However, to make it easier to work on larger projects or collaborate with others, your source of truth should be the R scripts. With your R scripts (and your data files), you can recreate the environment. With only your environment, it's much harder to recreate your R scripts: you'll either have to retype a lot of code from memory (inevitably making mistakes along the way) or you'll have to carefully mine your R history.

To help keep your R scripts as the source of truth for your analysis, we highly recommend that you instruct RStudio not to preserve your workspace between sessions. You can do this either by running `usethis::use_blank_slate()`[2] or by mimicking the options shown in Figure 6.2. This will cause you some short-term pain, because now when you restart RStudio, it will no longer remember the code that you ran last time nor will the objects you created or the datasets you read be available to use. But this short-term pain saves you long-term agony because it forces you to capture all important procedures in your code. There's nothing worse than discovering three months after the fact that you've only stored the results of an important calculation in your environment, not the calculation itself in your code.



Figure 6.2: Copy these options in your RStudio options to always start your RStudio session with a clean slate.

There is a great pair of keyboard shortcuts that will work together to make sure you've captured the important parts of your code in the editor:

1. Press Cmd/Ctrl + Shift + 0/F10 to restart R.
2. Press Cmd/Ctrl + Shift + S to re-run the current script.

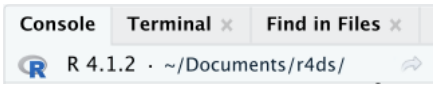
We collectively use this pattern hundreds of times a week.

Alternatively, if you don't use keyboard shortcuts, you can go to Session > Restart R and then highlight and re-run your current script.

If you're using RStudio server, your R session is never restarted by default. When you close your RStudio server tab, it might feel like you're closing R, but the server actually keeps it running in the background. The next time you return, you'll be in exactly the same place you left. This makes it even more important to regularly restart R so that you're starting with a clean slate.

6.2.2 Where does your analysis live?

R has a powerful notion of the **working directory**. This is where R looks for files that you ask it to load, and where it will put any files that you ask it to save. RStudio shows your current working directory at the top of the console:



And you can print this out in R code by running `getwd()`:

```
getwd()
#> [1] "/Users/hadley/Documents/r4ds"
```

In this R session, the current working directory (think of it as "home") is in hadley's Documents folder, in a subfolder called r4ds. This code will return a different result when you run it, because your computer has a different directory structure than Hadley's!

As a beginning R user, it's OK to let your working directory be your home directory, documents directory, or any other weird directory on your computer. But you're more than a handful of chapters into this book, and you're no longer a beginner. Very soon now you should evolve to organizing your projects into directories and, when working on a project, set R's working directory to the associated directory.

You can set the working directory from within R but **we do not recommend it**:

```
setwd("/path/to/my/CoolProject")
```

There's a better way; a way that also puts you on the path to managing your R work like an expert. That way is the **RStudio project**.

6.2.3 RStudio projects

Keeping all the files associated with a given project (input data, R scripts, analytical results, and figures) together in one directory is such a wise and common practice that RStudio has built-in support for this via **projects**. Let's make a project for you to use while you're working through the rest of this book. Click File > New Project, then follow the steps shown in Figure 6.3.

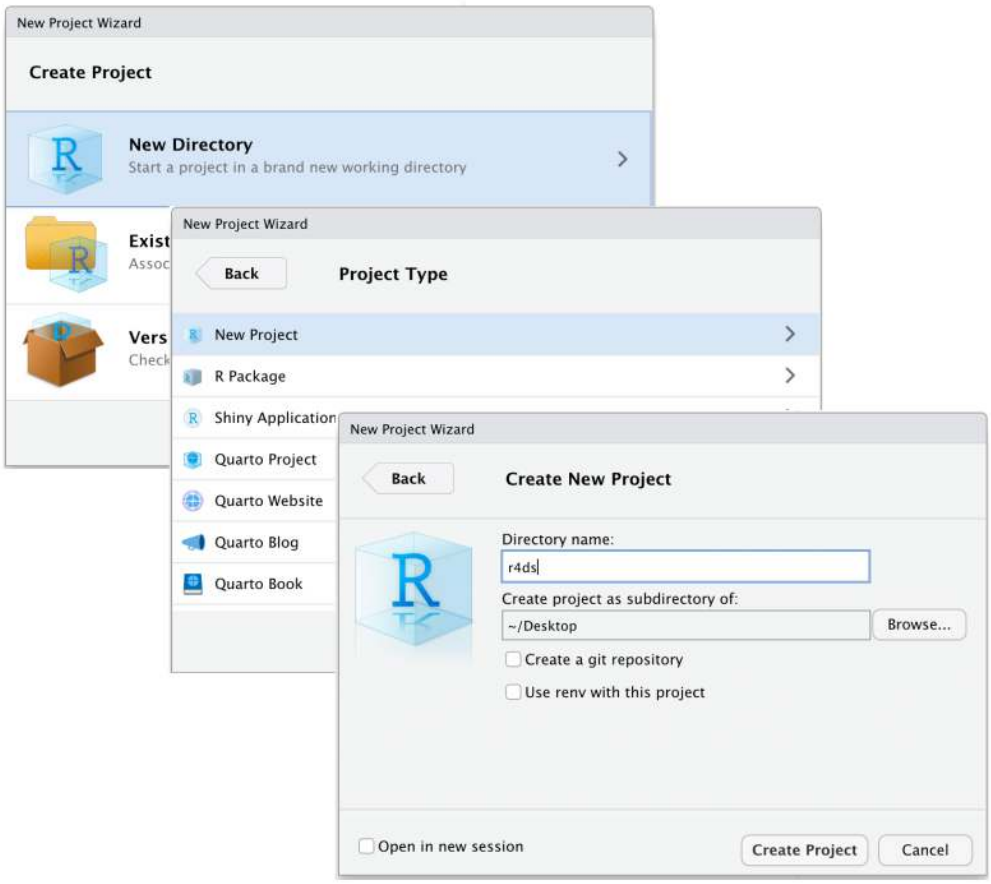


Figure 6.3: To create new project: (top) first click New Directory, then (middle) click New Project, then (bottom) fill in the directory (project) its home and click Create Project.

Call your project `r4ds` and think carefully about which subdirectory you put the project in. If you don't store it somewhere sensible, it will be hard to find it in the future!

Once this process is complete, you'll get a new RStudio project just for this book. Check that the "home" of your project is the current working directory:

```
getwd()
#> [1] /Users/hadley/Documents/r4ds
```

Now enter the following commands in the script editor, and save the file, calling it "diamonds.R". Then, create a new folder called "data". You can do this by clicking on the "New Folder" button in the Files pane in RStudio. Finally, run the complete script which will save a PNG and CSV file into your project directory. Don't worry about the details, you'll learn them later in the book.

Quit RStudio. Inspect the folder associated with your project — notice the `.Rproj` file. Double-click that file to re-open the project. Notice you get back to where you left off: it's the same working directory and command history, and all the files you were working on are still open. Because you followed our instructions above, you will, however, have a completely fresh environment, guaranteeing that you're starting with a clean slate.

In your favorite OS-specific way, search your computer for `diamonds.png` and you will find the PNG (no surprise) but *also the script that created it* (`diamonds.R`). This is a huge win! One day, you will want to remake a figure or just understand where it came from. If you rigorously save figures to files **with R code** and never with the mouse or the clipboard, you will be able to reproduce old work with ease!

6.2.4 Relative and absolute paths

Once you're inside a project, you should only ever use relative paths not absolute paths. What's the difference? A relative path is relative to the working directory, i.e. the project's home. When Hadley wrote `data/diamonds.csv` above it was a shortcut for `/Users/hadley/Documents/r4ds/data/diamonds.csv`. But importantly, if Mine ran this code on her computer, it would point to `/Users/Mine/Documents/r4ds/data/diamonds.csv`. This is why relative paths are important: they'll work regardless of where the R project folder ends up.

Absolute paths point to the same place regardless of your working directory. They look a little different depending on your operating system. On Windows they start with a drive letter (e.g., `C:`) or two backslashes (e.g., `\\servername`) and on Mac/Linux they start with a slash "/" (e.g., `/users/hadley`). You should **never** use absolute paths in your scripts, because they hinder sharing: no one else will have exactly the same directory configuration as you.

There's another important difference between operating systems: how you separate the components of the path. Mac and Linux uses slashes (e.g., `data/diamonds.csv`) and Windows uses backslashes (e.g., `data\diamonds.csv`). R can work with either type (no matter what platform you're currently using), but unfortunately, backslashes mean something special to R, and to get a single backslash in the path, you need to type two backslashes! That makes life frustrating, so we recommend always using the Linux/Mac style with forward slashes.

6.3 Exercises

1. Go to the RStudio Tips Twitter account, https://twitter.com/rstudiotips and find one tip that looks interesting. Practice using it!

2. What other common mistakes will RStudio diagnostics report? Read https://support.posit.co/hc/en-us/articles/205753617-Code-Diagnostics to find out.

6.4 Summary

In this chapter, you've learned how to organize your R code in scripts (files) and projects (directories). Much like code style, this may feel like busywork at first. But as you accumulate more code across multiple projects, you'll learn to appreciate how a little up front organisation can save you a bunch of time down the road.

In summary, scripts and projects give you a solid workflow that will serve you well in the future:

- Create one RStudio project for each data analysis project.
- Save your scripts (with informative names) in the project, edit them, run them in bits or as a whole. Restart R frequently to make sure you've captured everything in your scripts.
- Only ever use relative paths, not absolute paths.

Then everything you need is in one place and cleanly separated from all the other projects that you are working on.

So far, we've worked with datasets bundled inside of R packages. This makes it easier to get some practice on pre-prepared data, but obviously your data won't be available in this way. So in the next chapter, you're going to learn how load data from disk into your R session using the readr package.

7 Data import

7.1 Introduction

Working with data provided by R packages is a great way to learn data science tools, but you want to apply what you've learned to your own data at some point. In this chapter, you'll learn the basics of reading data files into R.

Specifically, this chapter will focus on reading plain-text rectangular files. We'll start with practical advice for handling features like column names, types, and missing data. You will then learn about reading data from multiple files at once and writing data from R to a file. Finally, you'll learn how to handcraft data frames in R.

7.1.1 Prerequisites

In this chapter, you'll learn how to load flat files in R with the **readr** package, which is part of the core tidyverse.

```
library(tidyverse)
```

7.2 Reading data from a file

To begin, we'll focus on the most common rectangular data file type: CSV, which is short for comma-separated values. Here is what a simple CSV file looks like. The first row, commonly called the header row, gives the column names, and the following six rows provide the data. The columns are separated, aka delimited, by commas.

```
Student ID,Full Name,favourite.food,mealPlan,AGE
1,Sunil Huffmann,Strawberry yoghurt,Lunch only,4
2,Barclay Lynn,French fries,Lunch only,5
3,Jayendra Lyne,N/A,Breakfast and lunch,7
4,Leon Rossini,Anchovies,Lunch only,
5,Chidiegwu Dunkel,Pizza,Breakfast and lunch,five
6,Güvenç Attila,Ice cream,Lunch only,6
```

Table 7.1 shows a representation of the same data as a table.

Table 7.1: Data from the students.csv file as a table.

Student ID	Full Name	favourite.food	mealPlan	AGE
1	Sunil Huffmann	Strawberry yoghurt	Lunch only	4

Student ID	Full Name	favourite.food	mealPlan	AGE
2	Barclay Lynn	French fries	Lunch only	5
3	Jayendra Lyne	N/A	Breakfast and lunch	7
4	Leon Rossini	Anchovies	Lunch only	NA
5	Chidiegwu Dunkel	Pizza	Breakfast and lunch	five
6	Güvenç Attila	Ice cream	Lunch only	6

We can read this file into R using `read_csv()`. The first argument is the most important: the path to the file. You can think about the path as the address of the file: the file is called `students.csv` and that it lives in the `data` folder.

```
students <- read_csv("data/students.csv")
#> Rows: 6 Columns: 5
#> ── Column specification
────────────────────────────────────────────────────────
#> Delimiter: ","
#> chr (4): Full Name, favourite.food, mealPlan, AGE
#> dbl (1): Student ID
#>
#> ℹ Use `spec()` to retrieve the full column specification for this data.
#> ℹ Specify the column types or set `show_col_types = FALSE` to quiet this
message.
```

The code above will work if you have the `students.csv` file in a `data` folder in your project. You can download the `students.csv` file from https://pos.it/r4ds-students-csv or you can read it directly from that URL with:

```
students <- read_csv("https://pos.it/r4ds-students-csv")
```

When you run `read_csv()`, it prints out a message telling you the number of rows and columns of data, the delimiter that was used, and the column specifications (names of columns organized by the type of data the column contains). It also prints out some information about retrieving the full column specification and how to quiet this message. This message is an integral part of readr, and we'll return to it in Section 7.3.

7.2.1 Practical advice

Once you read data in, the first step usually involves transforming it in some way to make it easier to work with in the rest of your analysis. Let's take another look at the `students` data with that in mind.

```
students
#> # A tibble: 6 × 5
#>   `Student ID` `Full Name`      favourite.food     mealPlan           AGE
```

```
#>          <dbl> <chr>            <chr>              <chr>
<chr>
#> 1            1 Sunil Huffmann   Strawberry yoghurt Lunch only          4
#> 2            2 Barclay Lynn     French fries       Lunch only          5
#> 3            3 Jayendra Lyne    N/A                Breakfast and lunch 7
#> 4            4 Leon Rossini     Anchovies          Lunch only
<NA>
#> 5            5 Chidiegwu Dunkel Pizza              Breakfast and lunch
five
#> 6            6 Güvenç Attila    Ice cream          Lunch only          6
```

In the `favourite.food` column, there are a bunch of food items, and then the character string `N/A`, which should have been a real `NA` that R will recognize as "not available". This is something we can address using the `na` argument. By default, `read_csv()` only recognizes empty strings (`""`) in this dataset as `NA`s, we want it to also recognize the character string `"N/A"`.

```
students <- read_csv("data/students.csv", na = c("N/A", ""))

students
#> # A tibble: 6 × 5
#>   `Student ID` `Full Name`      favourite.food     mealPlan            AGE
#>          <dbl> <chr>            <chr>              <chr>
<chr>
#> 1            1 Sunil Huffmann   Strawberry yoghurt Lunch only          4
#> 2            2 Barclay Lynn     French fries       Lunch only          5
#> 3            3 Jayendra Lyne    <NA>               Breakfast and lunch 7
#> 4            4 Leon Rossini     Anchovies          Lunch only
<NA>
#> 5            5 Chidiegwu Dunkel Pizza              Breakfast and lunch
five
#> 6            6 Güvenç Attila    Ice cream          Lunch only          6
```

You might also notice that the `Student ID` and `Full Name` columns are surrounded by backticks. That's because they contain spaces, breaking R's usual rules for variable names; they're **non-syntactic** names. To refer to these variables, you need to surround them with backticks, `` ` ``:

```
students |>
  rename(
    student_id = `Student ID`,
    full_name = `Full Name`
  )
```

```
#> # A tibble: 6 × 5
#>   student_id full_name        favourite.food     mealPlan            AGE
#>        <dbl> <chr>            <chr>              <chr>               <chr>
#> 1          1 Sunil Huffmann   Strawberry yoghurt Lunch only          4
#> 2          2 Barclay Lynn     French fries       Lunch only          5
#> 3          3 Jayendra Lyne    <NA>               Breakfast and lunch 7
#> 4          4 Leon Rossini     Anchovies          Lunch only          <NA>
#> 5          5 Chidiegwu Dunkel Pizza              Breakfast and lunch five
#> 6          6 Güvenç Attila    Ice cream          Lunch only          6
```

An alternative approach is to use `janitor::clean_names()` to use some heuristics to turn them all into snake case at once[1].

```
students |> janitor::clean_names()
#> # A tibble: 6 × 5
#>   student_id full_name        favourite_food     meal_plan           age
#>        <dbl> <chr>            <chr>              <chr>               <chr>
#> 1          1 Sunil Huffmann   Strawberry yoghurt Lunch only          4
#> 2          2 Barclay Lynn     French fries       Lunch only          5
#> 3          3 Jayendra Lyne    <NA>               Breakfast and lunch 7
#> 4          4 Leon Rossini     Anchovies          Lunch only          <NA>
#> 5          5 Chidiegwu Dunkel Pizza              Breakfast and lunch five
#> 6          6 Güvenç Attila    Ice cream          Lunch only          6
```

Another common task after reading in data is to consider variable types. For example, `meal_plan` is a categorical variable with a known set of possible values, which in R should be represented as a factor:

```
students |>
  janitor::clean_names() |>
  mutate(meal_plan = factor(meal_plan))
#> # A tibble: 6 × 5
#>   student_id full_name        favourite_food     meal_plan           age
#>        <dbl> <chr>            <chr>              <fct>               <chr>
#> 1          1 Sunil Huffmann   Strawberry yoghurt Lunch only          4
#> 2          2 Barclay Lynn     French fries       Lunch only          5
#> 3          3 Jayendra Lyne    <NA>               Breakfast and lunch 7
#> 4          4 Leon Rossini     Anchovies          Lunch only          <NA>
#> 5          5 Chidiegwu Dunkel Pizza              Breakfast and lunch five
#> 6          6 Güvenç Attila    Ice cream          Lunch only          6
```

Note that the values in the `meal_plan` variable have stayed the same, but the type of variable denoted underneath the variable name has changed from character (`<chr>`) to factor (`<fct>`). You'll learn more about factors in Chapter 16.

Before you analyze these data, you'll probably want to fix the `age` and `id` columns. Currently, `age` is a character variable because one of the observations is typed out as `five` instead of a numeric `5`. We discuss the details of fixing this issue in Chapter 20.

```
students <- students |>
  janitor::clean_names() |>
  mutate(
    meal_plan = factor(meal_plan),
    age = parse_number(if_else(age == "five", "5", age))
  )

students
#> # A tibble: 6 × 5
#>   student_id full_name        favourite_food     meal_plan             age
#>        <dbl> <chr>            <chr>              <fct>                 <dbl>
#> 1          1 Sunil Huffmann   Strawberry yoghurt Lunch only              4
#> 2          2 Barclay Lynn     French fries       Lunch only              5
#> 3          3 Jayendra Lyne    <NA>               Breakfast and lunch     7
#> 4          4 Leon Rossini     Anchovies          Lunch only             NA
#> 5          5 Chidiegwu Dunkel Pizza              Breakfast and lunch     5
#> 6          6 Güvenç Attila    Ice cream          Lunch only              6
```

A new function here is `if_else()`, which has three arguments. The first argument `test` should be a logical vector. The result will contain the value of the second argument, `yes`, when `test` is `TRUE`, and the value of the third argument, `no`, when it is `FALSE`. Here we're saying if `age` is the character string `"five"`, make it `"5"`, and if not leave it as `age`. You will learn more about `if_else()` and logical vectors in Chapter 12.

7.2.2 Other arguments

There are a couple of other important arguments that we need to mention, and they'll be easier to demonstrate if we first show you a handy trick: `read_csv()` can read text strings that you've created and formatted like a CSV file:

```
read_csv(
  "a,b,c
  1,2,3
  4,5,6"
)
```

```
#> # A tibble: 2 × 3
#>       a     b     c
#>   <dbl> <dbl> <dbl>
#> 1     1     2     3
#> 2     4     5     6
```

Usually, `read_csv()` uses the first line of the data for the column names, which is a very common convention. But it's not uncommon for a few lines of metadata to be included at the top of the file. You can use `skip = n` to skip the first `n` lines or use `comment = "#"` to drop all lines that start with (e.g.) `#`:

```
read_csv(
  "The first line of metadata
  The second line of metadata
  x,y,z
  1,2,3",
  skip = 2
)
#> # A tibble: 1 × 3
#>       x     y     z
#>   <dbl> <dbl> <dbl>
#> 1     1     2     3

read_csv(
  "# A comment I want to skip
  x,y,z
  1,2,3",
  comment = "#"
)
#> # A tibble: 1 × 3
#>       x     y     z
#>   <dbl> <dbl> <dbl>
#> 1     1     2     3
```

In other cases, the data might not have column names. You can use `col_names = FALSE` to tell `read_csv()` not to treat the first row as headings and instead label them sequentially from `X1` to `Xn`:

```
read_csv(
  "1,2,3
  4,5,6",
  col_names = FALSE
)
#> # A tibble: 2 × 3
```

```
#>      X1    X2    X3
#>   <dbl> <dbl> <dbl>
#> 1     1     2     3
#> 2     4     5     6
```

Alternatively, you can pass `col_names` a character vector which will be used as the column names:

```
read_csv(
  "1,2,3
  4,5,6",
  col_names = c("x", "y", "z")
)
#> # A tibble: 2 × 3
#>       x     y     z
#>   <dbl> <dbl> <dbl>
#> 1     1     2     3
#> 2     4     5     6
```

These arguments are all you need to know to read the majority of CSV files that you'll encounter in practice. (For the rest, you'll need to carefully inspect your `.csv` file and read the documentation for `read_csv()`'s many other arguments.)

7.2.3 Other file types

Once you've mastered `read_csv()`, using readr's other functions is straightforward; it's just a matter of knowing which function to reach for:

- `read_csv2()` reads semicolon-separated files. These use `;` instead of `,` to separate fields and are common in countries that use `,` as the decimal marker.
- `read_tsv()` reads tab-delimited files.
- `read_delim()` reads in files with any delimiter, attempting to automatically guess the delimiter if you don't specify it.
- `read_fwf()` reads fixed-width files. You can specify fields by their widths with `fwf_widths()` or by their positions with `fwf_positions()`.
- `read_table()` reads a common variation of fixed-width files where columns are separated by white space.
- `read_log()` reads Apache-style log files.

7.2.4 Exercises

1. What function would you use to read a file where fields were separated with “|”?

2. Apart from `file`, `skip`, and `comment`, what other arguments do `read_csv()` and `read_tsv()` have in common?

3. What are the most important arguments to `read_fwf()`?

4. Sometimes strings in a CSV file contain commas. To prevent them from causing problems, they need to be surrounded by a quoting character, like `"` or `'`. By default, `read_csv()` assumes that the quoting character will be `"`. To read the following text into a data frame, what argument to `read_csv()` do you need to specify?

```
"x,y\n1,'a,b'"
```

5. Identify what is wrong with each of the following inline CSV files. What happens when you run the code?

```
read_csv("a,b\n1,2,3\n4,5,6")
read_csv("a,b,c\n1,2\n1,2,3,4")
read_csv("a,b\n\"1")
read_csv("a,b\n1,2\na,b")
read_csv("a;b\n1;3")
```

6. Practice referring to non-syntactic names in the following data frame by:

    a. Extracting the variable called `1`.
    b. Plotting a scatterplot of `1` vs. `2`.
    c. Creating a new column called `3`, which is `2` divided by `1`.
    d. Renaming the columns to `one`, `two`, and `three`.

```
annoying <- tibble(
  `1` = 1:10,
  `2` = `1` * 2 + rnorm(length(`1`))
)
```

7.3 Controlling column types

A CSV file doesn’t contain any information about the type of each variable (i.e. whether it’s a logical, number, string, etc.), so readr will try to guess the type. This section describes how the guessing process works, how to resolve some common problems that cause it to fail, and, if needed, how to supply the column types yourself. Finally, we’ll mention a few general strategies that are useful if readr is failing catastrophically and you need to get more insight into the structure of your file.

7.3.1 Guessing types

readr uses a heuristic to figure out the column types. For each column, it pulls the values of 1,000[2] rows spaced evenly from the first row to the last, ignoring missing values. It then works through the following questions:

- Does it contain only `F`, `T`, `FALSE`, or `TRUE` (ignoring case)? If so, it's a logical.
- Does it contain only numbers (e.g., `1`, `-4.5`, `5e6`, `Inf`)? If so, it's a number.
- Does it match the ISO8601 standard? If so, it's a date or date-time. (We'll return to date-times in more detail in Section 17.2).
- Otherwise, it must be a string.

You can see that behavior in action in this simple example:

```
read_csv("
  logical,numeric,date,string
  TRUE,1,2021-01-15,abc
  false,4.5,2021-02-15,def
  T,Inf,2021-02-16,ghi
")
#> # A tibble: 3 × 4
#>   logical numeric date       string
#>   <lgl>     <dbl> <date>     <chr>
#> 1 TRUE        1   2021-01-15 abc
#> 2 FALSE       4.5 2021-02-15 def
#> 3 TRUE      Inf   2021-02-16 ghi
```

This heuristic works well if you have a clean dataset, but in real life, you'll encounter a selection of weird and beautiful failures.

7.3.2 Missing values, column types, and problems

The most common way column detection fails is that a column contains unexpected values, and you get a character column instead of a more specific type. One of the most common causes for this is a missing value, recorded using something other than the `NA` that readr expects.

Take this simple 1 column CSV file as an example:

```
simple_csv <- "
  x
  10
  .
```

```
  20
  30"
```

If we read it without any additional arguments, `x` becomes a character column:

```
read_csv(simple_csv)
#> # A tibble: 4 × 1
#>   x
#>   <chr>
#> 1 10
#> 2 .
#> 3 20
#> 4 30
```

In this very small case, you can easily see the missing value `.`. But what happens if you have thousands of rows with only a few missing values represented by `.`s sprinkled among them? One approach is to tell readr that `x` is a numeric column, and then see where it fails. You can do that with the `col_types` argument, which takes a named list where the names match the column names in the CSV file:

```
df <- read_csv(
  simple_csv,
  col_types = list(x = col_double())
)
#> Warning: One or more parsing issues, call `problems()` on your data frame
for
#> details, e.g.:
#>   dat <- vroom(...)
#>   problems(dat)
```

Now `read_csv()` reports that there was a problem, and tells us we can find out more with `problems()`:

```
problems(df)
#> # A tibble: 1 × 5
#>     row   col expected actual file
#>   <int> <int> <chr>    <chr>  <chr>
#> 1     3     1 a double .      /tmp/RtmpNvDeIn/file1d2219643a02
```

This tells us that there was a problem in row 3, col 1 where readr expected a double but got a `.`. That suggests this dataset uses `.` for missing values. So then we set `na = "."`, the automatic guessing succeeds, giving us the numeric column that we want:

```
read_csv(simple_csv, na = ".")
#> # A tibble: 4 × 1
#>       x
#>   <dbl>
#> 1    10
#> 2    NA
#> 3    20
#> 4    30
```

7.3.3 Column types

readr provides a total of nine column types for you to use:

- `col_integer()` reads integers. We seldom distinguish integers and doubles in this book because they're functionally equivalent, but reading integers explicitly can occasionally be useful because they occupy half the memory of doubles.
- `col_character()` reads strings. This can be useful to specify explicitly when you have a column that is a numeric identifier, i.e., long series of digits that identifies an object but doesn't make sense to apply mathematical operations to. Examples include phone numbers, social security numbers, credit card numbers, etc.
- `col_factor()`, `col_date()`, and `col_datetime()` create factors, dates, and date-times respectively; you'll learn more about those when we get to those data types in Chapter 16 and Chapter 17.
- `col_number()` is a permissive numeric parser that will ignore non-numeric components, and is particularly useful for currencies. You'll learn more about it in Chapter 13.
- `col_skip()` skips a column so it's not included in the result, which can be useful for speeding up reading the data if you have a large CSV file and you only want to use some of the columns.

It's also possible to override the default column by switching from `list()` to `cols()` and specifying `.default`:

```
another_csv <- "
x,y,z
1,2,3"

read_csv(
  another_csv,
  col_types = cols(.default = col_character())
)
#> # A tibble: 1 × 3
#>   x     y     z
```

```
#>   <chr> <chr> <chr>
#> 1 1     2     3
```

Another useful helper is `cols_only()` which will read in only the columns you specify:

```
read_csv(
  another_csv,
  col_types = cols_only(x = col_character())
)
#> # A tibble: 1 × 1
#>   x
#>   <chr>
#> 1 1
```

7.4 Reading data from multiple files

Sometimes your data is split across multiple files instead of being contained in a single file. For example, you might have sales data for multiple months, with each month's data in a separate file: `01-sales.csv` for January, `02-sales.csv` for February, and `03-sales.csv` for March. With `read_csv()` you can read these data in at once and stack them on top of each other in a single data frame.

```
sales_files <- c("data/01-sales.csv", "data/02-sales.csv", "data/03-
sales.csv")
read_csv(sales_files, id = "file")
#> # A tibble: 19 × 6
#>   file              month    year brand  item     n
#>   <chr>             <chr>   <dbl> <dbl> <dbl> <dbl>
#> 1 data/01-sales.csv January  2019     1  1234     3
#> 2 data/01-sales.csv January  2019     1  8721     9
#> 3 data/01-sales.csv January  2019     1  1822     2
#> 4 data/01-sales.csv January  2019     2  3333     1
#> 5 data/01-sales.csv January  2019     2  2156     9
#> 6 data/01-sales.csv January  2019     2  3987     6
#> # ℹ 13 more rows
```

Once again, the code above will work if you have the CSV files in a `data` folder in your project. You can download these files from https://pos.it/r4ds-01-sales, https://pos.it/r4ds-02-sales, and https://pos.it/r4ds-03-sales or you can read them directly with:

```
sales_files <- c(
  "https://pos.it/r4ds-01-sales",
  "https://pos.it/r4ds-02-sales",
```

```
  "https://pos.it/r4ds-03-sales"
)
read_csv(sales_files, id = "file")
```

The `id` argument adds a new column called `file` to the resulting data frame that identifies the file the data come from. This is especially helpful in circumstances where the files you're reading in do not have an identifying column that can help you trace the observations back to their original sources.

If you have many files you want to read in, it can get cumbersome to write out their names as a list. Instead, you can use the base `list.files()` function to find the files for you by matching a pattern in the file names. You'll learn more about these patterns in Chapter 15.

```
sales_files <- list.files("data", pattern = "sales\\.csv$", full.names =
TRUE)
sales_files
#> [1] "data/01-sales.csv" "data/02-sales.csv" "data/03-sales.csv"
```

7.5 Writing to a file

readr also comes with two useful functions for writing data back to disk: `write_csv()` and `write_tsv()`. The most important arguments to these functions are `x` (the data frame to save) and `file` (the location to save it). You can also specify how missing values are written with `na`, and if you want to `append` to an existing file.

```
write_csv(students, "students.csv")
```

Now let's read that csv file back in. Note that the variable type information that you just set up is lost when you save to CSV because you're starting over with reading from a plain text file again:

```
students
#> # A tibble: 6 × 5
#>   student_id full_name        favourite_food     meal_plan             age
#>        <dbl> <chr>            <chr>              <fct>                 <dbl>
#> 1          1 Sunil Huffmann   Strawberry yoghurt Lunch only              4
#> 2          2 Barclay Lynn     French fries       Lunch only              5
#> 3          3 Jayendra Lyne    <NA>               Breakfast and lunch     7
#> 4          4 Leon Rossini     Anchovies          Lunch only             NA
#> 5          5 Chidiegwu Dunkel Pizza              Breakfast and lunch     5
#> 6          6 Güvenç Attila    Ice cream          Lunch only              6
write_csv(students, "students-2.csv")
read_csv("students-2.csv")
#> # A tibble: 6 × 5
```

```
#>   student_id full_name        favourite_food     meal_plan              age
#>        <dbl> <chr>            <chr>              <chr>                <dbl>
#> 1          1 Sunil Huffmann   Strawberry yoghurt Lunch only               4
#> 2          2 Barclay Lynn     French fries       Lunch only               5
#> 3          3 Jayendra Lyne    <NA>               Breakfast and lunch      7
#> 4          4 Leon Rossini     Anchovies          Lunch only              NA
#> 5          5 Chidiegwu Dunkel Pizza              Breakfast and lunch      5
#> 6          6 Güvenç Attila    Ice cream          Lunch only               6
```

This makes CSVs a little unreliable for caching interim results—you need to recreate the column specification every time you load in. There are two main alternatives:

1. `write_rds()` and `read_rds()` are uniform wrappers around the base functions `readRDS()` and `saveRDS()`. These store data in R's custom binary format called RDS. This means that when you reload the object, you are loading the *exact same* R object that you stored.

```
write_rds(students, "students.rds")
read_rds("students.rds")
#> # A tibble: 6 × 5
#>   student_id full_name        favourite_food     meal_plan
age
#>        <dbl> <chr>            <chr>              <fct>
<dbl>
#> 1          1 Sunil Huffmann   Strawberry yoghurt Lunch only
4
#> 2          2 Barclay Lynn     French fries       Lunch only
5
#> 3          3 Jayendra Lyne    <NA>               Breakfast and lunch
7
#> 4          4 Leon Rossini     Anchovies          Lunch only
NA
#> 5          5 Chidiegwu Dunkel Pizza              Breakfast and lunch
5
#> 6          6 Güvenç Attila    Ice cream          Lunch only
6
```

2. The arrow package allows you to read and write parquet files, a fast binary file format that can be shared across programming languages. We'll return to arrow in more depth in Chapter 22.

```
library(arrow)
write_parquet(students, "students.parquet")
read_parquet("students.parquet")
```

```
#> # A tibble: 6 × 5
#>   student_id full_name        favourite_food     meal_plan
age
#>        <dbl> <chr>            <chr>              <fct>
<dbl>
#> 1          1 Sunil Huffmann   Strawberry yoghurt Lunch only
4
#> 2          2 Barclay Lynn     French fries       Lunch only
5
#> 3          3 Jayendra Lyne    NA                 Breakfast and lunch
7
#> 4          4 Leon Rossini     Anchovies          Lunch only
NA
#> 5          5 Chidiegwu Dunkel Pizza              Breakfast and lunch
5
#> 6          6 Güvenç Attila    Ice cream          Lunch only
6
```

Parquet tends to be much faster than RDS and is usable outside of R, but does require the arrow package.

7.6 Data entry

Sometimes you'll need to assemble a tibble "by hand" doing a little data entry in your R script. There are two useful functions to help you do this which differ in whether you layout the tibble by columns or by rows. `tibble()` works by column:

```
tibble(
  x = c(1, 2, 5),
  y = c("h", "m", "g"),
  z = c(0.08, 0.83, 0.60)
)
#> # A tibble: 3 × 3
#>       x y         z
#>   <dbl> <chr> <dbl>
#> 1     1 h      0.08
#> 2     2 m      0.83
#> 3     5 g      0.6
```

Laying out the data by column can make it hard to see how the rows are related, so an alternative is `tribble()`, short for **tr**ansposed t**ibble**, which lets you lay out your data row by row. `tribble()` is customized for data entry in code: column headings start with ~ and entries are separated by commas. This makes it possible to lay out small amounts of data in an easy to read form:

```
tribble(
  ~x, ~y, ~z,
  1, "h", 0.08,
  2, "m", 0.83,
  5, "g", 0.60
)
#> # A tibble: 3 × 3
#>       x y         z
#>   <dbl> <chr> <dbl>
#> 1     1 h      0.08
#> 2     2 m      0.83
#> 3     5 g      0.6
```

7.7 Summary

In this chapter, you've learned how to load CSV files with `read_csv()` and to do your own data entry with `tibble()` and `tribble()`. You've learned how csv files work, some of the problems you might encounter, and how to overcome them. We'll come to data import a few times in this book: Chapter 20 from Excel and Google Sheets, Chapter 21 will show you how to load data from databases, Chapter 22 from parquet files, Chapter 23 from JSON, and Chapter 24 from websites.

We're just about at the end of this section of the book, but there's one important last topic to cover: how to get help. So in the next chapter, you'll learn some good places to look for help, how to create a reprex to maximize your chances of getting good help, and some general advice on keeping up with the world of R.

8 Workflow: getting help

This book is not an island; there is no single resource that will allow you to master R. As you begin to apply the techniques described in this book to your own data, you will soon find questions that we do not answer. This section describes a few tips on how to get help and to help you keep learning.

8.1 Google is your friend

If you get stuck, start with Google. Typically adding "R" to a query is enough to restrict it to relevant results: if the search isn't useful, it often means that there aren't any R-specific results available. Additionally, adding package names like "tidyverse" or "ggplot2" will help narrow down the results to code that will feel more familiar to you as well, e.g., "how to make a boxplot in R" vs. "how to make a boxplot in R with ggplot2". Google is particularly useful for error messages. If you get an error message and you have no idea what it means, try googling it! Chances are that someone else has been confused by it in the past, and there will be help somewhere on the web. (If the error message isn't in English, run `Sys.setenv(LANGUAGE = "en")` and re-run the code; you're more likely to find help for English error messages.)

If Google doesn't help, try Stack Overflow. Start by spending a little time searching for an existing answer, including `[R]`, to restrict your search to questions and answers that use R.

8.2 Making a reprex

If your googling doesn't find anything useful, it's a really good idea to prepare a **reprex,** short for minimal **repr**oducible **ex**ample. A good reprex makes it easier for other people to help you, and often you'll figure out the problem yourself in the course of making it. There are two parts to creating a reprex:

- First, you need to make your code reproducible. This means that you need to capture everything, i.e. include any `library()` calls and create all necessary objects. The easiest way to make sure you've done this is using the reprex package.

- Second, you need to make it minimal. Strip away everything that is not directly related to your problem. This usually involves creating a much smaller and simpler R object than the one you're facing in real life or even using built-in data.

That sounds like a lot of work! And it can be, but it has a great payoff:

- 80% of the time, creating an excellent reprex reveals the source of your problem. It's amazing how often the process of writing up a self-contained and minimal example allows you to answer your own question.

- The other 20% of the time, you will have captured the essence of your problem in a way that is easy for others to play with. This substantially improves your chances of getting help!

When creating a reprex by hand, it's easy to accidentally miss something, meaning your code can't be run on someone else's computer. Avoid this problem by using the reprex package, which is installed as part of the tidyverse. Let's say you copy this code onto your clipboard (or, on RStudio Server or Cloud, select it):

```
y <- 1:4
mean(y)
```

Then call `reprex()`, where the default output is formatted for GitHub:

```
reprex::reprex()
```

A nicely rendered HTML preview will display in RStudio's Viewer (if you're in RStudio) or your default browser otherwise. The reprex is automatically copied to your clipboard (on RStudio Server or Cloud, you will need to copy this yourself):

````
``` r
y <- 1:4
mean(y)
#> [1] 2.5
```
````

This text is formatted in a special way, called Markdown, which can be pasted to sites like StackOverflow or Github and they will automatically render it to look like code. Here's what that Markdown would look like rendered on GitHub:

```
y <- 1:4
mean(y)
#> [1] 2.5
```

Anyone else can copy, paste, and run this immediately.

There are three things you need to include to make your example reproducible: required packages, data, and code.

1. **Packages** should be loaded at the top of the script so it's easy to see which ones the example needs. This is a good time to check that you're using the latest version of each package; you may have discovered a bug that's been fixed since you installed or last updated the package. For packages in the tidyverse, the easiest way to check is to run `tidyverse_update()`.

2. The easiest way to include **data** is to use `dput()` to generate the R code needed to recreate it. For example, to recreate the `mtcars` dataset in R, perform the following steps:

    1. Run `dput(mtcars)` in R
    2. Copy the output
    3. In reprex, type `mtcars <-`, then paste.

    Try to use the smallest subset of your data that still reveals the problem.

3. Spend a little bit of time ensuring that your **code** is easy for others to read:

    - Make sure you've used spaces and your variable names are concise yet informative.
    - Use comments to indicate where your problem lies.
    - Do your best to remove everything that is not related to the problem.

    The shorter your code is, the easier it is to understand and the easier it is to fix.

Finish by checking that you have actually made a reproducible example by starting a fresh R session and copying and pasting your script.

Creating reprexes is not trivial, and it will take some practice to learn to create good, truly minimal reprexes. However, learning to ask questions that include the code, and investing the time to make it reproducible will continue to pay off as you learn and master R.

8.3 Investing in yourself

You should also spend some time preparing yourself to solve problems before they occur. Investing a little time in learning R each day will pay off handsomely in the long run. One way is to follow what the tidyverse team is doing on the tidyverse blog. To keep up with the R community more broadly, we recommend reading R Weekly: it's a community effort to aggregate the most interesting news in the R community each week.

8.4 Summary

This chapter concludes the Whole Game part of the book. You've now seen the most important parts of the data science process: visualization, transformation, tidying and importing. Now you've got a holistic view of the whole process, and we start to get into the details of small pieces.

The next part of the book, Visualize, does a deeper dive into the grammar of graphics and creating data visualizations with ggplot2, showcases how to use the tools you've learned so far to conduct exploratory data analysis, and introduces good practices for creating plots for communication.

Visualize

After reading the first part of the book, you understand (at least superficially) the most important tools for doing data science. Now it's time to start diving into the details. In this part of the book, you'll learn about visualizing data in further depth.

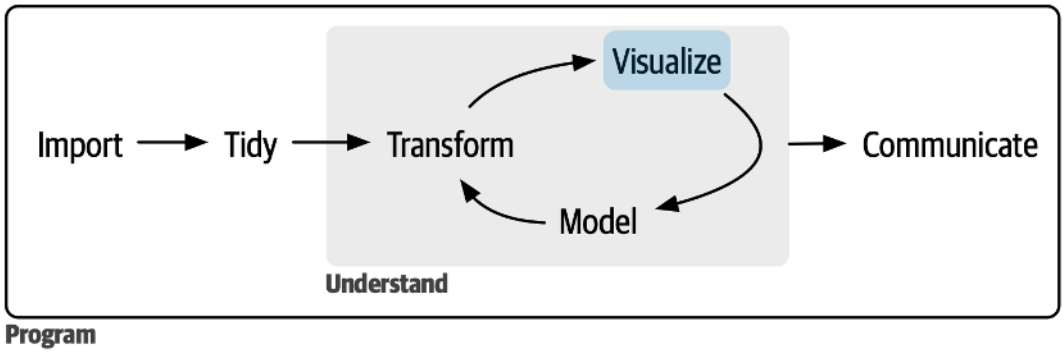


Figure 1: Data visualization is often the first step in data exploration.

Each chapter addresses one to a few aspects of creating a data visualization.

- In 9 Layers you will learn about the layered grammar of graphics.
- In 10 Exploratory data analysis, you'll combine visualization with your curiosity and skepticism to ask and answer interesting questions about data.
- Finally, in 11 Communication you will learn how to take your exploratory graphics, elevate them, and turn them into expository graphics, graphics that help the newcomer to your analysis understand what's going on as quickly and easily as possible.

These three chapters get you started in the world of visualization, but there is much more to learn. The absolute best place to learn more is the ggplot2 book: *ggplot2: Elegant graphics for data analysis*. It goes into much more depth about the underlying theory, and has many more examples of how to combine the individual pieces to solve practical problems. Another great resource is the ggplot2 extensions gallery https://exts.ggplot2.tidyverse.org/gallery/. This site lists many of the packages that extend ggplot2 with new geoms and scales. It's a great place to start if you're trying to do something that seems hard with ggplot2.

8 Workflow: getting help
9 Layers

9 Layers

1. Visualize
2. 9 Layers

9.1 Introduction

In Chapter 1, you learned much more than just how to make scatterplots, bar charts, and boxplots. You learned a foundation that you can use to make *any* type of plot with ggplot2.

In this chapter, you'll expand on that foundation as you learn about the layered grammar of graphics. We'll start with a deeper dive into aesthetic mappings, geometric objects, and facets. Then, you will learn about statistical transformations ggplot2 makes under the hood when creating a plot. These transformations are used to calculate new values to plot, such as the heights of bars in a bar plot or medians in a box plot. You will also learn about position adjustments, which modify how geoms are displayed in your plots. Finally, we'll briefly introduce coordinate systems.

We will not cover every single function and option for each of these layers, but we will walk you through the most important and commonly used functionality provided by ggplot2 as well as introduce you to packages that extend ggplot2.

9.1.1 Prerequisites

This chapter focuses on ggplot2. To access the datasets, help pages, and functions used in this chapter, load the tidyverse by running this code:

```
library(tidyverse)
```

9.2 Aesthetic mappings

> "The greatest value of a picture is when it forces us to notice what we never expected to see." — John Tukey

Remember that the `mpg` data frame bundled with the ggplot2 package contains 234 observations on 38 car models.

```
mpg
#> # A tibble: 234 × 11
#>   manufacturer model displ  year   cyl trans      drv     cty   hwy fl
#>   <chr>        <chr> <dbl> <int> <int> <chr>      <chr> <int> <int> <chr>
#> 1 audi         a4      1.8  1999     4 auto(l5)   f        18    29 p
#> 2 audi         a4      1.8  1999     4 manual(m5) f        21    29 p
#> 3 audi         a4      2    2008     4 manual(m6) f        20    31 p
#> 4 audi         a4      2    2008     4 auto(av)   f        21    30 p
#> 5 audi         a4      2.8  1999     6 auto(l5)   f        16    26 p
#> 6 audi         a4      2.8  1999     6 manual(m5) f        18    26 p
#> # ℹ 228 more rows
#> # ℹ 1 more variable: class <chr>
```

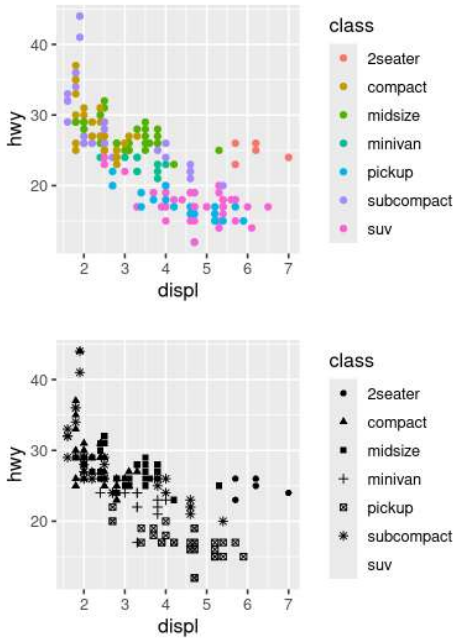
Among the variables in `mpg` are:

1. `displ`: A car's engine size, in liters. A numerical variable.
2. `hwy`: A car's fuel efficiency on the highway, in miles per gallon (mpg). A car with a low fuel efficiency consumes more fuel than a car with a high fuel efficiency when they travel the same distance. A numerical variable.
3. `class`: Type of car. A categorical variable.

Let's start by visualizing the relationship between `displ` and `hwy` for various `class`es of cars. We can do this with a scatterplot where the numerical variables are mapped to the `x` and `y` aesthetics and the categorical variable is mapped to an aesthetic like `color` or `shape`.

```
# Left
ggplot(mpg, aes(x = displ, y = hwy, color = class)) +
  geom_point()

# Right
ggplot(mpg, aes(x = displ, y = hwy, shape = class)) +
  geom_point()
#> Warning: The shape palette can deal with a maximum of 6 discrete values because
more
#> than 6 becomes difficult to discriminate
#> ℹ you have requested 7 values. Consider specifying shapes manually if you
#>   need that many have them.
#> Warning: Removed 62 rows containing missing values or values outside the scale
range
#> (`geom_point()`).
```



When `class` is mapped to `shape`, we get two warnings:

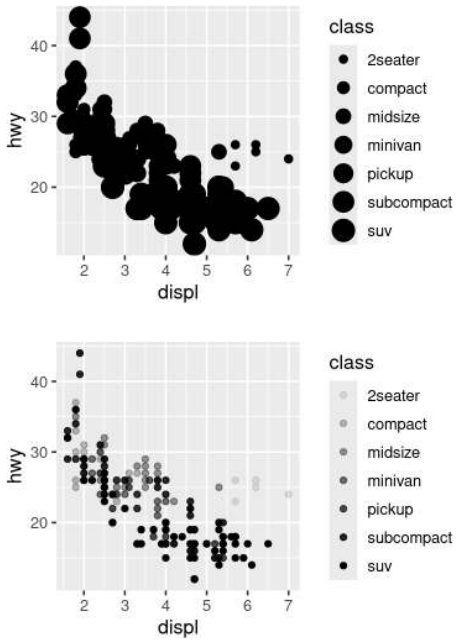
> 1: The shape palette can deal with a maximum of 6 discrete values because more than 6 becomes difficult to discriminate; you have 7. Consider specifying shapes manually if you must have them.
>
> 2: Removed 62 rows containing missing values (`geom_point()`).

Since ggplot2 will only use six shapes at a time, by default, additional groups will go unplotted when you use the shape aesthetic. The second warning is related – there are 62 SUVs in the dataset and they're not plotted.

Similarly, we can map `class` to `size` or `alpha` aesthetics as well, which control the shape and the transparency of the points, respectively.

```
# Left
ggplot(mpg, aes(x = displ, y = hwy, size = class)) +
  geom_point()
#> Warning: Using size for a discrete variable is not advised.

# Right
ggplot(mpg, aes(x = displ, y = hwy, alpha = class)) +
  geom_point()
#> Warning: Using alpha for a discrete variable is not advised.
```



Both of these produce warnings as well:

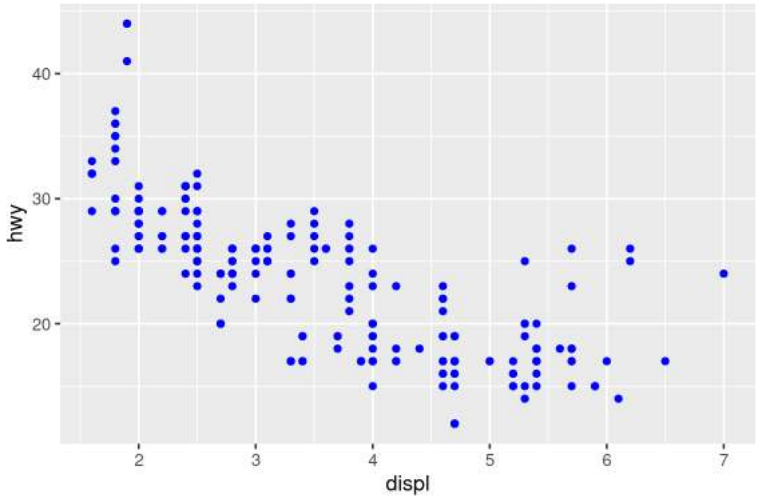
> Using alpha for a discrete variable is not advised.

Mapping an unordered discrete (categorical) variable (`class`) to an ordered aesthetic (`size` or `alpha`) is generally not a good idea because it implies a ranking that does not in fact exist.

Once you map an aesthetic, ggplot2 takes care of the rest. It selects a reasonable scale to use with the aesthetic, and it constructs a legend that explains the mapping between levels and values. For x and y aesthetics, ggplot2 does not create a legend, but it creates an axis line with tick marks and a label. The axis line provides the same information as a legend; it explains the mapping between locations and values.

You can also set the visual properties of your geom manually as an argument of your geom function (*outside* of `aes()`) instead of relying on a variable mapping to determine the appearance. For example, we can make all of the points in our plot blue:

```
ggplot(mpg, aes(x = displ, y = hwy)) +
  geom_point(color = "blue")
```



Here, the color doesn't convey information about a variable, but only changes the appearance of the plot. You'll need to pick a value that makes sense for that aesthetic:

- The name of a color as a character string, e.g., `color = "blue"`
- The size of a point in mm, e.g., `size = 1`

- The shape of a point as a number, e.g, `shape = 1`, as shown in Figure 9.1.

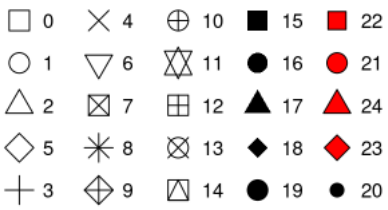


Figure 9.1: R has 25 built-in shapes that are identified by numbers. There are some seeming duplicates: for example, 0, 15, and 22 are all squares. The difference comes from the interaction of the `color` and `fill` aesthetics. The hollow shapes (0–14) have a border determined by `color`; the solid shapes (15–20) are filled with `color`; the filled shapes (21–24) have a border of `color` and are filled with `fill`. Shapes are arranged to keep similar shapes next to each other.

So far we have discussed aesthetics that we can map or set in a scatterplot, when using a point geom. You can learn more about all possible aesthetic mappings in the aesthetic specifications vignette at https://ggplot2.tidyverse.org/articles/ggplot2-specs.html.

The specific aesthetics you can use for a plot depend on the geom you use to represent the data. In the next section we dive deeper into geoms.

9.2.1 Exercises

1. Create a scatterplot of `hwy` vs. `displ` where the points are pink filled in triangles.

2. Why did the following code not result in a plot with blue points?

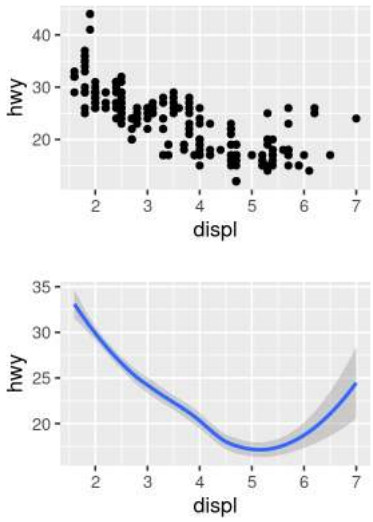
```
ggplot(mpg) +
  geom_point(aes(x = displ, y = hwy, color = "blue"))
```

3. What does the `stroke` aesthetic do? What shapes does it work with? (Hint: use `?geom_point`)

4. What happens if you map an aesthetic to something other than a variable name, like `aes(color = displ < 5)`? Note, you'll also need to specify x and y.

9.3 Geometric objects

How are these two plots similar?



Both plots contain the same x variable, the same y variable, and both describe the same data. But the plots are not identical. Each plot uses a different geometric object, geom, to represent the data. The plot on the left uses the point geom, and the plot on the right uses the smooth geom, a smooth line fitted to the data.

To change the geom in your plot, change the geom function that you add to `ggplot()`. For instance, to make the plots above, you can use the following code:

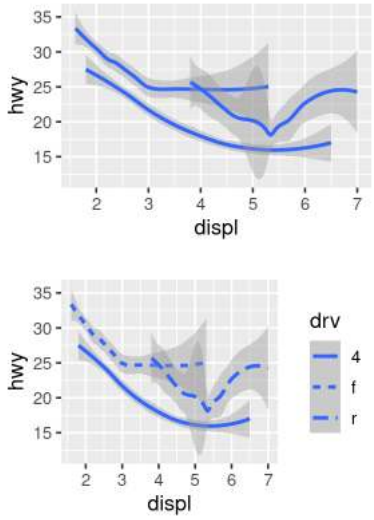
```
# Left
ggplot(mpg, aes(x = displ, y = hwy)) +
  geom_point()

# Right
ggplot(mpg, aes(x = displ, y = hwy)) +
  geom_smooth()
#> `geom_smooth()` using method = 'loess' and formula = 'y ~ x'
```

Every geom function in ggplot2 takes a `mapping` argument, either defined locally in the geom layer or globally in the `ggplot()` layer. However, not every aesthetic works with every geom. You could set the shape of a point, but you couldn't set the "shape" of a line. If you try, ggplot2 will silently ignore that aesthetic mapping. On the other hand, you *could* set the linetype of a line. `geom_smooth()` will draw a different line, with a different linetype, for each unique value of the variable that you map to linetype.

```
# Left
ggplot(mpg, aes(x = displ, y = hwy, shape = drv)) +
  geom_smooth()

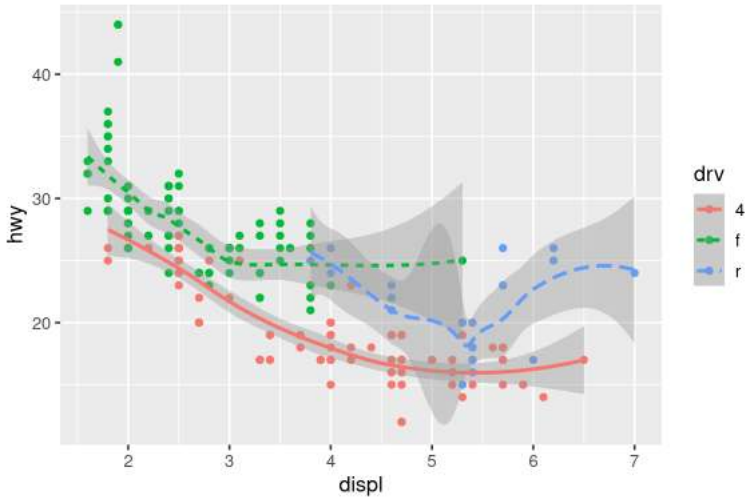
# Right
ggplot(mpg, aes(x = displ, y = hwy, linetype = drv)) +
  geom_smooth()
```



Here, `geom_smooth()` separates the cars into three lines based on their `drv` value, which describes a car's drive train. One line describes all of the points that have a `4` value, one line describes all of the points that have an `f` value, and one line describes all of the points that have an `r` value. Here, `4` stands for four-wheel drive, `f` for front-wheel drive, and `r` for rear-wheel drive.

If this sounds strange, we can make it clearer by overlaying the lines on top of the raw data and then coloring everything according to `drv`.

```
ggplot(mpg, aes(x = displ, y = hwy, color = drv)) +
  geom_point() +
  geom_smooth(aes(linetype = drv))
```



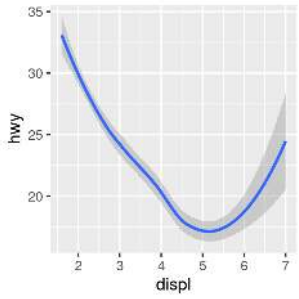
Notice that this plot contains two geoms in the same graph.

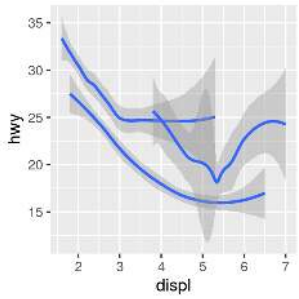
Many geoms, like `geom_smooth()`, use a single geometric object to display multiple rows of data. For these geoms, you can set the `group` aesthetic to a categorical variable to draw multiple objects. ggplot2 will draw a separate object for each unique value of the grouping variable. In practice, ggplot2 will automatically group the data for these geoms whenever you map an aesthetic to a discrete variable (as in the `linetype` example). It is convenient to rely on this feature because the `group` aesthetic by itself does not add a legend or distinguishing features to the geoms.

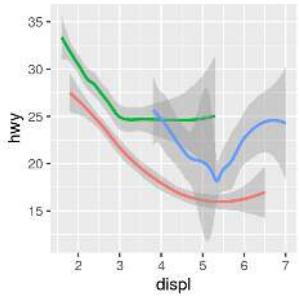
```
# Left
ggplot(mpg, aes(x = displ, y = hwy)) +
  geom_smooth()

# Middle
ggplot(mpg, aes(x = displ, y = hwy)) +
  geom_smooth(aes(group = drv))

# Right
ggplot(mpg, aes(x = displ, y = hwy)) +
  geom_smooth(aes(color = drv), show.legend = FALSE)
```

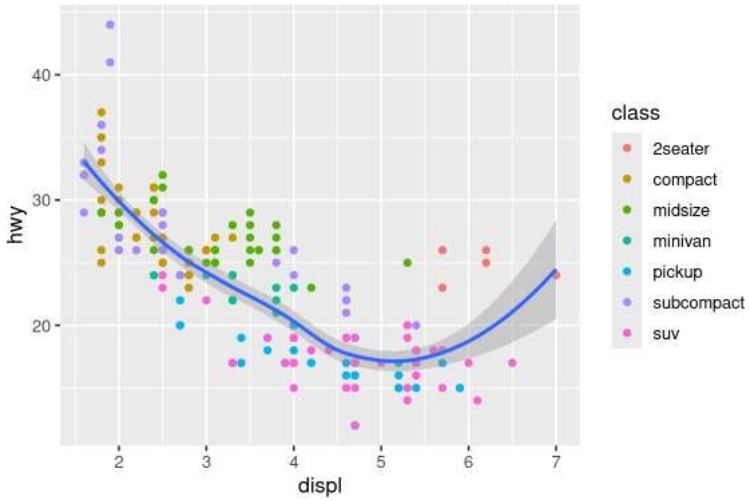






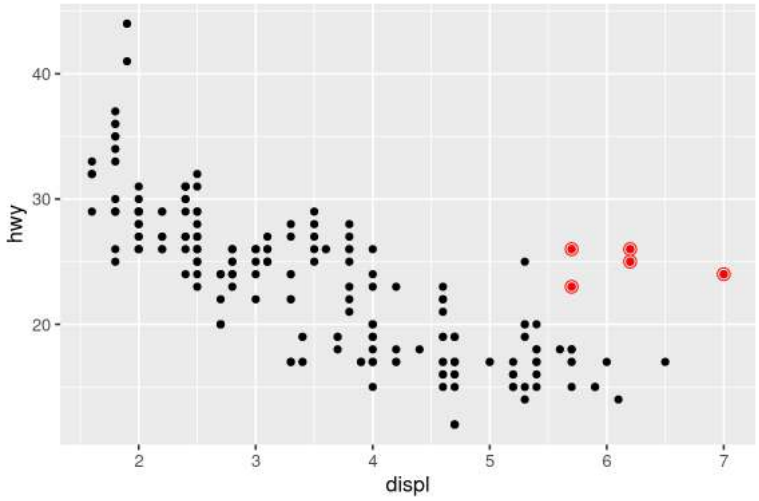
If you place mappings in a geom function, ggplot2 will treat them as local mappings for the layer. It will use these mappings to extend or overwrite the global mappings *for that layer only*. This makes it possible to display different aesthetics in different layers.

```
ggplot(mpg, aes(x = displ, y = hwy)) +
  geom_point(aes(color = class)) +
  geom_smooth()
```



You can use the same idea to specify different `data` for each layer. Here, we use red points as well as open circles to highlight two-seater cars. The local data argument in `geom_point()` overrides the global data argument in `ggplot()` for that layer only.

```
ggplot(mpg, aes(x = displ, y = hwy)) +
  geom_point() +
  geom_point(
    data = mpg |> filter(class == "2seater"),
    color = "red"
  ) +
  geom_point(
    data = mpg |> filter(class == "2seater"),
    shape = "circle open", size = 3, color = "red"
  )
```

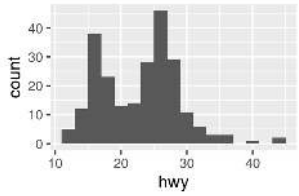


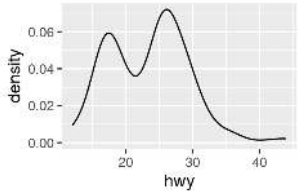
Geoms are the fundamental building blocks of ggplot2. You can completely transform the look of your plot by changing its geom, and different geoms can reveal different features of your data. For example, the histogram and density plot below reveal that the distribution of highway mileage is bimodal and right skewed while the boxplot reveals two potential outliers.

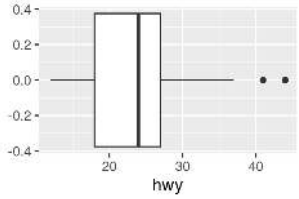
```
# Left
ggplot(mpg, aes(x = hwy)) +
  geom_histogram(binwidth = 2)

# Middle
ggplot(mpg, aes(x = hwy)) +
  geom_density()

# Right
ggplot(mpg, aes(x = hwy)) +
  geom_boxplot()
```





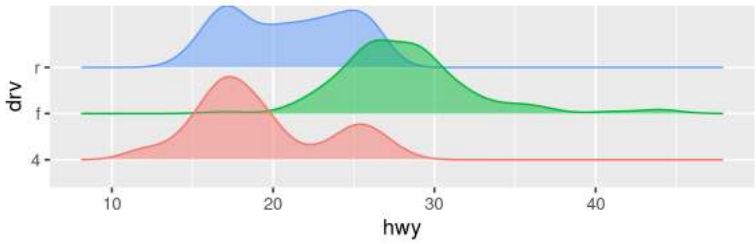


ggplot2 provides more than 40 geoms but these don't cover all possible plots one could make. If you need a different geom, we recommend looking into extension packages first to see if someone else has already implemented it (see https://exts.ggplot2.tidyverse.org/gallery/ for a sampling). For example, the **ggridges** package (https://wilkelab.org/ggridges) is useful for making ridgeline plots, which can be useful for visualizing the density of a numerical variable for different levels of a categorical variable. In the following plot not only did we use a new geom

(`geom_density_ridges()`), but we have also mapped the same variable to multiple aesthetics (`drv` to `y`, `fill`, and `color`) as well as set an aesthetic (`alpha = 0.5`) to make the density curves transparent.

```
library(ggridges)

ggplot(mpg, aes(x = hwy, y = drv, fill = drv, color = drv)) +
  geom_density_ridges(alpha = 0.5, show.legend = FALSE)
#> Picking joint bandwidth of 1.28
```



The best place to get a comprehensive overview of all of the geoms ggplot2 offers, as well as all functions in the package, is the reference page: https://ggplot2.tidyverse.org/reference. To learn more about any single geom, use the help (e.g., `?geom_smooth`).

9.3.1 Exercises

1. What geom would you use to draw a line chart? A boxplot? A histogram? An area chart?

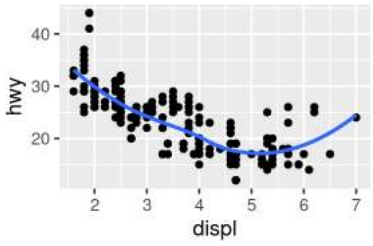
2. Earlier in this chapter we used `show.legend` without explaining it:

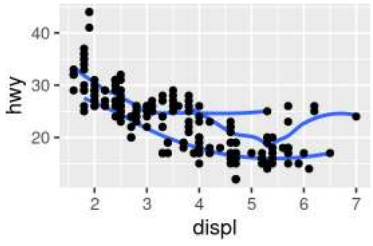
   ```
   geom_smooth(aes(color = drv), show.legend = FALSE)
   ```

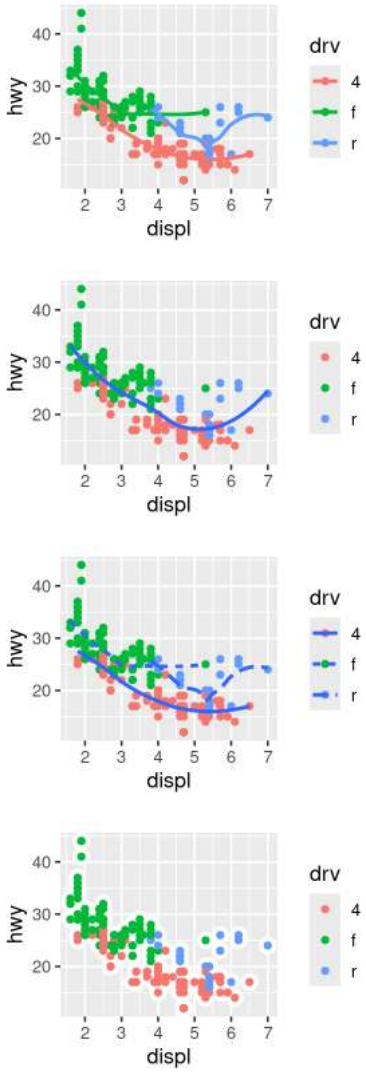
   What does `show.legend = FALSE` do here? What happens if you remove it? Why do you think we used it earlier?

3. What does the `se` argument to `geom_smooth()` do?

4. Recreate the R code necessary to generate the following graphs. Note that wherever a categorical variable is used in the plot, it's `drv`.



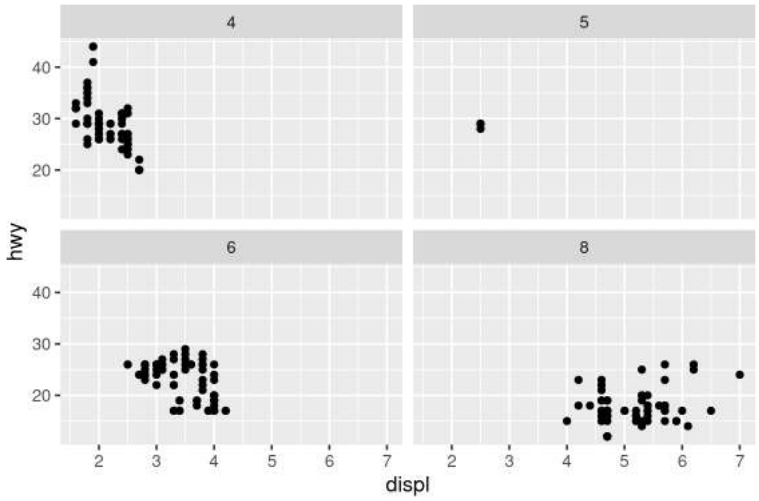




9.4 Facets

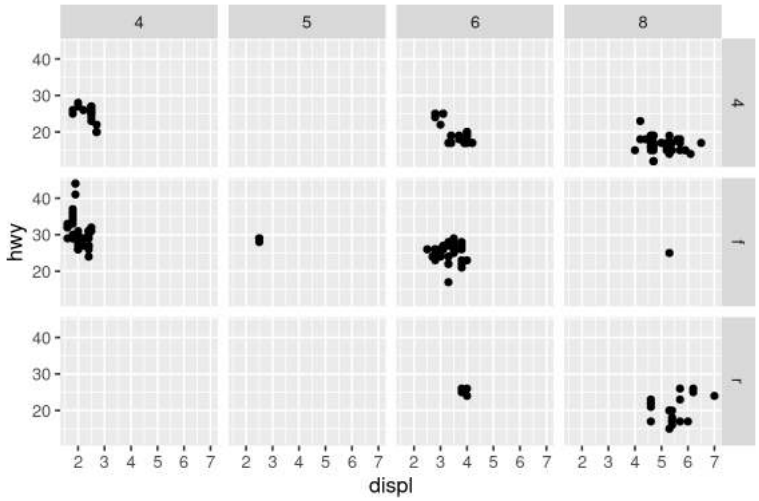
In Chapter 1 you learned about faceting with `facet_wrap()`, which splits a plot into subplots that each display one subset of the data based on a categorical variable.

```
geom_point() +
facet_wrap(~cyl)
```



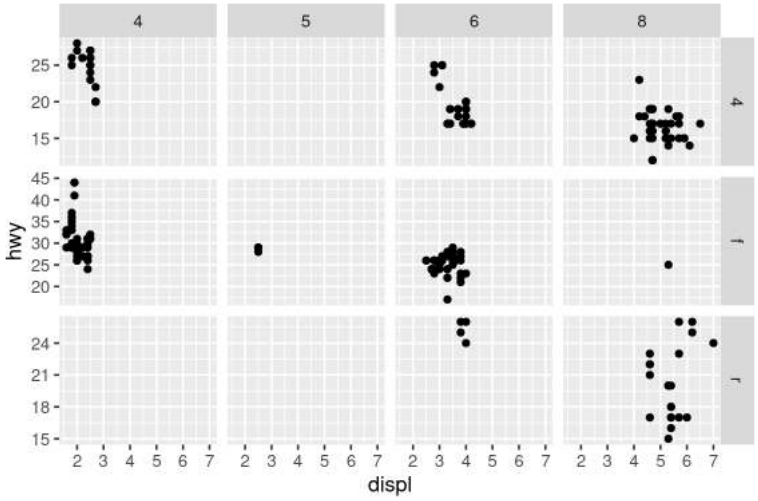
To facet your plot with the combination of two variables, switch from `facet_wrap()` to `facet_grid()`. The first argument of `facet_grid()` is also a formula, but now it's a double sided formula: `rows ~ cols`.

```
geom_point() +
facet_grid(drv ~ cyl)
```



By default each of the facets share the same scale and range for x and y axes. This is useful when you want to compare data across facets but it can be limiting when you want to visualize the relationship within each facet better. Setting the `scales` argument in a faceting function to `"free"` will allow for different axis scales across both rows and columns, `"free_x"` will allow for different scales across rows, and `"free_y"` will allow for different scales across columns.

```
geom_point() +
facet_grid(drv ~ cyl, scales = "free_y")
```



9.4.1 Exercises

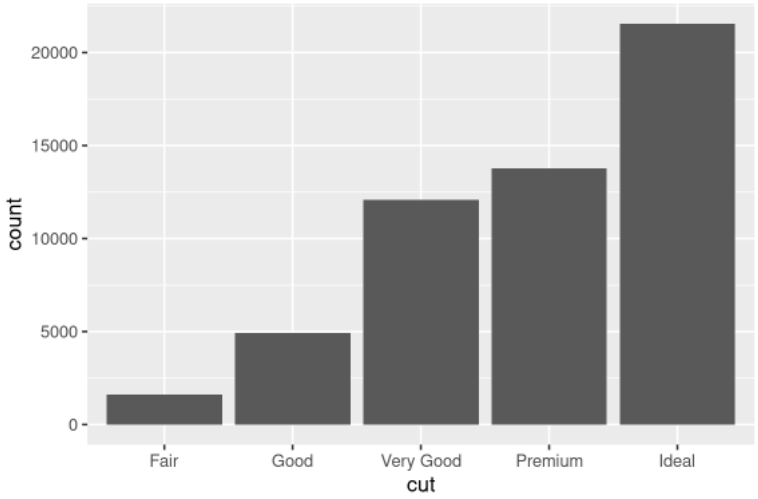
1. What happens if you facet on a continuous variable?
2. What do the empty cells in the plot above with `facet_grid(drv ~ cyl)` mean? Run the following code. How do they relate to the resulting plot?
3. What plots does the following code make? What does `.` do?
4. Take the first faceted plot in this section:

   What are the advantages to using faceting instead of the color aesthetic? What are the disadvantages? How might the balance change if you had a larger dataset?
5. Read `?facet_wrap`. What does `nrow` do? What does `ncol` do? What other options control the layout of the individual panels? Why doesn't `facet_grid()` have `nrow` and `ncol` arguments?
6. Which of the following plots makes it easier to compare engine size (`displ`) across cars with different drive trains? What does this say about when to place a faceting variable across rows or columns?
7. Recreate the following plot using `facet_wrap()` instead of `facet_grid()`. How do the positions of the facet labels change?

9.5 Statistical transformations

Consider a basic bar chart, drawn with `geom_bar()` or `geom_col()`. The following chart displays the total number of diamonds in the `diamonds` dataset, grouped by `cut`. The `diamonds` dataset is in the ggplot2 package and contains information on ~54,000 diamonds, including the `price`, `carat`, `color`, `clarity`, and `cut` of each diamond. The chart shows that more diamonds are available with high quality cuts than with low quality cuts.

```
ggplot(diamonds, aes(x = cut)) +
  geom_bar()
```



On the x-axis, the chart displays `cut`, a variable from `diamonds`. On the y-axis, it displays count, but count is not a variable in `diamonds`! Where does count come from? Many graphs, like scatterplots, plot the raw values of your dataset. Other graphs, like bar charts, calculate new values to plot:

- Bar charts, histograms, and frequency polygons bin your data and then plot bin counts, the number of points that fall in each bin.
- Smoothers fit a model to your data and then plot predictions from the model.
- Boxplots compute the five-number summary of the distribution and then display that summary as a specially formatted box.

The algorithm used to calculate new values for a graph is called a **stat**, short for statistical transformation. Figure 9.2 shows how this process works with `geom_bar()`.

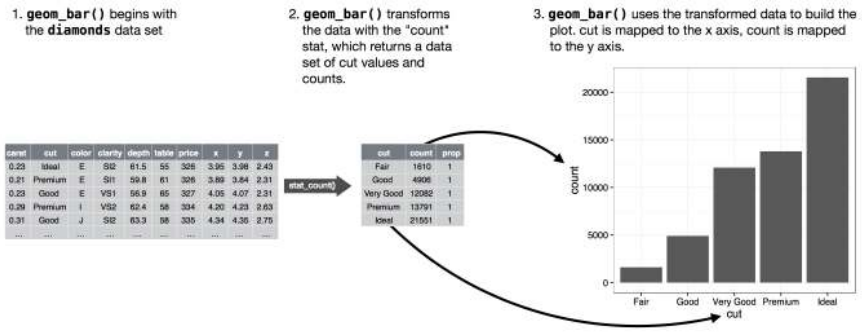


Figure 9.2: When creating a bar chart we first start with the raw data, then aggregate it to count the number of observations in each bar, and finally map those computed variables to plot aesthetics.

You can learn which stat a geom uses by inspecting the default value for the `stat` argument. For example, `?geom_bar` shows that the default value for `stat` is "count", which means that `geom_bar()` uses `stat_count()`. `stat_count()` is documented on the same page as `geom_bar()`. If you scroll down, the section called "Computed variables" explains that it computes two new variables: `count` and `prop`.

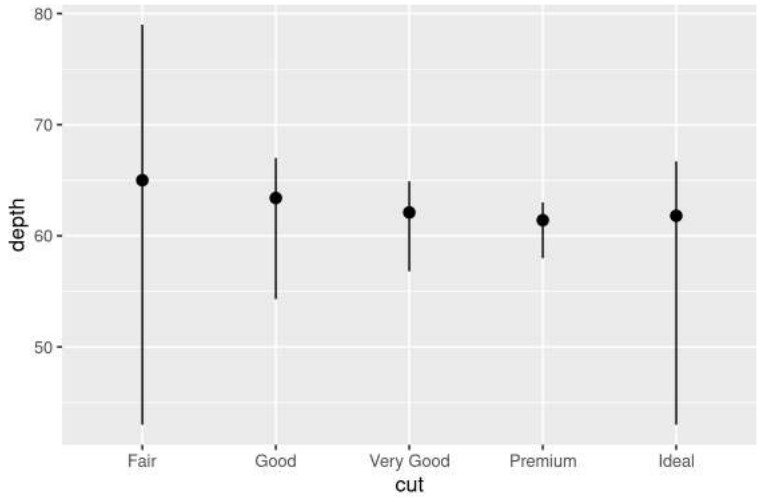
Every geom has a default stat; and every stat has a default geom. This means that you can typically use geoms without worrying about the underlying statistical transformation. However, there are three reasons why you might need to use a stat explicitly:

1. You might want to override the default stat. In the code below, we change the stat of `geom_bar()` from count (the default) to identity. This lets us map the height of the bars to the raw values of a y variable.
2. You might want to override the default mapping from transformed variables to aesthetics. For example, you might want to display a bar chart of proportions, rather than counts:

To find the possible variables that can be computed by the stat, look for the section titled “computed variables” in the help for `geom_bar()`.

3. You might want to draw greater attention to the statistical transformation in your code. For example, you might use `stat_summary()`, which summarizes the y values for each unique x value, to draw attention to the summary that you’re computing:

```
ggplot(diamonds) +
  stat_summary(
    aes(x = cut, y = depth),
    fun.min = min,
    fun.max = max,
    fun = median
  )
```



ggplot2 provides more than 20 stats for you to use. Each stat is a function, so you can get help in the usual way, e.g., `?stat_bin`.

9.5.1 Exercises

1. What is the default geom associated with `stat_summary()`? How could you rewrite the previous plot to use that geom function instead of the stat function?

2. What does `geom_col()` do? How is it different from `geom_bar()`?

3. Most geoms and stats come in pairs that are almost always used in concert. Make a list of all the pairs. What do they have in common? (Hint: Read through the documentation.)

4. What variables does `stat_smooth()` compute? What arguments control its behavior?

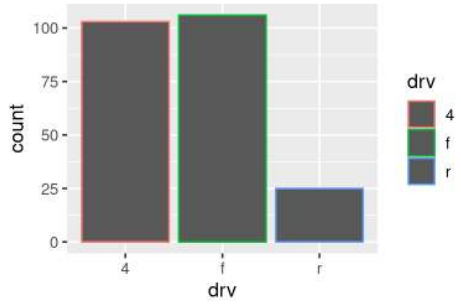
5. In our proportion bar chart, we needed to set `group = 1`. Why? In other words, what is the problem with these two graphs?

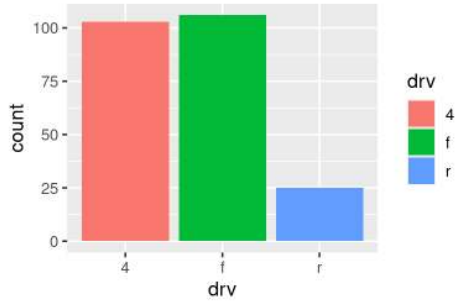
9.6 Position adjustments

There’s one more piece of magic associated with bar charts. You can color a bar chart using either the `color` aesthetic, or, more usefully, the `fill` aesthetic:

```
# Left
ggplot(mpg, aes(x = drv, color = drv)) +
  geom_bar()

# Right
ggplot(mpg, aes(x = drv, fill = drv)) +
  geom_bar()
```





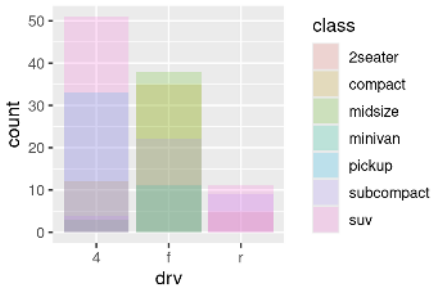
Note what happens if you map the fill aesthetic to another variable, like `class`: the bars are automatically stacked. Each colored rectangle represents a combination of `drv` and `class`.

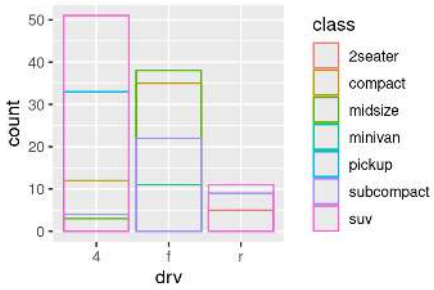
The stacking is performed automatically using the **position adjustment** specified by the `position` argument. If you don't want a stacked bar chart, you can use one of three other options: `"identity"`, `"dodge"` or `"fill"`.

- `position = "identity"` will place each object exactly where it falls in the context of the graph. This is not very useful for bars, because it overlaps them. To see that overlapping we either need to make the bars slightly transparent by setting `alpha` to a small value, or completely transparent by setting `fill = NA`.

```
# Left
ggplot(mpg, aes(x = drv, fill = class)) +
  geom_bar(alpha = 1/5, position = "identity")

# Right
ggplot(mpg, aes(x = drv, color = class)) +
  geom_bar(fill = NA, position = "identity")
```



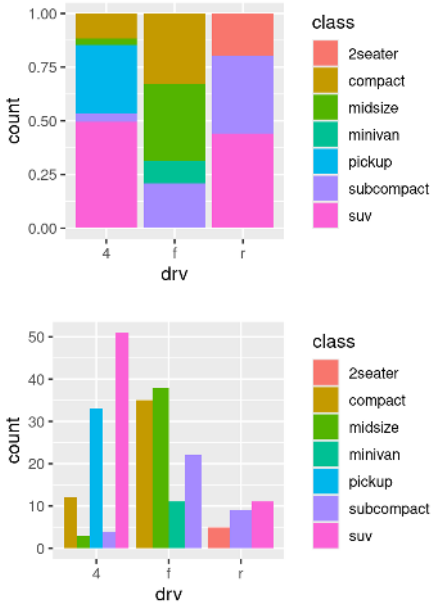


The identity position adjustment is more useful for 2d geoms, like points, where it is the default.

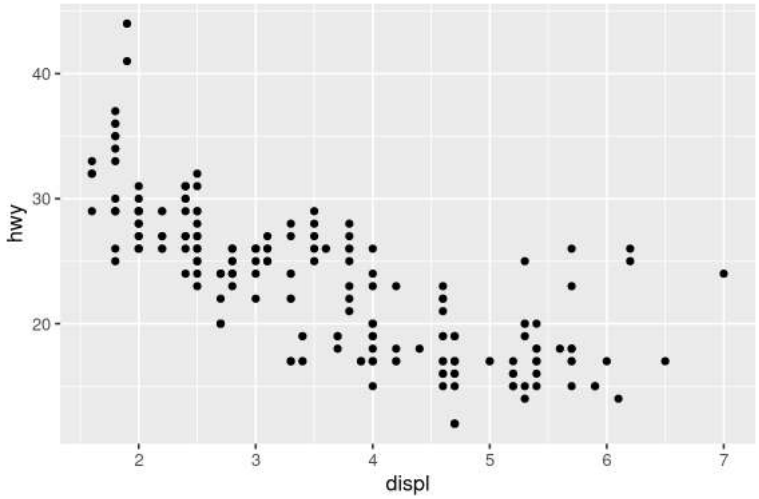
- `position = "fill"` works like stacking, but makes each set of stacked bars the same height. This makes it easier to compare proportions across groups.
- `position = "dodge"` places overlapping objects directly *beside* one another. This makes it easier to compare individual values.

```
# Left
ggplot(mpg, aes(x = drv, fill = class)) +
  geom_bar(position = "fill")

# Right
ggplot(mpg, aes(x = drv, fill = class)) +
  geom_bar(position = "dodge")
```



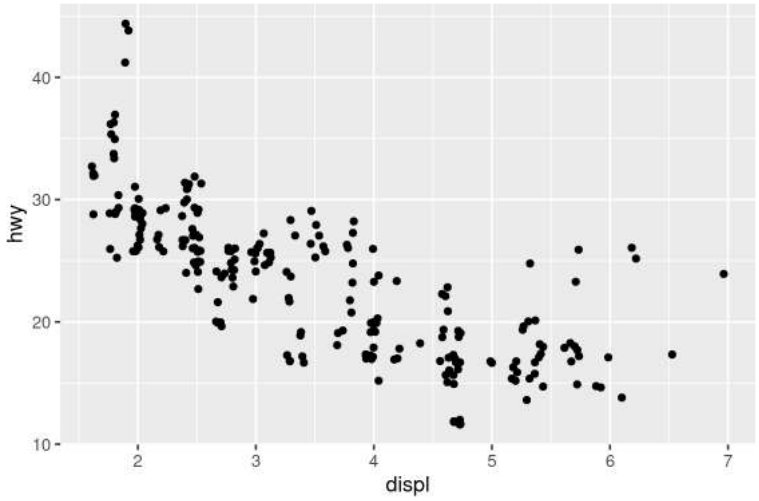
There's one other type of adjustment that's not useful for bar charts, but can be very useful for scatterplots. Recall our first scatterplot. Did you notice that the plot displays only 126 points, even though there are 234 observations in the dataset?



The underlying values of `hwy` and `displ` are rounded so the points appear on a grid and many points overlap each other. This problem is known as **overplotting**. This arrangement makes it difficult to see the distribution of the data. Are the data points spread equally throughout the graph, or is there one special combination of `hwy` and `displ` that contains 109 values?

You can avoid this gridding by setting the position adjustment to "jitter". `position = "jitter"` adds a small amount of random noise to each point. This spreads the points out because no two points are likely to receive the same amount of random noise.

```
geom_point(position = "jitter")
```



Adding randomness seems like a strange way to improve your plot, but while it makes your graph less accurate at small scales, it makes your graph *more* revealing at large scales. Because this is such a useful operation, ggplot2 comes with a shorthand for `geom_point(position = "jitter")`: `geom_jitter()`.

To learn more about a position adjustment, look up the help page associated with each adjustment: `?position_dodge`, `?position_fill`, `?position_identity`, `?position_jitter`, and `?position_stack`.

9.6.1 Exercises

1. What is the problem with the following plot? How could you improve it?
2. What, if anything, is the difference between the two plots? Why?
3. What parameters to `geom_jitter()` control the amount of jittering?
4. Compare and contrast `geom_jitter()` with `geom_count()`.
5. What's the default position adjustment for `geom_boxplot()`? Create a visualization of the `mpg` dataset that demonstrates it.

9.7 Coordinate systems

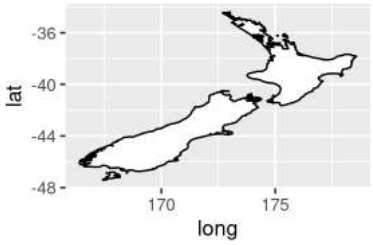
Coordinate systems are probably the most complicated part of ggplot2. The default coordinate system is the Cartesian coordinate system where the x and y positions act independently to determine the location of each point. There are two other coordinate systems that are occasionally helpful.

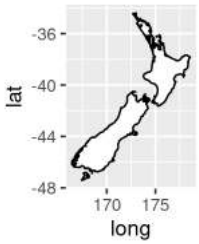
- `coord_quickmap()` sets the aspect ratio correctly for geographic maps. This is very important if you're plotting spatial data with ggplot2. We don't have the space to discuss maps in this book, but you can learn more in the Maps chapter of *ggplot2: Elegant graphics for data analysis*.

```
nz <- map_data("nz")

ggplot(nz, aes(x = long, y = lat, group = group)) +
  geom_polygon(fill = "white", color = "black")

ggplot(nz, aes(x = long, y = lat, group = group)) +
  geom_polygon(fill = "white", color = "black") +
  coord_quickmap()
```

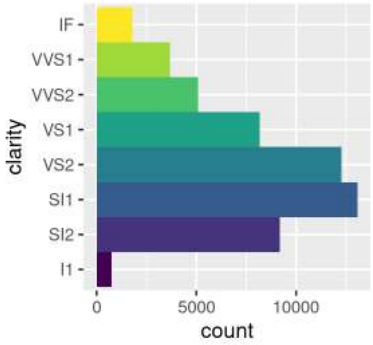


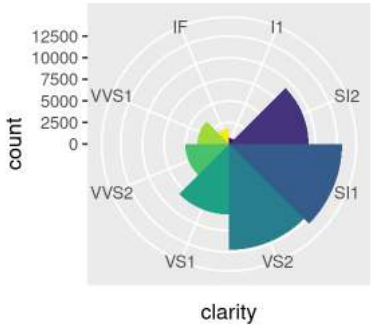


- `coord_polar()` uses polar coordinates. Polar coordinates reveal an interesting connection between a bar chart and a Coxcomb chart.

```
bar <- ggplot(data = diamonds) +
  geom_bar(
    mapping = aes(x = clarity, fill = clarity),
    show.legend = FALSE,
    width = 1
  ) +
  theme(aspect.ratio = 1)

bar + coord_flip()
bar + coord_polar()
```





9.7.1 Exercises

1. Turn a stacked bar chart into a pie chart using `coord_polar()`.

2. What's the difference between `coord_quickmap()` and `coord_map()`?

3. What does the following plot tell you about the relationship between city and highway mpg? Why is `coord_fixed()` important? What does `geom_abline()` do?

9.8 The layered grammar of graphics

We can expand on the graphing template you learned in Section 1.3 by adding position adjustments, stats, coordinate systems, and faceting:

```
ggplot(data = <DATA>) +
  <GEOM_FUNCTION>(
     mapping = aes(<MAPPINGS>),
     stat = <STAT>,
     position = <POSITION>
  ) +
  <COORDINATE_FUNCTION> +
  <FACET_FUNCTION>
```

Our new template takes seven parameters, the bracketed words that appear in the template. In practice, you rarely need to supply all seven parameters to make a graph because ggplot2 will provide useful defaults for everything except the data, the mappings, and the geom function.

The seven parameters in the template compose the grammar of graphics, a formal system for building plots. The grammar of graphics is based on the insight that you can uniquely describe *any* plot as a combination of a dataset, a geom, a set of mappings, a stat, a position adjustment, a coordinate system, a faceting scheme, and a theme.

To see how this works, consider how you could build a basic plot from scratch: you could start with a dataset and then transform it into the information that you want to display (with a stat). Next, you could choose a geometric object to represent each observation in the transformed data. You could then use the aesthetic properties of the geoms to represent variables in the data. You would map the values of each variable to the levels of an aesthetic. These steps are illustrated in Figure 9.3. You'd then select a coordinate system to place the geoms into, using the location of the objects (which is itself an aesthetic property) to display the values of the x and y variables.

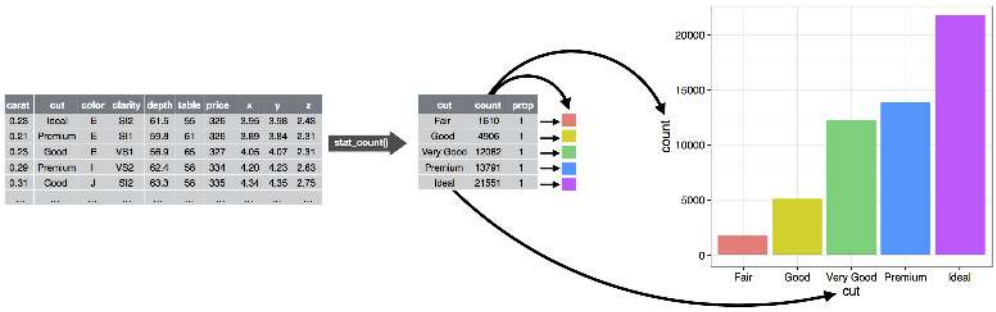


Figure 9.3: Steps for going from raw data to a table of frequencies to a bar plot where the heights of the bar represent the frequencies.

At this point, you would have a complete graph, but you could further adjust the positions of the geoms within the coordinate system (a position adjustment) or split the graph into subplots (faceting). You could also extend the plot by adding one or more additional layers, where each additional layer uses a dataset, a geom, a set of mappings, a stat, and a position adjustment.

You could use this method to build *any* plot that you imagine. In other words, you can use the code template that you've learned in this chapter to build hundreds of thousands of unique plots.

If you'd like to learn more about the theoretical underpinnings of ggplot2, you might enjoy reading "The Layered Grammar of Graphics", the scientific paper that describes the theory of ggplot2 in detail.

9.9 Summary

In this chapter you learned about the layered grammar of graphics starting with aesthetics and geometries to build a simple plot, facets for splitting the plot into subsets, statistics for understanding how geoms are calculated, position adjustments for controlling the fine details of position when geoms might otherwise overlap, and coordinate systems which allow you to fundamentally change what `x` and `y` mean. One layer we have not yet touched on is theme, which we will introduce in Section 11.5.

Two very useful resources for getting an overview of the complete ggplot2 functionality are the ggplot2 cheatsheet (which you can find at https://posit.co/resources/cheatsheets) and the ggplot2 package website (https://ggplot2.tidyverse.org).

An important lesson you should take from this chapter is that when you feel the need for a geom that is not provided by ggplot2, it's always a good idea to look into whether someone else has already solved your problem by creating a ggplot2 extension package that offers that geom.

10 Exploratory data analysis

10.1 Introduction

This chapter will show you how to use visualization and transformation to explore your data in a systematic way, a task that statisticians call exploratory data analysis, or EDA for short. EDA is an iterative cycle. You:

1. Generate questions about your data.

2. Search for answers by visualizing, transforming, and modelling your data.

3. Use what you learn to refine your questions and/or generate new questions.

EDA is not a formal process with a strict set of rules. More than anything, EDA is a state of mind. During the initial phases of EDA you should feel free to investigate every idea that occurs to you. Some of these ideas will pan out, and some will be dead ends. As your exploration continues, you will home in on a few particularly productive insights that you'll eventually write up and communicate to others.

EDA is an important part of any data analysis, even if the primary research questions are handed to you on a platter, because you always need to investigate the quality of your data. Data cleaning is just one application of EDA: you ask questions about whether your data meets your expectations or not. To do data cleaning, you'll need to deploy all the tools of EDA: visualization, transformation, and modelling.

10.1.1 Prerequisites

In this chapter we'll combine what you've learned about dplyr and ggplot2 to interactively ask questions, answer them with data, and then ask new questions.

```
library(tidyverse)
```

10.2 Questions

> "There are no routine statistical questions, only questionable statistical routines." — Sir David Cox

> "Far better an approximate answer to the right question, which is often vague, than an exact answer to the wrong question, which can always be made precise." — John Tukey

Your goal during EDA is to develop an understanding of your data. The easiest way to do this is to use questions as tools to guide your investigation. When you ask a question, the question focuses your attention

on a specific part of your dataset and helps you decide which graphs, models, or transformations to make.

EDA is fundamentally a creative process. And like most creative processes, the key to asking *quality* questions is to generate a large *quantity* of questions. It is difficult to ask revealing questions at the start of your analysis because you do not know what insights can be gleaned from your dataset. On the other hand, each new question that you ask will expose you to a new aspect of your data and increase your chance of making a discovery. You can quickly drill down into the most interesting parts of your data—and develop a set of thought-provoking questions—if you follow up each question with a new question based on what you find.

There is no rule about which questions you should ask to guide your research. However, two types of questions will always be useful for making discoveries within your data. You can loosely word these questions as:

1. What type of variation occurs within my variables?

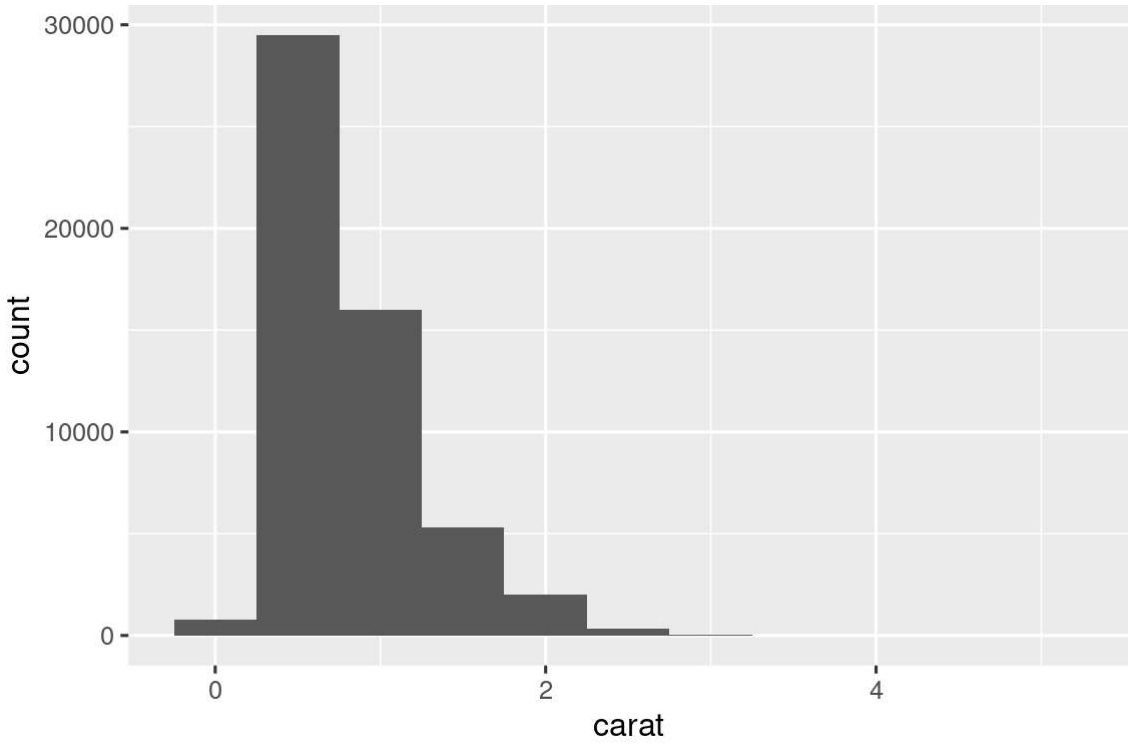
2. What type of covariation occurs between my variables?

The rest of this chapter will look at these two questions. We'll explain what variation and covariation are, and we'll show you several ways to answer each question.

10.3 Variation

Variation is the tendency of the values of a variable to change from measurement to measurement. You can see variation easily in real life; if you measure any continuous variable twice, you will get two different results. This is true even if you measure quantities that are constant, like the speed of light. Each of your measurements will include a small amount of error that varies from measurement to measurement. Variables can also vary if you measure across different subjects (e.g., the eye colors of different people) or at different times (e.g., the energy levels of an electron at different moments). Every variable has its own pattern of variation, which can reveal interesting information about how that it varies between measurements on the same observation as well as across observations. The best way to understand that pattern is to visualize the distribution of the variable's values, which you've learned about in Chapter 1.

We'll start our exploration by visualizing the distribution of weights (`carat`) of ~54,000 diamonds from the `diamonds` dataset. Since `carat` is a numerical variable, we can use a histogram:



Now that you can visualize variation, what should you look for in your plots? And what type of follow-up questions should you ask? We've put together a list below of the most useful types of information that you will find in your graphs, along with some follow-up questions for each type of information. The key to asking good follow-up questions will be to rely on your curiosity (What do you want to learn more about?) as well as your skepticism (How could this be misleading?).

10.3.1 Typical values

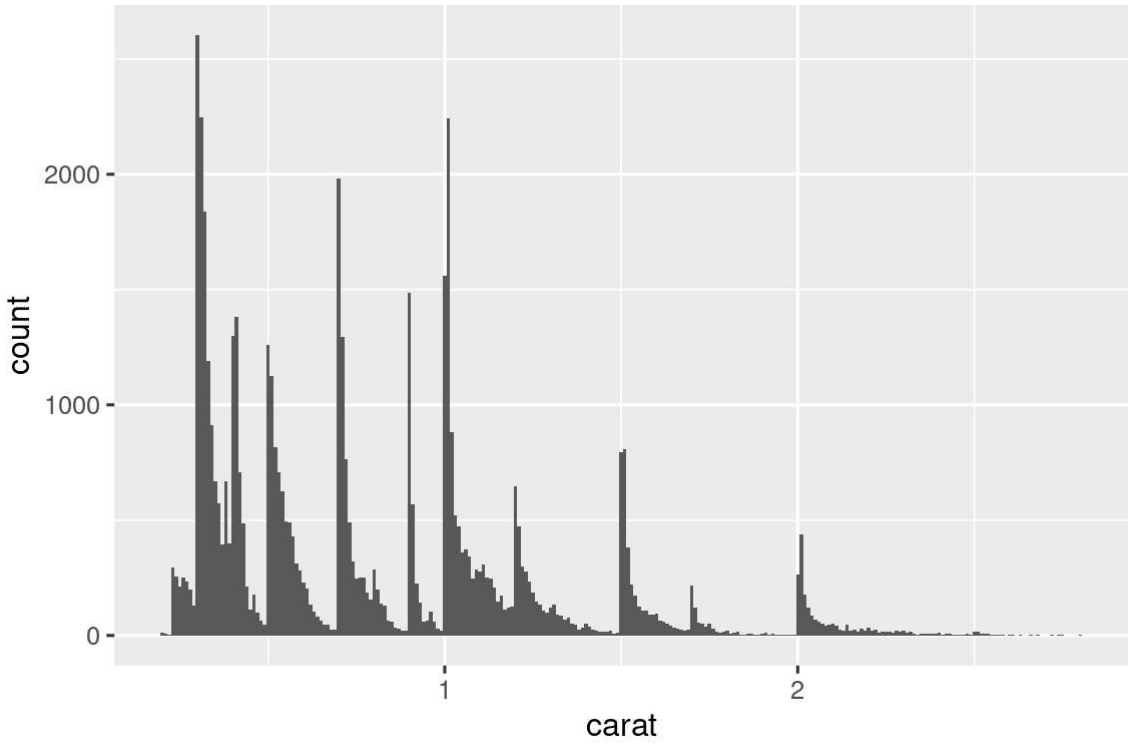
In both bar charts and histograms, tall bars show the common values of a variable, and shorter bars show less-common values. Places that do not have bars reveal values that were not seen in your data. To turn this information into useful questions, look for anything unexpected:

- Which values are the most common? Why?
- Which values are rare? Why? Does that match your expectations?
- Can you see any unusual patterns? What might explain them?

Let's take a look at the distribution of `carat` for smaller diamonds.

```
smaller <- diamonds |>
  filter(carat < 3)

ggplot(smaller, aes(x = carat)) +
  geom_histogram(binwidth = 0.01)
```



This histogram suggests several interesting questions:

- Why are there more diamonds at whole carats and common fractions of carats?
- Why are there more diamonds slightly to the right of each peak than there are slightly to the left of each peak?

Visualizations can also reveal clusters, which suggest that subgroups exist in your data. To understand the subgroups, ask:

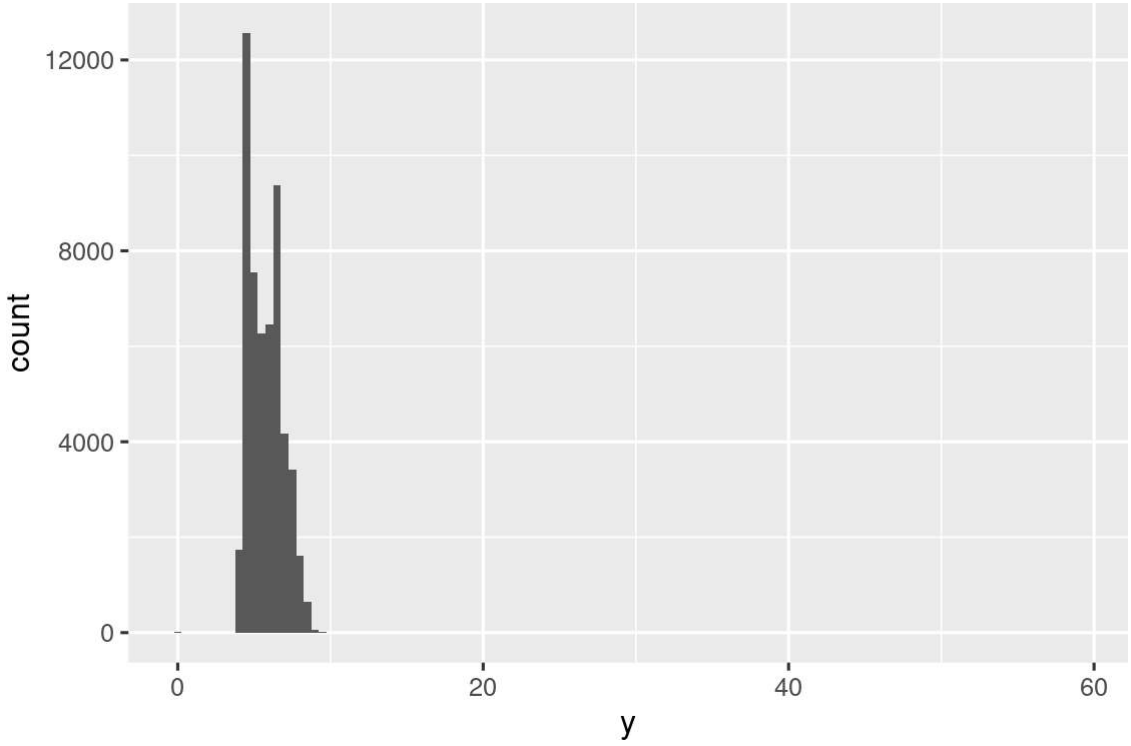
- How are the observations within each subgroup similar to each other?
- How are the observations in separate clusters different from each other?
- How can you explain or describe the clusters?
- Why might the appearance of clusters be misleading?

Some of these questions can be answered with the data while some will require domain expertise about the data. Many of them will prompt you to explore a relationship *between* variables, for example, to see if the values of one variable can explain the behavior of another variable. We'll get to that shortly.

10.3.2 Unusual values

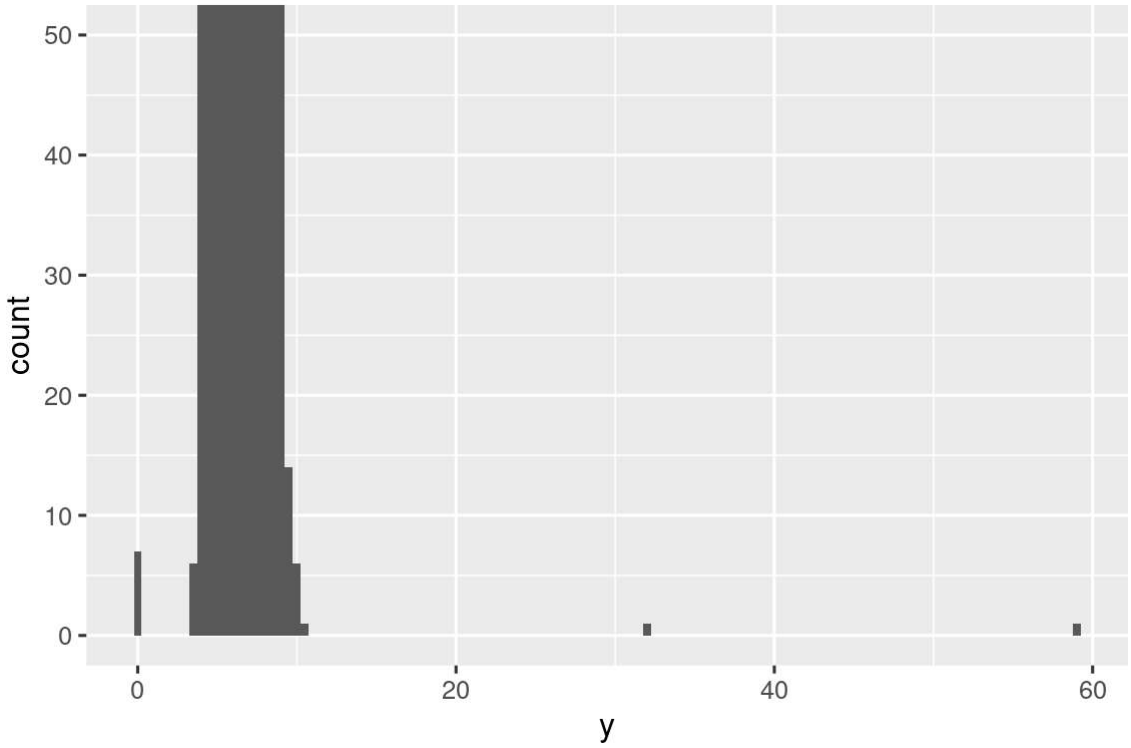
Outliers are observations that are unusual; data points that don't seem to fit the pattern. Sometimes outliers are data entry errors, sometimes they are simply values at the extremes that happened to be observed in this data collection, and other times they suggest important new discoveries. When you have a lot of data,

outliers are sometimes difficult to see in a histogram. For example, take the distribution of the `y` variable from the diamonds dataset. The only evidence of outliers is the unusually wide limits on the x-axis.



There are so many observations in the common bins that the rare bins are very short, making it very difficult to see them (although maybe if you stare intently at 0 you'll spot something). To make it easy to see the unusual values, we need to zoom to small values of the y-axis with `coord_cartesian()`:

```
ggplot(diamonds, aes(x = y)) +
  geom_histogram(binwidth = 0.5) +
  coord_cartesian(ylim = c(0, 50))
```



`coord_cartesian()` also has an `xlim()` argument for when you need to zoom into the x-axis. ggplot2 also has `xlim()` and `ylim()` functions that work slightly differently: they throw away the data outside the limits.

This allows us to see that there are three unusual values: 0, ~30, and ~60. We pluck them out with dplyr:

```
unusual <- diamonds |>
  filter(y < 3 | y > 20) |>
  select(price, x, y, z) |>
  arrange(y)
unusual
#> # A tibble: 9 × 4
#>   price     x     y     z
#>   <int> <dbl> <dbl> <dbl>
#> 1  5139  0      0    0
#> 2  6381  0      0    0
#> 3 12800  0      0    0
#> 4 15686  0      0    0
#> 5 18034  0      0    0
#> 6  2130  0      0    0
#> 7  2130  0      0    0
#> 8  2075  5.15  31.8  5.12
#> 9 12210  8.09  58.9  8.06
```

The `y` variable measures one of the three dimensions of these diamonds, in mm. We know that diamonds can't have a width of 0mm, so these values must be incorrect. By doing EDA, we have discovered missing data that was coded as 0, which we never would have found by simply searching for `NA`s. Going forward we might choose to re-code these values as `NA`s in order to prevent misleading calculations. We might also suspect that measurements of 32mm and 59mm are implausible: those diamonds are over an inch long, but don't cost hundreds of thousands of dollars!

It's good practice to repeat your analysis with and without the outliers. If they have minimal effect on the results, and you can't figure out why they're there, it's reasonable to omit them, and move on. However, if they have a substantial effect on your results, you shouldn't drop them without justification. You'll need to figure out what caused them (e.g., a data entry error) and disclose that you removed them in your write-up.

10.3.3 Exercises

1. Explore the distribution of each of the `x`, `y`, and `z` variables in `diamonds`. What do you learn? Think about a diamond and how you might decide which dimension is the length, width, and depth.

2. Explore the distribution of `price`. Do you discover anything unusual or surprising? (Hint: Carefully think about the `binwidth` and make sure you try a wide range of values.)

3. How many diamonds are 0.99 carat? How many are 1 carat? What do you think is the cause of the difference?

4. Compare and contrast `coord_cartesian()` vs. `xlim()` or `ylim()` when zooming in on a histogram. What happens if you leave `binwidth` unset? What happens if you try and zoom so only half a bar shows?

10.4 Unusual values

If you've encountered unusual values in your dataset, and simply want to move on to the rest of your analysis, you have two options.

1. Drop the entire row with the strange values:

   ```
   diamonds2 <- diamonds |>
     filter(between(y, 3, 20))
   ```

   We don't recommend this option because one invalid value doesn't imply that all the other values for that observation are also invalid. Additionally, if you have low quality data, by the time that you've applied this approach to every variable you might find that you don't have any data left!

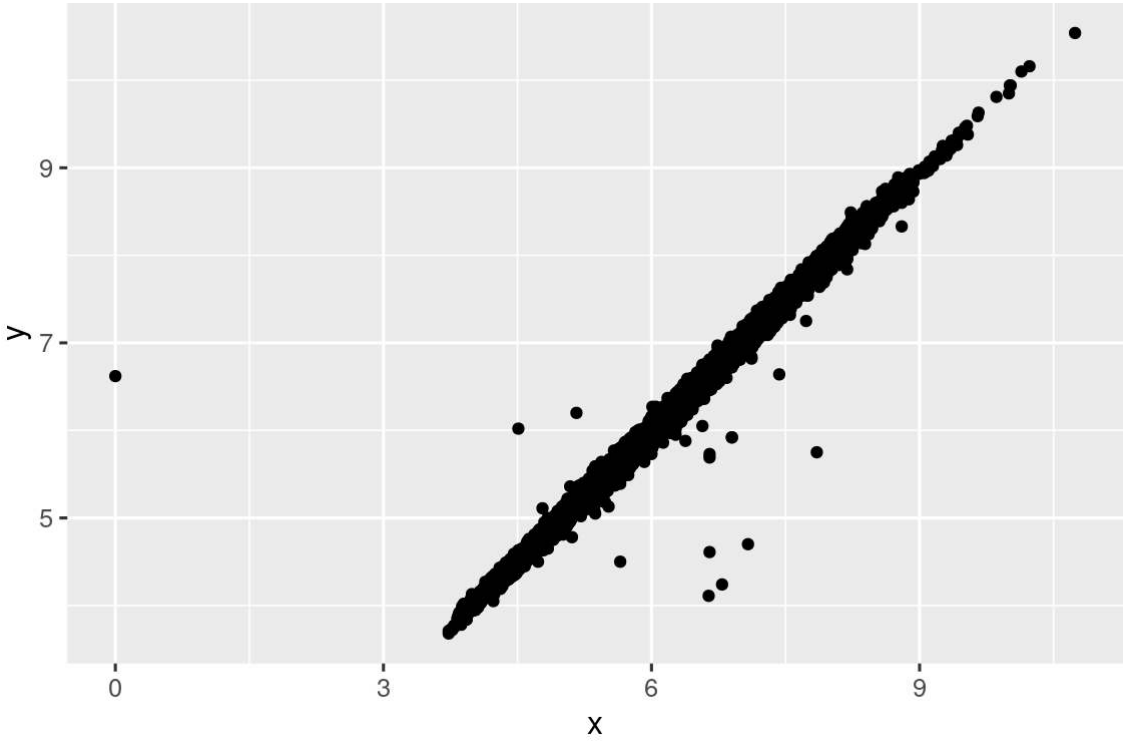
2. Instead, we recommend replacing the unusual values with missing values. The easiest way to do this is to use `mutate()` to replace the variable with a modified copy. You can use the `if_else()` function

to replace unusual values with `NA`:

```
diamonds2 <- diamonds |>
  mutate(y = if_else(y < 3 | y > 20, NA, y))
```

It's not obvious where you should plot missing values, so ggplot2 doesn't include them in the plot, but it does warn that they've been removed:

```
ggplot(diamonds2, aes(x = x, y = y)) +
  geom_point()
#> Warning: Removed 9 rows containing missing values or values outside the
scale range
#> (`geom_point()`).
```

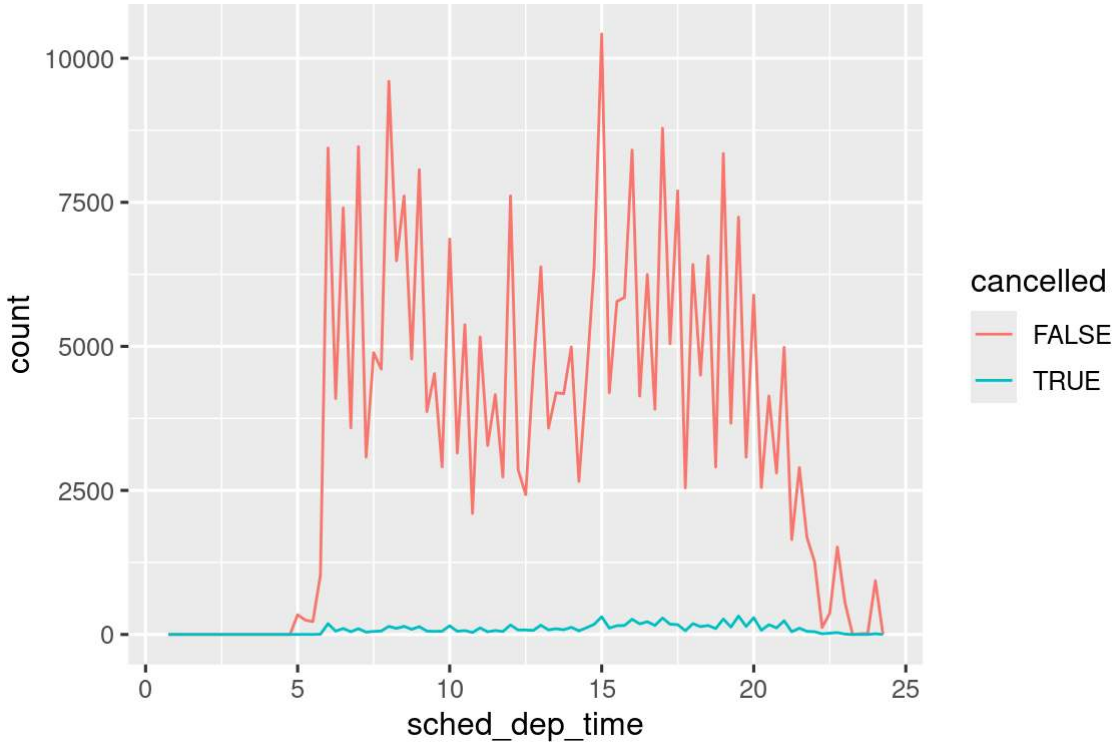


To suppress that warning, set `na.rm = TRUE`:

```
ggplot(diamonds2, aes(x = x, y = y)) +
  geom_point(na.rm = TRUE)
```

Other times you want to understand what makes observations with missing values different to observations with recorded values. For example, in `nycflights13::flights`[1], missing values in the `dep_time` variable indicate that the flight was cancelled. So you might want to compare the scheduled departure times for cancelled and non-cancelled times. You can do this by making a new variable, using `is.na()` to check if `dep_time` is missing.

```r
nycflights13::flights |>
  mutate(
    cancelled = is.na(dep_time),
    sched_hour = sched_dep_time %/% 100,
    sched_min = sched_dep_time %% 100,
    sched_dep_time = sched_hour + (sched_min / 60)
  ) |>
  ggplot(aes(x = sched_dep_time)) +
  geom_freqpoly(aes(color = cancelled), binwidth = 1/4)
```



However this plot isn't great because there are many more non-cancelled flights than cancelled flights. In the next section we'll explore some techniques for improving this comparison.

10.4.1 Exercises

1. What happens to missing values in a histogram? What happens to missing values in a bar chart? Why is there a difference in how missing values are handled in histograms and bar charts?

2. What does `na.rm = TRUE` do in `mean()` and `sum()`?

3. Recreate the frequency plot of `scheduled_dep_time` colored by whether the flight was cancelled or not. Also facet by the `cancelled` variable. Experiment with different values of the `scales` variable in the faceting function to mitigate the effect of more non-cancelled flights than cancelled flights.

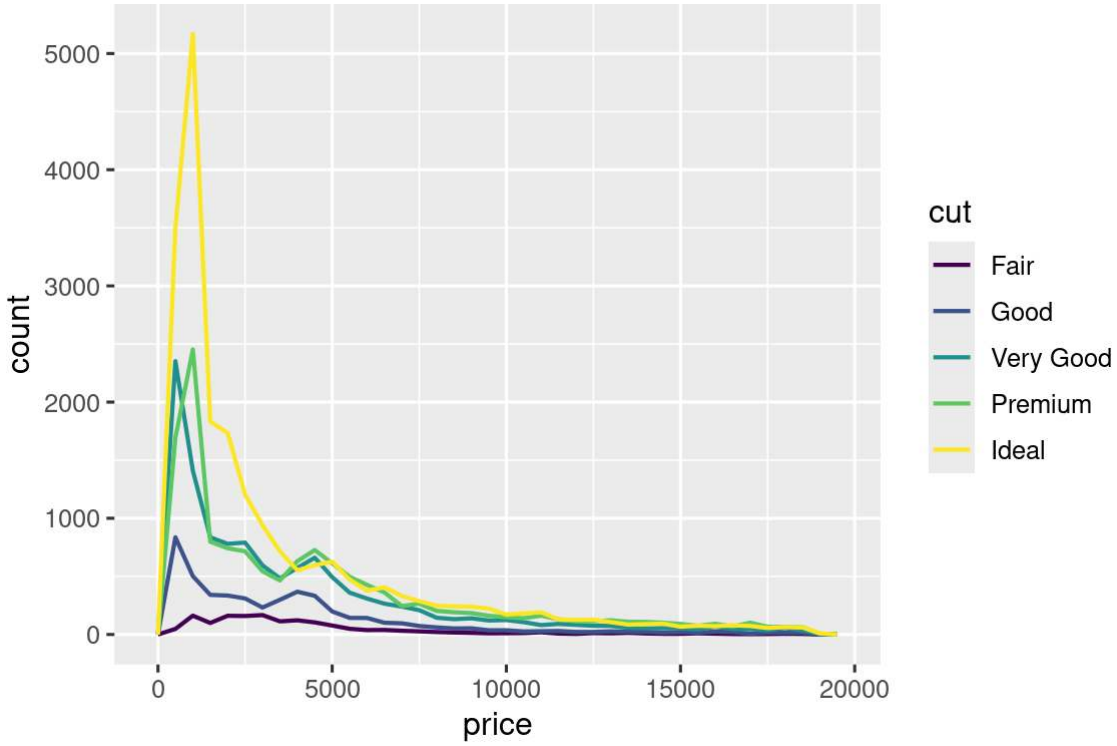
10.5 Covariation

If variation describes the behavior *within* a variable, covariation describes the behavior *between* variables. **Covariation** is the tendency for the values of two or more variables to vary together in a related way. The best way to spot covariation is to visualize the relationship between two or more variables.

10.5.1 A categorical and a numerical variable

For example, let's explore how the price of a diamond varies with its quality (measured by `cut`) using `geom_freqpoly()`:

```
ggplot(diamonds, aes(x = price)) +
  geom_freqpoly(aes(color = cut), binwidth = 500, linewidth = 0.75)
```

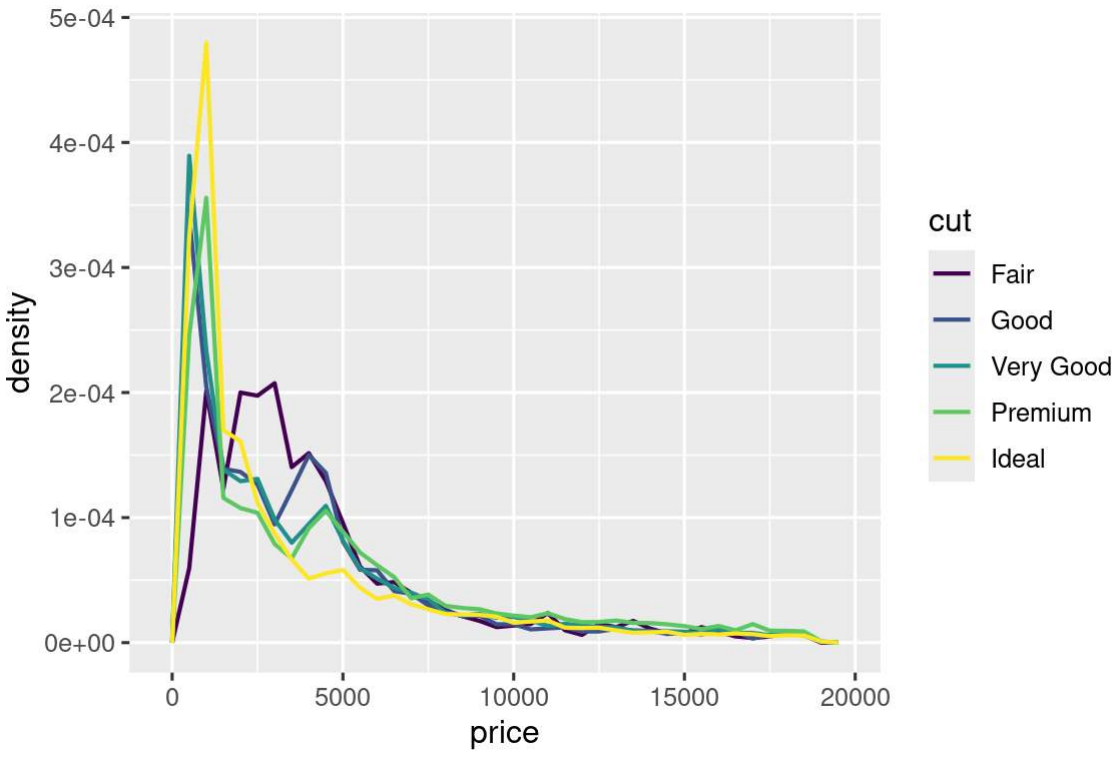


Note that ggplot2 uses an ordered color scale for `cut` because it's defined as an ordered factor variable in the data. You'll learn more about these in Section 16.6.

The default appearance of `geom_freqpoly()` is not that useful here because the height, determined by the overall count, differs so much across `cut`s, making it hard to see the differences in the shapes of their distributions.

To make the comparison easier we need to swap what is displayed on the y-axis. Instead of displaying count, we'll display the **density**, which is the count standardized so that the area under each frequency polygon is one.

```
ggplot(diamonds, aes(x = price, y = after_stat(density))) +
  geom_freqpoly(aes(color = cut), binwidth = 500, linewidth = 0.75)
```

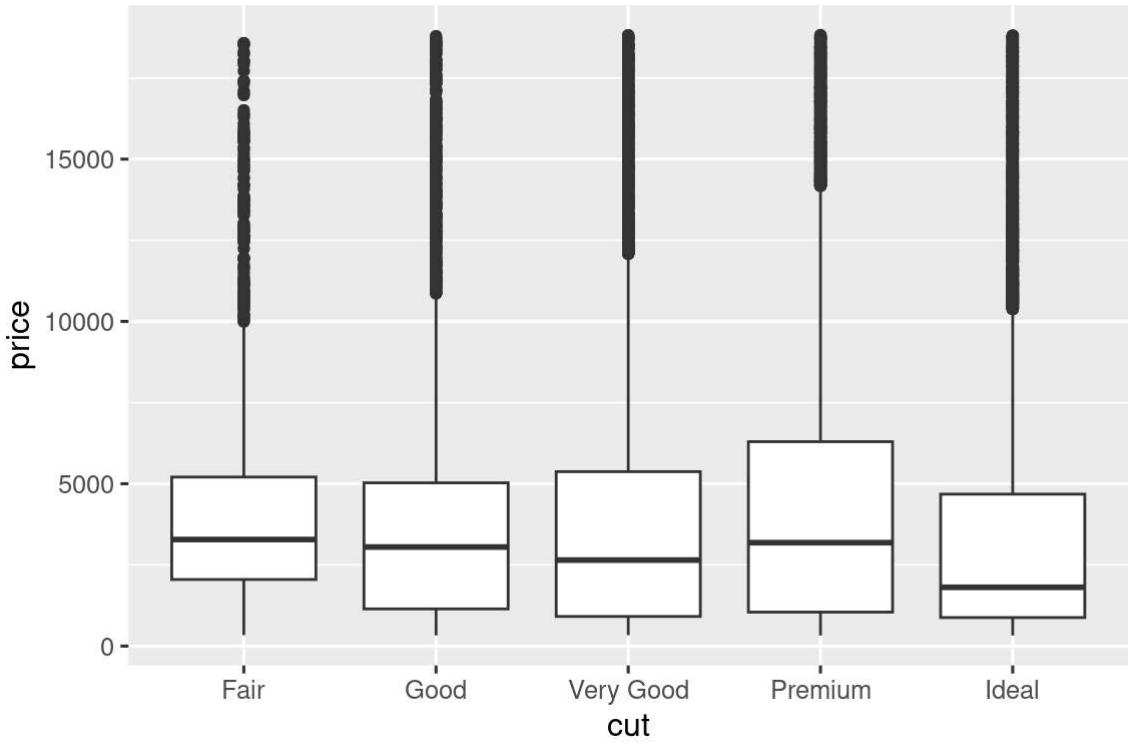


Note that we're mapping the density to `y`, but since `density` is not a variable in the `diamonds` dataset, we need to first calculate it. We use the `after_stat()` function to do so.

There's something rather surprising about this plot - it appears that fair diamonds (the lowest quality) have the highest average price! But maybe that's because frequency polygons are a little hard to interpret - there's a lot going on in this plot.

A visually simpler plot for exploring this relationship is using side-by-side boxplots.

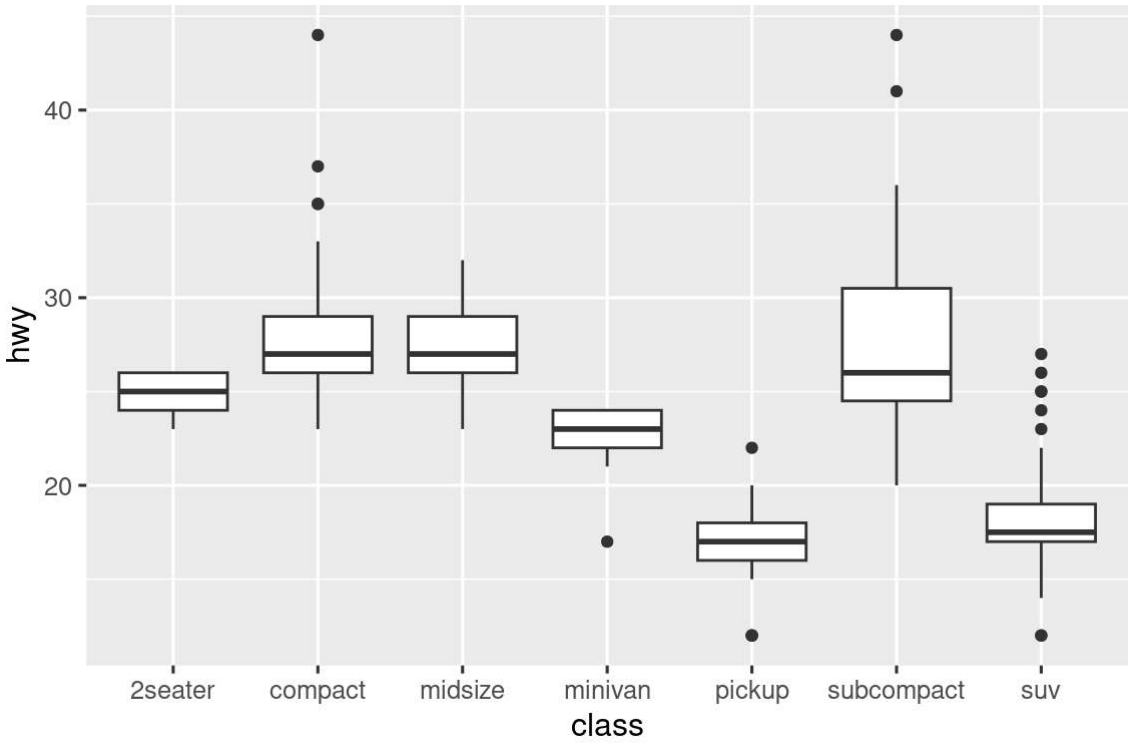
```
ggplot(diamonds, aes(x = cut, y = price)) +
  geom_boxplot()
```



We see much less information about the distribution, but the boxplots are much more compact so we can more easily compare them (and fit more on one plot). It supports the counter-intuitive finding that better quality diamonds are typically cheaper! In the exercises, you'll be challenged to figure out why.

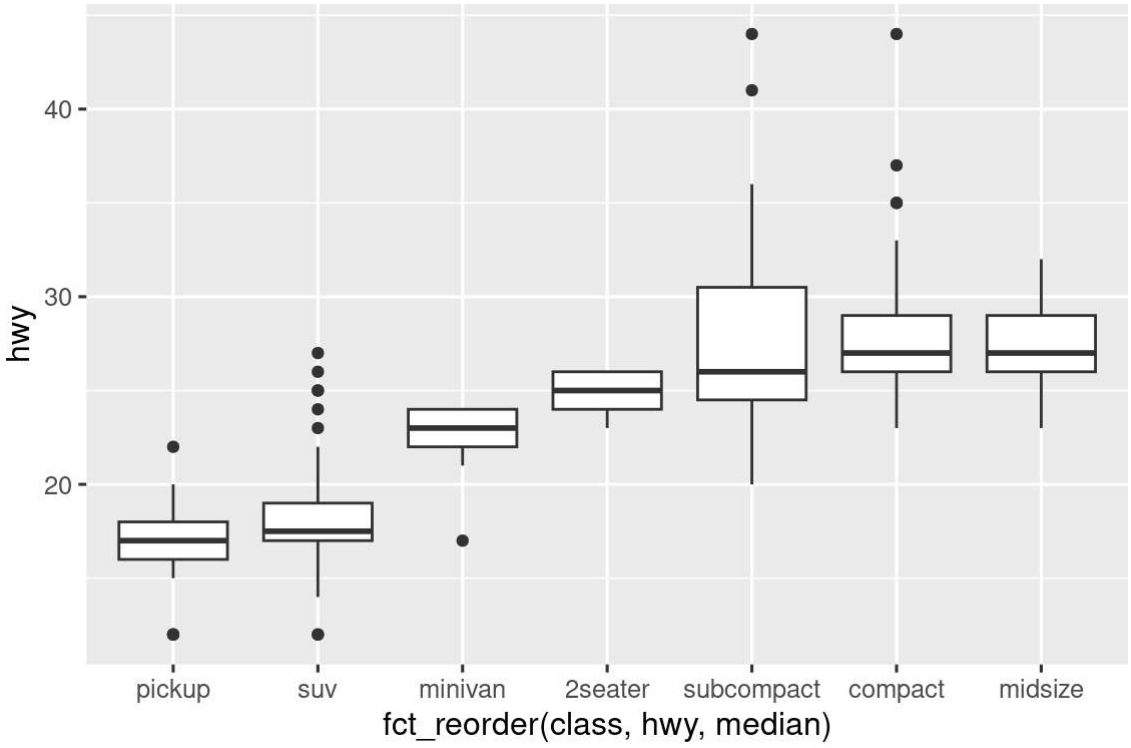
`cut` is an ordered factor: fair is worse than good, which is worse than very good and so on. Many categorical variables don't have such an intrinsic order, so you might want to reorder them to make a more informative display. One way to do that is with `fct_reorder()`. You'll learn more about that function in Section 16.4, but we want to give you a quick preview here because it's so useful. For example, take the `class` variable in the `mpg` dataset. You might be interested to know how highway mileage varies across classes:

```
ggplot(mpg, aes(x = class, y = hwy)) +
  geom_boxplot()
```



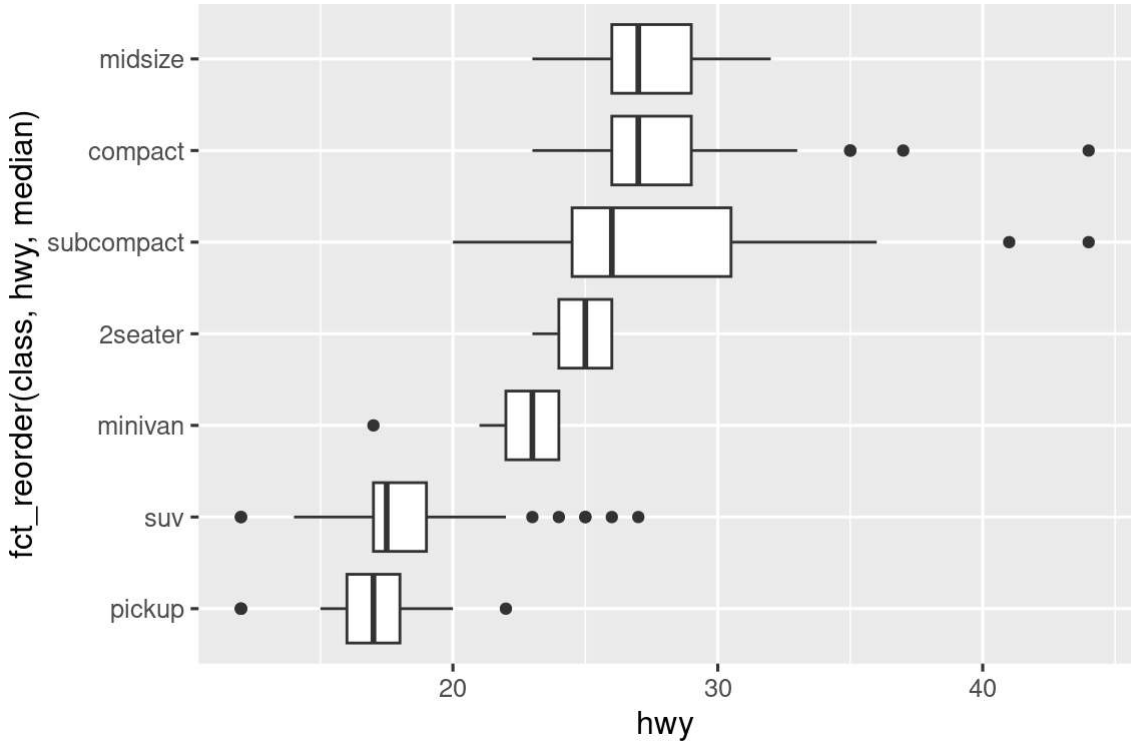
To make the trend easier to see, we can reorder `class` based on the median value of `hwy`:

```
ggplot(mpg, aes(x = fct_reorder(class, hwy, median), y = hwy)) +
  geom_boxplot()
```



If you have long variable names, `geom_boxplot()` will work better if you flip it 90°. You can do that by exchanging the x and y aesthetic mappings.

```
ggplot(mpg, aes(x = hwy, y = fct_reorder(class, hwy, median))) +
  geom_boxplot()
```



10.5.1.1 Exercises

1. Use what you've learned to improve the visualization of the departure times of cancelled vs. non-cancelled flights.

2. Based on EDA, what variable in the diamonds dataset appears to be most important for predicting the price of a diamond? How is that variable correlated with cut? Why does the combination of those two relationships lead to lower quality diamonds being more expensive?

3. Instead of exchanging the x and y variables, add `coord_flip()` as a new layer to the vertical boxplot to create a horizontal one. How does this compare to exchanging the variables?

4. One problem with boxplots is that they were developed in an era of much smaller datasets and tend to display a prohibitively large number of "outlying values". One approach to remedy this problem is the letter value plot. Install the lvplot package, and try using `geom_lv()` to display the distribution of price vs. cut. What do you learn? How do you interpret the plots?

5. Create a visualization of diamond prices vs. a categorical variable from the `diamonds` dataset using `geom_violin()`, then a faceted `geom_histogram()`, then a colored `geom_freqpoly()`, and then

a colored `geom_density()`. Compare and contrast the four plots. What are the pros and cons of each method of visualizing the distribution of a numerical variable based on the levels of a categorical variable?

6. If you have a small dataset, it's sometimes useful to use `geom_jitter()` to avoid overplotting to more easily see the relationship between a continuous and categorical variable. The ggbeeswarm package provides a number of methods similar to `geom_jitter()`. List them and briefly describe what each one does.

10.5.2 Two categorical variables

To visualize the covariation between categorical variables, you'll need to count the number of observations for each combination of levels of these categorical variables. One way to do that is to rely on the built-in `geom_count()`:

The size of each circle in the plot displays how many observations occurred at each combination of values. Covariation will appear as a strong correlation between specific x values and specific y values.

Another approach for exploring the relationship between these variables is computing the counts with dplyr:

```
diamonds |>
  count(color, cut)
#> # A tibble: 35 × 3
#>   color cut           n
#>   <ord> <ord>     <int>
#> 1 D     Fair        163
#> 2 D     Good        662
#> 3 D     Very Good  1513
#> 4 D     Premium    1603
#> 5 D     Ideal      2834
#> 6 E     Fair        224
#> # ℹ 29 more rows
```

Then visualize with `geom_tile()` and the fill aesthetic:

If the categorical variables are unordered, you might want to use the seriation package to simultaneously reorder the rows and columns in order to more clearly reveal interesting patterns. For larger plots, you might want to try the heatmaply package, which creates interactive plots.

10.5.2.1 Exercises

1. How could you rescale the count dataset above to more clearly show the distribution of cut within color, or color within cut?

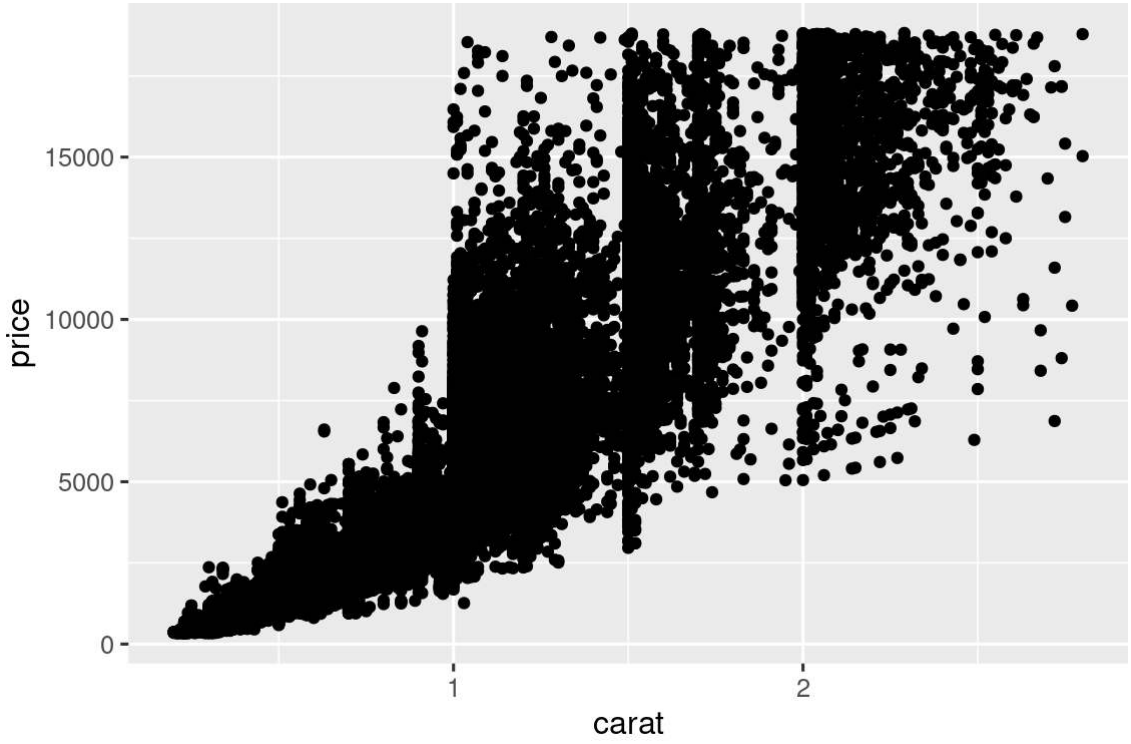
2. What different data insights do you get with a segmented bar chart if color is mapped to the `x` aesthetic and `cut` is mapped to the `fill` aesthetic? Calculate the counts that fall into each of the segments.

3. Use `geom_tile()` together with dplyr to explore how average flight departure delays vary by destination and month of year. What makes the plot difficult to read? How could you improve it?

10.5.3 Two numerical variables

You've already seen one great way to visualize the covariation between two numerical variables: draw a scatterplot with `geom_point()`. You can see covariation as a pattern in the points. For example, you can see a positive relationship between the carat size and price of a diamond: diamonds with more carats have a higher price. The relationship is exponential.

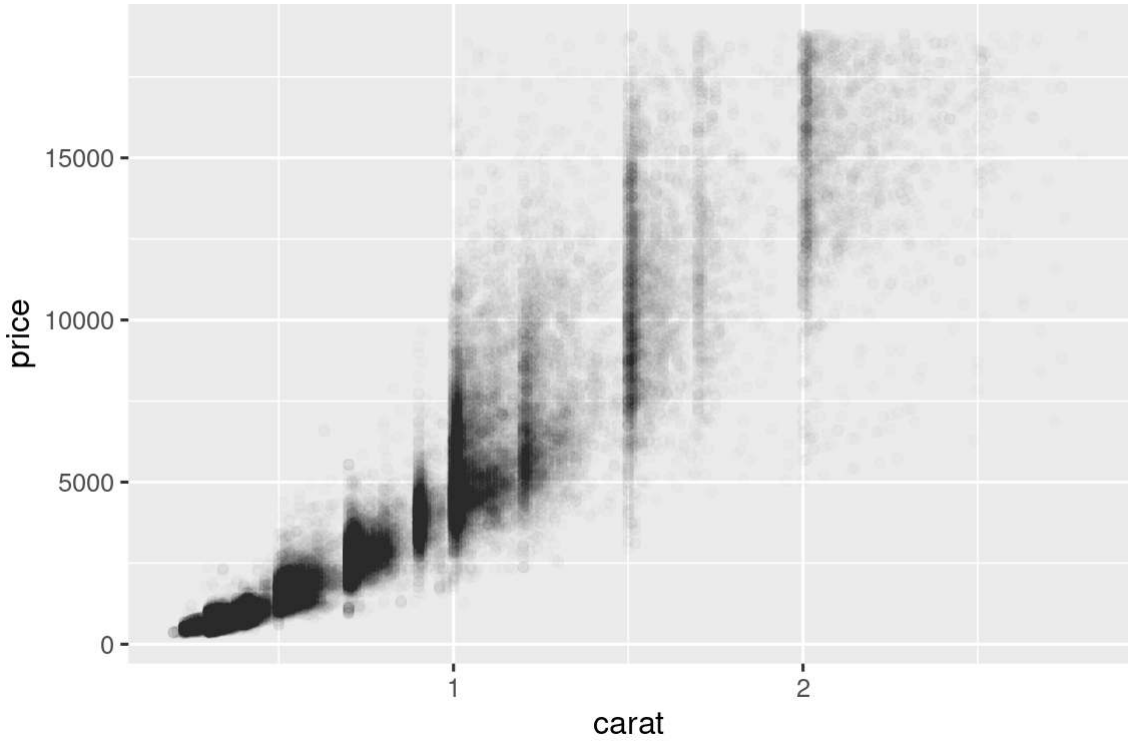
```
ggplot(smaller, aes(x = carat, y = price)) +
  geom_point()
```



(In this section we'll use the `smaller` dataset to stay focused on the bulk of the diamonds that are smaller than 3 carats)

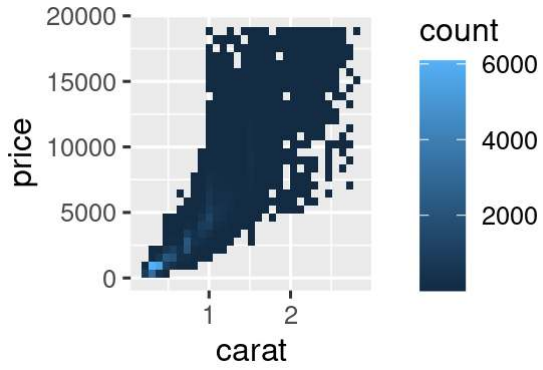
Scatterplots become less useful as the size of your dataset grows, because points begin to overplot, and pile up into areas of uniform black, making it hard to judge differences in the density of the data across the 2-dimensional space as well as making it hard to spot the trend. You've already seen one way to fix the problem: using the `alpha` aesthetic to add transparency.

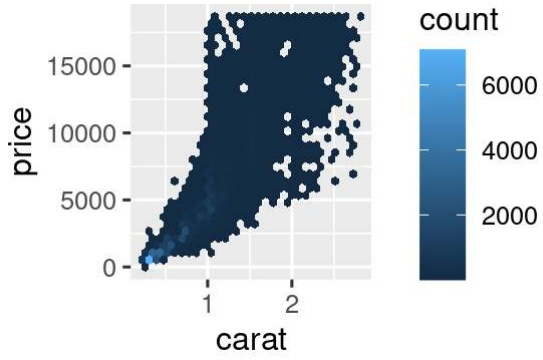
```
ggplot(smaller, aes(x = carat, y = price)) +
  geom_point(alpha = 1 / 100)
```



But using transparency can be challenging for very large datasets. Another solution is to use bin. Previously you used `geom_histogram()` and `geom_freqpoly()` to bin in one dimension. Now you'll learn how to use `geom_bin2d()` and `geom_hex()` to bin in two dimensions.

`geom_bin2d()` and `geom_hex()` divide the coordinate plane into 2d bins and then use a fill color to display how many points fall into each bin. `geom_bin2d()` creates rectangular bins. `geom_hex()` creates hexagonal bins. You will need to install the hexbin package to use `geom_hex()`.





Another option is to bin one continuous variable so it acts like a categorical variable. Then you can use one of the techniques for visualizing the combination of a categorical and a continuous variable that you learned about. For example, you could bin `carat` and then for each group, display a boxplot:

`cut_width(x, width)`, as used above, divides `x` into bins of width `width`. By default, boxplots look roughly the same (apart from number of outliers) regardless of how many observations there are, so it's difficult to tell that each boxplot summaries a different number of points. One way to show that is to make the width of the boxplot proportional to the number of points with `varwidth = TRUE`.

10.5.3.1 Exercises

1. Instead of summarizing the conditional distribution with a boxplot, you could use a frequency polygon. What do you need to consider when using `cut_width()` vs. `cut_number()`? How does that impact a visualization of the 2d distribution of `carat` and `price`?

2. Visualize the distribution of `carat`, partitioned by `price`.

3. How does the price distribution of very large diamonds compare to small diamonds? Is it as you expect, or does it surprise you?

4. Combine two of the techniques you've learned to visualize the combined distribution of cut, carat, and price.

5. Two dimensional plots reveal outliers that are not visible in one dimensional plots. For example, some points in the following plot have an unusual combination of `x` and `y` values, which makes the points outliers even though their `x` and `y` values appear normal when examined separately. Why is a scatterplot a better display than a binned plot for this case?

6. Instead of creating boxes of equal width with `cut_width()`, we could create boxes that contain roughly equal number of points with `cut_number()`. What are the advantages and disadvantages of this approach?

10.6 Patterns and models

If a systematic relationship exists between two variables it will appear as a pattern in the data. If you spot a pattern, ask yourself:

- Could this pattern be due to coincidence (i.e. random chance)?
- How can you describe the relationship implied by the pattern?
- How strong is the relationship implied by the pattern?
- What other variables might affect the relationship?
- Does the relationship change if you look at individual subgroups of the data?

Patterns in your data provide clues about relationships, i.e., they reveal covariation. If you think of variation as a phenomenon that creates uncertainty, covariation is a phenomenon that reduces it. If two variables covary, you can use the values of one variable to make better predictions about the values of the second. If the covariation is due to a causal relationship (a special case), then you can use the value of one variable to control the value of the second.

Models are a tool for extracting patterns out of data. For example, consider the diamonds data. It's hard to understand the relationship between cut and price, because cut and carat, and carat and price are tightly related. It's possible to use a model to remove the very strong relationship between price and carat so we can explore the subtleties that remain. The following code fits a model that predicts `price` from `carat` and then computes the residuals (the difference between the predicted value and the actual value). The residuals give us a view of the price of the diamond, once the effect of carat has been removed. Note that instead of using the raw values of `price` and `carat`, we log transform them first, and fit a model to the log-transformed values. Then, we exponentiate the residuals to put them back in the scale of raw prices.

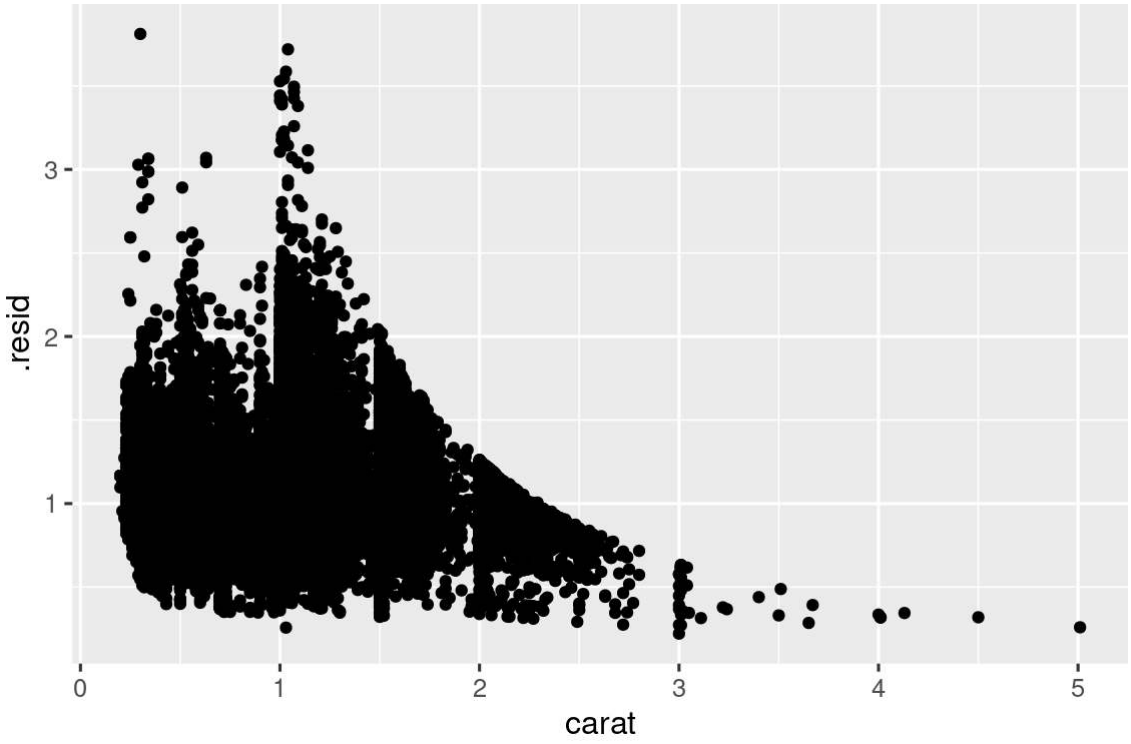
```
library(tidymodels)

diamonds <- diamonds |>
  mutate(
    log_price = log(price),
    log_carat = log(carat)
  )

diamonds_fit <- linear_reg() |>
  fit(log_price ~ log_carat, data = diamonds)

diamonds_aug <- augment(diamonds_fit, new_data = diamonds) |>
  mutate(.resid = exp(.resid))


  geom_point()
```



Once you've removed the strong relationship between carat and price, you can see what you expect in the relationship between cut and price: relative to their size, better quality diamonds are more expensive.

We're not discussing modelling in this book because understanding what models are and how they work is easiest once you have tools of data wrangling and programming in hand.

10.7 Summary

In this chapter you've learned a variety of tools to help you understand the variation within your data. You've seen techniques that work with a single variable at a time and with a pair of variables. This might seem painfully restrictive if you have tens or hundreds of variables in your data, but they're the foundation upon which all other techniques are built.

In the next chapter, we'll focus on the tools we can use to communicate our results.

11 Communication

11.1 Introduction

In Chapter 10, you learned how to use plots as tools for *exploration*. When you make exploratory plots, you know—even before looking—which variables the plot will display. You made each plot for a purpose, could quickly look at it, and then move on to the next plot. In the course of most analyses, you'll produce tens or hundreds of plots, most of which are immediately thrown away.

Now that you understand your data, you need to *communicate* your understanding to others. Your audience will likely not share your background knowledge and will not be deeply invested in the data. To help others quickly build up a good mental model of the data, you will need to invest considerable effort in making your plots as self-explanatory as possible. In this chapter, you'll learn some of the tools that ggplot2 provides to do so.

This chapter focuses on the tools you need to create good graphics. We assume that you know what you want, and just need to know how to do it. For that reason, we highly recommend pairing this chapter with a good general visualization book. We particularly like The Truthful Art, by Albert Cairo. It doesn't teach the mechanics of creating visualizations, but instead focuses on what you need to think about in order to create effective graphics.

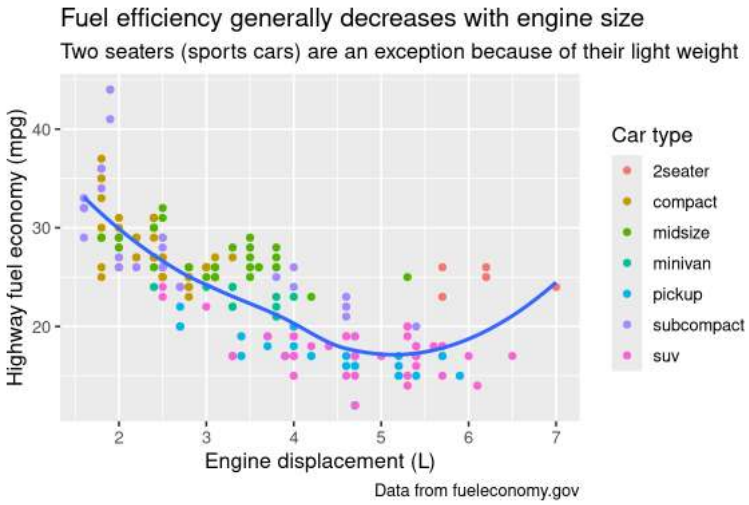
11.1.1 Prerequisites

In this chapter, we'll focus once again on ggplot2. We'll also use a little dplyr for data manipulation, **scales** to override the default breaks, labels, transformations and palettes, and a few ggplot2 extension packages, including **ggrepel** (https://ggrepel.slowkow.com) by Kamil Slowikowski and **patchwork** (https://patchwork.data-imaginist.com) by Thomas Lin Pedersen. Don't forget that you'll need to install those packages with `install.packages()` if you don't already have them.

11.2 Labels

The easiest place to start when turning an exploratory graphic into an expository graphic is with good labels. You add labels with the `labs()` function.

```
  geom_point(aes(color = class)) +
  geom_smooth(se = FALSE) +
  labs(
    x = "Engine displacement (L)",
    y = "Highway fuel economy (mpg)",
    color = "Car type",
    title = "Fuel efficiency generally decreases with engine size",
    subtitle = "Two seaters (sports cars) are an exception because of their light
weight",
    caption = "Data from fueleconomy.gov"
  )
```



The purpose of a plot title is to summarize the main finding. Avoid titles that just describe what the plot is, e.g., "A scatterplot of engine displacement vs. fuel economy".

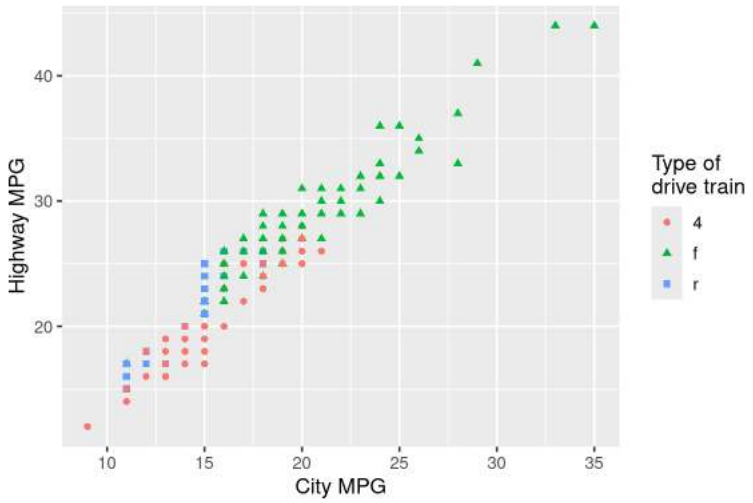
If you need to add more text, there are two other useful labels: `subtitle` adds additional detail in a smaller font beneath the title and `caption` adds text at the bottom right of the plot, often used to describe the source of the data. You can also use `labs()` to replace the axis and legend titles. It's usually a good idea to replace short variable names with more detailed descriptions, and to include the units.

It's possible to use mathematical equations instead of text strings. Just switch `""` out for `quote()` and read about the available options in `?plotmath`:

11.2.1 Exercises

1. Create one plot on the fuel economy data with customized `title`, `subtitle`, `caption`, `x`, `y`, and `color` labels.

2. Recreate the following plot using the fuel economy data. Note that both the colors and shapes of points vary by type of drive train.



3. Take an exploratory graphic that you've created in the last month, and add informative titles to make it easier for others to understand.

11.3 Annotations

In addition to labelling major components of your plot, it's often useful to label individual observations or groups of observations. The first tool you have at your disposal is `geom_text()`. `geom_text()` is similar to `geom_point()`, but it has an additional aesthetic: `label`. This makes it possible to add textual labels to your plots.

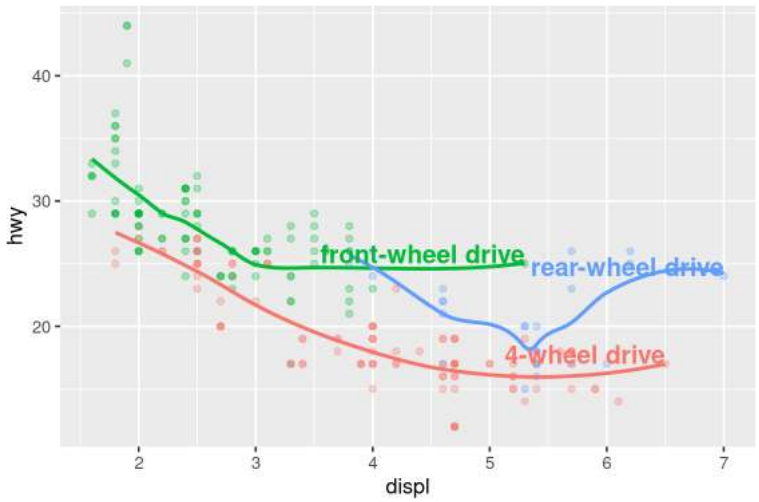
There are two possible sources of labels. First, you might have a tibble that provides labels. In the following plot we pull out the cars with the highest engine size in each drive type and save their information as a new data frame called `label_info`.

```
label_info <- mpg |>
  group_by(drv) |>
  arrange(desc(displ)) |>
  slice_head(n = 1) |>
  mutate(
    drive_type = case_when(
      drv == "f" ~ "front-wheel drive",
      drv == "r" ~ "rear-wheel drive",
      drv == "4" ~ "4-wheel drive"
    )
  ) |>
  select(displ, hwy, drv, drive_type)

label_info
#> # A tibble: 3 × 4
#> # Groups:   drv [3]
#>   displ   hwy drv   drive_type
#>   <dbl> <int> <chr> <chr>
#> 1   6.5    17 4     4-wheel drive
#> 2   5.3    25 f     front-wheel drive
#> 3   7      24 r     rear-wheel drive
```

Then, we use this new data frame to directly label the three groups to replace the legend with labels placed directly on the plot. Using the `fontface` and `size` arguments we can customize the look of the text labels. They're larger than the rest of the text on the plot and bolded. (`theme(legend.position = "none")` turns all the legends off — we'll talk about it more shortly.)

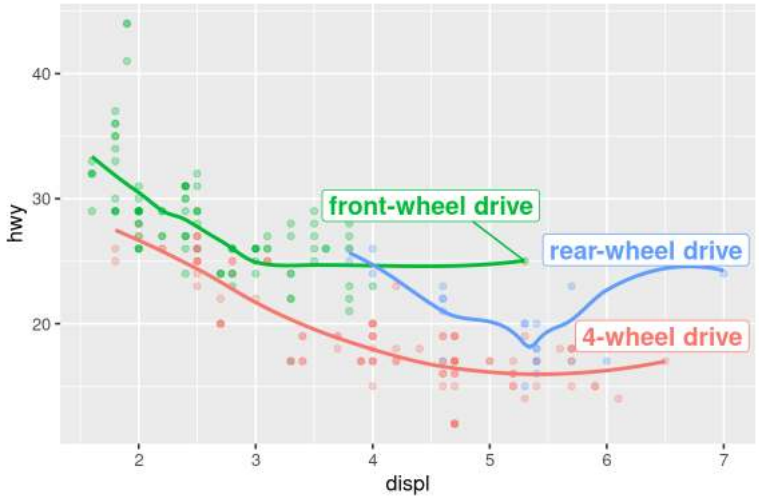
```
ggplot(mpg, aes(x = displ, y = hwy, color = drv)) +
  geom_point(alpha = 0.3) +
  geom_smooth(se = FALSE) +
  geom_text(
    data = label_info,
    aes(x = displ, y = hwy, label = drive_type),
    fontface = "bold", size = 5, hjust = "right", vjust = "bottom"
  ) +
  theme(legend.position = "none")
#> `geom_smooth()` using method = 'loess' and formula = 'y ~ x'
```



Note the use of `hjust` (horizontal justification) and `vjust` (vertical justification) to control the alignment of the label.

However the annotated plot we made above is hard to read because the labels overlap with each other, and with the points. We can use the `geom_label_repel()` function from the ggrepel package to address both of these issues. This useful package will automatically adjust labels so that they don't overlap:

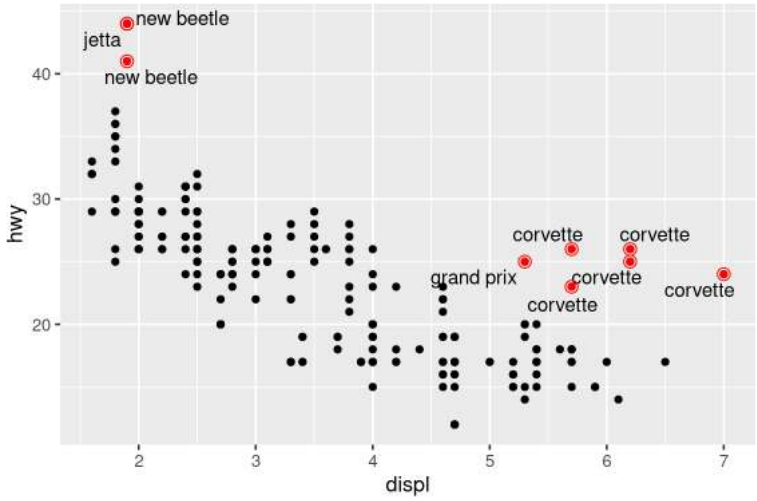
```
ggplot(mpg, aes(x = displ, y = hwy, color = drv)) +
  geom_point(alpha = 0.3) +
  geom_smooth(se = FALSE) +
  geom_label_repel(
    data = label_info,
    aes(x = displ, y = hwy, label = drive_type),
    fontface = "bold", size = 5, nudge_y = 2
  ) +
  theme(legend.position = "none")
#> `geom_smooth()` using method = 'loess' and formula = 'y ~ x'
```



You can also use the same idea to highlight certain points on a plot with `geom_text_repel()` from the ggrepel package. Note another handy technique used here: we added a second layer of large, hollow points to further highlight the labelled points.

```
potential_outliers <- mpg |>
  filter(hwy > 40 | (hwy > 20 & displ > 5))

ggplot(mpg, aes(x = displ, y = hwy)) +
  geom_point() +
  geom_text_repel(data = potential_outliers, aes(label = model)) +
  geom_point(data = potential_outliers, color = "red") +
  geom_point(
    data = potential_outliers,
    color = "red", size = 3, shape = "circle open"
  )
```



Remember, in addition to `geom_text()` and `geom_label()`, you have many other geoms in ggplot2 available to help annotate your plot. A couple ideas:

- Use `geom_hline()` and `geom_vline()` to add reference lines. We often make them thick (`linewidth = 2`) and white (`color = white`), and draw them underneath the primary data layer. That makes them easy to see, without drawing attention away from the data.
- Use `geom_rect()` to draw a rectangle around points of interest. The boundaries of the rectangle are defined by aesthetics `xmin`, `xmax`, `ymin`, `ymax`. Alternatively, look into the ggforce package, specifically `geom_mark_hull()`, which allows you to annotate subsets of points with hulls.
- Use `geom_segment()` with the `arrow` argument to draw attention to a point with an arrow. Use aesthetics `x` and `y` to define the starting location, and `xend` and `yend` to define the end location.

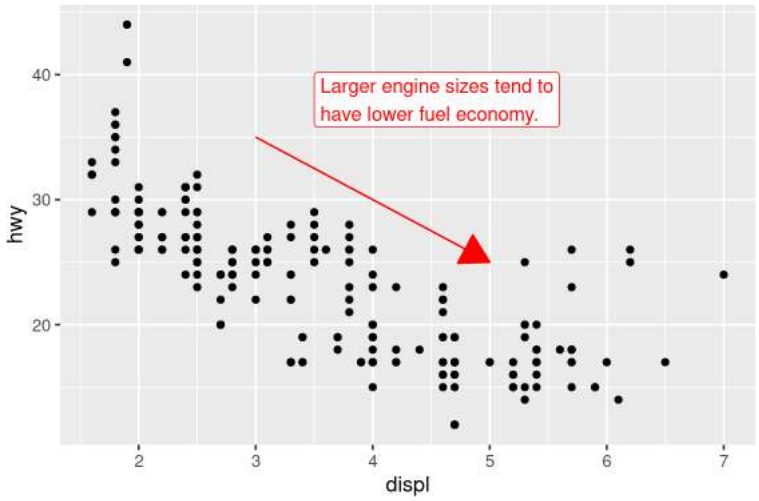
Another handy function for adding annotations to plots is `annotate()`. As a rule of thumb, geoms are generally useful for highlighting a subset of the data while `annotate()` is useful for adding one or few annotation elements to a plot.

To demonstrate using `annotate()`, let's create some text to add to our plot. The text is a bit long, so we'll use `stringr::str_wrap()` to automatically add line breaks to it given the number of characters you want per line:

```
trend_text <- "Larger engine sizes tend to have lower fuel economy." |>
  str_wrap(width = 30)
trend_text
#> [1] "Larger engine sizes tend to\nhave lower fuel economy."
```

Then, we add two layers of annotation: one with a label geom and the other with a segment geom. The `x` and `y` aesthetics in both define where the annotation should start, and the `xend` and `yend` aesthetics in the segment annotation define the end location of the segment. Note also that the segment is styled as an arrow.

```
ggplot(mpg, aes(x = displ, y = hwy)) +
  geom_point() +
  annotate(
    geom = "label", x = 3.5, y = 38,
    label = trend_text,
    hjust = "left", color = "red"
  ) +
  annotate(
    geom = "segment",
    x = 3, y = 35, xend = 5, yend = 25, color = "red",
    arrow = arrow(type = "closed")
  )
```



Annotation is a powerful tool for communicating main takeaways and interesting features of your visualizations. The only limit is your imagination (and your patience with positioning annotations to be aesthetically pleasing)!

11.3.1 Exercises

1. Use `geom_text()` with infinite positions to place text at the four corners of the plot.
2. Use `annotate()` to add a point geom in the middle of your last plot without having to create a tibble. Customize the shape, size, or color of the point.
3. How do labels with `geom_text()` interact with faceting? How can you add a label to a single facet? How can you put a different label in each facet? (Hint: Think about the dataset that is being passed to `geom_text()`.)
4. What arguments to `geom_label()` control the appearance of the background box?
5. What are the four arguments to `arrow()`? How do they work? Create a series of plots that demonstrate the most important options.

11.4 Scales

The third way you can make your plot better for communication is to adjust the scales. Scales control how the aesthetic mappings manifest visually.

11.4.1 Default scales

Normally, ggplot2 automatically adds scales for you. For example, when you type:

```
ggplot(mpg, aes(x = displ, y = hwy)) +
  geom_point(aes(color = class))
```

ggplot2 automatically adds default scales behind the scenes:

```
ggplot(mpg, aes(x = displ, y = hwy)) +
  geom_point(aes(color = class)) +
  scale_x_continuous() +
  scale_y_continuous() +
  scale_color_discrete()
```

Note the naming scheme for scales: `scale_` followed by the name of the aesthetic, then `_`, then the name of the scale. The default scales are named according to the type of variable they align with: continuous, discrete, datetime, or date. `scale_x_continuous()` puts the numeric values from `displ` on a continuous number line on the x-axis, `scale_color_discrete()` chooses colors for each of the `class` of car, etc. There are lots of non-default scales which you'll learn about below.

The default scales have been carefully chosen to do a good job for a wide range of inputs. Nevertheless, you might want to override the defaults for two reasons:

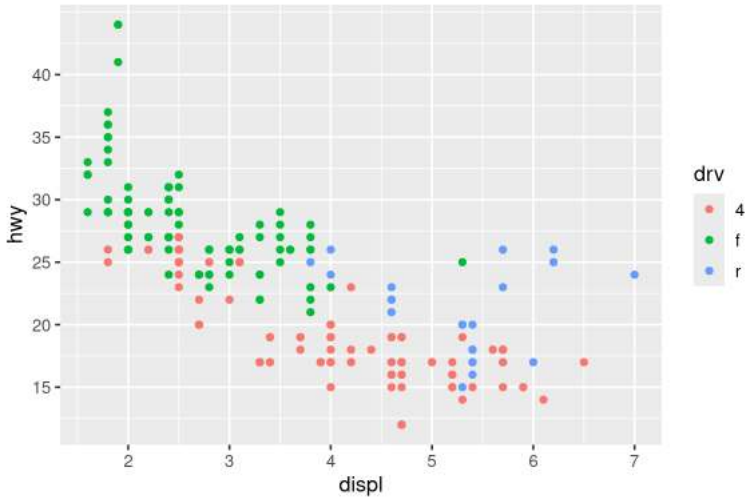
- You might want to tweak some of the parameters of the default scale. This allows you to do things like change the breaks on the axes, or the key labels on the legend.
- You might want to replace the scale altogether, and use a completely different algorithm. Often you can do better than the default because you know more about the data.

11.4.2 Axis ticks and legend keys

Collectively axes and legends are called **guides**. Axes are used for x and y aesthetics; legends are used for everything else.

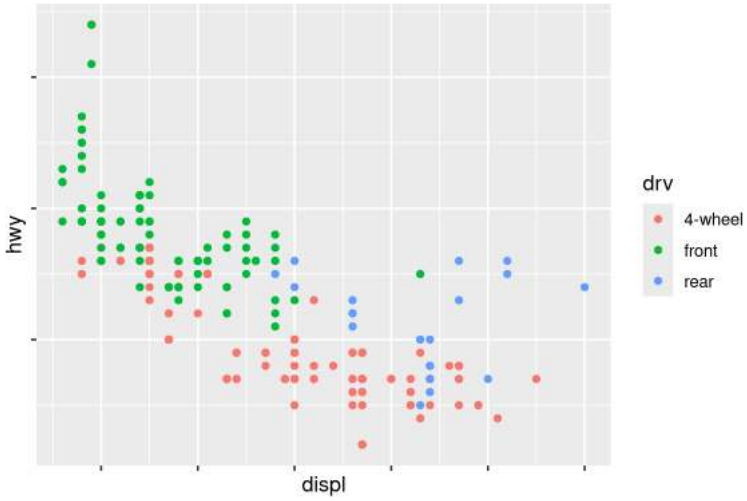
There are two primary arguments that affect the appearance of the ticks on the axes and the keys on the legend: `breaks` and `labels`. Breaks controls the position of the ticks, or the values associated with the keys. Labels controls the text label associated with each tick/key. The most common use of `breaks` is to override the default choice:

```
ggplot(mpg, aes(x = displ, y = hwy, color = drv)) +
  geom_point() +
  scale_y_continuous(breaks = seq(15, 40, by = 5))
```



You can use `labels` in the same way (a character vector the same length as `breaks`), but you can also set it to `NULL` to suppress the labels altogether. This can be useful for maps, or for publishing plots where you can't share the absolute numbers. You can also use `breaks` and `labels` to control the appearance of legends. For discrete scales for categorical variables, `labels` can be a named list of the existing levels names and the desired labels for them.

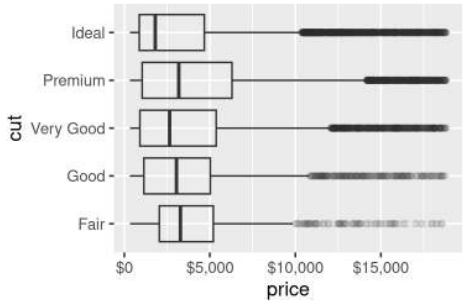
```
ggplot(mpg, aes(x = displ, y = hwy, color = drv)) +
  geom_point() +
  scale_x_continuous(labels = NULL) +
  scale_y_continuous(labels = NULL) +
  scale_color_discrete(labels = c("4" = "4-wheel", "f" = "front", "r" = "rear"))
```

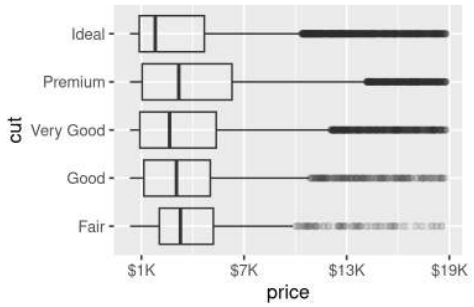


The `labels` argument coupled with labelling functions from the scales package is also useful for formatting numbers as currency, percent, etc. The plot on the left shows default labelling with `label_dollar()`, which adds a dollar sign as well as a thousand separator comma. The plot on the right adds further customization by dividing dollar values by 1,000 and adding a suffix "K" (for "thousands") as well as adding custom breaks. Note that `breaks` is in the original scale of the data.

```
# Left
ggplot(diamonds, aes(x = price, y = cut)) +
  geom_boxplot(alpha = 0.05) +
  scale_x_continuous(labels = label_dollar())

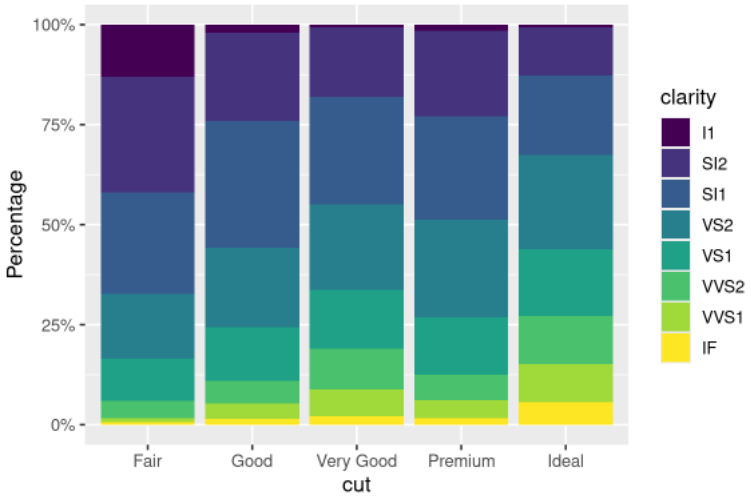
# Right
ggplot(diamonds, aes(x = price, y = cut)) +
  geom_boxplot(alpha = 0.05) +
  scale_x_continuous(
    labels = label_dollar(scale = 1/1000, suffix = "K"),
    breaks = seq(1000, 19000, by = 6000)
  )
```





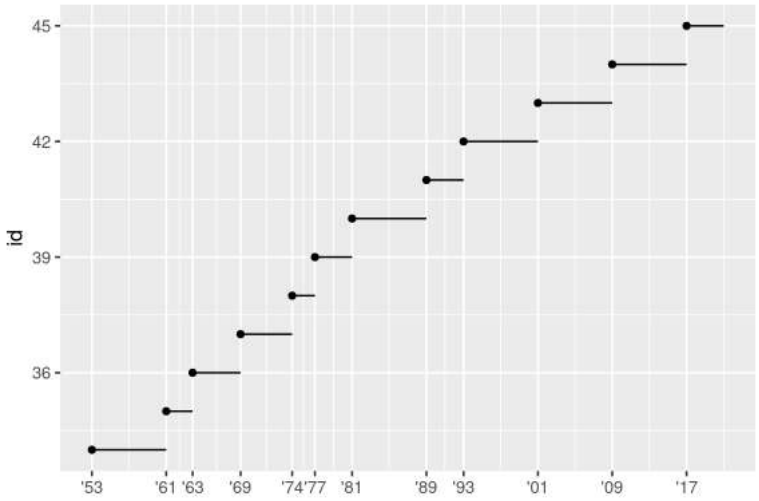
Another handy label function is `label_percent()`:

```
ggplot(diamonds, aes(x = cut, fill = clarity)) +
  geom_bar(position = "fill") +
  scale_y_continuous(name = "Percentage", labels = label_percent())
```



Another use of `breaks` is when you have relatively few data points and want to highlight exactly where the observations occur. For example, take this plot that shows when each US president started and ended their term.

```
presidential |>
  mutate(id = 33 + row_number()) |>
  ggplot(aes(x = start, y = id)) +
  geom_point() +
  geom_segment(aes(xend = end, yend = id)) +
  scale_x_date(name = NULL, breaks = presidential$start, date_labels = "'%y")
```



Note that for the `breaks` argument we pulled out the `start` variable as a vector with `presidential$start` because we can't do an aesthetic mapping for this argument. Also note that the specification of breaks and labels for date and datetime scales is a little different:

- `date_labels` takes a format specification, in the same form as `parse_datetime()`.
- `date_breaks` (not shown here), takes a string like "2 days" or "1 month".

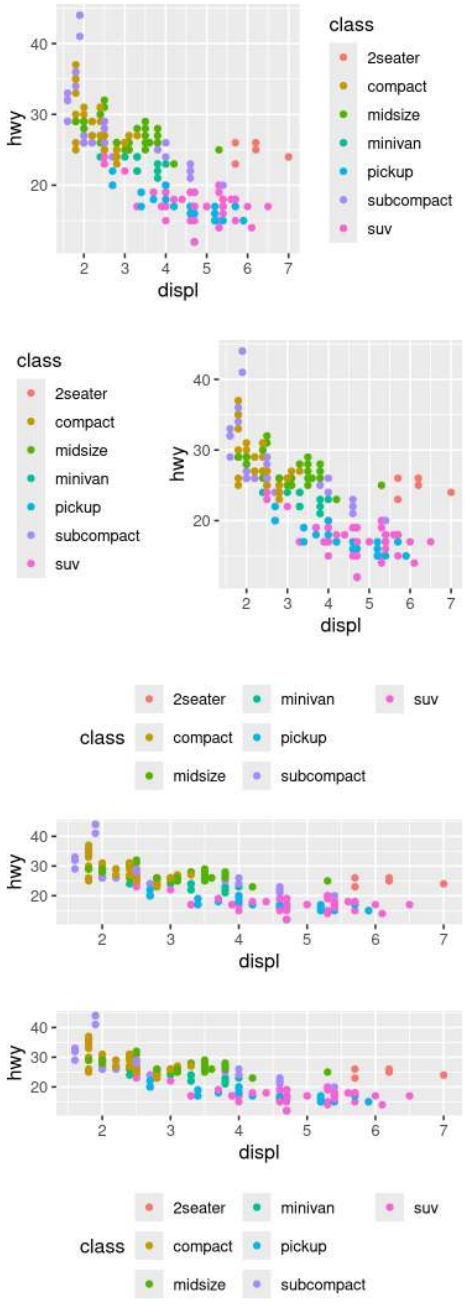
11.4.3 Legend layout

You will most often use `breaks` and `labels` to tweak the axes. While they both also work for legends, there are a few other techniques you are more likely to use.

To control the overall position of the legend, you need to use a `theme()` setting. We'll come back to themes at the end of the chapter, but in brief, they control the non-data parts of the plot. The theme setting `legend.position` controls where the legend is drawn:

```
base <- ggplot(mpg, aes(x = displ, y = hwy)) +
  geom_point(aes(color = class))

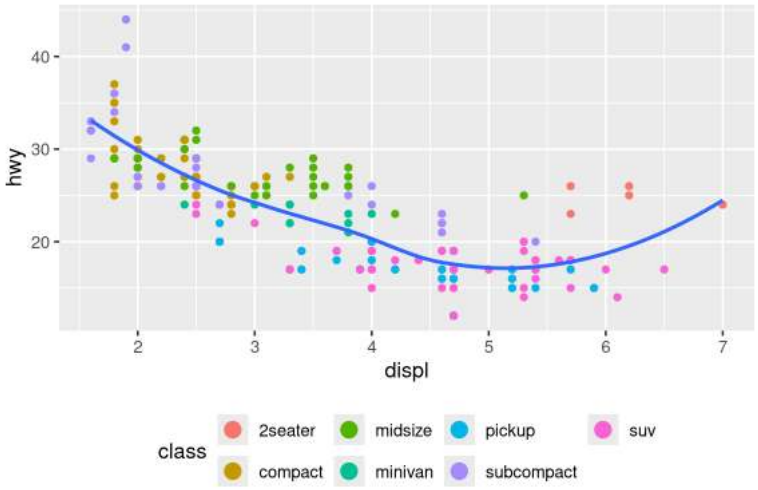
base + theme(legend.position = "right") # the default
base + theme(legend.position = "left")
base +
  theme(legend.position = "top") +
  guides(color = guide_legend(nrow = 3))
base +
  theme(legend.position = "bottom") +
  guides(color = guide_legend(nrow = 3))
```



If your plot is short and wide, place the legend at the top or bottom, and if it's tall and narrow, place the legend at the left or right. You can also use `legend.position = "none"` to suppress the display of the legend altogether.

To control the display of individual legends, use `guides()` along with `guide_legend()` or `guide_colorbar()`. The following example shows two important settings: controlling the number of rows the legend uses with `nrow`, and overriding one of the aesthetics to make the points bigger. This is particularly useful if you have used a low `alpha` to display many points on a plot.

```
ggplot(mpg, aes(x = displ, y = hwy)) +
  geom_point(aes(color = class)) +
  geom_smooth(se = FALSE) +
  theme(legend.position = "bottom") +
  guides(color = guide_legend(nrow = 2, override.aes = list(size = 4)))
#> `geom_smooth()` using method = 'loess' and formula = 'y ~ x'
```



Note that the name of the argument in `guides()` matches the name of the aesthetic, just like in `labs()`.

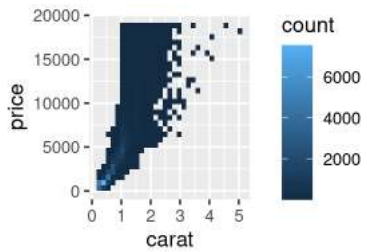
11.4.4 Replacing a scale

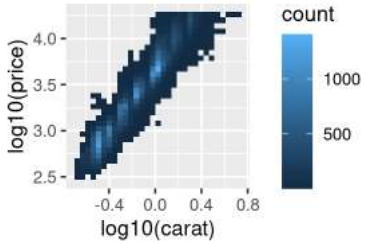
Instead of just tweaking the details a little, you can instead replace the scale altogether. There are two types of scales you're mostly likely to want to switch out: continuous position scales and color scales. Fortunately, the same principles apply to all the other aesthetics, so once you've mastered position and color, you'll be able to quickly pick up other scale replacements.

It's very useful to plot transformations of your variable. For example, it's easier to see the precise relationship between `carat` and `price` if we log transform them:

```
# Left
ggplot(diamonds, aes(x = carat, y = price)) +
  geom_bin2d()

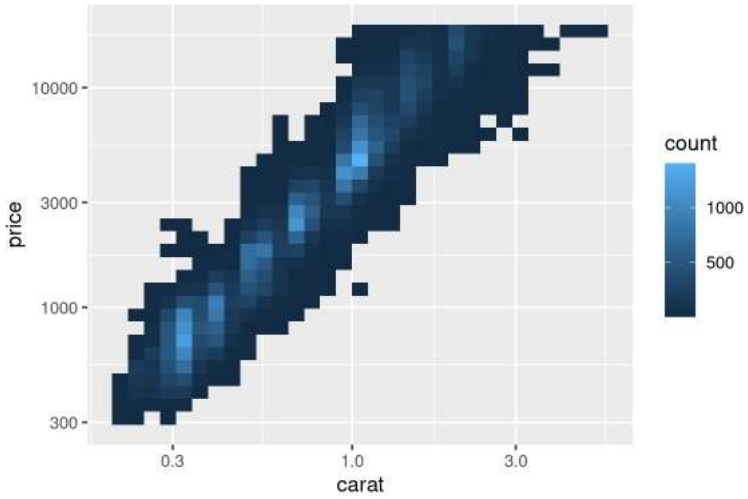
# Right
ggplot(diamonds, aes(x = log10(carat), y = log10(price))) +
  geom_bin2d()
```





However, the disadvantage of this transformation is that the axes are now labelled with the transformed values, making it hard to interpret the plot. Instead of doing the transformation in the aesthetic mapping, we can instead do it with the scale. This is visually identical, except the axes are labelled on the original data scale.

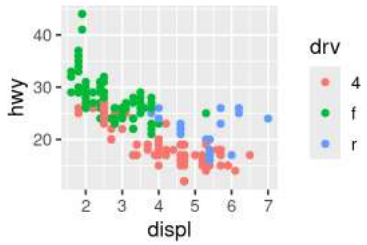
```
ggplot(diamonds, aes(x = carat, y = price)) +
  geom_bin2d() +
  scale_x_log10() +
  scale_y_log10()
```

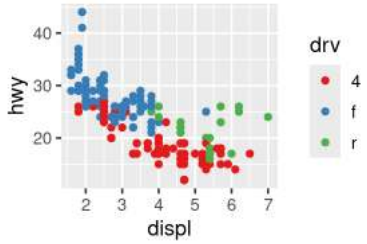


Another scale that is frequently customized is color. The default categorical scale picks colors that are evenly spaced around the color wheel. Useful alternatives are the ColorBrewer scales which have been hand tuned to work better for people with common types of color blindness. The two plots below look similar, but there is enough difference in the shades of red and green that the dots on the right can be distinguished even by people with red-green color blindness.[1]

```
ggplot(mpg, aes(x = displ, y = hwy)) +
  geom_point(aes(color = drv))

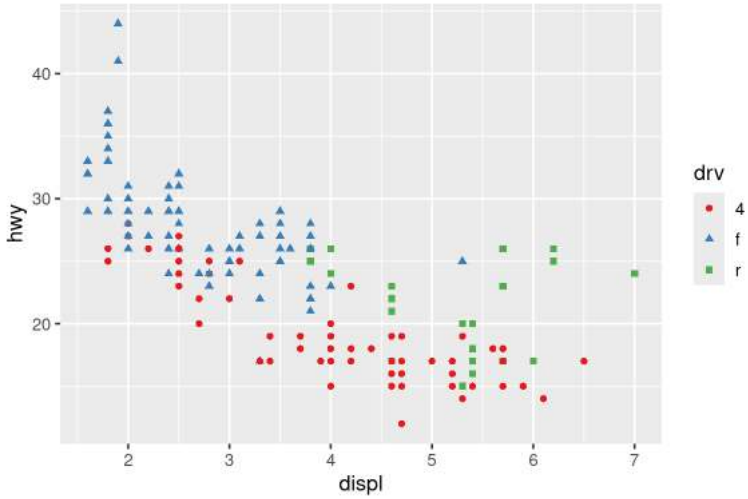
ggplot(mpg, aes(x = displ, y = hwy)) +
  geom_point(aes(color = drv)) +
  scale_color_brewer(palette = "Set1")
```





Don't forget simpler techniques for improving accessibility. If there are just a few colors, you can add a redundant shape mapping. This will also help ensure your plot is interpretable in black and white.

```
ggplot(mpg, aes(x = displ, y = hwy)) +
  geom_point(aes(color = drv, shape = drv)) +
  scale_color_brewer(palette = "Set1")
```



The ColorBrewer scales are documented online at https://colorbrewer2.org/ and made available in R via the **RColorBrewer** package, by Erich Neuwirth. Figure 11.1 shows the complete list of all palettes. The sequential (top) and diverging (bottom) palettes are particularly useful if your categorical values are ordered, or have a "middle". This often arises if you've used `cut()` to make a continuous variable into a categorical variable.



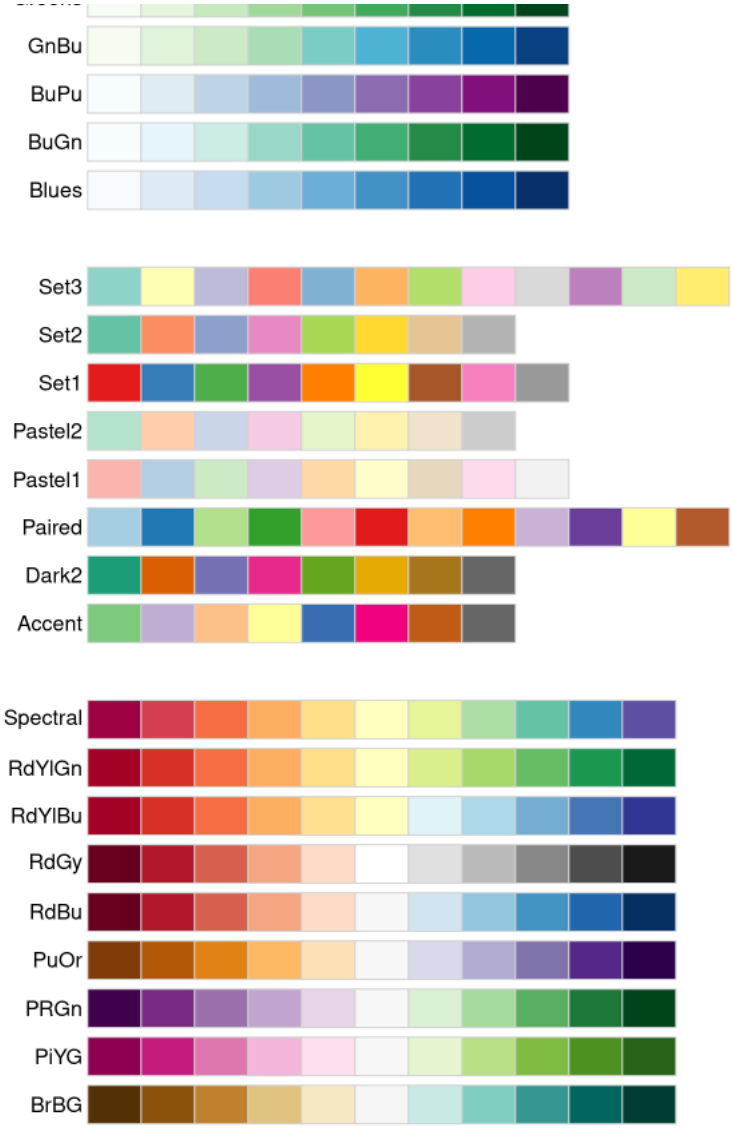
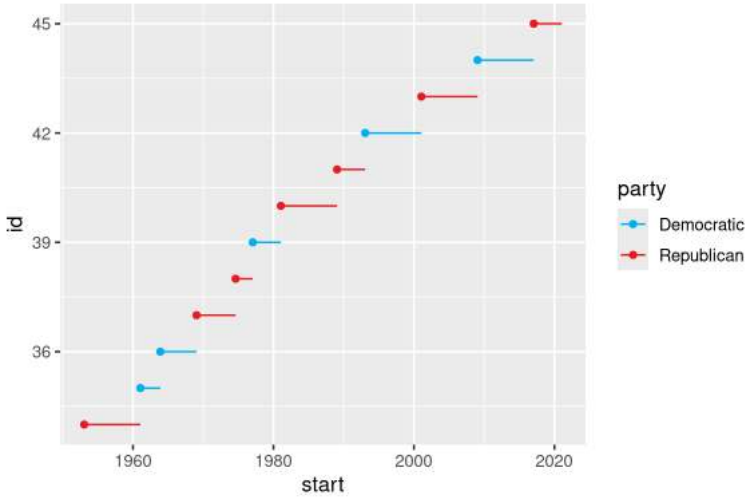


Figure 11.1: All colorBrewer scales.

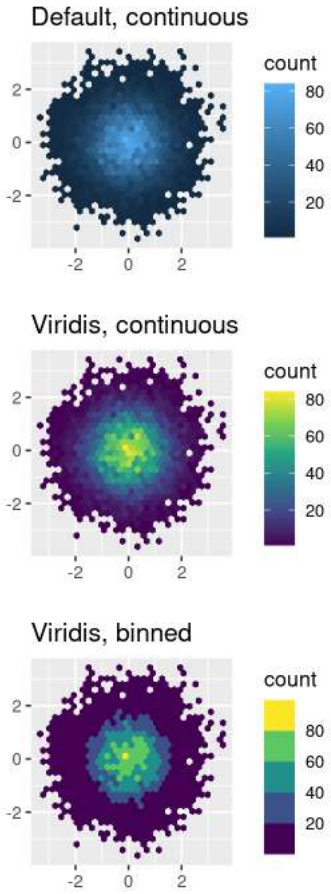
When you have a predefined mapping between values and colors, use `scale_color_manual()`. For example, if we map presidential party to color, we want to use the standard mapping of red for Republicans and blue for Democrats. One approach for assigning these colors is using hex color codes:

```
presidential |>
  mutate(id = 33 + row_number()) |>
  ggplot(aes(x = start, y = id, color = party)) +
  geom_point() +
  geom_segment(aes(xend = end, yend = id)) +
  scale_color_manual(values = c(Republican = "#E81B23", Democratic = "#00AEF3"))
```



For continuous color, you can use the built-in `scale_color_gradient()` or `scale_fill_gradient()`. If you have a diverging scale, you can use `scale_color_gradient2()`. That allows you to give, for example, positive and negative values different colors. That's sometimes also useful if you want to distinguish points above or below the mean.

Another option is to use the viridis color scales. The designers, Nathaniel Smith and Stéfan van der Walt, carefully tailored continuous color schemes that are perceptible to people with various forms of color blindness as well as perceptually uniform in both color and black and white. These scales are available as continuous (`c`), discrete (`d`), and binned (`b`) palettes in ggplot2.



Note that all color scales come in two varieties: `scale_color_*()` and `scale_fill_*()` for the `color` and `fill` aesthetics respectively (the color scales are available in both UK and US spellings).

11.4.5 Zooming

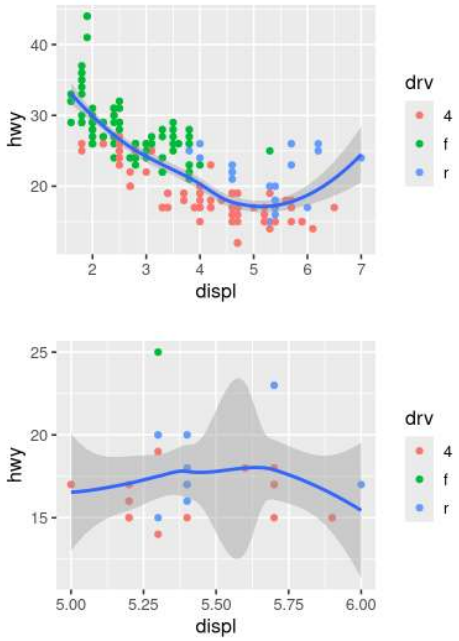
There are three ways to control the plot limits:

1. Adjusting what data are plotted.
2. Setting the limits in each scale.
3. Setting `xlim` and `ylim` in `coord_cartesian()`.

We'll demonstrate these options in a series of plots. The plot on the left shows the relationship between engine size and fuel efficiency, colored by type of drive train. The plot on the right shows the same variables, but subsets the data that are plotted. Subsetting the data has affected the x and y scales as well as the smooth curve.

```
# Left
ggplot(mpg, aes(x = displ, y = hwy)) +
  geom_point(aes(color = drv)) +
  geom_smooth()

# Right
mpg |>
  filter(displ >= 5 & displ <= 6 & hwy >= 10 & hwy <= 25) |>
  ggplot(aes(x = displ, y = hwy)) +
  geom_point(aes(color = drv)) +
  geom_smooth()
```



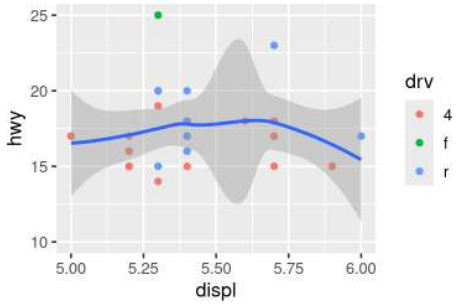
Let's compare these to the two plots below where the plot on the left sets the `limits` on individual scales and the plot on the right sets them in `coord_cartesian()`. We can see that reducing the limits is equivalent to subsetting the data. Therefore, to zoom in on a region of the plot, it's generally best to use `coord_cartesian()`.

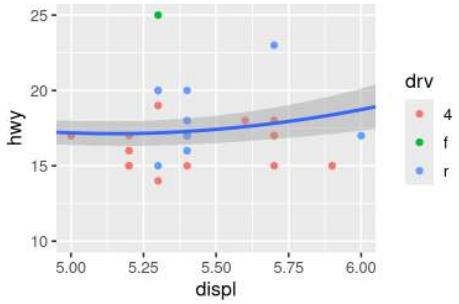
```
# Left

  geom_point(aes(color = drv)) +
  geom_smooth() +
  scale_x_continuous(limits = c(5, 6)) +
  scale_y_continuous(limits = c(10, 25))

# Right

  geom_point(aes(color = drv)) +
  geom_smooth() +
```



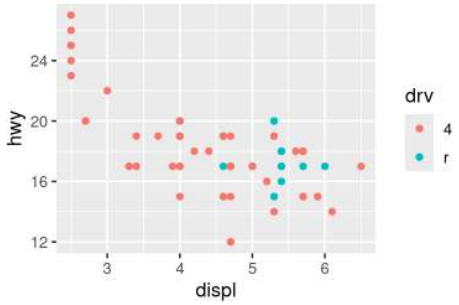


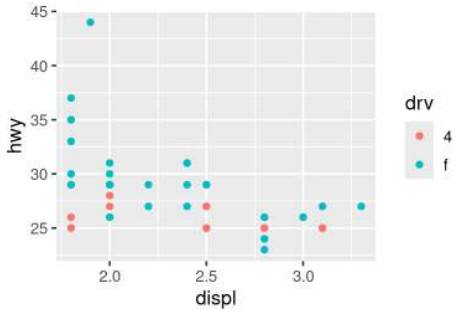
On the other hand, setting the `limits` on individual scales is generally more useful if you want to *expand* the limits, e.g., to match scales across different plots. For example, if we extract two classes of cars and plot them separately, it's difficult to compare the plots because all three scales (the x-axis, the y-axis, and the color aesthetic) have different ranges.

```
suv <- mpg |> filter(class == "suv")
compact <- mpg |> filter(class == "compact")

# Left
ggplot(suv, aes(x = displ, y = hwy, color = drv)) +
  geom_point()

# Right
ggplot(compact, aes(x = displ, y = hwy, color = drv)) +
  geom_point()
```



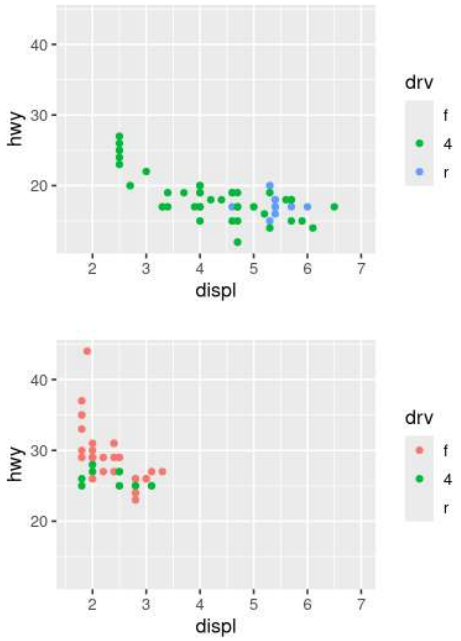


One way to overcome this problem is to share scales across multiple plots, training the scales with the `limits` of the full data.

```
x_scale <- scale_x_continuous(limits = range(mpg$displ))
y_scale <- scale_y_continuous(limits = range(mpg$hwy))
col_scale <- scale_color_discrete(limits = unique(mpg$drv))

# Left
ggplot(suv, aes(x = displ, y = hwy, color = drv)) +
  geom_point() +
  x_scale +
  y_scale +
  col_scale

# Right
ggplot(compact, aes(x = displ, y = hwy, color = drv)) +
  geom_point() +
  x_scale +
  y_scale +
  col_scale
```



In this particular case, you could have simply used faceting, but this technique is useful more generally, if for instance, you want to spread plots over multiple pages of a report.

11.4.6 Exercises

1. Why doesn't the following code override the default scale?

2. What is the first argument to every scale? How does it compare to `labs()`?

3. Change the display of the presidential terms by:

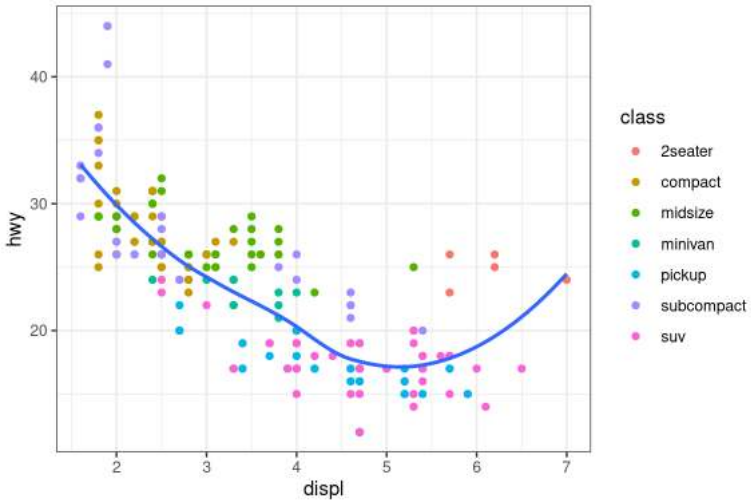
    a. Combining the two variants that customize colors and x axis breaks.
    b. Improving the display of the y axis.
    c. Labelling each term with the name of the president.
    d. Adding informative plot labels.
    e. Placing breaks every 4 years (this is trickier than it seems!).

4. First, create the following plot. Then, modify the code using `override.aes` to make the legend easier to see.

```
ggplot(diamonds, aes(x = carat, y = price)) +
  geom_point(aes(color = cut), alpha = 1/20)
```

11.5 Themes

Finally, you can customize the non-data elements of your plot with a theme:



ggplot2 includes the eight themes shown in Figure 11.2, with `theme_gray()` as the default.[2] Many more are included in add-on packages like **ggthemes** (https://jrnold.github.io/ggthemes), by Jeffrey Arnold. You can also create your own themes, if you are trying to match a particular corporate or journal style.

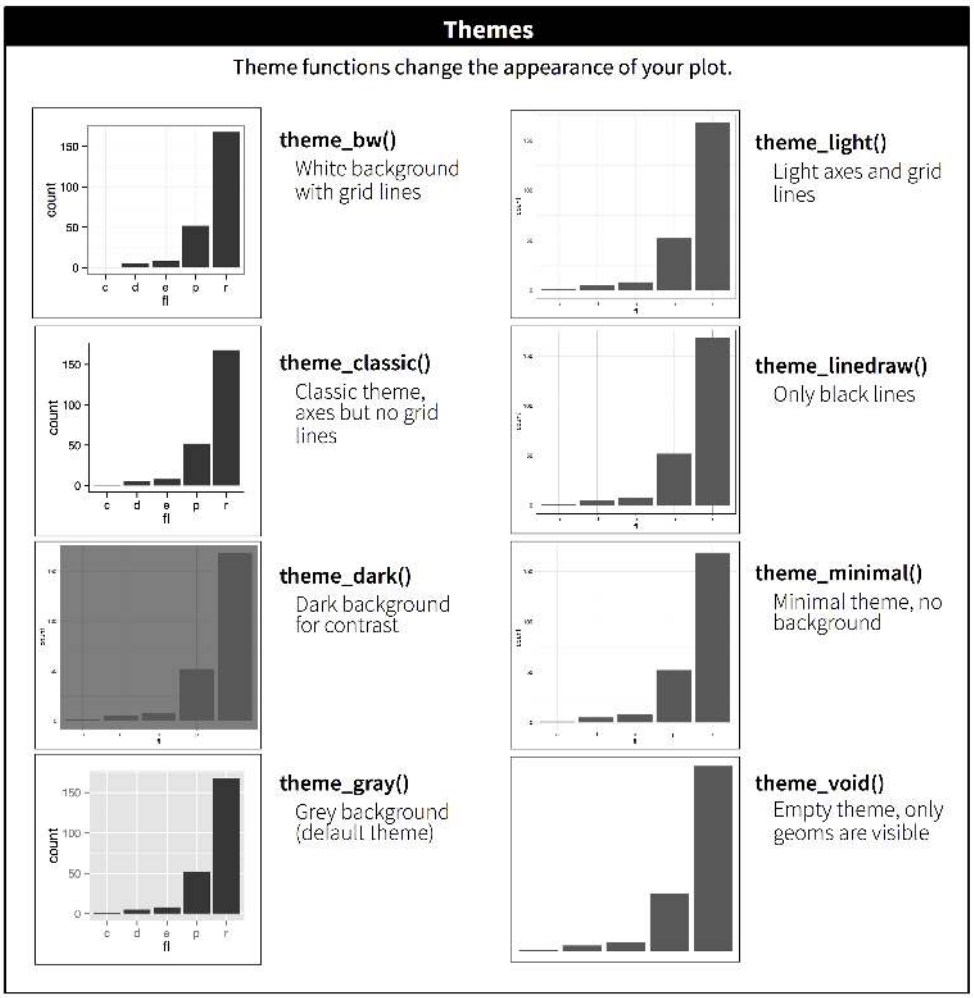
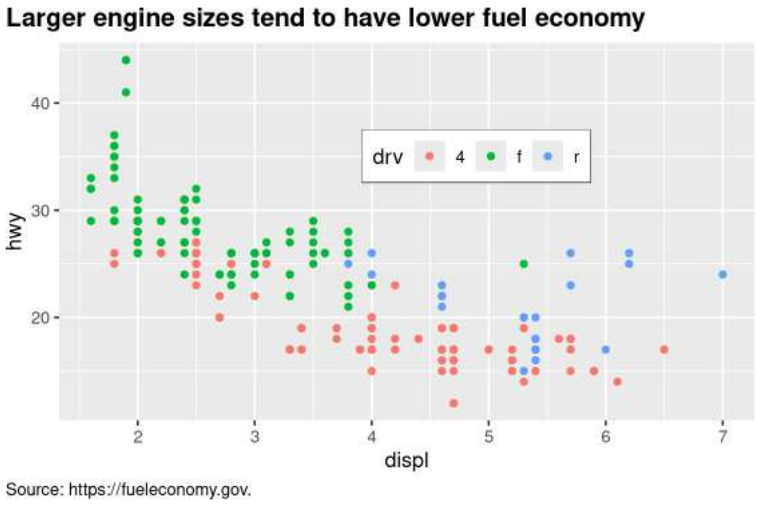


Figure 11.2: The eight themes built-in to ggplot2.

It's also possible to control individual components of each theme, like the size and color of the font used for the y axis. We've already seen that `legend.position` controls where the legend is drawn. There are many other aspects of the legend that can be customized with `theme()`. For example, in the plot below we change the direction of the

legend as well as put a black border around it. Note that customization of the legend box and plot title elements of the theme are done with `element_*()` functions. These functions specify the styling of non-data components, e.g., the title text is bolded in the `face` argument of `element_text()` and the legend border color is defined in the `color` argument of `element_rect()`. The theme elements that control the position of the title and the caption are `plot.title.position` and `plot.caption.position`, respectively. In the following plot these are set to `"plot"` to indicate these elements are aligned to the entire plot area, instead of the plot panel (the default). A few other helpful `theme()` components are used to change the placement for format of the title and caption text.

```
ggplot(mpg, aes(x = displ, y = hwy, color = drv)) +
  geom_point() +
  labs(
    title = "Larger engine sizes tend to have lower fuel economy",
    caption = "Source: https://fueleconomy.gov."
  ) +
  theme(
    legend.position = c(0.6, 0.7),
    legend.direction = "horizontal",
    legend.box.background = element_rect(color = "black"),
    plot.title = element_text(face = "bold"),
    plot.title.position = "plot",
    plot.caption.position = "plot",
    plot.caption = element_text(hjust = 0)
  )
#> Warning: A numeric `legend.position` argument in `theme()` was deprecated in
ggplot2
#> 3.5.0.
#> ℹ Please use the `legend.position.inside` argument of `theme()` instead.
```



For an overview of all `theme()` components, see help with `?theme`. The ggplot2 book is also a great place to go for the full details on theming.

11.5.1 Exercises

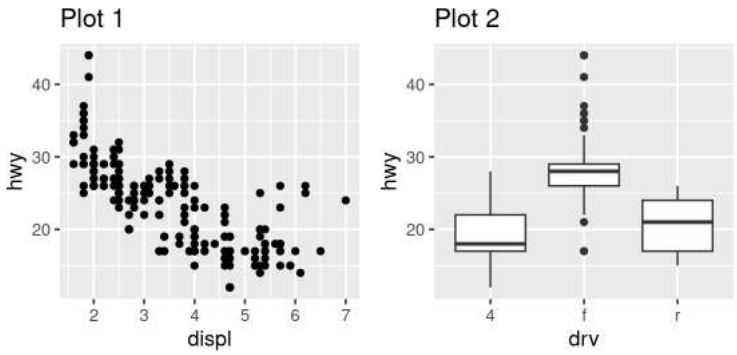
1. Pick a theme offered by the ggthemes package and apply it to the last plot you made.
2. Make the axis labels of your plot blue and bolded.

11.6 Layout

So far we talked about how to create and modify a single plot. What if you have multiple plots you want to lay out in a certain way? The patchwork package allows you to combine separate plots into the same graphic. We loaded this package earlier in the chapter.

To place two plots next to each other, you can simply add them to each other. Note that you first need to create the plots and save them as objects (in the following example they're called `p1` and `p2`). Then, you place them next to each other with `+`.

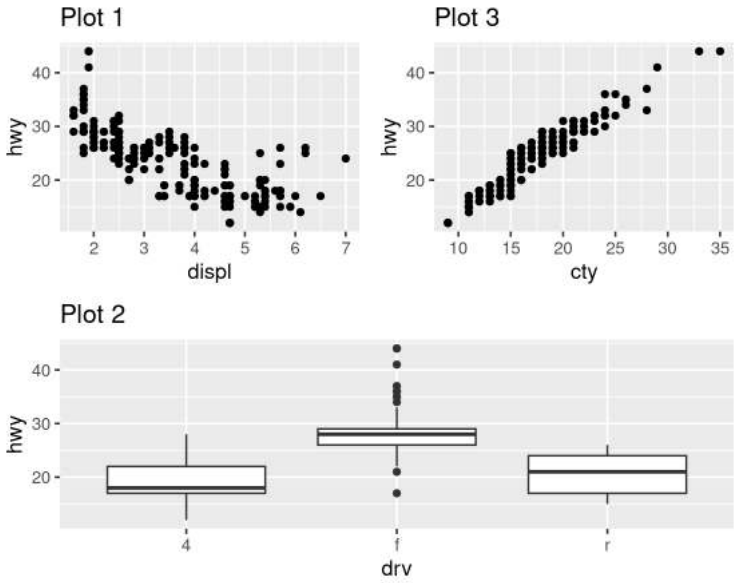
```
p1 <- ggplot(mpg, aes(x = displ, y = hwy)) +
  geom_point() +
  labs(title = "Plot 1")
p2 <- ggplot(mpg, aes(x = drv, y = hwy)) +
  geom_boxplot() +
  labs(title = "Plot 2")
p1 + p2
```



It's important to note that in the above code chunk we did not use a new function from the patchwork package. Instead, the package added a new functionality to the `+` operator.

You can also create complex plot layouts with patchwork. In the following, `|` places the `p1` and `p3` next to each other and `/` moves `p2` to the next line.

```
p3 <- ggplot(mpg, aes(x = cty, y = hwy)) +
  geom_point() +
  labs(title = "Plot 3")
(p1 | p3) / p2
```



Additionally, patchwork allows you to collect legends from multiple plots into one common legend, customize the placement of the legend as well as dimensions of the plots, and add a common title, subtitle, caption, etc. to your plots. Below we create 5 plots. We have turned off the legends on the box plots and the scatterplot and collected the legends for the density plots at the top of the plot with `& theme(legend.position = "top")`. Note the use of the `&` operator here instead of the usual `+`. This is because we're modifying the theme for the patchwork plot as opposed to the individual ggplots. The legend is placed on top, inside the `guide_area()`. Finally, we have also customized the heights of the various components of our patchwork – the guide has a height of 1, the box plots 3, density plots 2, and the faceted scatterplot 4. Patchwork divides up the area you have allotted for your plot using this scale and places the components accordingly.

```
p1 <- ggplot(mpg, aes(x = drv, y = cty, color = drv)) +
  geom_boxplot(show.legend = FALSE) +
  labs(title = "Plot 1")

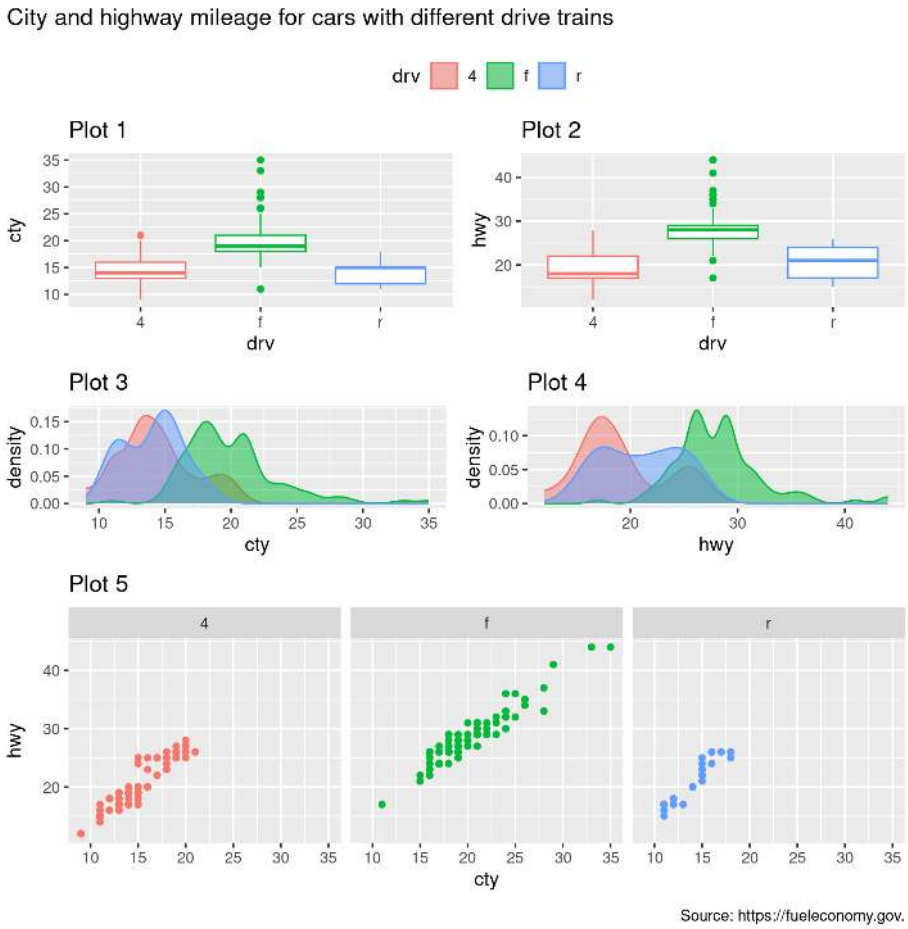
p2 <- ggplot(mpg, aes(x = drv, y = hwy, color = drv)) +
  geom_boxplot(show.legend = FALSE) +
  labs(title = "Plot 2")

p3 <- ggplot(mpg, aes(x = cty, color = drv, fill = drv)) +
  geom_density(alpha = 0.5) +
  labs(title = "Plot 3")

p4 <- ggplot(mpg, aes(x = hwy, color = drv, fill = drv)) +
  geom_density(alpha = 0.5) +
  labs(title = "Plot 4")

p5 <- ggplot(mpg, aes(x = cty, y = hwy, color = drv)) +
  geom_point(show.legend = FALSE) +
  facet_wrap(~drv) +
  labs(title = "Plot 5")

(guide_area() / (p1 + p2) / (p3 + p4) / p5) +
  plot_annotation(
    title = "City and highway mileage for cars with different drive trains",
    caption = "Source: https://fueleconomy.gov."
  ) +
  plot_layout(
    guides = "collect",
    heights = c(1, 3, 2, 4)
    ) &
  theme(legend.position = "top")
```



If you'd like to learn more about combining and layout out multiple plots with patchwork, we recommend looking through the guides on the package website: https://patchwork.data-imaginist.com.

11.6.1 Exercises

1. What happens if you omit the parentheses in the following plot layout. Can you explain why this happens?

```
p1 <- ggplot(mpg, aes(x = displ, y = hwy)) +
  geom_point() +
  labs(title = "Plot 1")
p2 <- ggplot(mpg, aes(x = drv, y = hwy)) +
  geom_boxplot() +
  labs(title = "Plot 2")
p3 <- ggplot(mpg, aes(x = cty, y = hwy)) +
  geom_point() +
  labs(title = "Plot 3")

(p1 | p2) / p3
```

2. Using the three plots from the previous exercise, recreate the following patchwork.

Fig. A:

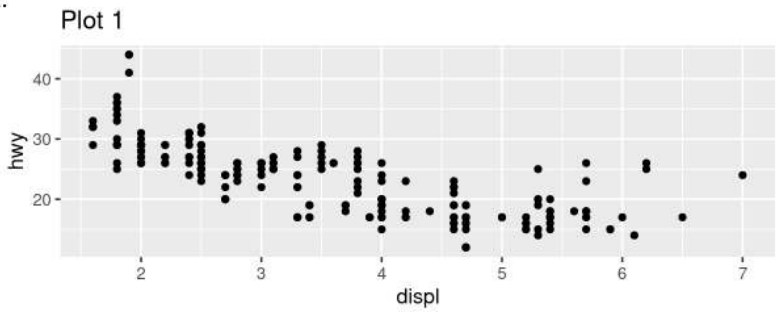


Fig. B:

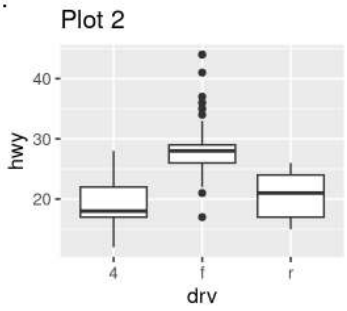
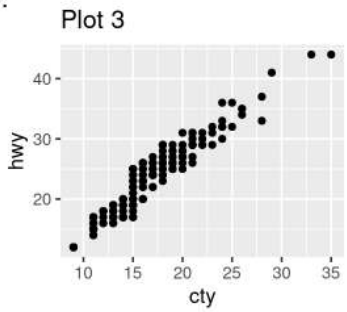


Fig. C:



11.7 Summary

In this chapter you've learned about adding plot labels such as title, subtitle, caption as well as modifying default axis labels, using annotation to add informational text to your plot or to highlight specific data points, customizing the axis scales, and changing the theme of your plot. You've also learned about combining multiple plots in a single graph using both simple and complex plot layouts.

While you've so far learned about how to make many different types of plots and how to customize them using a variety of techniques, we've barely scratched the surface of what you can create with ggplot2. If you want to get a comprehensive understanding of ggplot2, we recommend reading the book, *ggplot2: Elegant Graphics for Data Analysis*. Other useful resources are the *R Graphics Cookbook* by Winston Chang and *Fundamentals of Data Visualization* by Claus Wilke.

Transform

The second part of the book was a deep dive into data visualization. In this part of the book, you'll learn about the most important types of variables that you'll encounter inside a data frame and learn the tools you can use to work with them.

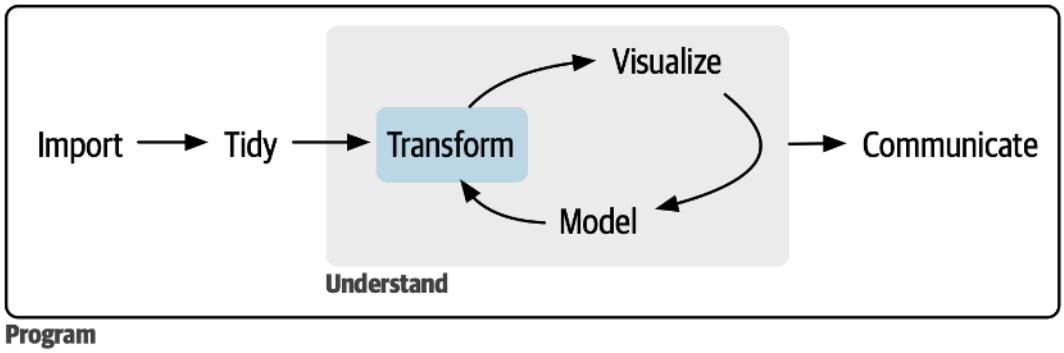


Figure 1: The options for data transformation depends heavily on the type of data involved, the subject of this part of the book.

You can read these chapters as you need them; they're designed to be largely standalone so that they can be read out of order.

- 12 Logical vectors teaches you about logical vectors. These are the simplest types of vectors, but are extremely powerful. You'll learn how to create them with numeric comparisons, how to combine them with Boolean algebra, how to use them in summaries, and how to use them for condition transformations.

- 13 Numbers dives into tools for vectors of numbers, the powerhouse of data science. You'll learn more about counting and a bunch of important transformation and summary functions.

- 14 Strings will give you the tools to work with strings: you'll slice them, you'll dice them, and you'll stick them back together again. This chapter mostly focuses on the stringr package, but you'll also learn some more tidyr functions devoted to extracting data from character strings.

- 15 Regular expressions introduces you to regular expressions, a powerful tool for manipulating strings. This chapter will take you from thinking that a cat walked over your keyboard to reading and writing complex string patterns.

- 16 Factors introduces factors: the data type that R uses to store categorical data. You use a factor when variable has a fixed set of possible values, or when you want to use a non-alphabetical ordering of a string.

- 17 Dates and times will give you the key tools for working with dates and date-times. Unfortunately, the more you learn about date-times, the more complicated they seem to get, but with the help of the lubridate package, you'll learn to how to overcome the most common challenges.
- 18 Missing values discusses missing values in depth. We've discussed them a couple of times in isolation, but now it's time to discuss them holistically, helping you come to grips with the difference between implicit and explicit missing values, and how and why you might convert between them.
- 19 Joins finishes up this part of the book by giving you tools to join two (or more) data frames together. Learning about joins will force you to grapple with the idea of keys, and think about how you identify each row in a dataset.

12 Logical vectors

12.1 Introduction

In this chapter, you'll learn tools for working with logical vectors. Logical vectors are the simplest type of vector because each element can only be one of three possible values: `TRUE`, `FALSE`, and `NA`. It's relatively rare to find logical vectors in your raw data, but you'll create and manipulate them in the course of almost every analysis.

We'll begin by discussing the most common way of creating logical vectors: with numeric comparisons. Then you'll learn about how you can use Boolean algebra to combine different logical vectors, as well as some useful summaries. We'll finish off with `if_else()` and `case_when()`, two useful functions for making conditional changes powered by logical vectors.

12.1.1 Prerequisites

Most of the functions you'll learn about in this chapter are provided by base R, so we don't need the tidyverse, but we'll still load it so we can use `mutate()`, `filter()`, and friends to work with data frames. We'll also continue to draw examples from the `nycflights13::flights` dataset.

```
library(tidyverse)
library(nycflights13)
```

However, as we start to cover more tools, there won't always be a perfect real example. So we'll start making up some dummy data with `c()`:

```
x <- c(1, 2, 3, 5, 7, 11, 13)
x * 2
#> [1]  2  4  6 10 14 22 26
```

This makes it easier to explain individual functions at the cost of making it harder to see how it might apply to your data problems. Just remember that any manipulation we do to a free-floating vector, you can do to a variable inside a data frame with `mutate()` and friends.

```
df <- tibble(x)
df |>
  mutate(y = x * 2)
#> # A tibble: 7 × 2
```

```
#>       x     y
#>   <dbl> <dbl>
#> 1     1     2
#> 2     2     4
#> 3     3     6
#> 4     5    10
#> 5     7    14
#> 6    11    22
#> # i 1 more row
```

12.2 Comparisons

A very common way to create a logical vector is via a numeric comparison with <, <=, >, >=, !=, and ==. So far, we've mostly created logical variables transiently within `filter()` — they are computed, used, and then thrown away. For example, the following filter finds all daytime departures that arrive roughly on time:

```
flights |>
  filter(dep_time > 600 & dep_time < 2000 & abs(arr_delay) < 20)
#> # A tibble: 172,286 × 19
#>    year month   day dep_time sched_dep_time dep_delay arr_time
sched_arr_time
#>   <int> <int> <int>    <int>          <int>     <dbl>    <int>
<int>
#> 1  2013     1     1      601            600         1      844
850
#> 2  2013     1     1      602            610        -8      812
820
#> 3  2013     1     1      602            605        -3      821
805
#> 4  2013     1     1      606            610        -4      858
910
#> 5  2013     1     1      606            610        -4      837
845
#> 6  2013     1     1      607            607         0      858
915
#> # i 172,280 more rows
#> # i 11 more variables: arr_delay <dbl>, carrier <chr>, flight <int>, …
```

It's useful to know that this is a shortcut and you can explicitly create the underlying logical variables with `mutate()`:

```
flights |>
  mutate(
    daytime = dep_time > 600 & dep_time < 2000,
    approx_ontime = abs(arr_delay) < 20,
    .keep = "used"
  )
#> # A tibble: 336,776 × 4
#>   dep_time arr_delay daytime approx_ontime
#>      <int>     <dbl> <lgl>   <lgl>
#> 1      517        11 FALSE   TRUE
#> 2      533        20 FALSE   FALSE
#> 3      542        33 FALSE   FALSE
#> 4      544       -18 FALSE   TRUE
#> 5      554       -25 FALSE   FALSE
#> 6      554        12 FALSE   TRUE
#> # ℹ 336,770 more rows
```

This is particularly useful for more complicated logic because naming the intermediate steps makes it easier to both read your code and check that each step has been computed correctly.

All up, the initial filter is equivalent to:

```
flights |>
  mutate(
    daytime = dep_time > 600 & dep_time < 2000,
    approx_ontime = abs(arr_delay) < 20,
  ) |>
  filter(daytime & approx_ontime)
```

12.2.1 Floating point comparison

Beware of using == with numbers. For example, it looks like this vector contains the numbers 1 and 2:

```
x <- c(1 / 49 * 49, sqrt(2) ^ 2)
x
#> [1] 1 2
```

But if you test them for equality, you get `FALSE`:

```
x == c(1, 2)
#> [1] FALSE FALSE
```

What's going on? Computers store numbers with a fixed number of decimal places so there's no way to exactly represent 1/49 or `sqrt(2)` and subsequent computations will be very slightly off. We can see the exact values by calling `print()` with the `digits`[1] argument:

```
print(x, digits = 16)
#> [1] 0.9999999999999999 2.0000000000000004
```

You can see why R defaults to rounding these numbers; they really are very close to what you expect.

Now that you've seen why == is failing, what can you do about it? One option is to use `dplyr::near()` which ignores small differences:

12.2.2 Missing values

Missing values represent the unknown so they are "contagious": almost any operation involving an unknown value will also be unknown:

```
NA > 5
#> [1] NA
10 == NA
#> [1] NA
```

The most confusing result is this one:

```
NA == NA
#> [1] NA
```

It's easiest to understand why this is true if we artificially supply a little more context:

```
# We don't know how old Mary is
age_mary <- NA

# We don't know how old John is
age_john <- NA

# Are Mary and John the same age?
age_mary == age_john
#> [1] NA
# We don't know!
```

So if you want to find all flights where `dep_time` is missing, the following code doesn't work because `dep_time == NA` will yield `NA` for every single row, and `filter()` automatically drops missing values:

```
flights |>
  filter(dep_time == NA)
#> # A tibble: 0 × 19
#> # ℹ 19 variables: year <int>, month <int>, day <int>, dep_time <int>,
#> #   sched_dep_time <int>, dep_delay <dbl>, arr_time <int>, …
```

Instead we'll need a new tool: `is.na()`.

12.2.3 `is.na()`

`is.na(x)` works with any type of vector and returns `TRUE` for missing values and `FALSE` for everything else:

```
is.na(c(TRUE, NA, FALSE))
#> [1] FALSE  TRUE FALSE
is.na(c(1, NA, 3))
#> [1] FALSE  TRUE FALSE
is.na(c("a", NA, "b"))
#> [1] FALSE  TRUE FALSE
```

We can use `is.na()` to find all the rows with a missing `dep_time`:

```
flights |>
  filter(is.na(dep_time))
#> # A tibble: 8,255 × 19
#>    year month   day dep_time sched_dep_time dep_delay arr_time
sched_arr_time
#>   <int> <int> <int>    <int>          <int>     <dbl>    <int>
<int>
#> 1  2013     1     1       NA           1630        NA       NA
1815
#> 2  2013     1     1       NA           1935        NA       NA
2240
#> 3  2013     1     1       NA           1500        NA       NA
1825
#> 4  2013     1     1       NA            600        NA       NA
901
#> 5  2013     1     2       NA           1540        NA       NA
1747
#> 6  2013     1     2       NA           1620        NA       NA
1746
```

```
#> # ℹ 8,249 more rows
#> # ℹ 11 more variables: arr_delay <dbl>, carrier <chr>, flight <int>, …
```

`is.na()` can also be useful in `arrange()`. `arrange()` usually puts all the missing values at the end but you can override this default by first sorting by `is.na()`:

```
flights |>
  filter(month == 1, day == 1) |>
  arrange(dep_time)
#> # A tibble: 842 × 19
#>    year month   day dep_time sched_dep_time dep_delay arr_time
sched_arr_time
#>   <int> <int> <int>    <int>          <int>     <dbl>    <int>
<int>
#> 1  2013     1     1      517            515         2      830
819
#> 2  2013     1     1      533            529         4      850
830
#> 3  2013     1     1      542            540         2      923
850
#> 4  2013     1     1      544            545        -1     1004
1022
#> 5  2013     1     1      554            600        -6      812
837
#> 6  2013     1     1      554            558        -4      740
728
#> # ℹ 836 more rows
#> # ℹ 11 more variables: arr_delay <dbl>, carrier <chr>, flight <int>, …

flights |>
  filter(month == 1, day == 1) |>
  arrange(desc(is.na(dep_time)), dep_time)
#> # A tibble: 842 × 19
#>    year month   day dep_time sched_dep_time dep_delay arr_time
sched_arr_time
#>   <int> <int> <int>    <int>          <int>     <dbl>    <int>
<int>
#> 1  2013     1     1       NA           1630        NA       NA
1815
#> 2  2013     1     1       NA           1935        NA       NA
2240
```

```
#> 3  2013     1     1       NA           1500        NA       NA
1825
#> 4  2013     1     1       NA            600        NA       NA
901
#> 5  2013     1     1      517            515         2      830
819
#> 6  2013     1     1      533            529         4      850
830
#> # i 836 more rows
#> # i 11 more variables: arr_delay <dbl>, carrier <chr>, flight <int>, …
```

We'll come back to cover missing values in more depth in Chapter 18.

12.2.4 Exercises

1. How does `dplyr::near()` work? Type `near` to see the source code. Is `sqrt(2)^2` near 2?
2. Use `mutate()`, `is.na()`, and `count()` together to describe how the missing values in `dep_time`, `sched_dep_time` and `dep_delay` are connected.

12.3 Boolean algebra

Once you have multiple logical vectors, you can combine them together using Boolean algebra. In R, `&` is "and", `|` is "or", `!` is "not", and `xor()` is exclusive or[2]. For example, `df |> filter(!is.na(x))` finds all rows where `x` is not missing and `df |> filter(x < -10 | x > 0)` finds all rows where `x` is smaller than -10 or bigger than 0. Figure 12.1 shows the complete set of Boolean operations and how they work.

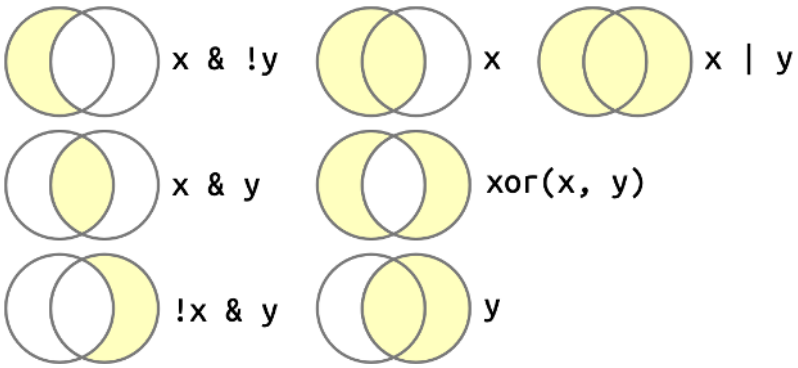


Figure 12.1: The complete set of Boolean operations. `x` is the left-hand circle, `y` is the right-hand circle, and the shaded region show which parts each operator selects.

As well as `&` and `|`, R also has `&&` and `||`. Don't use them in dplyr functions! These are called short-circuiting operators and only ever return a single `TRUE` or `FALSE`. They're important for programming, not data science.

12.3.1 Missing values

The rules for missing values in Boolean algebra are a little tricky to explain because they seem inconsistent at first glance:

```
df <- tibble(x = c(TRUE, FALSE, NA))

df |>
  mutate(
    and = x & NA,
    or = x | NA
  )
#> # A tibble: 3 × 3
#>   x     and   or
#>   <lgl> <lgl> <lgl>
#> 1 TRUE  NA    TRUE
#> 2 FALSE FALSE NA
#> 3 NA    NA    NA
```

To understand what's going on, think about `NA | TRUE` (`NA` or `TRUE`). A missing value in a logical vector means that the value could either be `TRUE` or `FALSE`. `TRUE | TRUE` and `FALSE | TRUE` are both `TRUE` because at least one of them is `TRUE`. `NA | TRUE` must also be `TRUE` because `NA` can either be `TRUE` or `FALSE`. However, `NA | FALSE` is `NA` because we don't know if `NA` is `TRUE` or `FALSE`. Similar reasoning applies with `NA & FALSE`.

12.3.2 Order of operations

Note that the order of operations doesn't work like English. Take the following code that finds all flights that departed in November or December:

```
flights |>
   filter(month == 11 | month == 12)
```

You might be tempted to write it like you'd say in English: "Find all flights that departed in November or December.":

```
flights |>
   filter(month == 11 | 12)
#> # A tibble: 336,776 × 19
#>    year month   day dep_time sched_dep_time dep_delay arr_time
sched_arr_time
#>   <int> <int> <int>    <int>          <int>     <dbl>    <int>
```

```
<int>
#> 1  2013     1     1      517            515         2      830
819
#> 2  2013     1     1      533            529         4      850
830
#> 3  2013     1     1      542            540         2      923
850
#> 4  2013     1     1      544            545        -1     1004
1022
#> 5  2013     1     1      554            600        -6      812
837
#> 6  2013     1     1      554            558        -4      740
728
#> # ℹ 336,770 more rows
#> # ℹ 11 more variables: arr_delay <dbl>, carrier <chr>, flight <int>, …
```

This code doesn't error but it also doesn't seem to have worked. What's going on? Here, R first evaluates `month == 11` creating a logical vector, which we call `nov`. It computes `nov | 12`. When you use a number with a logical operator it converts everything apart from 0 to `TRUE`, so this is equivalent to `nov | TRUE` which will always be `TRUE`, so every row will be selected:

```
flights |>
  mutate(
    nov = month == 11,
    final = nov | 12,
    .keep = "used"
  )
#> # A tibble: 336,776 × 3
#>   month nov   final
#>   <int> <lgl> <lgl>
#> 1     1 FALSE TRUE
#> 2     1 FALSE TRUE
#> 3     1 FALSE TRUE
#> 4     1 FALSE TRUE
#> 5     1 FALSE TRUE
#> 6     1 FALSE TRUE
#> # ℹ 336,770 more rows
```

12.3.3 `%in%`

An easy way to avoid the problem of getting your ==s and |s in the right order is to use `%in%`. `x %in% y` returns a logical vector the same length as `x` that is `TRUE` whenever a value in `x` is anywhere in `y`.

```
1:12 %in% c(1, 5, 11)
#>  [1]  TRUE FALSE FALSE FALSE  TRUE FALSE FALSE FALSE FALSE FALSE  TRUE
FALSE
letters[1:10] %in% c("a", "e", "i", "o", "u")
#>  [1]  TRUE FALSE FALSE FALSE  TRUE FALSE FALSE FALSE  TRUE FALSE
```

So to find all flights in November and December we could write:

```
flights |>
  filter(month %in% c(11, 12))
```

Note that `%in%` obeys different rules for `NA` to ==, as `NA %in% NA` is `TRUE`.

```
c(1, 2, NA) == NA
#> [1] NA NA NA
c(1, 2, NA) %in% NA
#> [1] FALSE FALSE  TRUE
```

This can make for a useful shortcut:

```
flights |>
  filter(dep_time %in% c(NA, 0800))
#> # A tibble: 8,803 × 19
#>    year month   day dep_time sched_dep_time dep_delay arr_time
sched_arr_time
#>   <int> <int> <int>    <int>          <int>     <dbl>    <int>
<int>
#> 1  2013     1     1      800            800         0     1022
1014
#> 2  2013     1     1      800            810       -10      949
955
#> 3  2013     1     1       NA           1630        NA       NA
1815
#> 4  2013     1     1       NA           1935        NA       NA
2240
#> 5  2013     1     1       NA           1500        NA       NA
1825
#> 6  2013     1     1       NA            600        NA       NA
901
```

```
#> # ℹ 8,797 more rows
#> # ℹ 11 more variables: arr_delay <dbl>, carrier <chr>, flight <int>, …
```

12.3.4 Exercises

1. Find all flights where `arr_delay` is missing but `dep_delay` is not. Find all flights where neither `arr_time` nor `sched_arr_time` are missing, but `arr_delay` is.
2. How many flights have a missing `dep_time`? What other variables are missing in these rows? What might these rows represent?
3. Assuming that a missing `dep_time` implies that a flight is cancelled, look at the number of cancelled flights per day. Is there a pattern? Is there a connection between the proportion of cancelled flights and the average delay of non-cancelled flights?

12.4 Summaries

The following sections describe some useful techniques for summarizing logical vectors. As well as functions that only work specifically with logical vectors, you can also use functions that work with numeric vectors.

12.4.1 Logical summaries

There are two main logical summaries: `any()` and `all()`. `any(x)` is the equivalent of `|`; it'll return `TRUE` if there are any `TRUE`'s in `x`. `all(x)` is equivalent of `&`; it'll return `TRUE` only if all values of `x` are `TRUE`'s. Like all summary functions, they'll return `NA` if there are any missing values present, and as usual you can make the missing values go away with `na.rm = TRUE`.

For example, we could use `all()` and `any()` to find out if every flight was delayed on departure by at most an hour or if any flights were delayed on arrival by five hours or more. And using `group_by()` allows us to do that by day:

```
flights |>
  group_by(year, month, day) |>
  summarize(
    all_delayed = all(dep_delay <= 60, na.rm = TRUE),
    any_long_delay = any(arr_delay >= 300, na.rm = TRUE),
    .groups = "drop"
  )
#> # A tibble: 365 × 5
#>    year month   day all_delayed any_long_delay
#>   <int> <int> <int> <lgl>       <lgl>
#> 1  2013     1     1 FALSE       TRUE
#> 2  2013     1     2 FALSE       TRUE
#> 3  2013     1     3 FALSE       FALSE
```

```
#> 4  2013     1     4 FALSE       FALSE
#> 5  2013     1     5 FALSE       TRUE
#> 6  2013     1     6 FALSE       FALSE
#> # ℹ 359 more rows
```

In most cases, however, `any()` and `all()` are a little too crude, and it would be nice to be able to get a little more detail about how many values are `TRUE` or `FALSE`. That leads us to the numeric summaries.

12.4.2 Numeric summaries of logical vectors

When you use a logical vector in a numeric context, `TRUE` becomes 1 and `FALSE` becomes 0. This makes `sum()` and `mean()` very useful with logical vectors because `sum(x)` gives the number of `TRUE`s and `mean(x)` gives the proportion of `TRUE`s (because `mean()` is just `sum()` divided by `length()`.

That, for example, allows us to see the proportion of flights that were delayed on departure by at most an hour and the number of flights that were delayed on arrival by five hours or more:

```
flights |>
  group_by(year, month, day) |>
  summarize(
    proportion_delayed = mean(dep_delay <= 60, na.rm = TRUE),
    count_long_delay = sum(arr_delay >= 300, na.rm = TRUE),
    .groups = "drop"
  )
#> # A tibble: 365 × 5
#>    year month   day proportion_delayed count_long_delay
#>   <int> <int> <int>              <dbl>            <int>
#> 1  2013     1     1              0.939                3
#> 2  2013     1     2              0.914                3
#> 3  2013     1     3              0.941                0
#> 4  2013     1     4              0.953                0
#> 5  2013     1     5              0.964                1
#> 6  2013     1     6              0.959                0
#> # ℹ 359 more rows
```

12.4.3 Logical subsetting

There's one final use for logical vectors in summaries: you can use a logical vector to filter a single variable to a subset of interest. This makes use of the base `[` (pronounced subset) operator, which you'll learn more about in Section 27.2.

Imagine we wanted to look at the average delay just for flights that were actually delayed. One way to do so would be to first filter the flights and then calculate the average delay:

```
flights |>
  filter(arr_delay > 0) |>
  group_by(year, month, day) |>
  summarize(
    behind = mean(arr_delay),
    n = n(),
    .groups = "drop"
  )
#> # A tibble: 365 × 5
#>    year month   day behind     n
#>   <int> <int> <int>  <dbl> <int>
#> 1  2013     1     1   32.5   461
#> 2  2013     1     2   32.0   535
#> 3  2013     1     3   27.7   460
#> 4  2013     1     4   28.3   297
#> 5  2013     1     5   22.6   238
#> 6  2013     1     6   24.4   381
#> # ℹ 359 more rows
```

This works, but what if we wanted to also compute the average delay for flights that arrived early? We'd need to perform a separate filter step, and then figure out how to combine the two data frames together[3]. Instead you could use `[` to perform an inline filtering: `arr_delay[arr_delay > 0]` will yield only the positive arrival delays.

This leads to:

```
flights |>
  group_by(year, month, day) |>
  summarize(
    behind = mean(arr_delay[arr_delay > 0], na.rm = TRUE),
    ahead = mean(arr_delay[arr_delay < 0], na.rm = TRUE),
    n = n(),
    .groups = "drop"
  )
#> # A tibble: 365 × 6
#>    year month   day behind ahead     n
#>   <int> <int> <int>  <dbl> <dbl> <int>
#> 1  2013     1     1   32.5 -12.5   842
#> 2  2013     1     2   32.0 -14.3   943
#> 3  2013     1     3   27.7 -18.2   914
#> 4  2013     1     4   28.3 -17.0   915
```

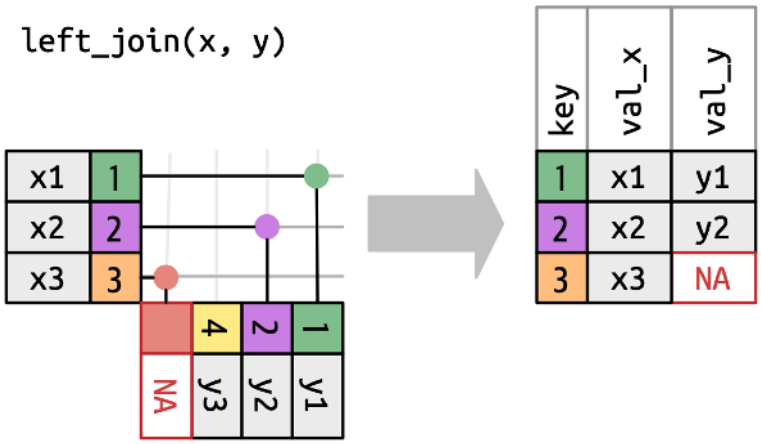


Figure 19.5: A visual representation of the left join where every row in `x` appears in the output.

- A **right join** keeps all observations in `y`, Figure 19.6. Every row of `y` is preserved in the output because it can fall back to matching a row of `NA`s in `x`. The output still matches `x` as much as possible; any extra rows from `y` are added to the end.

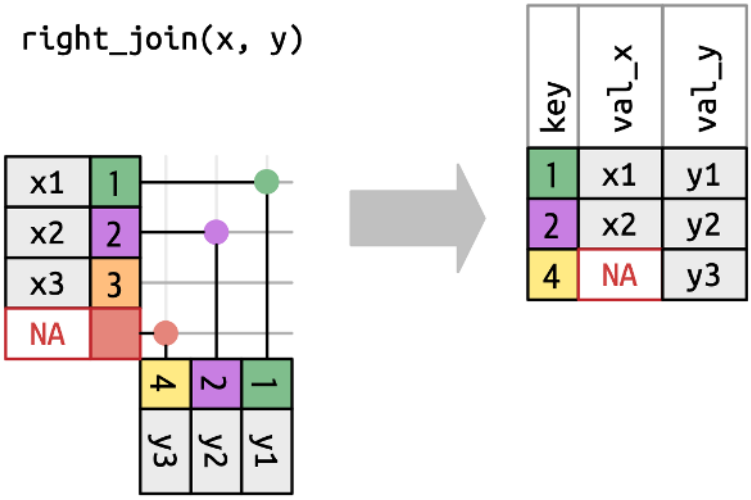


Figure 19.6: A visual representation of the right join where every row of `y` appears in the output.

- A **full join** keeps all observations that appear in `x` or `y`, Figure 19.7. Every row of `x` and `y` is included in the output because both `x` and `y` have a fall back row of `NA`s. Again, the output starts with all rows from `x`, followed by the remaining unmatched `y` rows.

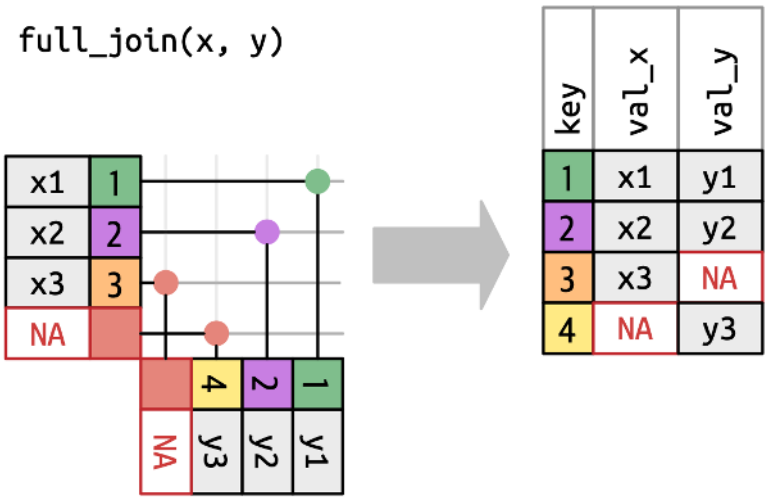


Figure 19.7: A visual representation of the full join where every row in `x` and `y` appears in the output.

Another way to show how the types of outer join differ is with a Venn diagram, as in Figure 19.8. However, this is not a great representation because while it might jog your memory about which rows are preserved, it fails to illustrate what's happening with the columns.

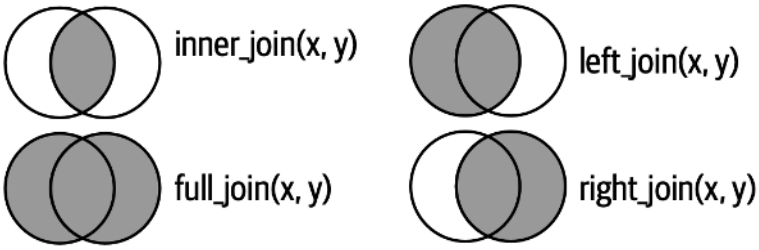


Figure 19.8: Venn diagrams showing the difference between inner, left, right, and full joins.

The joins shown here are the so-called **equi joins**, where rows match if the keys are equal. Equi joins are the most common type of join, so we'll typically omit the equi prefix, and just say "inner join" rather than "equi inner join". We'll come back to non-equi joins in Section 19.5.

19.4.1 Row matching

So far we've explored what happens if a row in `x` matches zero or one row in `y`. What happens if it matches more than one row? To understand what's going let's first narrow our focus to the `inner_join()` and then draw a picture, Figure 19.9.

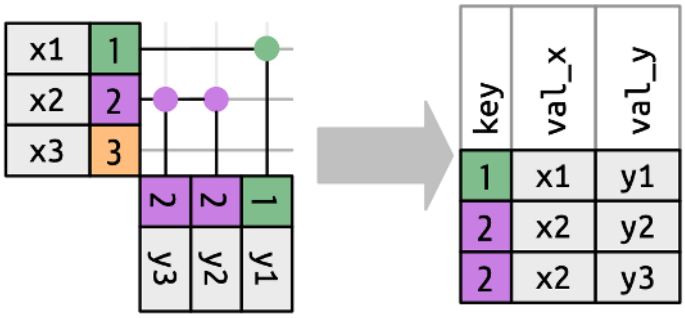


Figure 19.9: The three ways a row in `x` can match. `x1` matches one row in `y`, `x2` matches two rows in `y`, `x3` matches zero rows in y. Note that while there are three rows in `x` and three rows in the output, there isn't a direct correspondence between the rows.

There are three possible outcomes for a row in `x`:

- If it doesn't match anything, it's dropped.
- If it matches 1 row in `y`, it's preserved.
- If it matches more than 1 row in `y`, it's duplicated once for each match.

In principle, this means that there's no guaranteed correspondence between the rows in the output and the rows in `x`, but in practice, this rarely causes problems. There is, however, one particularly dangerous case which can cause a combinatorial explosion of rows. Imagine joining the following two tables:

```
df1 <- tibble(key = c(1, 2, 2), val_x = c("x1", "x2", "x3"))
df2 <- tibble(key = c(1, 2, 2), val_y = c("y1", "y2", "y3"))
```

While the first row in `df1` only matches one row in `df2`, the second and third rows both match two rows. This is sometimes called a `many-to-many` join, and will cause dplyr to emit a warning:

```
df1 |>
  inner_join(df2, join_by(key))
#> Warning in inner_join(df1, df2, join_by(key)): Detected an unexpected
many-to-many relationship between `x` and `y`.
#> ℹ Row 2 of `x` matches multiple rows in `y`.
#> ℹ Row 2 of `y` matches multiple rows in `x`.
#> ℹ If a many-to-many relationship is expected, set `relationship =
#>   "many-to-many"` to silence this warning.
#> # A tibble: 5 × 3
#>     key val_x val_y
#>   <dbl> <chr> <chr>
#> 1     1 x1    y1
```

```
#> 2     2 x2    y2
#> 3     2 x2    y3
#> 4     2 x3    y2
#> 5     2 x3    y3
```

If you are doing this deliberately, you can set `relationship = "many-to-many"`, as the warning suggests.

19.4.2 Filtering joins

The number of matches also determines the behavior of the filtering joins. The semi-join keeps rows in `x` that have one or more matches in `y`, as in Figure 19.10. The anti-join keeps rows in `x` that match zero rows in `y`, as in Figure 19.11. In both cases, only the existence of a match is important; it doesn't matter how many times it matches. This means that filtering joins never duplicate rows like mutating joins do.

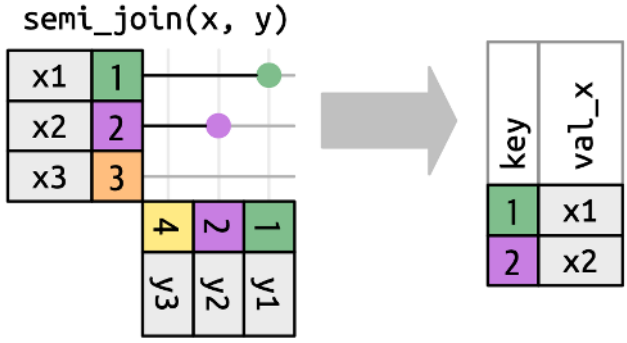


Figure 19.10: In a semi-join it only matters that there is a match; otherwise values in `y` don't affect the output.

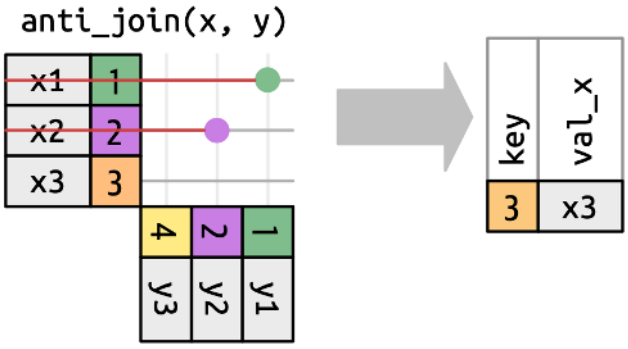


Figure 19.11: An anti-join is the inverse of a semi-join, dropping rows from `x` that have a match in `y`.

19.5 Non-equi joins

So far you've only seen equi joins, joins where the rows match if the `x` key equals the `y` key. Now we're going to relax that restriction and discuss other ways of determining if a pair of rows match.

But before we can do that, we need to revisit a simplification we made above. In equi joins the `x` keys and `y` are always equal, so we only need to show one in the output. We can request that dplyr keep both keys with `keep = TRUE`, leading to the code below and the re-drawn `inner_join()` in Figure 19.12.

```
x |> inner_join(y, join_by(key == key), keep = TRUE)
#> # A tibble: 2 × 4
#>   key.x val_x key.y val_y
#>   <dbl> <chr> <dbl> <chr>
#> 1     1 x1        1 y1
#> 2     2 x2        2 y2
```

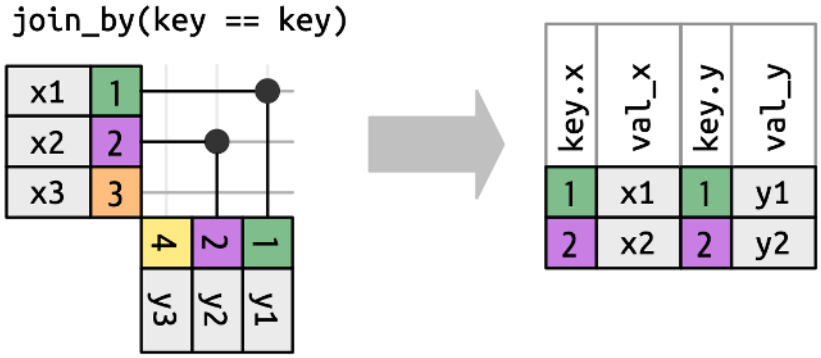


Figure 19.12: An inner join showing both `x` and `y` keys in the output.

When we move away from equi joins we'll always show the keys, because the key values will often be different. For example, instead of matching only when the `x$key` and `y$key` are equal, we could match whenever the `x$key` is greater than or equal to the `y$key`, leading to Figure 19.13. dplyr's join functions understand this distinction equi and non-equi joins so will always show both keys when you perform a non-equi join.

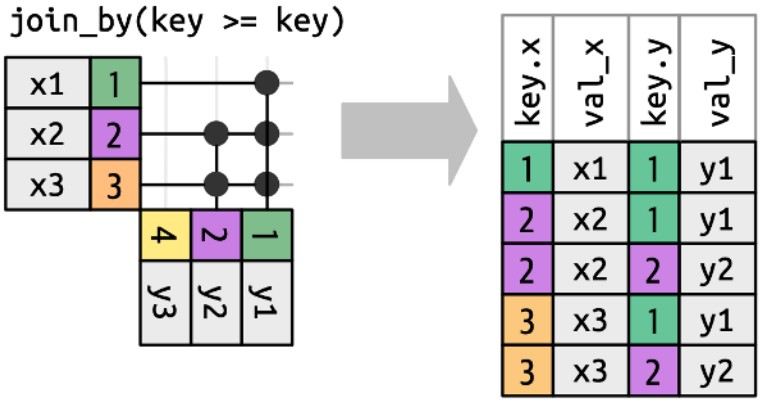


Figure 19.13: A non-equi join where the `x` key must be greater than or equal to the `y` key. Many rows generate multiple matches.

Non-equi join isn't a particularly useful term because it only tells you what the join is not, not what it is. dplyr helps by identifying four particularly useful types of non-equi join:

- **Cross joins** match every pair of rows.
- **Inequality joins** use `<`, `<=`, `>`, and `>=` instead of `==`.
- **Rolling joins** are similar to inequality joins but only find the closest match.
- **Overlap joins** are a special type of inequality join designed to work with ranges.

Each of these is described in more detail in the following sections.

19.5.1 Cross joins

A cross join matches everything, as in Figure 19.14, generating the Cartesian product of rows. This means the output will have `nrow(x) * nrow(y)` rows.

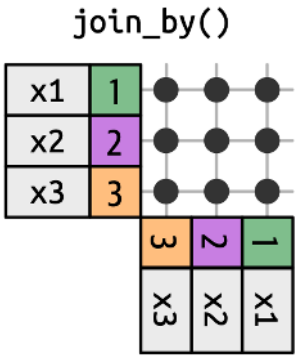


Figure 19.14: A cross join matches each row in `x` with every row in `y`.

Cross joins are useful when generating permutations. For example, the code below generates every possible pair of names. Since we're joining `df` to itself, this is sometimes called a **self-join**. Cross joins use a different join function because there's no distinction between inner/left/right/full when you're matching every row.

```
df <- tibble(name = c("John", "Simon", "Tracy", "Max"))
df |> cross_join(df)
#> # A tibble: 16 × 2
#>   name.x name.y
#>   <chr>  <chr>
#> 1 John   John
```

```
#> 2 John   Simon
#> 3 John   Tracy
#> 4 John   Max
#> 5 Simon  John
#> 6 Simon  Simon
#> # ℹ 10 more rows
```

19.5.2 Inequality joins

Inequality joins use <, <=, >=, or > to restrict the set of possible matches, as in Figure 19.13 and Figure 19.15.

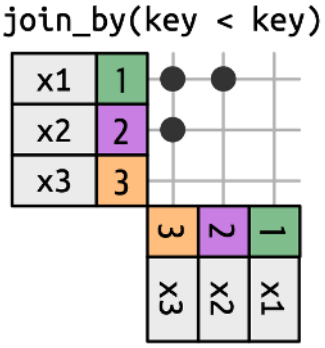


Figure 19.15: An inequality join where `x` is joined to `y` on rows where the key of `x` is less than the key of `y`. This makes a triangular shape in the top-left corner.

Inequality joins are extremely general, so general that it's hard to come up with meaningful specific use cases. One small useful technique is to use them to restrict the cross join so that instead of generating all permutations, we generate all combinations:

```
df <- tibble(id = 1:4, name = c("John", "Simon", "Tracy", "Max"))

df |> inner_join(df, join_by(id < id))
#> # A tibble: 6 × 4
#>    id.x name.x  id.y name.y
#>   <int> <chr>  <int> <chr>
#> 1     1 John       2 Simon
#> 2     1 John       3 Tracy
#> 3     1 John       4 Max
#> 4     2 Simon      3 Tracy
```

```
#> 5      2 Simon       4 Max
#> 6      3 Tracy       4 Max
```

19.5.3 Rolling joins

Rolling joins are a special type of inequality join where instead of getting *every* row that satisfies the inequality, you get just the closest row, as in Figure 19.16. You can turn any inequality join into a rolling join by adding `closest()`. For example `join_by(closest(x <= y))` matches the smallest `y` that's greater than or equal to x, and `join_by(closest(x > y))` matches the biggest `y` that's less than `x`.

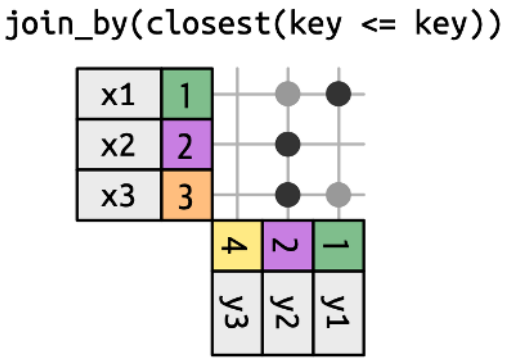


Figure 19.16: A rolling join is similar to a greater-than-or-equal inequality join but only matches the first value.

Rolling joins are particularly useful when you have two tables of dates that don't perfectly line up and you want to find (e.g.) the closest date in table 1 that comes before (or after) some date in table 2.

For example, imagine that you're in charge of the party planning commission for your office. Your company is rather cheap so instead of having individual parties, you only have a party once each quarter. The rules for determining when a party will be held are a little complex: parties are always on a Monday, you skip the first week of January since a lot of people are on holiday, and the first Monday of Q3 2022 is July 4, so that has to be pushed back a week. That leads to the following party days:

```
parties <- tibble(
  q = 1:4,
  party = ymd(c("2022-01-10", "2022-04-04", "2022-07-11", "2022-10-03"))
)
```

Now imagine that you have a table of employee birthdays:

```
set.seed(123)
employees <- tibble(
  name = sample(babynames::babynames$name, 100),
  birthday = ymd("2022-01-01") + (sample(365, 100, replace = TRUE) - 1)
)
```

```
employees
#> # A tibble: 100 × 2
#>   name     birthday
#>   <chr>    <date>
#> 1 Kemba    2022-01-22
#> 2 Orean    2022-06-26
#> 3 Kirstyn  2022-02-11
#> 4 Amparo   2022-11-11
#> 5 Belen    2022-03-25
#> 6 Rayshaun 2022-01-11
#> # ℹ 94 more rows
```

And for each employee we want to find the first party date that comes after (or on) their birthday. We can express that with a rolling join:

```
employees |>
  left_join(parties, join_by(closest(birthday >= party)))
#> # A tibble: 100 × 4
#>   name     birthday       q party
#>   <chr>    <date>     <int> <date>
#> 1 Kemba    2022-01-22     1 2022-01-10
#> 2 Orean    2022-06-26     2 2022-04-04
#> 3 Kirstyn  2022-02-11     1 2022-01-10
#> 4 Amparo   2022-11-11     4 2022-10-03
#> 5 Belen    2022-03-25     1 2022-01-10
#> 6 Rayshaun 2022-01-11     1 2022-01-10
#> # ℹ 94 more rows
```

There is, however, one problem with this approach: the folks with birthdays before January 10 don't get a party:

```
employees |>
  anti_join(parties, join_by(closest(birthday >= party)))
#> # A tibble: 2 × 2
#>   name   birthday
#>   <chr>  <date>
#> 1 Maks   2022-01-07
#> 2 Nalani 2022-01-04
```

To resolve that issue we'll need to tackle the problem a different way, with overlap joins.

19.5.4 Overlap joins

Overlap joins provide three helpers that use inequality joins to make it easier to work with intervals:

- `between(x, y_lower, y_upper)` is short for `x >= y_lower, x <= y_upper`.
- `within(x_lower, x_upper, y_lower, y_upper)` is short for `x_lower >= y_lower, x_upper <= y_upper`.
- `overlaps(x_lower, x_upper, y_lower, y_upper)` is short for `x_lower <= y_upper, x_upper >= y_lower`.

Let's continue the birthday example to see how you might use them. There's one problem with the strategy we used above: there's no party preceding the birthdays Jan 1-9. So it might be better to be explicit about the date ranges that each party spans, and make a special case for those early birthdays:

```
parties <- tibble(
  q = 1:4,
  party = ymd(c("2022-01-10", "2022-04-04", "2022-07-11", "2022-10-03")),
  start = ymd(c("2022-01-01", "2022-04-04", "2022-07-11", "2022-10-03")),
  end = ymd(c("2022-04-03", "2022-07-11", "2022-10-02", "2022-12-31"))
)
parties
#> # A tibble: 4 × 4
#>       q party      start      end
#>   <int> <date>     <date>     <date>
#> 1     1 2022-01-10 2022-01-01 2022-04-03
#> 2     2 2022-04-04 2022-04-04 2022-07-11
#> 3     3 2022-07-11 2022-07-11 2022-10-02
#> 4     4 2022-10-03 2022-10-03 2022-12-31
```

Hadley is hopelessly bad at data entry so he also wanted to check that the party periods don't overlap. One way to do this is by using a self-join to check if any start-end interval overlap with another:

```
parties |>
  inner_join(parties, join_by(overlaps(start, end, start, end), q < q)) |>
  select(start.x, end.x, start.y, end.y)
#> # A tibble: 1 × 4
#>   start.x    end.x      start.y    end.y
#>   <date>     <date>     <date>     <date>
#> 1 2022-04-04 2022-07-11 2022-07-11 2022-10-02
```

Ooops, there is an overlap, so let's fix that problem and continue:

```
parties <- tibble(
  q = 1:4,
```

```
  party = ymd(c("2022-01-10", "2022-04-04", "2022-07-11", "2022-10-03")),
  start = ymd(c("2022-01-01", "2022-04-04", "2022-07-11", "2022-10-03")),
  end = ymd(c("2022-04-03", "2022-07-10", "2022-10-02", "2022-12-31"))
)
```

Now we can match each employee to their party. This is a good place to use `unmatched = "error"` because we want to quickly find out if any employees didn't get assigned a party.

```
employees |>
  inner_join(parties, join_by(between(birthday, start, end)), unmatched =
"error")
#> # A tibble: 100 × 6
#>   name     birthday       q party      start      end
#>   <chr>    <date>     <int> <date>     <date>     <date>
#> 1 Kemba    2022-01-22     1 2022-01-10 2022-01-01 2022-04-03
#> 2 Orean    2022-06-26     2 2022-04-04 2022-04-04 2022-07-10
#> 3 Kirstyn  2022-02-11     1 2022-01-10 2022-01-01 2022-04-03
#> 4 Amparo   2022-11-11     4 2022-10-03 2022-10-03 2022-12-31
#> 5 Belen    2022-03-25     1 2022-01-10 2022-01-01 2022-04-03
#> 6 Rayshaun 2022-01-11     1 2022-01-10 2022-01-01 2022-04-03
#> # ℹ 94 more rows
```

19.5.5 Exercises

1. Can you explain what's happening with the keys in this equi join? Why are they different?

```
#> # A tibble: 4 × 3
#>     key val_x val_y
#>   <dbl> <chr> <chr>
#> 1     1 x1    y1
#> 2     2 x2    y2
#> 3     3 x3    <NA>
#> 4     4 <NA>  y3

x |> full_join(y, join_by(key == key), keep = TRUE)
#> # A tibble: 4 × 4
#>   key.x val_x key.y val_y
#>   <dbl> <chr> <dbl> <chr>
#> 1     1 x1        1 y1
#> 2     2 x2        2 y2
```

```
#> 3     3 x3       NA <NA>
#> 4    NA <NA>      4 y3
```

2. When finding if any party period overlapped with another party period we used `q < q` in the `join_by()`? Why? What happens if you remove this inequality?

19.6 Summary

In this chapter, you've learned how to use mutating and filtering joins to combine data from a pair of data frames. Along the way you learned how to identify keys, and the difference between primary and foreign keys. You also understand how joins work and how to figure out how many rows the output will have. Finally, you've gained a glimpse into the power of non-equi joins and seen a few interesting use cases.

This chapter concludes the "Transform" part of the book where the focus was on the tools you could use with individual columns and tibbles. You learned about dplyr and base functions for working with logical vectors, numbers, and complete tables, stringr functions for working with strings, lubridate functions for working with date-times, and forcats functions for working with factors.

In the next part of the book, you'll learn more about getting various types of data into R in a tidy form.

18 Missing values
Import

Import

In this part of the book, you'll learn how to import a wider range of data into R, as well as how to get it into a form useful for analysis. Sometimes this is just a matter of calling a function from the appropriate data import package. But in more complex cases it might require both tidying and transformation in order to get to the tidy rectangle that you'd prefer to work with.

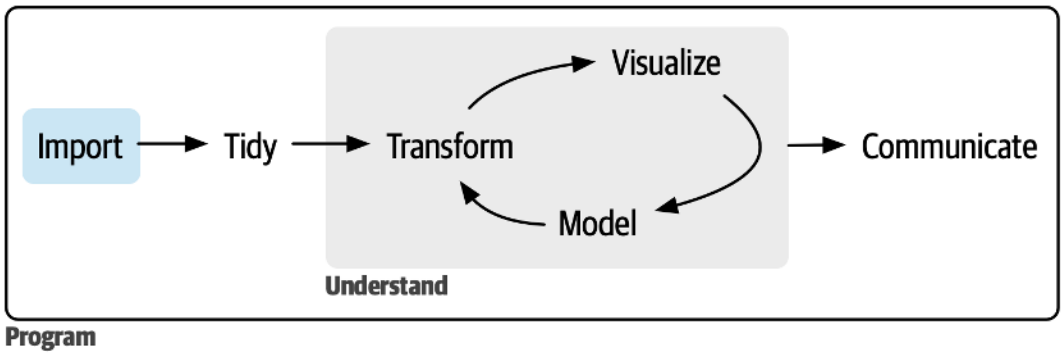


Figure 1: Data import is the beginning of the data science process; without data you can't do data science!

In this part of the book you'll learn how to access data stored in the following ways:

- In 20 Spreadsheets, you'll learn how to import data from Excel spreadsheets and Google Sheets.
- In 21 Databases, you'll learn about getting data out of a database and into R (and you'll also learn a little about how to get data out of R and into a database).
- In 22 Arrow, you'll learn about Arrow, a powerful tool for working with out-of-memory data, particularly when it's stored in the parquet format.
- In 23 Hierarchical data, you'll learn how to work with hierarchical data, including the deeply nested lists produced by data stored in the JSON format.
- In 24 Web scraping, you'll learn web "scraping", the art and science of extracting data from web pages.

There are two important tidyverse packages that we don't discuss here: haven and xml2. If you're working with data from SPSS, Stata, and SAS files, check out the **haven** package, https://haven.tidyverse.org. If you're working with XML data, check out the **xml2** package, https://xml2.r-lib.org. Otherwise, you'll need to do some research to figure which package you'll need to use; google is your friend here 😃.

20 Spreadsheets

20 Spreadsheets

20.1 Introduction

In Chapter 7 you learned about importing data from plain text files like `.csv` and `.tsv`. Now it's time to learn how to get data out of a spreadsheet, either an Excel spreadsheet or a Google Sheet. This will build on much of what you've learned in Chapter 7, but we will also discuss additional considerations and complexities when working with data from spreadsheets.

If you or your collaborators are using spreadsheets for organizing data, we strongly recommend reading the paper "Data Organization in Spreadsheets" by Karl Broman and Kara Woo: https://doi.org/10.1080/00031305.2017.1375989. The best practices presented in this paper will save you much headache when you import data from a spreadsheet into R to analyze and visualize.

20.2 Excel

Microsoft Excel is a widely used spreadsheet software program where data are organized in worksheets inside of spreadsheet files.

20.2.1 Prerequisites

In this section, you'll learn how to load data from Excel spreadsheets in R with the **readxl** package. This package is non-core tidyverse, so you need to load it explicitly, but it is installed automatically when you install the tidyverse package. Later, we'll also use the writexl package, which allows us to create Excel spreadsheets.

20.2.2 Getting started

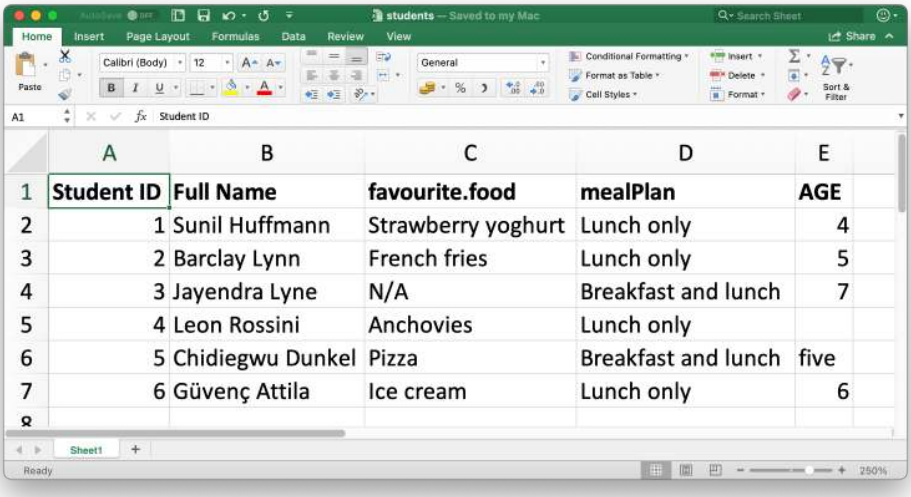
Most of readxl's functions allow you to load Excel spreadsheets into R:

- `read_xls()` reads Excel files with `xls` format.
- `read_xlsx()` read Excel files with `xlsx` format.
- `read_excel()` can read files with both `xls` and `xlsx` format. It guesses the file type based on the input.

These functions all have similar syntax just like other functions we have previously introduced for reading other types of files, e.g., `read_csv()`, `read_table()`, etc. For the rest of the chapter we will focus on using `read_excel()`.

20.2.3 Reading Excel spreadsheets

Figure 20.1 shows what the spreadsheet we're going to read into R looks like in Excel. This spreadsheet can be downloaded an Excel file from https://docs.google.com/spreadsheets/d/1V1nPp1tzOuutXFLb3G9Eyxi3qxeEhnOXUzL5_BcCQ0w/.



	A	B	C	D	E
1	**Student ID**	**Full Name**	**favourite.food**	**mealPlan**	**AGE**
2	1	Sunil Huffmann	Strawberry yoghurt	Lunch only	4
3	2	Barclay Lynn	French fries	Lunch only	5
4	3	Jayendra Lyne	N/A	Breakfast and lunch	7
5	4	Leon Rossini	Anchovies	Lunch only	
6	5	Chidiegwu Dunkel	Pizza	Breakfast and lunch	five
7	6	Güvenç Attila	Ice cream	Lunch only	6

Figure 20.1: Spreadsheet called students.xlsx in Excel.

The first argument to `read_excel()` is the path to the file to read.

```
students <- read_excel("data/students.xlsx")
```

`read_excel()` will read the file in as a tibble.

```
students
#> # A tibble: 6 × 5
#>   `Student ID` `Full Name`      favourite.food     mealPlan            AGE
#>          <dbl> <chr>            <chr>              <chr>               <chr>
#> 1            1 Sunil Huffmann   Strawberry yoghurt Lunch only          4
#> 2            2 Barclay Lynn     French fries       Lunch only          5
#> 3            3 Jayendra Lyne    N/A                Breakfast and lunch 7
#> 4            4 Leon Rossini     Anchovies          Lunch only          <NA>
#> 5            5 Chidiegwu Dunkel Pizza              Breakfast and lunch five
#> 6            6 Güvenç Attila    Ice cream          Lunch only          6
```

We have six students in the data and five variables on each student. However there are a few things we might want to address in this dataset:

1. The column names are all over the place. You can provide column names that follow a consistent format; we recommend `snake_case` using the `col_names` argument.

   ```
   read_excel(
     "data/students.xlsx",
     col_names = c("student_id", "full_name", "favourite_food", "meal_plan",
   "age")
   )
   #> # A tibble: 7 × 5
   #>   student_id full_name        favourite_food     meal_plan           age
   #>   <chr>      <chr>            <chr>              <chr>               <chr>
   #> 1 Student ID Full Name        favourite.food     mealPlan            AGE
   #> 2 1          Sunil Huffmann   Strawberry yoghurt Lunch only          4
   #> 3 2          Barclay Lynn     French fries       Lunch only          5
   #> 4 3          Jayendra Lyne    N/A                Breakfast and lunch 7
   #> 5 4          Leon Rossini     Anchovies          Lunch only          <NA>
   #> 6 5          Chidiegwu Dunkel Pizza              Breakfast and lunch five
   #> 7 6          Güvenç Attila    Ice cream          Lunch only          6
   ```

   Unfortunately, this didn't quite do the trick. We now have the variable names we want, but what was previously the header row now shows up as the first observation in the data. You can explicitly skip that row using the `skip` argument.

   ```
   read_excel(
     "data/students.xlsx",
     col_names = c("student_id", "full_name", "favourite_food", "meal_plan",
   "age"),
     skip = 1
   )
   #> # A tibble: 6 × 5
   #>   student_id full_name        favourite_food     meal_plan           age
   #>        <dbl> <chr>            <chr>              <chr>               <chr>
   #> 1          1 Sunil Huffmann   Strawberry yoghurt Lunch only          4
   #> 2          2 Barclay Lynn     French fries       Lunch only          5
   #> 3          3 Jayendra Lyne    N/A                Breakfast and lunch 7
   #> 4          4 Leon Rossini     Anchovies          Lunch only          <NA>
   #> 5          5 Chidiegwu Dunkel Pizza              Breakfast and lunch five
   #> 6          6 Güvenç Attila    Ice cream          Lunch only          6
   ```

2. In the `favourite_food` column, one of the observations is `N/A`, which stands for "not available" but it's currently not recognized as an `NA` (note the contrast between this `N/A` and the age of the fourth student in the

list). You can specify which character strings should be recognized as `NA`s with the `na` argument. By default, only `""` (empty string, or, in the case of reading from a spreadsheet, an empty cell or a cell with the formula `=NA()`) is recognized as an `NA`.

```
read_excel(
  "data/students.xlsx",
  col_names = c("student_id", "full_name", "favourite_food", "meal_plan",
"age"),
  skip = 1,
  na = c("", "N/A")
)
#> # A tibble: 6 × 5
#>   student_id full_name        favourite_food     meal_plan           age
#>        <dbl> <chr>            <chr>              <chr>               <chr>
#> 1          1 Sunil Huffmann   Strawberry yoghurt Lunch only          4
#> 2          2 Barclay Lynn     French fries       Lunch only          5
#> 3          3 Jayendra Lyne    <NA>               Breakfast and lunch 7
#> 4          4 Leon Rossini     Anchovies          Lunch only          <NA>
#> 5          5 Chidiegwu Dunkel Pizza              Breakfast and lunch five
#> 6          6 Güvenç Attila    Ice cream          Lunch only          6
```

3. One other remaining issue is that `age` is read in as a character variable, but it really should be numeric. Just like with `read_csv()` and friends for reading data from flat files, you can supply a `col_types` argument to `read_excel()` and specify the column types for the variables you read in. The syntax is a bit different, though. Your options are `"skip"`, `"guess"`, `"logical"`, `"numeric"`, `"date"`, `"text"` or `"list"`.

```
read_excel(
  "data/students.xlsx",
  col_names = c("student_id", "full_name", "favourite_food", "meal_plan",
"age"),
  skip = 1,
  na = c("", "N/A"),
  col_types = c("numeric", "text", "text", "text", "numeric")
)
#> Warning: Expecting numeric in E6 / R6C5: got 'five'
#> # A tibble: 6 × 5
#>   student_id full_name        favourite_food     meal_plan             age
#>        <dbl> <chr>            <chr>              <chr>               <dbl>
#> 1          1 Sunil Huffmann   Strawberry yoghurt Lunch only              4
#> 2          2 Barclay Lynn     French fries       Lunch only              5
#> 3          3 Jayendra Lyne    <NA>               Breakfast and lunch     7
#> 4          4 Leon Rossini     Anchovies          Lunch only             NA
#> 5          5 Chidiegwu Dunkel Pizza              Breakfast and lunch    NA
#> 6          6 Güvenç Attila    Ice cream          Lunch only              6
```

However, this didn't quite produce the desired result either. By specifying that `age` should be numeric, we have turned the one cell with the non-numeric entry (which had the value `five`) into an `NA`. In this case, we should read age in as `"text"` and then make the change once the data is loaded in R.

```
students <- read_excel(
  "data/students.xlsx",
  col_names = c("student_id", "full_name", "favourite_food", "meal_plan",
"age"),
  skip = 1,
  na = c("", "N/A"),
  col_types = c("numeric", "text", "text", "text", "text")
)

students <- students |>
  mutate(
    age = if_else(age == "five", "5", age),
    age = parse_number(age)
```

```
  )

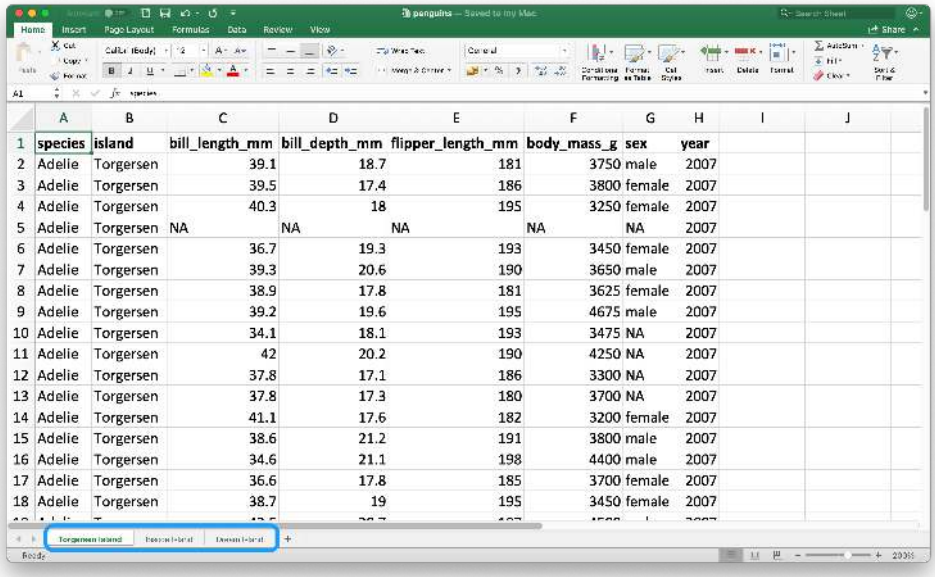
students
#> # A tibble: 6 × 5
#>   student_id full_name        favourite_food     meal_plan             age
#>        <dbl> <chr>            <chr>              <chr>                 <dbl>
#> 1          1 Sunil Huffmann   Strawberry yoghurt Lunch only              4
#> 2          2 Barclay Lynn     French fries       Lunch only              5
#> 3          3 Jayendra Lyne    <NA>               Breakfast and lunch     7
#> 4          4 Leon Rossini     Anchovies          Lunch only             NA
#> 5          5 Chidiegwu Dunkel Pizza              Breakfast and lunch     5
#> 6          6 Güvenç Attila    Ice cream          Lunch only              6
```

It took us multiple steps and trial-and-error to load the data in exactly the format we want, and this is not unexpected. Data science is an iterative process, and the process of iteration can be even more tedious when reading data in from spreadsheets compared to other plain text, rectangular data files because humans tend to input data into spreadsheets and use them not just for data storage but also for sharing and communication.

There is no way to know exactly what the data will look like until you load it and take a look at it. Well, there is one way, actually. You can open the file in Excel and take a peek. If you're going to do so, we recommend making a copy of the Excel file to open and browse interactively while leaving the original data file untouched and reading into R from the untouched file. This will ensure you don't accidentally overwrite anything in the spreadsheet while inspecting it. You should also not be afraid of doing what we did here: load the data, take a peek, make adjustments to your code, load it again, and repeat until you're happy with the result.

20.2.4 Reading worksheets

An important feature that distinguishes spreadsheets from flat files is the notion of multiple sheets, called worksheets. Figure 20.2 shows an Excel spreadsheet with multiple worksheets. The data come from the **palmerpenguins** package, and you can download this spreadsheet as an Excel file from https://docs.google.com/spreadsheets/d/1aFu8lnD_g0yjF5O-K6SFgSEWiHPpgvFCF0NY9D6LXnY/. Each worksheet contains information on penguins from a different island where data were collected.



	A	B	C	D	E	F	G	H	I	J
1	species	island	bill_length_mm	bill_depth_mm	flipper_length_mm	body_mass_g	sex	year		
2	Adelie	Torgersen	39.1	18.7	181	3750	male	2007		
3	Adelie	Torgersen	39.5	17.4	186	3800	female	2007		
4	Adelie	Torgersen	40.3	18	195	3250	female	2007		
5	Adelie	Torgersen	NA	NA	NA	NA	NA	2007		
6	Adelie	Torgersen	36.7	19.3	193	3450	female	2007		
7	Adelie	Torgersen	39.3	20.6	190	3650	male	2007		
8	Adelie	Torgersen	38.9	17.8	181	3625	female	2007		
9	Adelie	Torgersen	39.2	19.6	195	4675	male	2007		
10	Adelie	Torgersen	34.1	18.1	193	3475	NA	2007		
11	Adelie	Torgersen	42	20.2	190	4250	NA	2007		
12	Adelie	Torgersen	37.8	17.1	186	3300	NA	2007		
13	Adelie	Torgersen	37.8	17.3	180	3700	NA	2007		
14	Adelie	Torgersen	41.1	17.6	182	3200	female	2007		
15	Adelie	Torgersen	38.6	21.2	191	3800	male	2007		
16	Adelie	Torgersen	34.6	21.1	198	4400	male	2007		
17	Adelie	Torgersen	36.6	17.8	185	3700	female	2007		
18	Adelie	Torgersen	38.7	19	195	3450	female	2007		

Figure 20.2: Spreadsheet called penguins.xlsx in Excel containing three worksheets.

You can read a single worksheet from a spreadsheet with the `sheet` argument in `read_excel()`. The default, which we've been relying on up until now, is the first sheet.

```
read_excel("data/penguins.xlsx", sheet = "Torgersen Island")
#> # A tibble: 52 × 8
#>   species island    bill_length_mm     bill_depth_mm      flipper_length_mm
#>   <chr>   <chr>     <chr>              <chr>              <chr>
#> 1 Adelie  Torgersen 39.1               18.7               181
```

```
#> 2 Adelie  Torgersen 39.5               17.399999999999999 186
#> 3 Adelie  Torgersen 40.299999999999997 18                 195
#> 4 Adelie  Torgersen NA                NA                 NA
#> 5 Adelie  Torgersen 36.700000000000003 19.3               193
#> 6 Adelie  Torgersen 39.299999999999997 20.6               190
#> # ℹ 46 more rows
#> # ℹ 3 more variables: body_mass_g <chr>, sex <chr>, year <dbl>
```

Some variables that appear to contain numerical data are read in as characters due to the character string `"NA"` not being recognized as a true `NA`.

```
penguins_torgersen <- read_excel("data/penguins.xlsx", sheet = "Torgersen Island",
na = "NA")

penguins_torgersen
#> # A tibble: 52 × 8
#>   species island    bill_length_mm bill_depth_mm flipper_length_mm
#>   <chr>   <chr>              <dbl>         <dbl>             <dbl>
#> 1 Adelie  Torgersen           39.1          18.7               181
#> 2 Adelie  Torgersen           39.5          17.4               186
#> 3 Adelie  Torgersen           40.3          18                 195
#> 4 Adelie  Torgersen           NA            NA                  NA
#> 5 Adelie  Torgersen           36.7          19.3               193
#> 6 Adelie  Torgersen           39.3          20.6               190
#> # ℹ 46 more rows
#> # ℹ 3 more variables: body_mass_g <dbl>, sex <chr>, year <dbl>
```

Alternatively, you can use `excel_sheets()` to get information on all worksheets in an Excel spreadsheet, and then read the one(s) you're interested in.

```
excel_sheets("data/penguins.xlsx")
#> [1] "Torgersen Island" "Biscoe Island"    "Dream Island"
```

Once you know the names of the worksheets, you can read them in individually with `read_excel()`.

```
penguins_biscoe <- read_excel("data/penguins.xlsx", sheet = "Biscoe Island", na =
"NA")
penguins_dream  <- read_excel("data/penguins.xlsx", sheet = "Dream Island", na =
"NA")
```

In this case the full penguins dataset is spread across three worksheets in the spreadsheet. Each worksheet has the same number of columns but different numbers of rows.

```
dim(penguins_torgersen)
#> [1] 52  8
dim(penguins_biscoe)
#> [1] 168   8
dim(penguins_dream)
#> [1] 124   8
```

We can put them together with `bind_rows()`.

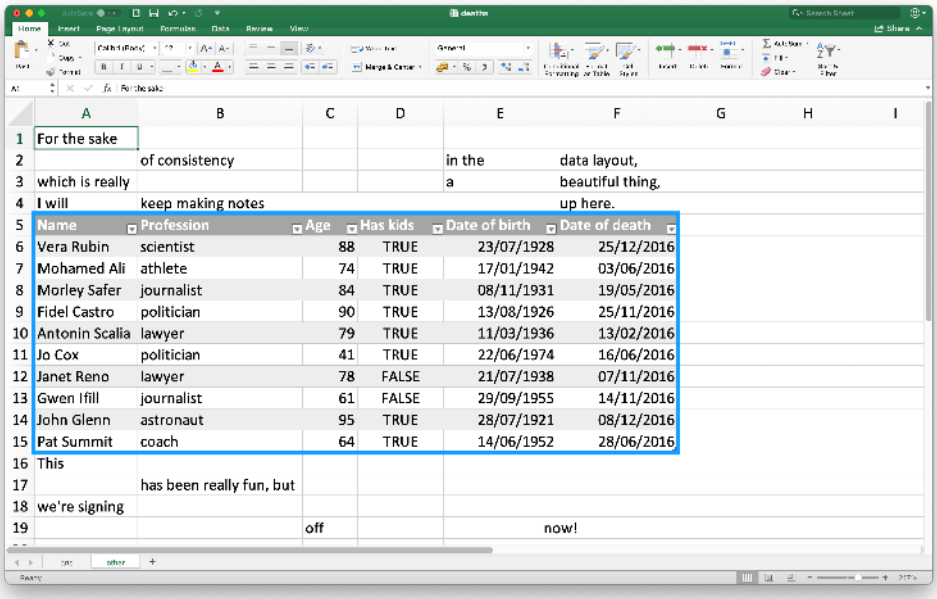
```
penguins <- bind_rows(penguins_torgersen, penguins_biscoe, penguins_dream)
penguins
#> # A tibble: 344 × 8
#>   species island    bill_length_mm bill_depth_mm flipper_length_mm
#>   <chr>   <chr>              <dbl>         <dbl>             <dbl>
#> 1 Adelie  Torgersen           39.1          18.7               181
#> 2 Adelie  Torgersen           39.5          17.4               186
#> 3 Adelie  Torgersen           40.3          18                 195
#> 4 Adelie  Torgersen           NA            NA                  NA
#> 5 Adelie  Torgersen           36.7          19.3               193
#> 6 Adelie  Torgersen           39.3          20.6               190
```

```
#> # ℹ 338 more rows
#> # ℹ 3 more variables: body_mass_g <dbl>, sex <chr>, year <dbl>
```

In Chapter 26 we'll talk about ways of doing this sort of task without repetitive code.

20.2.5 Reading part of a sheet

Since many use Excel spreadsheets for presentation as well as for data storage, it's quite common to find cell entries in a spreadsheet that are not part of the data you want to read into R. Figure 20.3 shows such a spreadsheet: in the middle of the sheet is what looks like a data frame but there is extraneous text in cells above and below the data.



	A	B	C	D	E	F	G	H	I
1	For the sake								
2		of consistency			in the	data layout,			
3	which is really				a	beautiful thing,			
4	I will	keep making notes				up here.			
5	Name	Profession	Age	Has kids	Date of birth	Date of death			
6	Vera Rubin	scientist	88	TRUE	23/07/1928	25/12/2016			
7	Mohamed Ali	athlete	74	TRUE	17/01/1942	03/06/2016			
8	Morley Safer	journalist	84	TRUE	08/11/1931	19/05/2016			
9	Fidel Castro	politician	90	TRUE	13/08/1926	25/11/2016			
10	Antonin Scalia	lawyer	79	TRUE	11/03/1936	13/02/2016			
11	Jo Cox	politician	41	TRUE	22/06/1974	16/06/2016			
12	Janet Reno	lawyer	78	FALSE	21/07/1938	07/11/2016			
13	Gwen Ifill	journalist	61	FALSE	29/09/1955	14/11/2016			
14	John Glenn	astronaut	95	TRUE	28/07/1921	08/12/2016			
15	Pat Summit	coach	64	TRUE	14/06/1952	28/06/2016			
16	This								
17		has been really fun, but							
18	we're signing								
19			off			now!			

Figure 20.3: Spreadsheet called deaths.xlsx in Excel.

This spreadsheet is one of the example spreadsheets provided in the readxl package. You can use the `readxl_example()` function to locate the spreadsheet on your system in the directory where the package is installed. This function returns the path to the spreadsheet, which you can use in `read_excel()` as usual.

```
deaths_path <- readxl_example("deaths.xlsx")
deaths <- read_excel(deaths_path)
#> New names:
#> • `` -> `...2`
#> • `` -> `...3`
#> • `` -> `...4`
#> • `` -> `...5`
#> • `` -> `...6`
deaths
#> # A tibble: 18 × 6
#>   `Lots of people`    ...2       ...3  ...4     ...5          ...6
#>   <chr>               <chr>      <chr> <chr>    <chr>         <chr>
#> 1 simply cannot resi… <NA>       <NA>  <NA>     <NA>          some notes
#> 2 at                  the        top   <NA>     of            their spreadsh…
#> 3 or                  merging    <NA>  <NA>     <NA>          cells
#> 4 Name                Profession Age   Has kids Date of birth Date of death
#> 5 David Bowie         musician   69    TRUE     17175         42379
#> 6 Carrie Fisher       actor      60    TRUE     20749         42731
#> # ℹ 12 more rows
```

The top three rows and the bottom four rows are not part of the data frame. It's possible to eliminate these extraneous rows using the `skip` and `n_max` arguments, but we recommend using cell ranges. In Excel, the top left cell is `A1`. As you move across columns to the right, the cell label moves down the alphabet, i.e. `B1`, `C1`, etc. And as you move down a column, the number in the cell label increases, i.e. `A2`, `A3`, etc.

Here the data we want to read in starts in cell `A5` and ends in cell `F15`. In spreadsheet notation, this is `A5:F15`, which we supply to the `range` argument:

```
read_excel(deaths_path, range = "A5:F15")
#> # A tibble: 10 × 6
#>   Name          Profession   Age `Has kids` `Date of birth`
#>   <chr>         <chr>      <dbl> <lgl>      <dttm>
#> 1 David Bowie   musician      69 TRUE       1947-01-08 00:00:00
#> 2 Carrie Fisher actor         60 TRUE       1956-10-21 00:00:00
#> 3 Chuck Berry   musician      90 TRUE       1926-10-18 00:00:00
#> 4 Bill Paxton   actor         61 TRUE       1955-05-17 00:00:00
#> 5 Prince        musician      57 TRUE       1958-06-07 00:00:00
#> 6 Alan Rickman  actor         69 FALSE      1946-02-21 00:00:00
#> # ℹ 4 more rows
#> # ℹ 1 more variable: `Date of death` <dttm>
```

20.2.6 Data types

In CSV files, all values are strings. This is not particularly true to the data, but it is simple: everything is a string.

The underlying data in Excel spreadsheets is more complex. A cell can be one of four things:

- A boolean, like `TRUE`, `FALSE`, or `NA`.
- A number, like “10” or “10.5”.
- A datetime, which can also include time like “11/1/21” or “11/1/21 3:00 PM”.
- A text string, like “ten”.

When working with spreadsheet data, it’s important to keep in mind that the underlying data can be very different than what you see in the cell. For example, Excel has no notion of an integer. All numbers are stored as floating points, but you can choose to display the data with a customizable number of decimal points. Similarly, dates are actually stored as numbers, specifically the number of seconds since January 1, 1970. You can customize how you display the date by applying formatting in Excel. Confusingly, it’s also possible to have something that looks like a number but is actually a string (e.g., type `'10` into a cell in Excel).

These differences between how the underlying data are stored vs. how they’re displayed can cause surprises when the data are loaded into R. By default readxl will guess the data type in a given column. A recommended workflow is to let readxl guess the column types, confirm that you’re happy with the guessed column types, and if not, go back and re-import specifying `col_types` as shown in Section 20.2.3.

Another challenge is when you have a column in your Excel spreadsheet that has a mix of these types, e.g., some cells are numeric, others text, others dates. When importing the data into R readxl has to make some decisions. In these cases you can set the type for this column to `"list"`, which will load the column as a list of length 1 vectors, where the type of each element of the vector is guessed.

Sometimes data is stored in more exotic ways, like the color of the cell background, or whether or not the text is bold. In such cases, you might find the tidyxl package useful. See https://nacnudus.github.io/spreadsheet-munging-strategies/ for more on strategies for working with non-tabular data from Excel.

20.2.7 Writing to Excel

Let’s create a small data frame that we can then write out. Note that `item` is a factor and `quantity` is an integer.

```
bake_sale <- tibble(
  item     = factor(c("brownie", "cupcake", "cookie")),
  quantity = c(10, 5, 8)
)

bake_sale
#> # A tibble: 3 × 2
#>   item    quantity
#>   <fct>      <dbl>
#> 1 brownie       10
```

```
#> 2 cupcake         5
#> 3 cookie          8
```

You can write data back to disk as an Excel file using the `write_xlsx()` from the writexl package:

```
write_xlsx(bake_sale, path = "data/bake-sale.xlsx")
```

Figure 20.4 shows what the data looks like in Excel. Note that column names are included and bolded. These can be turned off by setting `col_names` and `format_headers` arguments to `FALSE`.

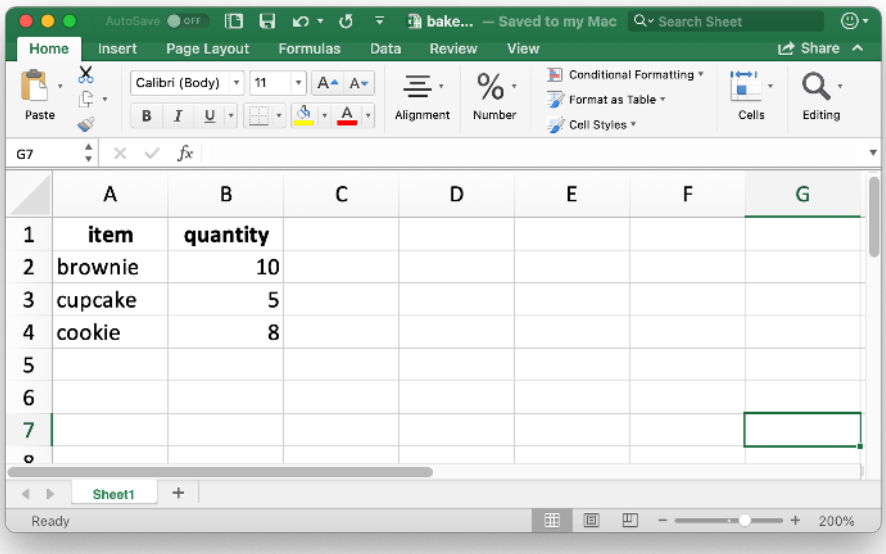


Figure 20.4: Spreadsheet called bake_sale.xlsx in Excel.

Just like reading from a CSV, information on data type is lost when we read the data back in. This makes Excel files unreliable for caching interim results as well. For alternatives, see Section 7.5.

```
read_excel("data/bake-sale.xlsx")
#> # A tibble: 3 × 2
#>   item     quantity
#>   <chr>       <dbl>
#> 1 brownie        10
#> 2 cupcake         5
#> 3 cookie          8
```

20.2.8 Formatted output

The writexl package is a light-weight solution for writing a simple Excel spreadsheet, but if you're interested in additional features like writing to sheets within a spreadsheet and styling, you will want to use the openxlsx package. We won't go into the details of using this package here, but we recommend reading https://ycphs.github.io/openxlsx/articles/Formatting.html for an extensive discussion on further formatting functionality for data written from R to Excel with openxlsx.

Note that this package is not part of the tidyverse so the functions and workflows may feel unfamiliar. For example, function names are camelCase, multiple functions can't be composed in pipelines, and arguments are in a different order than they tend to be in the tidyverse. However, this is ok. As your R learning and usage expands outside of this book you will encounter lots of different styles used in various R packages that you might use to accomplish specific goals in R. A good way of familiarizing yourself with the coding style used in a new package is to run the examples provided in function documentation to get a feel for the syntax and the output formats as well as reading any vignettes that might come with the package.

20.2.9 Exercises

1. In an Excel file, create the following dataset and save it as `survey.xlsx`. Alternatively, you can download it as an Excel file from here.

	A	B
1	survey_id	n_pets
2	1	0
3	2	1
4	3	N/A
5	4	two
6	5	2
7	6	

Then, read it into R, with `survey_id` as a character variable and `n_pets` as a numerical variable.

```
#> # A tibble: 6 × 2
#>   survey_id n_pets
#>   <chr>      <dbl>
#> 1 1              0
#> 2 2              1
#> 3 3             NA
#> 4 4              2
#> 5 5              2
#> 6 6             NA
```

2. In another Excel file, create the following dataset and save it as `roster.xlsx`. Alternatively, you can download it as an Excel file from here.

	A	B	C
1	**group**	**subgroup**	**id**
2	1	A	1
3			2
4			3
5		B	4
6			5
7			6
8			7
9	2	A	8
10			9
11		B	10
12			11
13			12

Then, read it into R. The resulting data frame should be called `roster` and should look like the following.

```
#> # A tibble: 12 × 3
#>    group subgroup    id
#>    <dbl> <chr>    <dbl>
#>  1     1 A            1
#>  2     1 A            2
#>  3     1 A            3
#>  4     1 B            4
#>  5     1 B            5
#>  6     1 B            6
#>  7     1 B            7
#>  8     2 A            8
#>  9     2 A            9
#> 10     2 B           10
#> 11     2 B           11
#> 12     2 B           12
```

3. In a new Excel file, create the following dataset and save it as `sales.xlsx`. Alternatively, you can download it as an Excel file from here.

	A	B
1	This file contains information on sales.	
2	Data are organized by brand name, and for each brand, we have the ID number for the item sold, and how many are sold.	
3		
4		
5	Brand 1	n
6	1234	8
7	8721	2
8	1822	3
9	Brand 2	n
10	3333	1
11	2156	3
12	3987	6
13	3216	5

a. Read `sales.xlsx` in and save as `sales`. The data frame should look like the following, with `id` and `n` as column names and with 9 rows.

```
#> # A tibble: 9 × 2
#>   id      n
#>   <chr>   <chr>
#> 1 Brand 1 n
#> 2 1234    8
#> 3 8721    2
#> 4 1822    3
#> 5 Brand 2 n
#> 6 3333    1
#> 7 2156    3
#> 8 3987    6
#> 9 3216    5
```

b. Modify `sales` further to get it into the following tidy format with three columns (`brand`, `id`, and `n`) and 7 rows of data. Note that `id` and `n` are numeric, `brand` is a character variable.

```
#> # A tibble: 7 × 3
#>   brand      id     n
#>   <chr>   <dbl> <dbl>
#> 1 Brand 1  1234     8
#> 2 Brand 1  8721     2
#> 3 Brand 1  1822     3
#> 4 Brand 2  3333     1
#> 5 Brand 2  2156     3
#> 6 Brand 2  3987     6
#> 7 Brand 2  3216     5
```

4. Recreate the `bake_sale` data frame, write it out to an Excel file using the `write.xlsx()` function from the openxlsx package.

5. In Chapter 7 you learned about the `janitor::clean_names()` function to turn column names into snake case. Read the `students.xlsx` file that we introduced earlier in this section and use this function to "clean" the column names.

6. What happens if you try to read in a file with `.xlsx` extension with `read_xls()`?

20.3 Google Sheets

Google Sheets is another widely used spreadsheet program. It's free and web-based. Just like with Excel, in Google Sheets data are organized in worksheets (also called sheets) inside of spreadsheet files.

20.3.1 Prerequisites

This section will also focus on spreadsheets, but this time you'll be loading data from a Google Sheet with the **googlesheets4** package. This package is non-core tidyverse as well, you need to load it explicitly.

```
library(googlesheets4)
library(tidyverse)
```

A quick note about the name of the package: googlesheets4 uses v4 of the Sheets API v4 to provide an R interface to Google Sheets, hence the name.

20.3.2 Getting started

The main function of the googlesheets4 package is `read_sheet()`, which reads a Google Sheet from a URL or a file id. This function also goes by the name `range_read()`.

You can also create a brand new sheet with `gs4_create()` or write to an existing sheet with `sheet_write()` and friends.

In this section we'll work with the same datasets as the ones in the Excel section to highlight similarities and differences between workflows for reading data from Excel and Google Sheets. readxl and googlesheets4 packages are both designed to mimic the functionality of the readr package, which provides the `read_csv()` function you've seen in Chapter 7. Therefore, many of the tasks can be accomplished with simply swapping out `read_excel()` for `read_sheet()`. However you'll also see that Excel and Google Sheets don't behave in exactly the same way, therefore other tasks may require further updates to the function calls.

20.3.3 Reading Google Sheets

Figure 20.5 shows what the spreadsheet we're going to read into R looks like in Google Sheets. This is the same dataset as in Figure 20.1, except it's stored in a Google Sheet instead of Excel.

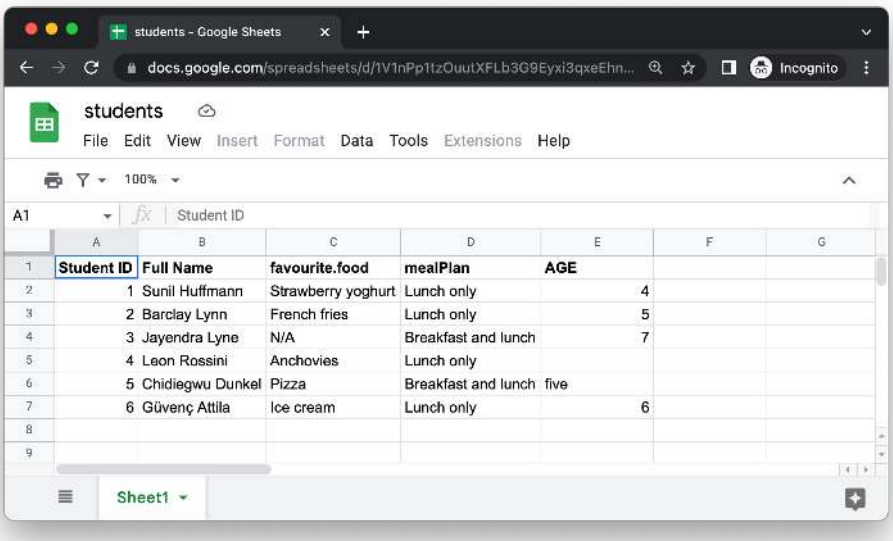


Figure 20.5: Google Sheet called students in a browser window.

The first argument to `read_sheet()` is the URL of the file to read, and it returns a tibble: https://docs.google.com/spreadsheets/d/1V1nPp1tzOuutXFLb3G9Eyxi3qxeEhnOXUzL5_BcCQ0w. These URLs are not pleasant to work with, so you'll often want to identify a sheet by its ID.

```
gs4_deauth()
```

```
students_sheet_id <- "1V1nPp1tzOuutXFLb3G9Eyxi3qxeEhnOXUzL5_BcCQ0w"
students <- read_sheet(students_sheet_id)
#> ✔ Reading from students.
#> ✔ Range Sheet1.
students
```

```
#> # A tibble: 6 × 5
#>   `Student ID` `Full Name`      favourite.food     mealPlan            AGE
#>          <dbl> <chr>            <chr>              <chr>               <list>
#> 1            1 Sunil Huffmann   Strawberry yoghurt Lunch only          <dbl>
#> 2            2 Barclay Lynn     French fries       Lunch only          <dbl>
#> 3            3 Jayendra Lyne    N/A                Breakfast and lunch <dbl>
#> 4            4 Leon Rossini     Anchovies          Lunch only          <NULL>
#> 5            5 Chidiegwu Dunkel Pizza              Breakfast and lunch <chr>
#> 6            6 Güvenç Attila    Ice cream          Lunch only          <dbl>
```

Just like we did with `read_excel()`, we can supply column names, NA strings, and column types to `read_sheet()`.

```
students <- read_sheet(
  students_sheet_id,
  col_names = c("student_id", "full_name", "favourite_food", "meal_plan", "age"),
  skip = 1,
  na = c("", "N/A"),
  col_types = "dcccc"
)
#> ✔ Reading from students.
#> ✔ Range 2:10000000.

students
#> # A tibble: 6 × 5
#>   student_id full_name        favourite_food     meal_plan           age
#>        <dbl> <chr>            <chr>              <chr>               <chr>
#> 1          1 Sunil Huffmann   Strawberry yoghurt Lunch only          4
#> 2          2 Barclay Lynn     French fries       Lunch only          5
#> 3          3 Jayendra Lyne    <NA>               Breakfast and lunch 7
#> 4          4 Leon Rossini     Anchovies          Lunch only          <NA>
#> 5          5 Chidiegwu Dunkel Pizza              Breakfast and lunch five
#> 6          6 Güvenç Attila    Ice cream          Lunch only          6
```

Note that we defined column types a bit differently here, using short codes. For example, “dcccc” stands for “double, character, character, character, character”.

It’s also possible to read individual sheets from Google Sheets as well. Let’s read the “Torgersen Island” sheet from the penguins Google Sheet:

```
penguins_sheet_id <- "1aFu8lnD_g0yjF5O-K6SFgSEWiHPpgvFCF0NY9D6LXnY"
read_sheet(penguins_sheet_id, sheet = "Torgersen Island")
#> ✔ Reading from penguins.
#> ✔ Range ''Torgersen Island''.
#> # A tibble: 52 × 8
#>   species island    bill_length_mm bill_depth_mm flipper_length_mm
#>   <chr>   <chr>     <list>         <list>        <list>
#> 1 Adelie  Torgersen <dbl [1]>      <dbl [1]>     <dbl [1]>
#> 2 Adelie  Torgersen <dbl [1]>      <dbl [1]>     <dbl [1]>
#> 3 Adelie  Torgersen <dbl [1]>      <dbl [1]>     <dbl [1]>
#> 4 Adelie  Torgersen <chr [1]>      <chr [1]>     <chr [1]>
#> 5 Adelie  Torgersen <dbl [1]>      <dbl [1]>     <dbl [1]>
#> 6 Adelie  Torgersen <dbl [1]>      <dbl [1]>     <dbl [1]>
#> # ℹ 46 more rows
#> # ℹ 3 more variables: body_mass_g <list>, sex <chr>, year <dbl>
```

You can obtain a list of all sheets within a Google Sheet with `sheet_names()`:

```
sheet_names(penguins_sheet_id)
#> [1] "Torgersen Island" "Biscoe Island"    "Dream Island"
```

Finally, just like with `read_excel()`, we can read in a portion of a Google Sheet by defining a `range` in `read_sheet()`. Note that we’re also using the `gs4_example()` function below to locate an example Google Sheet

that comes with the googlesheets4 package.

```
deaths_url <- gs4_example("deaths")
deaths <- read_sheet(deaths_url, range = "A5:F15")
#> ✔ Reading from deaths.
#> ✔ Range A5:F15.
deaths
#> # A tibble: 10 × 6
#>   Name          Profession   Age `Has kids` `Date of birth`
#>   <chr>         <chr>      <dbl> <lgl>      <dttm>
#> 1 David Bowie   musician      69 TRUE       1947-01-08 00:00:00
#> 2 Carrie Fisher actor         60 TRUE       1956-10-21 00:00:00
#> 3 Chuck Berry   musician      90 TRUE       1926-10-18 00:00:00
#> 4 Bill Paxton   actor         61 TRUE       1955-05-17 00:00:00
#> 5 Prince        musician      57 TRUE       1958-06-07 00:00:00
#> 6 Alan Rickman  actor         69 FALSE      1946-02-21 00:00:00
#> # ℹ 4 more rows
#> # ℹ 1 more variable: `Date of death` <dttm>
```

20.3.4 Writing to Google Sheets

You can write from R to Google Sheets with `write_sheet()`. The first argument is the data frame to write, and the second argument is the name (or other identifier) of the Google Sheet to write to:

```
write_sheet(bake_sale, ss = "bake-sale")
```

If you'd like to write your data to a specific (work)sheet inside a Google Sheet, you can specify that with the `sheet` argument as well.

```
write_sheet(bake_sale, ss = "bake-sale", sheet = "Sales")
```

20.3.5 Authentication

While you can read from a public Google Sheet without authenticating with your Google account and with `gs4_deauth()`, reading a private sheet or writing to a sheet requires authentication so that googlesheets4 can view and manage *your* Google Sheets.

When you attempt to read in a sheet that requires authentication, googlesheets4 will direct you to a web browser with a prompt to sign in to your Google account and grant permission to operate on your behalf with Google Sheets. However, if you want to specify a specific Google account, authentication scope, etc. you can do so with `gs4_auth()`, e.g., `gs4_auth(email = "mine@example.com")`, which will force the use of a token associated with a specific email. For further authentication details, we recommend reading the documentation googlesheets4 auth vignette: https://googlesheets4.tidyverse.org/articles/auth.html.

20.3.6 Exercises

1. Read the `students` dataset from earlier in the chapter from Excel and also from Google Sheets, with no additional arguments supplied to the `read_excel()` and `read_sheet()` functions. Are the resulting data frames in R exactly the same? If not, how are they different?

2. Read the Google Sheet titled survey from https://pos.it/r4ds-survey, with `survey_id` as a character variable and `n_pets` as a numerical variable.

3. Read the Google Sheet titled roster from https://pos.it/r4ds-roster. The resulting data frame should be called `roster` and should look like the following.

```
#> # A tibble: 12 × 3
#>    group subgroup    id
#>    <dbl> <chr>    <dbl>
#>  1     1 A            1
#>  2     1 A            2
#>  3     1 A            3
#>  4     1 B            4
#>  5     1 B            5
```

```
#>  6     1 B            6
#>  7     1 B            7
#>  8     2 A            8
#>  9     2 A            9
#> 10     2 B           10
#> 11     2 B           11
#> 12     2 B           12
```

20.4 Summary

Microsoft Excel and Google Sheets are two of the most popular spreadsheet systems. Being able to interact with data stored in Excel and Google Sheets files directly from R is a superpower! In this chapter you learned how to read data into R from spreadsheets from Excel with `read_excel()` from the readxl package and from Google Sheets with `read_sheet()` from the googlesheets4 package. These functions work very similarly to each other and have similar arguments for specifying column names, NA strings, rows to skip on top of the file you're reading in, etc. Additionally, both functions make it possible to read a single sheet from a spreadsheet as well.

On the other hand, writing to an Excel file requires a different package and function (`writexl::write_xlsx()`) while you can write to a Google Sheet with the googlesheets4 package, with `write_sheet()`.

In the next chapter, you'll learn about a different data source and how to read data from that source into R: databases.

21 Databases

21.1 Introduction

A huge amount of data lives in databases, so it's essential that you know how to access it. Sometimes you can ask someone to download a snapshot into a `.csv` for you, but this gets painful quickly: every time you need to make a change you'll have to communicate with another human. You want to be able to reach into the database directly to get the data you need, when you need it.

In this chapter, you'll first learn the basics of the DBI package: how to use it to connect to a database and then retrieve data with a SQL[1] query. **SQL**, short for **s**tructured **q**uery **l**anguage, is the lingua franca of databases, and is an important language for all data scientists to learn. That said, we're not going to start with SQL, but instead we'll teach you dbplyr, which can translate your dplyr code to the SQL. We'll use that as a way to teach you some of the most important features of SQL. You won't become a SQL master by the end of the chapter, but you will be able to identify the most important components and understand what they do.

21.1.1 Prerequisites

In this chapter, we'll introduce DBI and dbplyr. DBI is a low-level interface that connects to databases and executes SQL; dbplyr is a high-level interface that translates your dplyr code to SQL queries then executes them with DBI.

21.2 Database basics

At the simplest level, you can think about a database as a collection of data frames, called **tables** in database terminology. Like a data frame, a database table is a collection of named columns, where every value in the column is the same type. There are three high level differences between data frames and database tables:

- Database tables are stored on disk and can be arbitrarily large. Data frames are stored in memory, and are fundamentally limited (although that limit is still plenty large for many problems).
- Database tables almost always have indexes. Much like the index of a book, a database index makes it possible to quickly find rows of interest without having to look at every single row. Data frames and tibbles don't have indexes, but data.tables do, which is one of the reasons that they're so fast.
- Most classical databases are optimized for rapidly collecting data, not analyzing existing data. These databases are called **row-oriented** because the data is stored row-by-row, rather than column-by-

column like R. More recently, there's been much development of **column-oriented** databases that make analyzing the existing data much faster.

Databases are run by database management systems (**DBMS**'s for short), which come in three basic forms:

- **Client-server** DBMS's run on a powerful central server, which you connect from your computer (the client). They are great for sharing data with multiple people in an organization. Popular client-server DBMS's include PostgreSQL, MariaDB, SQL Server, and Oracle.
- **Cloud** DBMS's, like Snowflake, Amazon's RedShift, and Google's BigQuery, are similar to client server DBMS's, but they run in the cloud. This means that they can easily handle extremely large datasets and can automatically provide more compute resources as needed.
- **In-process** DBMS's, like SQLite or duckdb, run entirely on your computer. They're great for working with large datasets where you're the primary user.

21.3 Connecting to a database

To connect to the database from R, you'll use a pair of packages:

- You'll always use DBI (**d**ata**b**ase **i**nterface) because it provides a set of generic functions that connect to the database, upload data, run SQL queries, etc.

- You'll also use a package tailored for the DBMS you're connecting to. This package translates the generic DBI commands into the specifics needed for a given DBMS. There's usually one package for each DBMS, e.g. RPostgres for PostgreSQL and RMariaDB for MySQL.

If you can't find a specific package for your DBMS, you can usually use the odbc package instead. This uses the ODBC protocol supported by many DBMS. odbc requires a little more setup because you'll also need to install an ODBC driver and tell the odbc package where to find it.

Concretely, you create a database connection using `DBI::dbConnect()`. The first argument selects the DBMS[2], then the second and subsequent arguments describe how to connect to it (i.e. where it lives and the credentials that you need to access it). The following code shows a couple of typical examples:

```
con <- DBI::dbConnect(
  RMariaDB::MariaDB(),
  username = "foo"
)
con <- DBI::dbConnect(
  RPostgres::Postgres(),
  hostname = "databases.mycompany.com",
  port = 1234
)
```

The precise details of the connection vary a lot from DBMS to DBMS so unfortunately we can't cover all the details here. This means you'll need to do a little research on your own. Typically you can ask the other data scientists in your team or talk to your DBA (**d**ata**b**ase **a**dministrator). The initial setup will often take a little fiddling (and maybe some googling) to get it right, but you'll generally only need to do it once.

21.3.1 In this book

Setting up a client-server or cloud DBMS would be a pain for this book, so we'll instead use an in-process DBMS that lives entirely in an R package: duckdb. Thanks to the magic of DBI, the only difference between using duckdb and any other DBMS is how you'll connect to the database. This makes it great to teach with because you can easily run this code as well as easily take what you learn and apply it elsewhere.

Connecting to duckdb is particularly simple because the defaults create a temporary database that is deleted when you quit R. That's great for learning because it guarantees that you'll start from a clean slate every time you restart R:

```
con <- DBI::dbConnect(duckdb::duckdb())
```

duckdb is a high-performance database that's designed very much for the needs of a data scientist. We use it here because it's very easy to get started with, but it's also capable of handling gigabytes of data with great speed. If you want to use duckdb for a real data analysis project, you'll also need to supply the `dbdir` argument to make a persistent database and tell duckdb where to save it. Assuming you're using a project (Chapter 6), it's reasonable to store it in the `duckdb` directory of the current project:

```
con <- DBI::dbConnect(duckdb::duckdb(), dbdir = "duckdb")
```

21.3.2 Load some data

Since this is a new database, we need to start by adding some data. Here we'll add `mpg` and `diamonds` datasets from ggplot2 using `DBI::dbWriteTable()`. The simplest usage of `dbWriteTable()` needs three arguments: a database connection, the name of the table to create in the database, and a data frame of data.

```
dbWriteTable(con, "mpg", ggplot2::mpg)
dbWriteTable(con, "diamonds", ggplot2::diamonds)
```

If you're using duckdb in a real project, we highly recommend learning about `duckdb_read_csv()` and `duckdb_register_arrow()`. These give you powerful and performant ways to quickly load data directly into duckdb, without having to first load it into R. We'll also show off a useful technique for loading multiple files into a database in Section 26.4.1.

21.3.3 DBI basics

You can check that the data is loaded correctly by using a couple of other DBI functions: `dbListTables()` lists all tables in the database[3] and `dbReadTable()` retrieves the contents of a table.

```
dbListTables(con)
#> [1] "diamonds" "mpg"

con |>
  dbReadTable("diamonds") |>
  as_tibble()
#> # A tibble: 53,940 × 10
#>   carat cut       color clarity depth table price     x     y     z
#>   <dbl> <fct>     <fct> <fct>   <dbl> <dbl> <int> <dbl> <dbl> <dbl>
#> 1  0.23 Ideal     E     SI2      61.5    55   326  3.95  3.98  2.43
#> 2  0.21 Premium   E     SI1      59.8    61   326  3.89  3.84  2.31
#> 3  0.23 Good      E     VS1      56.9    65   327  4.05  4.07  2.31
#> 4  0.29 Premium   I     VS2      62.4    58   334  4.2   4.23  2.63
#> 5  0.31 Good      J     SI2      63.3    58   335  4.34  4.35  2.75
#> 6  0.24 Very Good J     VVS2     62.8    57   336  3.94  3.96  2.48
#> # ℹ 53,934 more rows
```

`dbReadTable()` returns a `data.frame` so we use `as_tibble()` to convert it into a tibble so that it prints nicely.

If you already know SQL, you can use `dbGetQuery()` to get the results of running a query on the database:

```
sql <- "
  SELECT carat, cut, clarity, color, price
  FROM diamonds
  WHERE price > 15000
"
as_tibble(dbGetQuery(con, sql))
#> # A tibble: 1,655 × 5
#>   carat cut       clarity color price
#>   <dbl> <fct>     <fct>   <fct> <int>
#> 1  1.54 Premium   VS2     E     15002
#> 2  1.19 Ideal     VVS1    F     15005
#> 3  2.1  Premium   SI1     I     15007
#> 4  1.69 Ideal     SI1     D     15011
#> 5  1.5  Very Good VVS2    G     15013
#> 6  1.73 Very Good VS1     G     15014
#> # ℹ 1,649 more rows
```

If you've never seen SQL before, don't worry! You'll learn more about it shortly. But if you read it carefully, you might guess that it selects five columns of the diamonds dataset and all the rows where `price` is greater than 15,000.

21.4 dbplyr basics

Now that we've connected to a database and loaded up some data, we can start to learn about dbplyr. dbplyr is a dplyr **backend**, which means that you keep writing dplyr code but the backend executes it differently. In this, dbplyr translates to SQL; other backends include dtplyr which translates to data.table, and multidplyr which executes your code on multiple cores.

To use dbplyr, you must first use `tbl()` to create an object that represents a database table:

```
diamonds_db <- tbl(con, "diamonds")
diamonds_db
#> # Source:   table<diamonds> [?? x 10]
#> # Database: DuckDB v0.10.2 [unknown@Linux 6.5.0-1021-azure:R
4.4.0/:memory:]
#>   carat cut       color clarity depth table price     x     y     z
#>   <dbl> <fct>     <fct> <fct>   <dbl> <dbl> <int> <dbl> <dbl> <dbl>
#> 1  0.23 Ideal     E     SI2      61.5    55   326  3.95  3.98  2.43
#> 2  0.21 Premium   E     SI1      59.8    61   326  3.89  3.84  2.31
#> 3  0.23 Good      E     VS1      56.9    65   327  4.05  4.07  2.31
#> 4  0.29 Premium   I     VS2      62.4    58   334  4.2   4.23  2.63
#> 5  0.31 Good      J     SI2      63.3    58   335  4.34  4.35  2.75
#> 6  0.24 Very Good J     VVS2     62.8    57   336  3.94  3.96  2.48
#> # ℹ more rows
```

There are two other common ways to interact with a database. First, many corporate databases are very large so you need some hierarchy to keep all the tables organized. In that case you might need to supply a schema, or a catalog and a schema, in order to pick the table you're interested in:

```
diamonds_db <- tbl(con, in_schema("sales", "diamonds"))
diamonds_db <- tbl(con, in_catalog("north_america", "sales", "diamonds"))
```

Other times you might want to use your own SQL query as a starting point:

```
diamonds_db <- tbl(con, sql("SELECT * FROM diamonds"))
```

This object is **lazy**; when you use dplyr verbs on it, dplyr doesn't do any work: it just records the sequence of operations that you want to perform and only performs them when needed. For example, take the following pipeline:

```
big_diamonds_db <- diamonds_db |>
  filter(price > 15000) |>
  select(carat:clarity, price)

big_diamonds_db
#> # Source:   SQL [?? x 5]
#> # Database: DuckDB v0.10.2 [unknown@Linux 6.5.0-1021-azure:R
4.4.0/:memory:]
#>   carat cut       color clarity price
#>   <dbl> <fct>     <fct> <fct>   <int>
#> 1  1.54 Premium   E     VS2     15002
#> 2  1.19 Ideal     F     VVS1    15005
#> 3  2.1  Premium   I     SI1     15007
#> 4  1.69 Ideal     D     SI1     15011
#> 5  1.5  Very Good G     VVS2    15013
#> 6  1.73 Very Good G     VS1     15014
#> # ℹ more rows
```

You can tell this object represents a database query because it prints the DBMS name at the top, and while it tells you the number of columns, it typically doesn't know the number of rows. This is because finding the total number of rows usually requires executing the complete query, something we're trying to avoid.

You can see the SQL code generated by the dplyr function `show_query()`. If you know dplyr, this is a great way to learn SQL! Write some dplyr code, get dbplyr to translate it to SQL, and then try to figure out how the two languages match up.

```
big_diamonds_db |>
  show_query()
#> <SQL>
#> SELECT carat, cut, color, clarity, price
#> FROM diamonds
#> WHERE (price > 15000.0)
```

To get all the data back into R, you call `collect()`. Behind the scenes, this generates the SQL, calls `dbGetQuery()` to get the data, then turns the result into a tibble:

```
big_diamonds <- big_diamonds_db |>
  collect()
big_diamonds
#> # A tibble: 1,655 × 5
#>   carat cut       color clarity price
#>   <dbl> <fct>     <fct> <fct>   <int>
```

```
#> 1  1.54 Premium   E     VS2     15002
#> 2  1.19 Ideal     F     VVS1    15005
#> 3  2.1  Premium   I     SI1     15007
#> 4  1.69 Ideal     D     SI1     15011
#> 5  1.5  Very Good G     VVS2    15013
#> 6  1.73 Very Good G     VS1     15014
#> # ℹ 1,649 more rows
```

Typically, you'll use dbplyr to select the data you want from the database, performing basic filtering and aggregation using the translations described below. Then, once you're ready to analyse the data with functions that are unique to R, you'll `collect()` the data to get an in-memory tibble, and continue your work with pure R code.

21.5 SQL

The rest of the chapter will teach you a little SQL through the lens of dbplyr. It's a rather non-traditional introduction to SQL but we hope it will get you quickly up to speed with the basics. Luckily, if you understand dplyr you're in a great place to quickly pick up SQL because so many of the concepts are the same.

We'll explore the relationship between dplyr and SQL using a couple of old friends from the nycflights13 package: `flights` and `planes`. These datasets are easy to get into our learning database because dbplyr comes with a function that copies the tables from nycflights13 to our database:

```
dbplyr::copy_nycflights13(con)
#> Creating table: airlines
#> Creating table: airports
#> Creating table: flights
#> Creating table: planes
#> Creating table: weather
flights <- tbl(con, "flights")
planes <- tbl(con, "planes")
```

21.5.1 SQL basics

The top-level components of SQL are called **statements**. Common statements include `CREATE` for defining new tables, `INSERT` for adding data, and `SELECT` for retrieving data. We will focus on `SELECT` statements, also called **queries**, because they are almost exclusively what you'll use as a data scientist.

A query is made up of **clauses**. There are five important clauses: `SELECT`, `FROM`, `WHERE`, `ORDER BY`, and `GROUP BY`. Every query must have the `SELECT`[4] and `FROM`[5] clauses and the simplest query is `SELECT * FROM table`, which selects all columns from the specified table . This is what dbplyr generates for an unadulterated table :

```
flights |> show_query()
#> <SQL>
#> SELECT *
#> FROM flights
planes |> show_query()
#> <SQL>
#> SELECT *
#> FROM planes
```

`WHERE` and `ORDER BY` control which rows are included and how they are ordered:

```
flights |>
  filter(dest == "IAH") |>
  arrange(dep_delay) |>
  show_query()
#> <SQL>
#> SELECT flights.*
#> FROM flights
#> WHERE (dest = 'IAH')
#> ORDER BY dep_delay
```

`GROUP BY` converts the query to a summary, causing aggregation to happen:

```
flights |>
  group_by(dest) |>
  summarize(dep_delay = mean(dep_delay, na.rm = TRUE)) |>
  show_query()
#> <SQL>
#> SELECT dest, AVG(dep_delay) AS dep_delay
#> FROM flights
#> GROUP BY dest
```

There are two important differences between dplyr verbs and SELECT clauses:

- In SQL, case doesn't matter: you can write `select`, `SELECT`, or even `SeLeCt`. In this book we'll stick with the common convention of writing SQL keywords in uppercase to distinguish them from table or variables names.
- In SQL, order matters: you must always write the clauses in the order `SELECT`, `FROM`, `WHERE`, `GROUP BY`, `ORDER BY`. Confusingly, this order doesn't match how the clauses actually evaluated which is first `FROM`, then `WHERE`, `GROUP BY`, `SELECT`, and `ORDER BY`.

The following sections explore each clause in more detail.

Note that while SQL is a standard, it is extremely complex and no database follows it exactly. While the main components that we'll focus on in this book are very similar between DBMS's, there are many minor variations. Fortunately, dbplyr is designed to handle this problem and generates different translations for different databases. It's not perfect, but it's continually improving, and if you hit a problem you can file an issue on GitHub to help us do better.

21.5.2 SELECT

The `SELECT` clause is the workhorse of queries and performs the same job as `select()`, `mutate()`, `rename()`, `relocate()`, and, as you'll learn in the next section, `summarize()`.

`select()`, `rename()`, and `relocate()` have very direct translations to `SELECT` as they just affect where a column appears (if at all) along with its name:

```
planes |>
  select(tailnum, type, manufacturer, model, year) |>
  show_query()
#> <SQL>
#> SELECT tailnum, "type", manufacturer, model, "year"
#> FROM planes

planes |>
  select(tailnum, type, manufacturer, model, year) |>
  rename(year_built = year) |>
  show_query()
#> <SQL>
#> SELECT tailnum, "type", manufacturer, model, "year" AS year_built
#> FROM planes

planes |>
  select(tailnum, type, manufacturer, model, year) |>
  relocate(manufacturer, model, .before = type) |>
  show_query()
#> <SQL>
#> SELECT tailnum, manufacturer, model, "type", "year"
#> FROM planes
```

This example also shows you how SQL does renaming. In SQL terminology renaming is called **aliasing** and is done with `AS`. Note that unlike `mutate()`, the old name is on the left and the new name is on the right.

In the examples above note that `"year"` and `"type"` are wrapped in double quotes. That's because these are **reserved words** in duckdb, so dbplyr quotes them to avoid any potential confusion between

column/table names and SQL operators.

When working with other databases you're likely to see every variable name quoted because only a handful of client packages, like duckdb, know what all the reserved words are, so they quote everything to be safe.

```
SELECT "tailnum", "type", "manufacturer", "model", "year"
FROM "planes"
```

Some other database systems use backticks instead of quotes:

```
SELECT `tailnum`, `type`, `manufacturer`, `model`, `year`
FROM `planes`
```

The translations for `mutate()` are similarly straightforward: each variable becomes a new expression in `SELECT`:

```
flights |>
  mutate(
    speed = distance / (air_time / 60)
  ) |>
  show_query()
#> <SQL>
#> SELECT flights.*, distance / (air_time / 60.0) AS speed
#> FROM flights
```

We'll come back to the translation of individual components (like `/`) in Section 21.6.

21.5.3 FROM

The `FROM` clause defines the data source. It's going to be rather uninteresting for a little while, because we're just using single tables. You'll see more complex examples once we hit the join functions.

21.5.4 GROUP BY

`group_by()` is translated to the `GROUP BY`[6] clause and `summarize()` is translated to the `SELECT` clause:

```
diamonds_db |>
  group_by(cut) |>
  summarize(
    n = n(),
    avg_price = mean(price, na.rm = TRUE)
  ) |>
  show_query()
```

```
#> <SQL>
#> SELECT cut, COUNT(*) AS n, AVG(price) AS avg_price
#> FROM diamonds
#> GROUP BY cut
```

We'll come back to what's happening with translation `n()` and `mean()` in Section 21.6.

21.5.5 WHERE

`filter()` is translated to the `WHERE` clause:

```
flights |>
  filter(dest == "IAH" | dest == "HOU") |>
  show_query()
#> <SQL>
#> SELECT flights.*
#> FROM flights
#> WHERE (dest = 'IAH' OR dest = 'HOU')

flights |>
  filter(arr_delay > 0 & arr_delay < 20) |>
  show_query()
#> <SQL>
#> SELECT flights.*
#> FROM flights
#> WHERE (arr_delay > 0.0 AND arr_delay < 20.0)
```

There are a few important details to note here:

- `|` becomes `OR` and `&` becomes `AND`.
- SQL uses `=` for comparison, not `==`. SQL doesn't have assignment, so there's no potential for confusion there.
- SQL uses only `''` for strings, not `""`. In SQL, `""` is used to identify variables, like R's `` ``.

Another useful SQL operator is `IN`, which is very close to R's `%in%`:

```
flights |>
  filter(dest %in% c("IAH", "HOU")) |>
  show_query()
#> <SQL>
#> SELECT flights.*
```

```
#> FROM flights
#> WHERE (dest IN ('IAH', 'HOU'))
```

SQL uses `NULL` instead of `NA`. `NULL`s behave similarly to `NA`s. The main difference is that while they're "infectious" in comparisons and arithmetic, they are silently dropped when summarizing. dbplyr will remind you about this behavior the first time you hit it:

```
flights |>
  group_by(dest) |>
  summarize(delay = mean(arr_delay))
#> Warning: Missing values are always removed in SQL aggregation functions.
#> Use `na.rm = TRUE` to silence this warning
#> This warning is displayed once every 8 hours.
#> # Source:   SQL [?? x 2]
#> # Database: DuckDB v0.10.2 [unknown@Linux 6.5.0-1021-azure:R
4.4.0/:memory:]
#>   dest  delay
#>   <chr> <dbl>
#> 1 SFO    2.67
#> 2 GSP   15.9
#> 3 SJU    2.52
#> 4 DCA    9.07
#> 5 SNA   -7.87
#> 6 GSO   14.1
#> # i more rows
```

If you want to learn more about how `NULL`s work, you might enjoy "The Three-Valued Logic of SQL" by Markus Winand.

In general, you can work with `NULL`s using the functions you'd use for `NA`s in R:

```
flights |>
  filter(!is.na(dep_delay)) |>
  show_query()
#> <SQL>
#> SELECT flights.*
#> FROM flights
#> WHERE (NOT((dep_delay IS NULL)))
```

This SQL query illustrates one of the drawbacks of dbplyr: while the SQL is correct, it isn't as simple as you might write by hand. In this case, you could drop the parentheses and use a special operator that's easier to read:

```
WHERE "dep_delay" IS NOT NULL
```

Note that if you `filter()` a variable that you created using a summarize, dbplyr will generate a `HAVING` clause, rather than a `WHERE` clause. This is a one of the idiosyncrasies of SQL: `WHERE` is evaluated before `SELECT` and `GROUP BY`, so SQL needs another clause that's evaluated afterwards.

```
diamonds_db |>
  group_by(cut) |>
  summarize(n = n()) |>
  filter(n > 100) |>
  show_query()
#> <SQL>
#> SELECT cut, COUNT(*) AS n
#> FROM diamonds
#> GROUP BY cut
#> HAVING (COUNT(*) > 100.0)
```

21.5.6 ORDER BY

Ordering rows involves a straightforward translation from `arrange()` to the `ORDER BY` clause:

```
flights |>
  arrange(year, month, day, desc(dep_delay)) |>
  show_query()
#> <SQL>
#> SELECT flights.*
#> FROM flights
#> ORDER BY "year", "month", "day", dep_delay DESC
```

Notice how `desc()` is translated to `DESC`: this is one of the many dplyr functions whose name was directly inspired by SQL.

21.5.7 Subqueries

Sometimes it's not possible to translate a dplyr pipeline into a single `SELECT` statement and you need to use a subquery. A **subquery** is just a query used as a data source in the `FROM` clause, instead of the usual table.

dbplyr typically uses subqueries to work around limitations of SQL. For example, expressions in the `SELECT` clause can't refer to columns that were just created. That means that the following (silly) dplyr pipeline needs to happen in two steps: the first (inner) query computes `year1` and then the second (outer) query can compute `year2`.

```
flights |>
  mutate(
    year1 = year + 1,
    year2 = year1 + 1
  ) |>
  show_query()
#> <SQL>
#> SELECT q01.*, year1 + 1.0 AS year2
#> FROM (
#>   SELECT flights.*, "year" + 1.0 AS year1
#>   FROM flights
#> ) q01
```

You'll also see this if you attempted to `filter()` a variable that you just created. Remember, even though `WHERE` is written after `SELECT`, it's evaluated before it, so we need a subquery in this (silly) example:

```
flights |>
  mutate(year1 = year + 1) |>
  filter(year1 == 2014) |>
  show_query()
#> <SQL>
#> SELECT q01.*
#> FROM (
#>   SELECT flights.*, "year" + 1.0 AS year1
#>   FROM flights
#> ) q01
#> WHERE (year1 = 2014.0)
```

Sometimes dbplyr will create a subquery where it's not needed because it doesn't yet know how to optimize that translation. As dbplyr improves over time, these cases will get rarer but will probably never go away.

21.5.8 Joins

If you're familiar with dplyr's joins, SQL joins are very similar. Here's a simple example:

```
flights |>
  left_join(planes |> rename(year_built = year), by = "tailnum") |>
  show_query()
#> <SQL>
#> SELECT
#>   flights.*,
#>   planes."year" AS year_built,
```

```
#>   "type",
#>   manufacturer,
#>   model,
#>   engines,
#>   seats,
#>   speed,
#>   engine
#> FROM flights
#> LEFT JOIN planes
#>   ON (flights.tailnum = planes.tailnum)
```

The main thing to notice here is the syntax: SQL joins use sub-clauses of the `FROM` clause to bring in additional tables, using `ON` to define how the tables are related.

dplyr's names for these functions are so closely connected to SQL that you can easily guess the equivalent SQL for `inner_join()`, `right_join()`, and `full_join()`:

```
SELECT flights.*, "type", manufacturer, model, engines, seats, speed
FROM flights
INNER JOIN planes ON (flights.tailnum = planes.tailnum)

SELECT flights.*, "type", manufacturer, model, engines, seats, speed
FROM flights
RIGHT JOIN planes ON (flights.tailnum = planes.tailnum)

SELECT flights.*, "type", manufacturer, model, engines, seats, speed
FROM flights
FULL JOIN planes ON (flights.tailnum = planes.tailnum)
```

You're likely to need many joins when working with data from a database. That's because database tables are often stored in a highly normalized form, where each "fact" is stored in a single place and to keep a complete dataset for analysis you need to navigate a complex network of tables connected by primary and foreign keys. If you hit this scenario, the dm package, by Tobias Schieferdecker, Kirill Müller, and Darko Bergant, is a life saver. It can automatically determine the connections between tables using the constraints that DBAs often supply, visualize the connections so you can see what's going on, and generate the joins you need to connect one table to another.

21.5.9 Other verbs

dbplyr also translates other verbs like `distinct()`, `slice_*()`, and `intersect()`, and a growing selection of tidyr functions like `pivot_longer()` and `pivot_wider()`. The easiest way to see the full set of what's currently available is to visit the dbplyr website: https://dbplyr.tidyverse.org/reference/.

21.5.10 Exercises

1. What is `distinct()` translated to? How about `head()`?

2. Explain what each of the following SQL queries do and try recreate them using dbplyr.

```
SELECT *
FROM flights
WHERE dep_delay < arr_delay

SELECT *, distance / (air_time / 60) AS speed
FROM flights
```

21.6 Function translations

So far we've focused on the big picture of how dplyr verbs are translated to the clauses of a query. Now we're going to zoom in a little and talk about the translation of the R functions that work with individual columns, e.g., what happens when you use `mean(x)` in a `summarize()`?

To help see what's going on, we'll use a couple of little helper functions that run a `summarize()` or `mutate()` and show the generated SQL. That will make it a little easier to explore a few variations and see how summaries and transformations can differ.

```
summarize_query <- function(df, ...) {
  df |>
    summarize(...) |>
    show_query()
}
mutate_query <- function(df, ...) {
  df |>
    mutate(..., .keep = "none") |>
    show_query()
}
```

Let's dive in with some summaries! Looking at the code below you'll notice that some summary functions, like `mean()`, have a relatively simple translation while others, like `median()`, are much more complex. The complexity is typically higher for operations that are common in statistics but less common in databases.

```
flights |>
  group_by(year, month, day) |>
  summarize_query(
    mean = mean(arr_delay, na.rm = TRUE),
```

```
    median = median(arr_delay, na.rm = TRUE)
  )
#> `summarise()` has grouped output by "year" and "month". You can override
#> using the `.groups` argument.
#> <SQL>
#> SELECT
#>   "year",
#>   "month",
#>   "day",
#>   AVG(arr_delay) AS mean,
#>   PERCENTILE_CONT(0.5) WITHIN GROUP (ORDER BY arr_delay) AS median
#> FROM flights
#> GROUP BY "year", "month", "day"
```

The translation of summary functions becomes more complicated when you use them inside a `mutate()` because they have to turn into so-called **window** functions. In SQL, you turn an ordinary aggregation function into a window function by adding `OVER` after it:

```
flights |>
  group_by(year, month, day) |>
  mutate_query(
    mean = mean(arr_delay, na.rm = TRUE),
  )
#> <SQL>
#> SELECT
#>   "year",
#>   "month",
#>   "day",
#>   AVG(arr_delay) OVER (PARTITION BY "year", "month", "day") AS mean
#> FROM flights
```

In SQL, the `GROUP BY` clause is used exclusively for summaries so here you can see that the grouping has moved from the `PARTITION BY` argument to `OVER`.

Window functions include all functions that look forward or backwards, like `lead()` and `lag()` which look at the “previous” or “next” value respectively:

```
flights |>
  group_by(dest) |>
  arrange(time_hour) |>
  mutate_query(
    lead = lead(arr_delay),
```

```
    lag = lag(arr_delay)
  )
#> <SQL>
#> SELECT
#>   dest,
#>   LEAD(arr_delay, 1, NULL) OVER (PARTITION BY dest ORDER BY time_hour) AS
lead,
#>   LAG(arr_delay, 1, NULL) OVER (PARTITION BY dest ORDER BY time_hour) AS
lag
#> FROM flights
#> ORDER BY time_hour
```

Here it's important to `arrange()` the data, because SQL tables have no intrinsic order. In fact, if you don't use `arrange()` you might get the rows back in a different order every time! Notice for window functions, the ordering information is repeated: the `ORDER BY` clause of the main query doesn't automatically apply to window functions.

Another important SQL function is `CASE WHEN`. It's used as the translation of `if_else()` and `case_when()`, the dplyr function that it directly inspired. Here are a couple of simple examples:

```
flights |>
  mutate_query(
    description = if_else(arr_delay > 0, "delayed", "on-time")
  )
#> <SQL>
#> SELECT CASE WHEN (arr_delay > 0.0) THEN 'delayed' WHEN NOT (arr_delay >
0.0) THEN 'on-time' END AS description
#> FROM flights
flights |>
  mutate_query(
    description =
      case_when(
        arr_delay < -5 ~ "early",
        arr_delay < 5 ~ "on-time",
        arr_delay >= 5 ~ "late"
      )
  )
#> <SQL>
#> SELECT CASE
#> WHEN (arr_delay < -5.0) THEN 'early'
#> WHEN (arr_delay < 5.0) THEN 'on-time'
#> WHEN (arr_delay >= 5.0) THEN 'late'
```

```
#> END AS description
#> FROM flights
```

`CASE WHEN` is also used for some other functions that don't have a direct translation from R to SQL. A good example of this is `cut()`:

```
flights |>
  mutate_query(
    description =  cut(
      arr_delay,
      breaks = c(-Inf, -5, 5, Inf),
      labels = c("early", "on-time", "late")
    )
  )
#> <SQL>
#> SELECT CASE
#> WHEN (arr_delay <= -5.0) THEN 'early'
#> WHEN (arr_delay <= 5.0) THEN 'on-time'
#> WHEN (arr_delay > 5.0) THEN 'late'
#> END AS description
#> FROM flights
```

dbplyr also translates common string and date-time manipulation functions, which you can learn about in `vignette("translation-function", package = "dbplyr")`. dbplyr's translations are certainly not perfect, and there are many R functions that aren't translated yet, but dbplyr does a surprisingly good job covering the functions that you'll use most of the time.

21.7 Summary

In this chapter you learned how to access data from databases. We focused on dbplyr, a dplyr "backend" that allows you to write the dplyr code you're familiar with, and have it be automatically translated to SQL. We used that translation to teach you a little SQL; it's important to learn some SQL because it's *the* most commonly used language for working with data and knowing some will make it easier for you to communicate with other data folks who don't use R. If you've finished this chapter and would like to learn more about SQL. We have two recommendations:

- *SQL for Data Scientists* by Renée M. P. Teate is an introduction to SQL designed specifically for the needs of data scientists, and includes examples of the sort of highly interconnected data you're likely to encounter in real organizations.
- *Practical SQL* by Anthony DeBarros is written from the perspective of a data journalist (a data scientist specialized in telling compelling stories) and goes into more detail about getting your data into a database and running your own DBMS.

In the next chapter, we'll learn about another dplyr backend for working with large data: arrow. Arrow is designed for working with large files on disk, and is a natural complement to databases.

22 Arrow

22.1 Introduction

CSV files are designed to be easily read by humans. They're a good interchange format because they're very simple and they can be read by every tool under the sun. But CSV files aren't very efficient: you have to do quite a lot of work to read the data into R. In this chapter, you'll learn about a powerful alternative: the parquet format, an open standards-based format widely used by big data systems.

We'll pair parquet files with Apache Arrow, a multi-language toolbox designed for efficient analysis and transport of large datasets. We'll use Apache Arrow via the arrow package, which provides a dplyr backend allowing you to analyze larger-than-memory datasets using familiar dplyr syntax. As an additional benefit, arrow is extremely fast: you'll see some examples later in the chapter.

Both arrow and dbplyr provide dplyr backends, so you might wonder when to use each. In many cases, the choice is made for you, as the data is already in a database or in parquet files, and you'll want to work with it as is. But if you're starting with your own data (perhaps CSV files), you can either load it into a database or convert it to parquet. In general, it's hard to know what will work best, so in the early stages of your analysis we'd encourage you to try both and pick the one that works the best for you.

(A big thanks to Danielle Navarro who contributed the initial version of this chapter.)

22.1.1 Prerequisites

In this chapter, we'll continue to use the tidyverse, particularly dplyr, but we'll pair it with the arrow package which is designed specifically for working with large data.

```
library(tidyverse)
library(arrow)
```

Later in the chapter, we'll also see some connections between arrow and duckdb, so we'll also need dbplyr and duckdb.

22.2 Getting the data

We begin by getting a dataset worthy of these tools: a dataset of item checkouts from Seattle public libraries, available online at data.seattle.gov/Community/Checkouts-by-Title/tmmm-ytt6. This dataset contains 41,389,465 rows that tell you how many times each book was checked out each month from April 2005 to October 2022.

The following code will get you a cached copy of the data. The data is a 9GB CSV file, so it will take some time to download. I highly recommend using `curl::multi_download()` to get very large files as it's built for exactly this purpose: it gives you a progress bar and it can resume the download if its interrupted.

```
dir.create("data", showWarnings = FALSE)

curl::multi_download(
  "https://r4ds.s3.us-west-2.amazonaws.com/seattle-library-checkouts.csv",
  "data/seattle-library-checkouts.csv",
  resume = TRUE
)
#> # A tibble: 1 × 10
#>   success status_code resumefrom url                       destfile
error
#>   <lgl>         <int>      <dbl> <chr>                     <chr>
<chr>
#> 1 TRUE            200          0 https://r4ds.s3.us-we… data/seattle-l…
<NA>
#> # ℹ 4 more variables: type <chr>, modified <dttm>, time <dbl>,
#> #   headers <list>
```

22.3 Opening a dataset

Let's start by taking a look at the data. At 9 GB, this file is large enough that we probably don't want to load the whole thing into memory. A good rule of thumb is that you usually want at least twice as much memory as the size of the data, and many laptops top out at 16 GB. This means we want to avoid `read_csv()` and instead use the `arrow::open_dataset()`:

```
seattle_csv <- open_dataset(
  sources = "data/seattle-library-checkouts.csv",
  col_types = schema(ISBN = string()),
  format = "csv"
)
```

What happens when this code is run? `open_dataset()` will scan a few thousand rows to figure out the structure of the dataset. The `ISBN` column contains blank values for the first 80,000 rows, so we have to specify the column type to help arrow work out the data structure. Once the data has been scanned by `open_dataset()`, it records what it's found and stops; it will only read further rows as you specifically request them. This metadata is what we see if we print `seattle_csv`:

```
seattle_csv
#> FileSystemDataset with 1 csv file
#> UsageClass: string
#> CheckoutType: string
#> MaterialType: string
#> CheckoutYear: int64
#> CheckoutMonth: int64
#> Checkouts: int64
#> Title: string
#> ISBN: string
#> Creator: string
#> Subjects: string
#> Publisher: string
#> PublicationYear: string
```

The first line in the output tells you that `seattle_csv` is stored locally on-disk as a single CSV file; it will only be loaded into memory as needed. The remainder of the output tells you the column type that arrow has imputed for each column.

We can see what's actually in with `glimpse()`. This reveals that there are ~41 million rows and 12 columns, and shows us a few values.

```
seattle_csv |> glimpse()
#> FileSystemDataset with 1 csv file
#> 41,389,465 rows x 12 columns
#> $ UsageClass      <string> "Physical", "Physical", "Digital", "Physical",
"Ph…
#> $ CheckoutType    <string> "Horizon", "Horizon", "OverDrive", "Horizon",
"Hor…
#> $ MaterialType    <string> "BOOK", "BOOK", "EBOOK", "BOOK", "SOUNDDISC",
"BOO…
#> $ CheckoutYear     <int64> 2016, 2016, 2016, 2016, 2016, 2016, 2016, 2016,
20…
#> $ CheckoutMonth    <int64> 6, 6, 6, 6, 6, 6, 6, 6, 6, 6, 6, 6, 6, 6, 6, 6,
6,…
#> $ Checkouts        <int64> 1, 1, 1, 1, 1, 1, 1, 1, 4, 1, 1, 2, 3, 2, 1, 3,
2,…
#> $ Title           <string> "Super rich : a guide to having it all /
Russell S…
#> $ ISBN            <string> "", "", "", "", "", "", "", "", "", "", "", "",
""…
#> $ Creator         <string> "Simmons, Russell", "Barclay, James, 1965-",
```

```
"Tim …
#> $ Subjects        <string> "Self realization, Conduct of life, Attitude
Psych…
#> $ Publisher       <string> "Gotham Books,", "Pyr,", "Random House, Inc.",
"Di…
#> $ PublicationYear <string> "c2011.", "2010.", "2015", "2005.", "c2004.",
"c20…
```

We can start to use this dataset with dplyr verbs, using `collect()` to force arrow to perform the computation and return some data. For example, this code tells us the total number of checkouts per year:

```
seattle_csv |>
  group_by(CheckoutYear) |>
  summarise(Checkouts = sum(Checkouts)) |>
  arrange(CheckoutYear) |>
  collect()
#> # A tibble: 18 × 2
#>   CheckoutYear Checkouts
#>          <int>     <int>
#> 1         2005   3798685
#> 2         2006   6599318
#> 3         2007   7126627
#> 4         2008   8438486
#> 5         2009   9135167
#> 6         2010   8608966
#> # ℹ 12 more rows
```

Thanks to arrow, this code will work regardless of how large the underlying dataset is. But it's currently rather slow: on Hadley's computer, it took ~10s to run. That's not terrible given how much data we have, but we can make it much faster by switching to a better format.

22.4 The parquet format

To make this data easier to work with, let's switch to the parquet file format and split it up into multiple files. The following sections will first introduce you to parquet and partitioning, and then apply what we learned to the Seattle library data.

22.4.1 Advantages of parquet

Like CSV, parquet is used for rectangular data, but instead of being a text format that you can read with any file editor, it's a custom binary format designed specifically for the needs of big data. This means that:

- Parquet files are usually smaller than the equivalent CSV file. Parquet relies on efficient encodings to keep file size down, and supports file compression. This helps make parquet files fast because there's less data to move from disk to memory.
- Parquet files have a rich type system. As we talked about in Section 7.3, a CSV file does not provide any information about column types. For example, a CSV reader has to guess whether `"08-10-2022"` should be parsed as a string or a date. In contrast, parquet files store data in a way that records the type along with the data.
- Parquet files are "column-oriented". This means that they're organized column-by-column, much like R's data frame. This typically leads to better performance for data analysis tasks compared to CSV files, which are organized row-by-row.
- Parquet files are "chunked", which makes it possible to work on different parts of the file at the same time, and, if you're lucky, to skip some chunks altogether.

There's one primary disadvantage to parquet files: they are no longer "human readable", i.e. if you look at a parquet file using `readr::read_file()`, you'll just see a bunch of gibberish.

22.4.2 Partitioning

As datasets get larger and larger, storing all the data in a single file gets increasingly painful and it's often useful to split large datasets across many files. When this structuring is done intelligently, this strategy can lead to significant improvements in performance because many analyses will only require a subset of the files.

There are no hard and fast rules about how to partition your dataset: the results will depend on your data, access patterns, and the systems that read the data. You're likely to need to do some experimentation before you find the ideal partitioning for your situation. As a rough guide, arrow suggests that you avoid files smaller than 20MB and larger than 2GB and avoid partitions that produce more than 10,000 files. You should also try to partition by variables that you filter by; as you'll see shortly, that allows arrow to skip a lot of work by reading only the relevant files.

22.4.3 Rewriting the Seattle library data

Let's apply these ideas to the Seattle library data to see how they play out in practice. We're going to partition by `CheckoutYear`, since it's likely some analyses will only want to look at recent data and partitioning by year yields 18 chunks of a reasonable size.

To rewrite the data we define the partition using `dplyr::group_by()` and then save the partitions to a directory with `arrow::write_dataset()`. `write_dataset()` has two important arguments: a directory where we'll create the files and the format we'll use.

```
pq_path <- "data/seattle-library-checkouts"
```

```
seattle_csv |>
  group_by(CheckoutYear) |>
  write_dataset(path = pq_path, format = "parquet")
```

This takes about a minute to run; as we'll see shortly this is an initial investment that pays off by making future operations much much faster.

Let's take a look at what we just produced:

```
tibble(
  files = list.files(pq_path, recursive = TRUE),
  size_MB = file.size(file.path(pq_path, files)) / 1024^2
)
#> # A tibble: 18 × 2
#>   files                              size_MB
#>   <chr>                                <dbl>
#> 1 CheckoutYear=2005/part-0.parquet     109.
#> 2 CheckoutYear=2006/part-0.parquet     164.
#> 3 CheckoutYear=2007/part-0.parquet     178.
#> 4 CheckoutYear=2008/part-0.parquet     195.
#> 5 CheckoutYear=2009/part-0.parquet     214.
#> 6 CheckoutYear=2010/part-0.parquet     222.
#> # ℹ 12 more rows
```

Our single 9GB CSV file has been rewritten into 18 parquet files. The file names use a "self-describing" convention used by the Apache Hive project. Hive-style partitions name folders with a "key=value" convention, so as you might guess, the `CheckoutYear=2005` directory contains all the data where `CheckoutYear` is 2005. Each file is between 100 and 300 MB and the total size is now around 4 GB, a little over half the size of the original CSV file. This is as we expect since parquet is a much more efficient format.

22.5 Using dplyr with arrow

Now we've created these parquet files, we'll need to read them in again. We use `open_dataset()` again, but this time we give it a directory:

```
seattle_pq <- open_dataset(pq_path)
```

Now we can write our dplyr pipeline. For example, we could count the total number of books checked out in each month for the last five years:

```
query <- seattle_pq |>
  filter(CheckoutYear >= 2018, MaterialType == "BOOK") |>
```

```
  group_by(CheckoutYear, CheckoutMonth) |>
  summarize(TotalCheckouts = sum(Checkouts)) |>
  arrange(CheckoutYear, CheckoutMonth)
```

Writing dplyr code for arrow data is conceptually similar to dbplyr, Chapter 21: you write dplyr code, which is automatically transformed into a query that the Apache Arrow C++ library understands, which is then executed when you call `collect()`. If we print out the `query` object we can see a little information about what we expect Arrow to return when the execution takes place:

```
query
#> FileSystemDataset (query)
#> CheckoutYear: int32
#> CheckoutMonth: int64
#> TotalCheckouts: int64
#>
#> * Grouped by CheckoutYear
#> * Sorted by CheckoutYear [asc], CheckoutMonth [asc]
#> See $.data for the source Arrow object
```

And we can get the results by calling `collect()`:

```
query |> collect()
#> # A tibble: 58 × 3
#> # Groups:   CheckoutYear [5]
#>   CheckoutYear CheckoutMonth TotalCheckouts
#>          <int>         <int>          <int>
#> 1         2018             1         355101
#> 2         2018             2         309813
#> 3         2018             3         344487
#> 4         2018             4         330988
#> 5         2018             5         318049
#> 6         2018             6         341825
#> # ℹ 52 more rows
```

Like dbplyr, arrow only understands some R expressions, so you may not be able to write exactly the same code you usually would. However, the list of operations and functions supported is fairly extensive and continues to grow; find a complete list of currently supported functions in `?acero`.

22.5.1 Performance

Let's take a quick look at the performance impact of switching from CSV to parquet. First, let's time how long it takes to calculate the number of books checked out in each month of 2021, when the data is stored as a

single large csv:

```
seattle_csv |>
  filter(CheckoutYear == 2021, MaterialType == "BOOK") |>
  group_by(CheckoutMonth) |>
  summarize(TotalCheckouts = sum(Checkouts)) |>
  arrange(desc(CheckoutMonth)) |>
  collect() |>
  system.time()
#>    user  system elapsed
#>  11.951   1.297  11.387
```

Now let's use our new version of the dataset in which the Seattle library checkout data has been partitioned into 18 smaller parquet files:

```
seattle_pq |>
  filter(CheckoutYear == 2021, MaterialType == "BOOK") |>
  group_by(CheckoutMonth) |>
  summarize(TotalCheckouts = sum(Checkouts)) |>
  arrange(desc(CheckoutMonth)) |>
  collect() |>
  system.time()
#>    user  system elapsed
#>   0.263   0.058   0.063
```

The ~100x speedup in performance is attributable to two factors: the multi-file partitioning, and the format of individual files:

- Partitioning improves performance because this query uses `CheckoutYear == 2021` to filter the data, and arrow is smart enough to recognize that it only needs to read 1 of the 18 parquet files.
- The parquet format improves performance by storing data in a binary format that can be read more directly into memory. The column-wise format and rich metadata means that arrow only needs to read the four columns actually used in the query (`CheckoutYear`, `MaterialType`, `CheckoutMonth`, and `Checkouts`).

This massive difference in performance is why it pays off to convert large CSVs to parquet!

22.5.2 Using duckdb with arrow

There's one last advantage of parquet and arrow — it's very easy to turn an arrow dataset into a DuckDB database (Chapter 21) by calling `arrow::to_duckdb()`:

```
seattle_pq |>
  to_duckdb() |>
  filter(CheckoutYear >= 2018, MaterialType == "BOOK") |>
  group_by(CheckoutYear) |>
  summarize(TotalCheckouts = sum(Checkouts)) |>
  arrange(desc(CheckoutYear)) |>
  collect()
#> Warning: Missing values are always removed in SQL aggregation functions.
#> Use `na.rm = TRUE` to silence this warning
#> This warning is displayed once every 8 hours.
#> # A tibble: 5 × 2
#>   CheckoutYear TotalCheckouts
#>          <int>          <dbl>
#> 1         2022        2431502
#> 2         2021        2266438
#> 3         2020        1241999
#> 4         2019        3931688
#> 5         2018        3987569
```

The neat thing about `to_duckdb()` is that the transfer doesn't involve any memory copying, and speaks to the goals of the arrow ecosystem: enabling seamless transitions from one computing environment to another.

22.5.3 Exercises

1. Figure out the most popular book each year.
2. Which author has the most books in the Seattle library system?
3. How has checkouts of books vs ebooks changed over the last 10 years?

22.6 Summary

In this chapter, you've been given a taste of the arrow package, which provides a dplyr backend for working with large on-disk datasets. It can work with CSV files, and it's much much faster if you convert your data to parquet. Parquet is a binary data format that's designed specifically for data analysis on modern computers. Far fewer tools can work with parquet files compared to CSV, but its partitioned, compressed, and columnar structure makes it much more efficient to analyze.

Next up you'll learn about your first non-rectangular data source, which you'll handle using tools provided by the tidyr package. We'll focus on data that comes from JSON files, but the general principles apply to tree-like data regardless of its source.

23 Hierarchical data

23.1 Introduction

In this chapter, you'll learn the art of data **rectangling**: taking data that is fundamentally hierarchical, or tree-like, and converting it into a rectangular data frame made up of rows and columns. This is important because hierarchical data is surprisingly common, especially when working with data that comes from the web.

To learn about rectangling, you'll need to first learn about lists, the data structure that makes hierarchical data possible. Then you'll learn about two crucial tidyr functions: `tidyr::unnest_longer()` and `tidyr::unnest_wider()`. We'll then show you a few case studies, applying these simple functions again and again to solve real problems. We'll finish off by talking about JSON, the most frequent source of hierarchical datasets and a common format for data exchange on the web.

23.1.1 Prerequisites

In this chapter, we'll use many functions from tidyr, a core member of the tidyverse. We'll also use repurrrsive to provide some interesting datasets for rectangling practice, and we'll finish by using jsonlite to read JSON files into R lists.

23.2 Lists

So far you've worked with data frames that contain simple vectors like integers, numbers, characters, date-times, and factors. These vectors are simple because they're homogeneous: every element is of the same data type. If you want to store elements of different types in the same vector, you'll need a **list**, which you create with `list()`:

```
x1 <- list(1:4, "a", TRUE)
x1
#> [[1]]
#> [1] 1 2 3 4
#>
#> [[2]]
#> [1] "a"
#>
#> [[3]]
#> [1] TRUE
```

It's often convenient to name the components, or **children**, of a list, which you can do in the same way as naming the columns of a tibble:

```
x2 <- list(a = 1:2, b = 1:3, c = 1:4)
x2
#> $a
#> [1] 1 2
#>
#> $b
#> [1] 1 2 3
#>
#> $c
#> [1] 1 2 3 4
```

Even for these very simple lists, printing takes up quite a lot of space. A useful alternative is `str()`, which generates a compact display of the **str**ucture, de-emphasizing the contents:

```
str(x1)
#> List of 3
#>  $ : int [1:4] 1 2 3 4
#>  $ : chr "a"
#>  $ : logi TRUE
str(x2)
#> List of 3
#>  $ a: int [1:2] 1 2
#>  $ b: int [1:3] 1 2 3
#>  $ c: int [1:4] 1 2 3 4
```

As you can see, `str()` displays each child of the list on its own line. It displays the name, if present, then an abbreviation of the type, then the first few values.

23.2.1 Hierarchy

Lists can contain any type of object, including other lists. This makes them suitable for representing hierarchical (tree-like) structures:

```
str(x3)
#> List of 2
#>  $ :List of 2
#>   ..$ : num 1
#>   ..$ : num 2
```

```
#>  $ :List of 2
#>   ..$ : num 3
#>   ..$ : num 4
```

This is notably different to `c()`, which generates a flat vector:

```
c(c(1, 2), c(3, 4))
#> [1] 1 2 3 4

x4 <- c(list(1, 2), list(3, 4))
str(x4)
#> List of 4
#>  $ : num 1
#>  $ : num 2
#>  $ : num 3
#>  $ : num 4
```

As lists get more complex, `str()` gets more useful, as it lets you see the hierarchy at a glance:

```
str(x5)
#> List of 2
#>  $ : num 1
#>  $ :List of 2
#>   ..$ : num 2
#>   ..$ :List of 2
#>   .. ..$ : num 3
#>   .. ..$ :List of 2
#>   .. .. ..$ : num 4
#>   .. .. ..$ :List of 1
#>   .. .. .. ..$ : num 5
```

As lists get even larger and more complex, `str()` eventually starts to fail, and you'll need to switch to `View()`[1]. Figure 23.1 shows the result of calling `View(x5)`. The viewer starts by showing just the top level of the list, but you can interactively expand any of the components to see more, as in Figure 23.2. RStudio will also show you the code you need to access that element, as in Figure 23.3. We'll come back to how this code works in Section 27.3.

experience using tools like Google Docs or MS Word, the easiest way to get started with Quarto in RStudio is the visual editor.

In the visual editor you can either use the buttons on the menu bar to insert images, tables, cross-references, etc. or you can use the catch-all ⌘ + / or `Ctrl` + / shortcut to insert just about anything. If you are at the beginning of a line (as shown in Figure 28.5), you can also enter just / to invoke the shortcut.

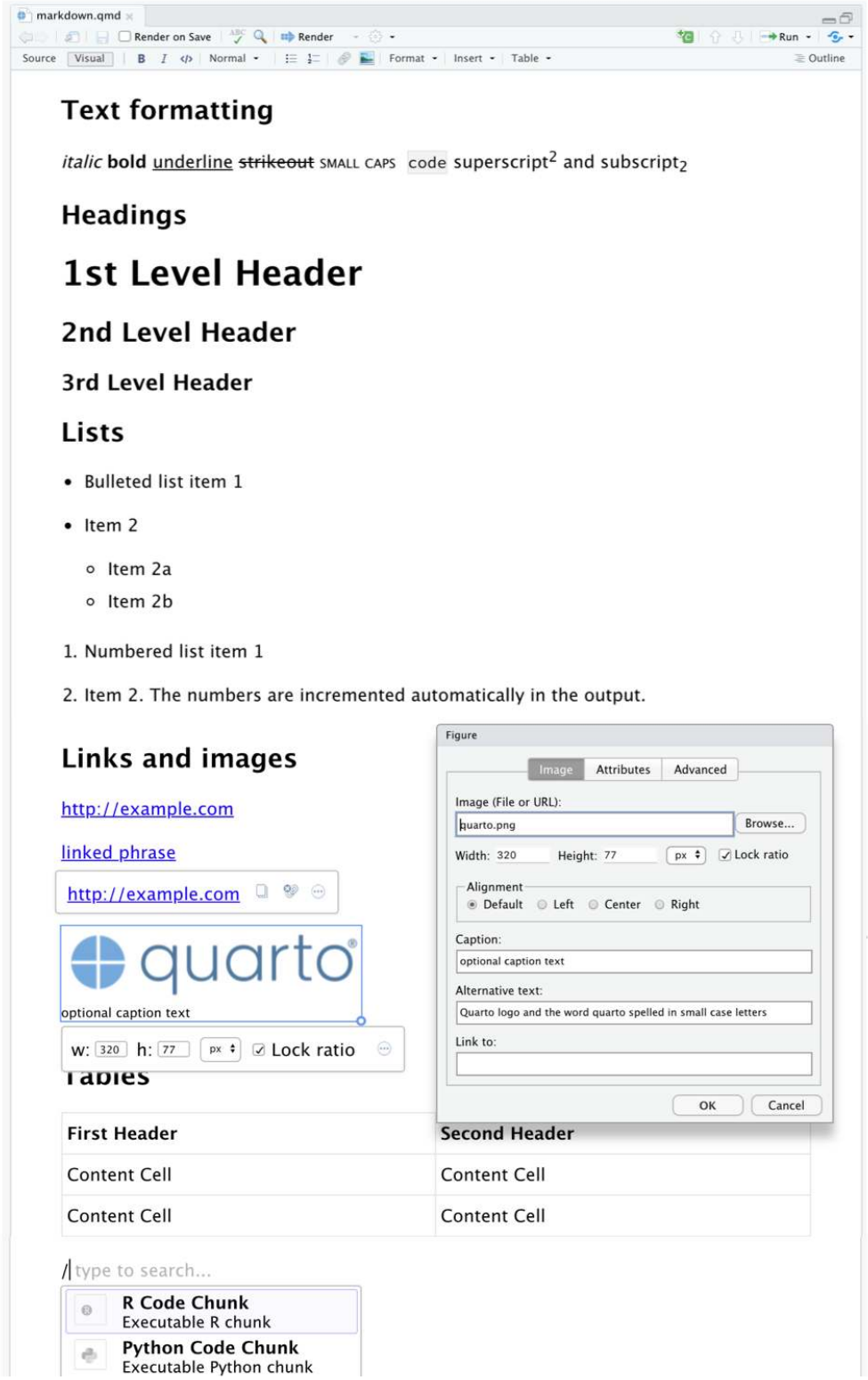




Figure 28.5: Quarto visual editor.

Inserting images and customizing how they are displayed is also facilitated with the visual editor. You can either paste an image from your clipboard directly into the visual editor (and RStudio will place a copy of that image in the project directory and link to it) or you can use the visual editor's Insert > Figure / Image menu to browse to the image you want to insert or paste it's URL. In addition, using the same menu you can resize the image as well as add a caption, alternative text, and a link.

The visual editor has many more features that we haven't enumerated here that you might find useful as you gain experience authoring with it.

Most importantly, while the visual editor displays your content with formatting, under the hood, it saves your content in plain Markdown and you can switch back and forth between the visual and source editors to view and edit your content using either tool.

28.3.1 Exercises

1. Re-create the document in Figure 28.5 using the visual editor.
2. Using the visual editor, insert a code chunk using the Insert menu and then the insert anything tool.
3. Using the visual editor, figure out how to:
    a. Add a footnote.
    b. Add a horizontal rule.
    c. Add a block quote.
4. In the visual editor, go to Insert > Citation and insert a citation to the paper titled Welcome to the Tidyverse using its DOI (digital object identifier), which is 10.21105/joss.01686. Render the document and observe how the reference shows up in the document. What change do you observe in the YAML of your document?

28.4 Source editor

You can also edit Quarto documents using the Source editor in RStudio, without the assist of the Visual editor. While the Visual editor will feel familiar to those with experience writing in tools like Google docs, the Source editor will feel familiar to those with experience writing R scripts or R Markdown documents. The

Source editor can also be useful for debugging any Quarto syntax errors since it's often easier to catch these in plain text.

The guide below shows how to use Pandoc's Markdown for authoring Quarto documents in the source editor.

```
## Text formatting

*italic* **bold** ~~strikeout~~ `code`

superscript^2^ subscript~2~

[underline]{.underline} [small caps]{.smallcaps}

## Headings

# 1st Level Header

## 2nd Level Header

### 3rd Level Header

## Lists

-   Bulleted list item 1

-   Item 2

    -   Item 2a

    -   Item 2b

1.  Numbered list item 1

2.  Item 2.
    The numbers are incremented automatically in the output.

## Links and images

<http://example.com>

[linked phrase](http://example.com)
```

```
![optional caption text](quarto.png){fig-alt="Quarto logo and the word quarto spelled in small case letters"}

## Tables

| First Header | Second Header |
|--------------|---------------|
| Content Cell | Content Cell  |
| Content Cell | Content Cell  |
```

The best way to learn these is simply to try them out. It will take a few days, but soon they will become second nature, and you won't need to think about them. If you forget, you can get to a handy reference sheet with *Help > Markdown Quick Reference*.

28.4.1 Exercises

1. Practice what you've learned by creating a brief CV. The title should be your name, and you should include headings for (at least) education or employment. Each of the sections should include a bulleted list of jobs/degrees. Highlight the year in bold.

2. Using the source editor and the Markdown quick reference, figure out how to:

    a. Add a footnote.
    b. Add a horizontal rule.
    c. Add a block quote.

3. Copy and paste the contents of `diamond-sizes.qmd` from https://github.com/hadley/r4ds/tree/main/quarto in to a local R Quarto document. Check that you can run it, then add text after the frequency polygon that describes its most striking features.

4. Create a document in a Google doc or MS Word (or locate a document you have created previously) with some content in it such as headings, hyperlinks, formatted text, etc. Copy the contents of this document and paste it into a Quarto document in the visual editor. Then, switch over to the source editor and inspect the source code.

28.5 Code chunks

To run code inside a Quarto document, you need to insert a chunk. There are three ways to do so:

1. The keyboard shortcut Cmd + Option + I / Ctrl + Alt + I.

2. The "Insert" button icon in the editor toolbar.

3. By manually typing the chunk delimiters ```` ```{r} ```` and ```` ``` ````.

We'd recommend you learn the keyboard shortcut. It will save you a lot of time in the long run!

You can continue to run the code using the keyboard shortcut that by now (we hope!) you know and love: Cmd/Ctrl + Enter. However, chunks get a new keyboard shortcut: Cmd/Ctrl + Shift + Enter, which runs all the code in the chunk. Think of a chunk like a function. A chunk should be relatively self-contained, and focused around a single task.

The following sections describe the chunk header which consists of ```` ```{r} ````, followed by an optional chunk label and various other chunk options, each on their own line, marked by `#|`.

28.5.1 Chunk label

Chunks can be given an optional label, e.g.

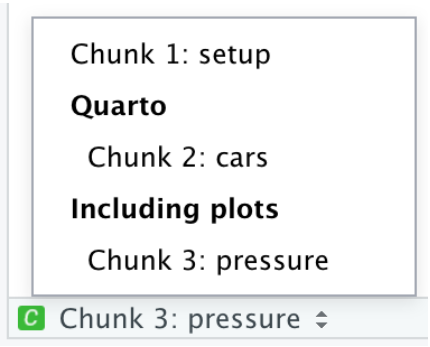
````
```{r}
#| label: simple-addition

1 + 1
```
````

```
#> [1] 2
```

This has three advantages:

1. You can more easily navigate to specific chunks using the drop-down code navigator in the bottom-left of the script editor:



2. Graphics produced by the chunks will have useful names that make them easier to use elsewhere. More on that in Section 28.6.

3. You can set up networks of cached chunks to avoid re-performing expensive computations on every run. More on that in Section 28.8.

Your chunk labels should be short but evocative and should not contain spaces. We recommend using dashes (`-`) to separate words (instead of underscores, `_`) and avoiding other special characters in chunk labels.

You are generally free to label your chunk however you like, but there is one chunk name that imbues special behavior: `setup`. When you're in a notebook mode, the chunk named setup will be run automatically once, before any other code is run.

Additionally, chunk labels cannot be duplicated. Each chunk label must be unique.

28.5.2 Chunk options

Chunk output can be customized with **options**, fields supplied to chunk header. Knitr provides almost 60 options that you can use to customize your code chunks. Here we'll cover the most important chunk options that you'll use frequently. You can see the full list at https://yihui.org/knitr/options.

The most important set of options controls if your code block is executed and what results are inserted in the finished report:

- `eval: false` prevents code from being evaluated. (And obviously if the code is not run, no results will be generated). This is useful for displaying example code, or for disabling a large block of code without commenting each line.
- `include: false` runs the code, but doesn't show the code or results in the final document. Use this for setup code that you don't want cluttering your report.
- `echo: false` prevents code, but not the results from appearing in the finished file. Use this when writing reports aimed at people who don't want to see the underlying R code.
- `message: false` or `warning: false` prevents messages or warnings from appearing in the finished file.
- `results: hide` hides printed output; `fig-show: hide` hides plots.
- `error: true` causes the render to continue even if code returns an error. This is rarely something you'll want to include in the final version of your report, but can be very useful if you need to debug exactly what is going on inside your `.qmd`. It's also useful if you're teaching R and want to deliberately include an error. The default, `error: false` causes rendering to fail if there is a single error in the document.

Each of these chunk options get added to the header of the chunk, following `#|`, e.g., in the following chunk the result is not printed since `eval` is set to false.

```
```{r}
#| label: simple-multiplication
#| eval: false

2 * 2
```
```

The following table summarizes which types of output each option suppresses:

Option	Run code	Show code	Output	Plots	Messages	Warnings
`eval: false`	X		X	X	X	X
`include: false`		X	X	X	X	X
`echo: false`		X				
`results: hide`			X			
`fig-show: hide`				X		
`message: false`					X	
`warning: false`						X

28.5.3 Global options

As you work more with knitr, you will discover that some of the default chunk options don't fit your needs and you want to change them.

You can do this by adding the preferred options in the document YAML, under `execute`. For example, if you are preparing a report for an audience who does not need to see your code but only your results and narrative, you might set `echo: false` at the document level. That will hide the code by default, so only showing the chunks you deliberately choose to show (with `echo: true`). You might consider setting `message: false` and `warning: false`, but that would make it harder to debug problems because you wouldn't see any messages in the final document.

```
title: "My report"
execute:
  echo: false
```

Since Quarto is designed to be multi-lingual (works with R as well as other languages like Python, Julia, etc.), all of the knitr options are not available at the document execution level since some of them only work with knitr and not other engines Quarto uses for running code in other languages (e.g., Jupyter). You can, however, still set these as global options for your document under the `knitr` field, under `opts_chunk`. For example, when writing books and tutorials we set:

```
title: "Tutorial"
knitr:

```

```
  opts_chunk:
    comment: "#>"
    collapse: true
```

This uses our preferred comment formatting and ensures that the code and output are kept closely entwined.

28.5.4 Inline code

There is one other way to embed R code into a Quarto document: directly into the text, with: `` `r ` ``. This can be very useful if you mention properties of your data in the text. For example, the example document used at the start of the chapter had:

> We have data about `` `r nrow(diamonds)` `` diamonds. Only `` `r nrow(diamonds) - nrow(smaller)` `` are larger than 2.5 carats. The distribution of the remainder is shown below:

When the report is rendered, the results of these computations are inserted into the text:

> We have data about 53940 diamonds. Only 126 are larger than 2.5 carats. The distribution of the remainder is shown below:

When inserting numbers into text, `format()` is your friend. It allows you to set the number of `digits` so you don't print to a ridiculous degree of precision, and a `big.mark` to make numbers easier to read. You might combine these into a helper function:

```
comma <- function(x) format(x, digits = 2, big.mark = ",")
comma(3452345)
#> [1] "3,452,345"
comma(.12358124331)
#> [1] "0.12"
```

28.5.5 Exercises

1. Add a section that explores how diamond sizes vary by cut, color, and clarity. Assume you're writing a report for someone who doesn't know R, and instead of setting `echo: false` on each chunk, set a global option.

2. Download `diamond-sizes.qmd` from https://github.com/hadley/r4ds/tree/main/quarto. Add a section that describes the largest 20 diamonds, including a table that displays their most important attributes.

3. Modify `diamonds-sizes.qmd` to use `label_comma()` to produce nicely formatted output. Also include the percentage of diamonds that are larger than 2.5 carats.

28.6 Figures

The figures in a Quarto document can be embedded (e.g., a PNG or JPEG file) or generated as a result of a code chunk.

To embed an image from an external file, you can use the Insert menu in the Visual Editor in RStudio and select Figure / Image. This will pop open a menu where you can browse to the image you want to insert as well as add alternative text or caption to it and adjust its size. In the visual editor you can also simply paste an image from your clipboard into your document and RStudio will place a copy of that image in your project folder.

If you include a code chunk that generates a figure (e.g., includes a `ggplot()` call), the resulting figure will be automatically included in your Quarto document.

28.6.1 Figure sizing

The biggest challenge of graphics in Quarto is getting your figures the right size and shape. There are five main options that control figure sizing: `fig-width`, `fig-height`, `fig-asp`, `out-width` and `out-height`. Image sizing is challenging because there are two sizes (the size of the figure created by R and the size at which it is inserted in the output document), and multiple ways of specifying the size (i.e. height, width, and aspect ratio: pick two of three).

We recommend three of the five options:

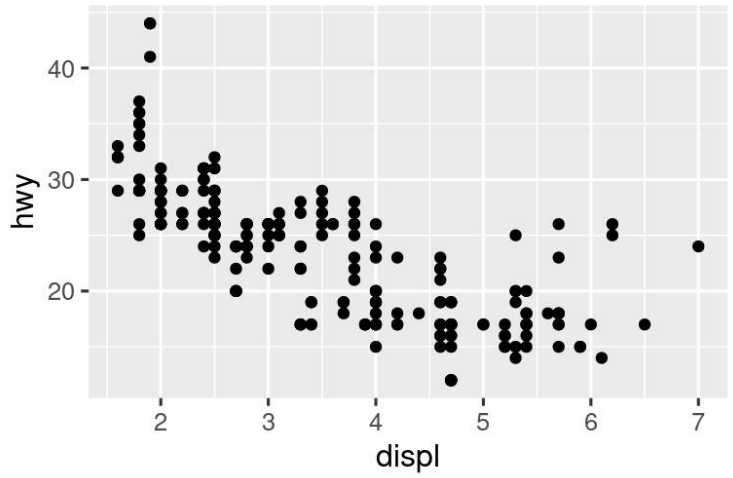
- Plots tend to be more aesthetically pleasing if they have consistent width. To enforce this, set `fig-width: 6` (6") and `fig-asp: 0.618` (the golden ratio) in the defaults. Then in individual chunks, only adjust `fig-asp`.

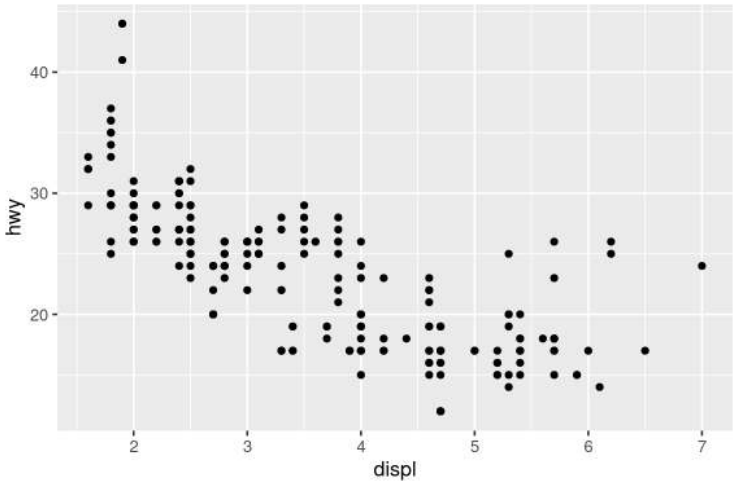
- Control the output size with `out-width` and set it to a percentage of the body width of the output document. We suggest to `out-width: "70%"` and `fig-align: center`.

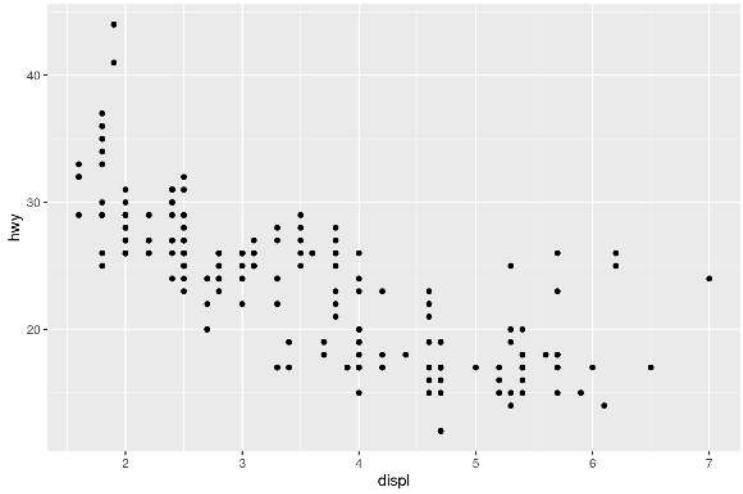
  That gives plots room to breathe, without taking up too much space.

- To put multiple plots in a single row, set the `layout-ncol` to 2 for two plots, 3 for three plots, etc. This effectively sets `out-width` to "50%" for each of your plots if `layout-ncol` is 2, "33%" if `layout-ncol` is 3, etc. Depending on what you're trying to illustrate (e.g., show data or show plot variations), you might also tweak `fig-width`, as discussed below.

If you find that you're having to squint to read the text in your plot, you need to tweak `fig-width`. If `fig-width` is larger than the size the figure is rendered in the final doc, the text will be too small; if `fig-width` is smaller, the text will be too big. You'll often need to do a little experimentation to figure out the right ratio between the `fig-width` and the eventual width in your document. To illustrate the principle, the following three plots have `fig-width` of 4, 6, and 8 respectively:







If you want to make sure the font size is consistent across all your figures, whenever you set `out-width`, you'll also need to adjust `fig-width` to maintain the same ratio with your default `out-width`. For example, if your default `fig-width` is 6 and `out-width` is "70%", when you set `out-width: "50%"` you'll need to set `fig-width` to 4.3 (6 * 0.5 / 0.7).

Figure sizing and scaling is an art and science and getting things right can require an iterative trial-and-error approach. You can learn more about figure sizing in the taking control of plot scaling blog post.

28.6.2 Other important options

When mingling code and text, like in this book, you can set `fig-show: hold` so that plots are shown after the code. This has the pleasant side effect of forcing you to break up large blocks of code with their explanations.

To add a caption to the plot, use `fig-cap`. In Quarto this will change the figure from inline to "floating".

If you're producing PDF output, the default graphics type is PDF. This is a good default because PDFs are high quality vector graphics. However, they can produce very large and slow plots if you are displaying thousands of points. In that case, set `fig-format: "png"` to force the use of PNGs. They are slightly lower quality, but will be much more compact.

It's a good idea to name code chunks that produce figures, even if you don't routinely label other chunks. The chunk label is used to generate the file name of the graphic on disk, so naming your chunks makes it much easier to pick out plots and reuse in other circumstances (e.g., if you want to quickly drop a single plot into an email).

28.6.3 Exercises

1. Open `diamond-sizes.qmd` in the visual editor, find an image of a diamond, copy it, and paste it into the document. Double click on the image and add a caption. Resize the image and render your document. Observe how the image is saved in your current working directory.
2. Edit the label of the code chunk in `diamond-sizes.qmd` that generates a plot to start with the prefix `fig-` and add a caption to the figure with the chunk option `fig-cap`. Then, edit the text above the code chunk to add a cross-reference to the figure with Insert > Cross Reference.
3. Change the size of the figure with the following chunk options, one at a time, render your document, and describe how the figure changes.

    a. `fig-width: 10`

    b. `fig-height: 3`

    c. `out-width: "100%"`

    d. `out-width: "20%"`

28.7 Tables

Similar to figures, you can include two types of tables in a Quarto document. They can be markdown tables that you create directly in your Quarto document (using the Insert Table menu) or they can be tables

generated as a result of a code chunk. In this section we will focus on the latter, tables generated via computation.

By default, Quarto prints data frames and matrices as you'd see them in the console:

```
mtcars[1:5, ]
#>                   mpg cyl disp  hp drat    wt  qsec vs am gear carb
#> Mazda RX4         21.0   6  160 110 3.90 2.620 16.46  0  1    4    4
#> Mazda RX4 Wag     21.0   6  160 110 3.90 2.875 17.02  0  1    4    4
#> Datsun 710        22.8   4  108  93 3.85 2.320 18.61  1  1    4    1
#> Hornet 4 Drive    21.4   6  258 110 3.08 3.215 19.44  1  0    3    1
#> Hornet Sportabout 18.7   8  360 175 3.15 3.440 17.02  0  0    3    2
```

If you prefer that data be displayed with additional formatting you can use the `knitr::kable()` function. The code below generates Table 28.1.

```
knitr::kable(mtcars[1:5, ], )
```

Table 28.1: A knitr kable.

	mpg	cyl	disp	hp	drat	wt	qsec	vs	am	gear	carb
Mazda RX4	21.0	6	160	110	3.90	2.620	16.46	0	1	4	4
Mazda RX4 Wag	21.0	6	160	110	3.90	2.875	17.02	0	1	4	4
Datsun 710	22.8	4	108	93	3.85	2.320	18.61	1	1	4	1
Hornet 4 Drive	21.4	6	258	110	3.08	3.215	19.44	1	0	3	1
Hornet Sportabout	18.7	8	360	175	3.15	3.440	17.02	0	0	3	2

Read the documentation for `?knitr::kable` to see the other ways in which you can customize the table. For even deeper customization, consider the **gt**, **huxtable**, **reactable**, **kableExtra**, **xtable**, **stargazer**, **pander**, **tables**, and **ascii** packages. Each provides a set of tools for returning formatted tables from R code.

28.7.1 Exercises

1. Open `diamond-sizes.qmd` in the visual editor, insert a code chunk, and add a table with `knitr::kable()` that shows the first 5 rows of the `diamonds` data frame.
2. Display the same table with `gt::gt()` instead.
3. Add a chunk label that starts with the prefix `tbl-` and add a caption to the table with the chunk option `tbl-cap`. Then, edit the text above the code chunk to add a cross-reference to the table with Insert > Cross Reference.

28.8 Caching

Normally, each render of a document starts from a completely clean slate. This is great for reproducibility, because it ensures that you've captured every important computation in code. However, it can be painful if

you have some computations that take a long time. The solution is `cache: true`.

You can enable the Knitr cache at the document level for caching the results of all computations in a document using standard YAML options:

```
---
title: "My Document"
execute:
  cache: true
---
```

You can also enable caching at the chunk level for caching the results of computation in a specific chunk:

````
```{r}
#| cache: true

# code for lengthy computation...
```
````

When set, this will save the output of the chunk to a specially named file on disk. On subsequent runs, knitr will check to see if the code has changed, and if it hasn't, it will reuse the cached results.

The caching system must be used with care, because by default it is based on the code only, not its dependencies. For example, here the `processed_data` chunk depends on the `raw-data` chunk:

````
```{r}
#| label: raw-data
#| cache: true

rawdata <- readr::read_csv("a_very_large_file.csv")
```
````

````
```{r}
#| label: processed_data
#| cache: true

processed_data <- rawdata |>
  filter(!is.na(import_var)) |>
  mutate(new_variable = complicated_transformation(x, y, z))
```
````

Caching the `processed_data` chunk means that it will get re-run if the dplyr pipeline is changed, but it won't get rerun if the `read_csv()` call changes. You can avoid that problem with the `dependson` chunk option:

```
```{r}
#| label: processed-data
#| cache: true
#| dependson: "raw-data"

processed_data <- rawdata |>
  filter(!is.na(import_var)) |>
  mutate(new_variable = complicated_transformation(x, y, z))
```
```

`dependson` should contain a character vector of *every* chunk that the cached chunk depends on. Knitr will update the results for the cached chunk whenever it detects that one of its dependencies have changed.

Note that the chunks won't update if `a_very_large_file.csv` changes, because knitr caching only tracks changes within the `.qmd` file. If you want to also track changes to that file you can use the `cache.extra` option. This is an arbitrary R expression that will invalidate the cache whenever it changes. A good function to use is `file.mtime()`: it returns when it was last modified. Then you can write:

```
```{r}
#| label: raw-data
#| cache: true
#| cache.extra: !expr file.mtime("a_very_large_file.csv")

rawdata <- readr::read_csv("a_very_large_file.csv")
```
```

We've followed the advice of David Robinson to name these chunks: each chunk is named after the primary object that it creates. This makes it easier to understand the `dependson` specification.

As your caching strategies get progressively more complicated, it's a good idea to regularly clear out all your caches with `knitr::clean_cache()`.

28.8.1 Exercises

1. Set up a network of chunks where `d` depends on `c` and `b`, and both `b` and `c` depend on `a`. Have each chunk print `lubridate::now()`, set `cache: true`, then verify your understanding of caching.

28.9 Troubleshooting

Troubleshooting Quarto documents can be challenging because you are no longer in an interactive R environment, and you will need to learn some new tricks. Additionally, the error could be due to issues with the Quarto document itself or due to the R code in the Quarto document.

One common error in documents with code chunks is duplicated chunk labels, which are especially pervasive if your workflow involves copying and pasting code chunks. To address this issue, all you need to do is to change one of your duplicated labels.

If the errors are due to the R code in the document, the first thing you should always try is to recreate the problem in an interactive session. Restart R, then "Run all chunks", either from the Code menu, under Run region or with the keyboard shortcut Ctrl + Alt + R. If you're lucky, that will recreate the problem, and you can figure out what's going on interactively.

If that doesn't help, there must be something different between your interactive environment and the Quarto environment. You're going to need to systematically explore the options. The most common difference is the working directory: the working directory of a Quarto is the directory in which it lives. Check the working directory is what you expect by including `getwd()` in a chunk.

Next, brainstorm all the things that might cause the bug. You'll need to systematically check that they're the same in your R session and your Quarto session. The easiest way to do that is to set `error: true` on the chunk causing the problem, then use `print()` and `str()` to check that settings are as you expect.

28.10 YAML header

You can control many other "whole document" settings by tweaking the parameters of the YAML header. You might wonder what YAML stands for: it's "YAML Ain't Markup Language", which is designed for representing hierarchical data in a way that's easy for humans to read and write. Quarto uses it to control many details of the output. Here we'll discuss three: self-contained documents, document parameters, and bibliographies.

28.10.1 Self-contained

HTML documents typically have a number of external dependencies (e.g., images, CSS style sheets, JavaScript, etc.) and, by default, Quarto places these dependencies in a `_files` folder in the same directory as your `.qmd` file. If you publish the HTML file on a hosting platform (e.g., QuartoPub, https://quartopub.com/), the dependencies in this directory are published with your document and hence are available in the published report. However, if you want to email the report to a colleague, you might prefer to have a single, self-contained, HTML document that embeds all of its dependencies. You can do this by specifying the `embed-resources` option:

```
format:
  html:
    embed-resources: true
```

The resulting file will be self-contained, such that it will need no external files and no internet access to be displayed properly by a browser.

28.10.2 Parameters

Quarto documents can include one or more parameters whose values can be set when you render the report. Parameters are useful when you want to re-render the same report with distinct values for various key inputs. For example, you might be producing sales reports per branch, exam results by student, or demographic summaries by country. To declare one or more parameters, use the `params` field.

This example uses a `my_class` parameter to determine which class of cars to display:

````
---
format: html
params:
  my_class: "suv"
---

```{r}
#| label: setup
#| include: false

library(tidyverse)

class <- mpg |> filter(class == params$my_class)
```

# Fuel economy for `r params$my_class`s

```{r}
#| message: false

ggplot(class, aes(x = displ, y = hwy)) +
  geom_point() +
  geom_smooth(se = FALSE)
```
````

As you can see, parameters are available within the code chunks as a read-only list named `params`.

You can write atomic vectors directly into the YAML header. You can also run arbitrary R expressions by prefacing the parameter value with `!expr`. This is a good way to specify date/time parameters.

```
params:
  start: !expr lubridate::ymd("2015-01-01")
  snapshot: !expr lubridate::ymd_hms("2015-01-01 12:30:00")
```

28.10.3 Bibliographies and Citations

Quarto can automatically generate citations and a bibliography in a number of styles. The most straightforward way of adding citations and bibliographies to a Quarto document is using the visual editor in RStudio.

To add a citation using the visual editor, go to Insert > Citation. Citations can be inserted from a variety of sources:

1. DOI (Document Object Identifier) references.
2. Zotero personal or group libraries.
3. Searches of Crossref, DataCite, or PubMed.
4. Your document bibliography (a `.bib` file in the directory of your document)

Under the hood, the visual mode uses the standard Pandoc markdown representation for citations (e.g., `[@citation]`).

If you add a citation using one of the first three methods, the visual editor will automatically create a `bibliography.bib` file for you and add the reference to it. It will also add a `bibliography` field to the document YAML. As you add more references, this file will get populated with their citations. You can also directly edit this file using many common bibliography formats including BibLaTeX, BibTeX, EndNote, Medline.

To create a citation within your .qmd file in the source editor, use a key composed of '@' + the citation identifier from the bibliography file. Then place the citation in square brackets. Here are some examples:

```
Separate multiple citations with a `;`: Blah blah [@smith04; @doe99].

You can add arbitrary comments inside the square brackets:
Blah blah [see @doe99, pp. 33-35; also @smith04, ch. 1].

Remove the square brackets to create an in-text citation: @smith04
says blah, or @smith04 [p. 33] says blah.

Add a `-` before the citation to suppress the author's name:
Smith says blah [-@smith04].
```

When Quarto renders your file, it will build and append a bibliography to the end of your document. The bibliography will contain each of the cited references from your bibliography file, but it will not contain a section heading. As a result it is common practice to end your file with a section header for the bibliography, such as `# References` or `# Bibliography`.

You can change the style of your citations and bibliography by referencing a CSL (citation style language) file in the `csl` field:

```
bibliography: rmarkdown.bib
csl: apa.csl
```

As with the bibliography field, your csl file should contain a path to the file. Here we assume that the csl file is in the same directory as the .qmd file. A good place to find CSL style files for common bibliography styles is https://github.com/citation-style-language/styles.

28.11 Workflow

Earlier, we discussed a basic workflow for capturing your R code where you work interactively in the *console*, then capture what works in the *script editor*. Quarto brings together the console and the script editor, blurring the lines between interactive exploration and long-term code capture. You can rapidly iterate within a chunk, editing and re-executing with Cmd/Ctrl + Shift + Enter. When you're happy, you move on and start a new chunk.

Quarto is also important because it so tightly integrates prose and code. This makes it a great **analysis notebook** because it lets you develop code and record your thoughts. An analysis notebook shares many of the same goals as a classic lab notebook in the physical sciences. It:

- Records what you did and why you did it. Regardless of how great your memory is, if you don't record what you do, there will come a time when you have forgotten important details. Write them down so you don't forget!
- Supports rigorous thinking. You are more likely to come up with a strong analysis if you record your thoughts as you go, and continue to reflect on them. This also saves you time when you eventually write up your analysis to share with others.
- Helps others understand your work. It is rare to do data analysis by yourself, and you'll often be working as part of a team. A lab notebook helps you share not only what you've done, but why you did it with your colleagues or lab mates.

Much of the good advice about using lab notebooks effectively can also be translated to analysis notebooks. We've drawn on our own experiences and Colin Purrington's advice on lab notebooks (https://colinpurrington.com/tips/lab-notebooks) to come up with the following tips:

- Ensure each notebook has a descriptive title, an evocative file name, and a first paragraph that briefly describes the aims of the analysis.
- Use the YAML header date field to record the date you started working on the notebook:

  ```
  date: 2016-08-23
  ```

  Use ISO8601 YYYY-MM-DD format so that's there no ambiguity. Use it even if you don't normally write dates that way!
- If you spend a lot of time on an analysis idea and it turns out to be a dead end, don't delete it! Write up a brief note about why it failed and leave it in the notebook. That will help you avoid going down the same dead end when you come back to the analysis in the future.
- Generally, you're better off doing data entry outside of R. But if you do need to record a small snippet of data, clearly lay it out using `tibble::tribble()`.
- If you discover an error in a data file, never modify it directly, but instead write code to correct the value. Explain why you made the fix.
- Before you finish for the day, make sure you can render the notebook. If you're using caching, make sure to clear the caches. That will let you fix any problems while the code is still fresh in your mind.
- If you want your code to be reproducible in the long-run (i.e. so you can come back to run it next month or next year), you'll need to track the versions of the packages that your code uses. A rigorous approach is to use **renv**, https://rstudio.github.io/renv/index.html, which stores packages in your project directory. A quick and dirty hack is to include a chunk that runs `sessionInfo()` — that won't let you easily recreate your packages as they are today, but at least you'll know what they were.
- You are going to create many, many, many analysis notebooks over the course of your career. How are you going to organize them so you can find them again in the future? We recommend storing them in individual projects, and coming up with a good naming scheme.

28.12 Summary

In this chapter we introduced you to Quarto for authoring and publishing reproducible computational documents that include your code and your prose in one place. You've learned about writing Quarto documents in RStudio with the visual or the source editor, how code chunks work and how to customize options for them, how to include figures and tables in your Quarto documents, and options for caching for computations. Additionally, you've learned about adjusting YAML header options for creating self-contained or parametrized documents as well as including citations and bibliography. We have also given you some troubleshooting and workflow tips.

While this introduction should be sufficient to get you started with Quarto, there is still a lot more to learn. Quarto is still relatively young, and is still growing rapidly. The best place to stay on top of innovations is the official Quarto website: https://quarto.org.

There are two important topics that we haven't covered here: collaboration and the details of accurately communicating your ideas to other humans. Collaboration is a vital part of modern data science, and you can make your life much easier by using version control tools, like Git and GitHub. We recommend "Happy Git with R", a user friendly introduction to Git and GitHub from R users, by Jenny Bryan. The book is freely available online: https://happygitwithr.com.

We have also not touched on what you should actually write in order to clearly communicate the results of your analysis. To improve your writing, we highly recommend reading either *Style: Lessons in Clarity and Grace* by Joseph M. Williams & Joseph Bizup, or *The Sense of Structure: Writing from the Reader's Perspective* by George Gopen. Both books will help you understand the structure of sentences and paragraphs, and give you the tools to make your writing more clear. (These books are rather expensive if purchased new, but they're used by many English classes so there are plenty of cheap second-hand copies). George Gopen also has a number of short articles on writing at https://www.georgegopen.com/litigation-articles.html. They are aimed at lawyers, but almost everything applies to data scientists too.

29 Quarto formats

29.1 Introduction

So far, you've seen Quarto used to produce HTML documents. This chapter gives a brief overview of some of the many other types of output you can produce with Quarto.

There are two ways to set the output of a document:

1. Permanently, by modifying the YAML header:

   ```
   title: "Diamond sizes"
   format: html
   ```

2. Transiently, by calling `quarto::quarto_render()` by hand:

   ```
   quarto::quarto_render("diamond-sizes.qmd", output_format = "docx")
   ```

   This is useful if you want to programmatically produce multiple types of output since the `output_format` argument can also take a list of values.

   ```
   quarto::quarto_render("diamond-sizes.qmd", output_format = c("docx",
   "pdf"))
   ```

29.2 Output options

Quarto offers a wide range of output formats. You can find the complete list at https://quarto.org/docs/output-formats/all-formats.html. Many formats share some output options (e.g., `toc: true` for including a table of contents), but others have options that are format specific (e.g., `code-fold: true` collapses code chunks into a `<details>` tag for HTML output so the user can display it on demand, it's not applicable in a PDF or Word document).

To override the default options, you need to use an expanded `format` field. For example, if you wanted to render an `html` with a floating table of contents, you'd use:

```
format:
  html:
```

```
    toc: true
    toc_float: true
```

You can even render to multiple outputs by supplying a list of formats:

```
format:
  html:
    toc: true
    toc_float: true
  pdf: default
  docx: default
```

Note the special syntax (`pdf: default`) if you don't want to override any default options.

To render to all formats specified in the YAML of a document, you can use `output_format = "all"`.

```
quarto::quarto_render("diamond-sizes.qmd", output_format = "all")
```

29.3 Documents

The previous chapter focused on the default `html` output. There are several basic variations on that theme, generating different types of documents. For example:

- `pdf` makes a PDF with LaTeX (an open-source document layout system), which you'll need to install. RStudio will prompt you if you don't already have it.
- `docx` for Microsoft Word (`.docx`) documents.
- `odt` for OpenDocument Text (`.odt`) documents.
- `rtf` for Rich Text Format (`.rtf`) documents.
- `gfm` for a GitHub Flavored Markdown (`.md`) document.
- `ipynb` for Jupyter Notebooks (`.ipynb`).

Remember, when generating a document to share with decision-makers, you can turn off the default display of code by setting global options in the document YAML:

```
execute:
  echo: false
```

For `html` documents another option is to make the code chunks hidden by default, but visible with a click:

```
format:
  html:
    code: true
```

29.4 Presentations

You can also use Quarto to produce presentations. You get less visual control than with a tool like Keynote or PowerPoint, but automatically inserting the results of your R code into a presentation can save a huge amount of time. Presentations work by dividing your content into slides, with a new slide beginning at each second (`##`) level header. Additionally, first (`#`) level headers indicate the beginning of a new section with a section title slide that is, by default, centered in the middle.

Quarto supports a variety of presentation formats, including:

1. `revealjs` - HTML presentation with revealjs
2. `pptx` - PowerPoint presentation
3. `beamer` - PDF presentation with LaTeX Beamer.

You can read more about creating presentations with Quarto at https://quarto.org/docs/presentations.

29.5 Interactivity

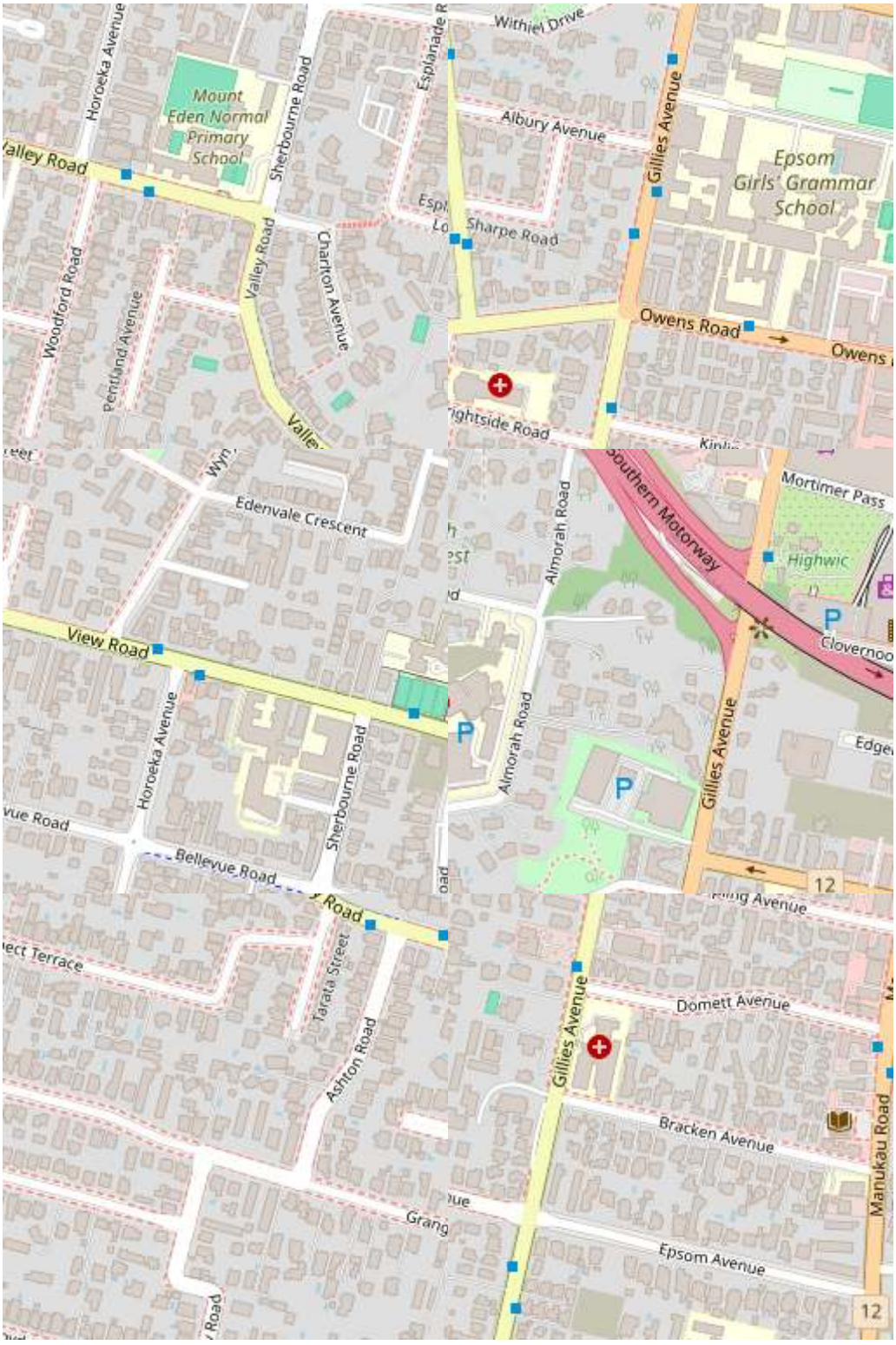
Just like any HTML document, HTML documents created with Quarto can contain interactive components as well. Here we introduce two options for including interactivity in your Quarto documents: htmlwidgets and Shiny.

29.5.1 htmlwidgets

HTML is an interactive format, and you can take advantage of that interactivity with **htmlwidgets**, R functions that produce interactive HTML visualizations. For example, take the **leaflet** map below. If you're viewing this page on the web, you can drag the map around, zoom in and out, etc. You obviously can't do that in a book, so Quarto automatically inserts a static screenshot for you.

```
library(leaflet)
leaflet() |>
  setView(174.764, -36.877, zoom = 16) |>
  addTiles() |>
  addMarkers(174.764, -36.877, popup = "Maungawhau")
```


Bellevue Road
Percy Street
Mount Eden Road
Puhi Huia Road
Wilkies Lookout
Omana Avenue
Batger Road
Owens Road
Barnett Crescent
Rockwood Place
Clive Road
Esplanade Road
Puka Street
Tahaki Reserve
Castle Drive
Mountain Road
View Road
Clifton House
Glenfell Place
Rautangi Road
Valley Road
Oaklands Road
Cecil Road
Maranui Avenue
Shipherds Avenue
Essex Road
Stokes Road
Mount Eden
Epsom Avenue
Ngauruhoe Street
Te Pou Hawaiki
Poronui Street



+−

Leaflet | © OpenStreetMap, ODbL

The great thing about htmlwidgets is that you don't need to know anything about HTML or JavaScript to use them. All the details are wrapped inside the package, so you don't need to worry about it.

There are many packages that provide htmlwidgets, including:

- **dygraphs** for interactive time series visualizations.
- **DT** for interactive tables.
- **threejs** for interactive 3d plots.
- **DiagrammeR** for diagrams (like flow charts and simple node-link diagrams).

To learn more about htmlwidgets and see a complete list of packages that provide them visit https://www.htmlwidgets.org.

29.5.2 Shiny

htmlwidgets provide **client-side** interactivity — all the interactivity happens in the browser, independently of R. On the one hand, that's great because you can distribute the HTML file without any connection to R. However, that fundamentally limits what you can do to things that have been implemented in HTML and JavaScript. An alternative approach is to use **shiny**, a package that allows you to create interactivity using R code, not JavaScript.

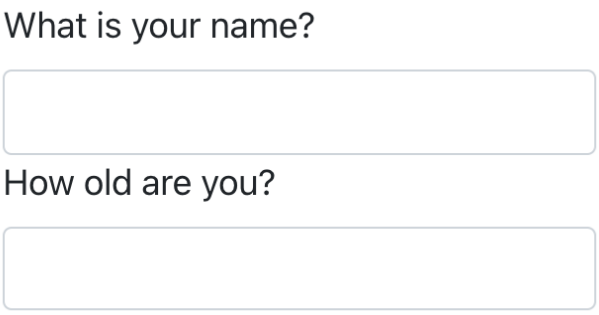
To call Shiny code from a Quarto document, add `server: shiny` to the YAML header:

```
title: "Shiny Web App"
format: html
server: shiny
```

Then you can use the "input" functions to add interactive components to the document:

```
library(shiny)

textInput("name", "What is your name?")
numericInput("age", "How old are you?", NA, min = 0, max = 150)
```



And you also need a code chunk with chunk option `context: server` which contains the code that needs to run in a Shiny server.

You can then refer to the values with `input$name` and `input$age`, and the code that uses them will be automatically re-run whenever they change.

We can't show you a live shiny app here because shiny interactions occur on the **server-side**. This means that you can write interactive apps without knowing JavaScript, but you need a server to run them on. This introduces a logistical issue: Shiny apps need a Shiny server to be run online. When you run Shiny apps on your own computer, Shiny automatically sets up a Shiny server for you, but you need a public-facing Shiny server if you want to publish this sort of interactivity online. That's the fundamental trade-off of shiny: you can do anything in a shiny document that you can do in R, but it requires someone to be running R.

For learning more about Shiny, we recommend reading Mastering Shiny by Hadley Wickham, https://mastering-shiny.org.

29.6 Websites and books

With a bit of additional infrastructure, you can use Quarto to generate a complete website or book:

- Put your `.qmd` files in a single directory. `index.qmd` will become the home page.
- Add a YAML file named `_quarto.yml` that provides the navigation for the site. In this file, set the `project` type to either `book` or `website`, e.g.:

```
project:
  type: book
```

For example, the following `_quarto.yml` file creates a website from three source files: `index.qmd` (the home page), `viridis-colors.qmd`, and `terrain-colors.qmd`.

```
project:
  type: website
```

```
website:
  title: "A website on color scales"
  navbar:
    left:
      - href: index.qmd
        text: Home
      - href: viridis-colors.qmd
        text: Viridis colors
      - href: terrain-colors.qmd
        text: Terrain colors
```

The `_quarto.yml` file you need for a book is very similarly structured. The following example shows how you can create a book with four chapters that renders to three different outputs (`html`, `pdf`, and `epub`). Once again, the source files are `.qmd` files.

```
project:
  type: book

book:
  title: "A book on color scales"
  author: "Jane Coloriste"
  chapters:
    - index.qmd
    - intro.qmd
    - viridis-colors.qmd
    - terrain-colors.qmd

format:
  html:
    theme: cosmo
  pdf: default
  epub: default
```

We recommend that you use an RStudio project for your websites and books. Based on the `_quarto.yml` file, RStudio will recognize the type of project you're working on, and add a Build tab to the IDE that you can use to render and preview your websites and books. Both websites and books can also be rendered using `quarto::quarto_render()`.

Read more at https://quarto.org/docs/websites about Quarto websites and https://quarto.org/docs/books about books.

29.7 Other formats

Quarto offers even more output formats:

- You can write journal articles using Quarto Journal Templates: https://quarto.org/docs/journals/templates.html.
- You can output Quarto documents to Jupyter Notebooks with `format: ipynb`: https://quarto.org/docs/reference/formats/ipynb.html.

See https://quarto.org/docs/output-formats/all-formats.html for a list of even more formats.

29.8 Summary

In this chapter we presented you a variety of options for communicating your results with Quarto, from static and interactive documents to presentations to websites and books.

To learn more about effective communication in these different formats, we recommend the following resources:

- To improve your presentation skills, try *Presentation Patterns* by Neal Ford, Matthew McCollough, and Nathaniel Schutta. It provides a set of effective patterns (both low- and high-level) that you can apply to improve your presentations.
- If you give academic talks, you might like the *Leek group guide to giving talks*.
- We haven't taken it ourselves, but we've heard good things about Matt McGarrity's online course on public speaking: https://www.coursera.org/learn/public-speaking.
- If you are creating many dashboards, make sure to read Stephen Few's *Information Dashboard Design: The Effective Visual Communication of Data*. It will help you create dashboards that are truly useful, not just pretty to look at.
- Effectively communicating your ideas often benefits from some knowledge of graphic design. Robin Williams' *The Non-Designer's Design Book* is a great place to start.